

CYBER ELIAS ACADEMY

TEACHING MANUAL

Web Design

Session Roadmap · Lesson Notes · Concept Diagrams · Answer Keys

READY TO TEACH

© 2026 Cyber Elias Academy · cea.ng

Table of Contents

Course Roadmap

Session 01 — HOW THE WEB WORKS

Session 02 — HTML BASICS

Session 03 — HTML INTERMEDIATE

Session 04 — CSS BASICS

Session 05 — CSS LAYOUT

Session 06 — CSS RESPONSIVE DESIGN

Session 07 — PLANNING A WEBSITE

Session 08 — BUILDING THE STRUCTURE

Session 09 — STYLING THE WEBSITE

Session 10 — INTERACTIVITY AND MEDIA

Session 11 — TESTING AND DEBUGGING

Session 12 — FINAL PROJECT: CAPSTONE (FINALISE AND PRESENT)

OVERVIEW

Course Roadmap

What we cover in every session

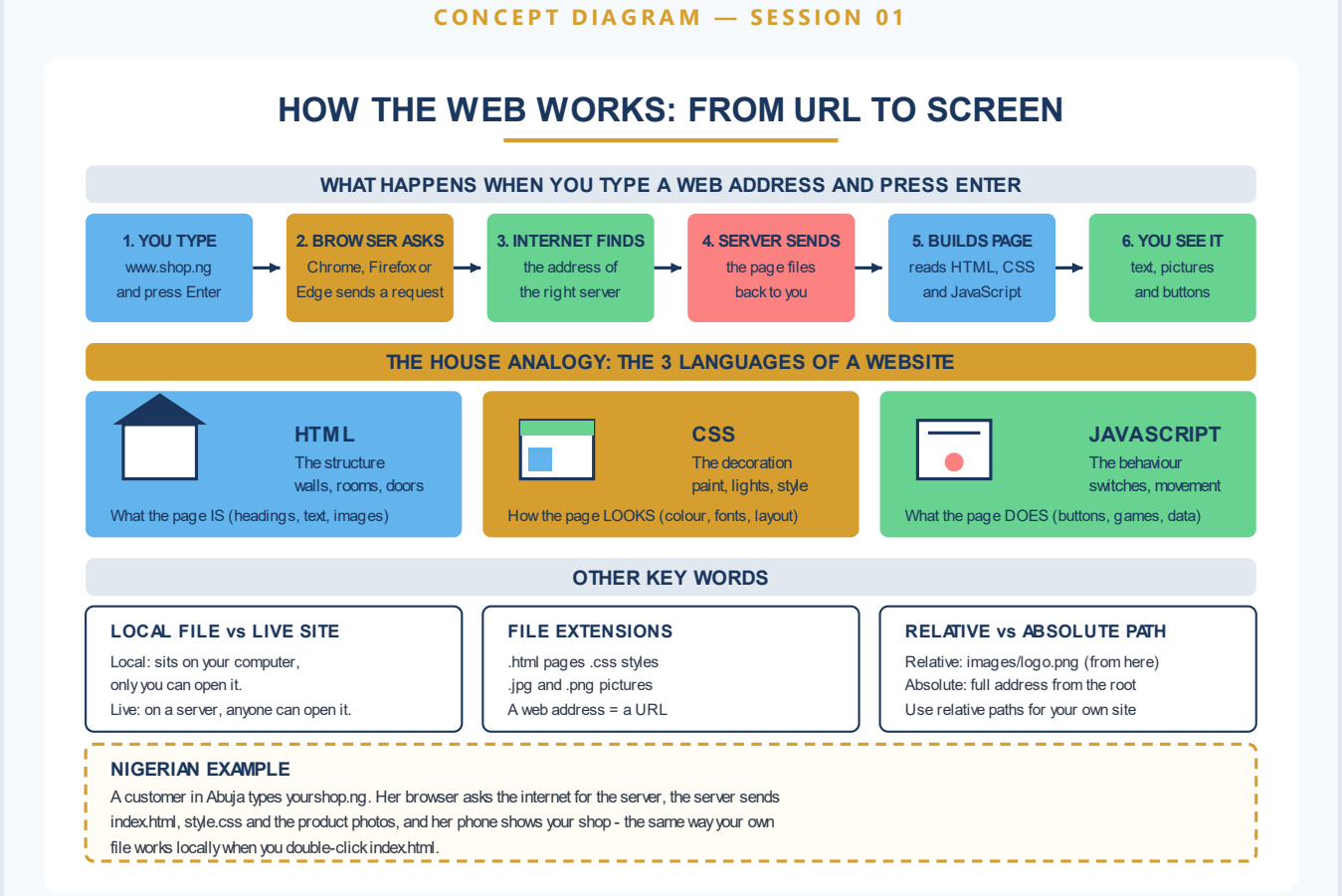
#	SESSION TOPIC	WEEK / MODULE	DURATION
01	HOW THE WEB WORKS	Week 1 — Module 1: HTML Foundations	105 minutes
02	HTML BASICS	Week 1 — Module 1: HTML Foundations	105 minutes
03	HTML INTERMEDIATE	Week 1 — Module 1: HTML Foundations	105 minutes
04	CSS BASICS	Week 2 — Module 2: CSS Foundations	105 minutes
05	CSS LAYOUT	Week 2 — Module 2: CSS Foundations	105 minutes
06	CSS RESPONSIVE DESIGN	Week 2 — Module 2: CSS Foundations	105 minutes
07	PLANNING A WEBSITE	Week 3 — Module 3: Building a Website	105 minutes
08	BUILDING THE STRUCTURE	Week 3 — Module 3: Building a Website	105 minutes
09	STYLING THE WEBSITE	Week 3 — Module: Building a Website	105 minutes
10	INTERACTIVITY AND MEDIA	Week 4 — Module: Publishing and Capstone	105 minutes
11	TESTING AND DEBUGGING	Week 4 — Module: Publishing and Capstone	105 minutes
12	FINAL PROJECT: CAPSTONE (FINALISE AND PRESENT)	Week 4 — Module: Publishing and Capstone	105 minutes

How to use this book: Each session below is a complete, ready-to-teach lesson note. Section A tells you what to prepare, B lists the outcomes, D–F are the teaching content, G–H are your demos and practicals, and the knowledge check with answer key is at the end. The concept diagram for each session is embedded as a visual aid.

SESSION 01 OF 12

HOW THE WEB WORKS

CONCEPT DIAGRAM — SESSION 01



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 1 — HOW THE WEB WORKS

A. SESSION IDENTIFICATION

Item	Detail
Session number	1 of 12
Session title	How the Web Works
Week / Module	Week 1 — Module 1: HTML Foundations
Recommended duration	105 minutes
Difficulty level	Beginner (first contact with web design)

Item	Detail
Prerequisite knowledge	Basic computer skills: creating folders, saving files, using a browser, typing. A learner with no prior coding experience is expected and welcome.
Connection to previous work	This is the first session of Web Design. It sets the foundation every later session builds on.
Preparation before class	Academy PCs on and logged in; Google Chrome installed; Visual Studio Code installed; projector connected to the instructor machine; a folder `webdesign-practice` created on each learner Desktop (or the learners create it in Section H); a simple demo HTML file prepared for Section G.
Running project	Across the whole course we are building a small business website for **Adaeze Fashion**, a clothing business in Port Harcourt. This session we simply meet the idea of a website and set up the tools.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in their own words** what happens, step by step, when a person types a website address (URL) into a browser and presses Enter.
2. **Describe what a web browser is** and name at least three popular browsers used today.
3. **Describe what a web server is** and explain, in simple language, how it delivers a webpage to a browser.
4. **Distinguish** between *local files* (files sitting on your own computer) and *live websites* (files served to anybody on the internet).
5. **Identify the three core languages** of a webpage — HTML, CSS and JavaScript — and state what each one does, using the house analogy.
6. **Explain what a file extension is** and correctly name the extensions `.html`, `.css`, `.jpg` / `.png` and recognise a website address (URL) when they see one.
7. **Explain the difference** between a *relative path* and an *absolute path*, and read a simple file path such as `images/logo.png`.
8. **Set up a workspace** in Visual Studio Code using the folder `webdesign-practice`.
9. **Create, save and open** a simple `index.html` file in Google Chrome without help.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Sharing one machine between two learners is acceptable — pair them so one types and the other reads the screen aloud, then swap halfway.
- Instructor computer connected to a projector or large screen.
- Optional: one smartphone or tablet per table, for looking at a real website later.

Software

- Google Chrome (any recent version). The wording of menus is stable across modern versions.
- Visual Studio Code (VS Code). Any version from 2022 onward works.
- Windows 10 or Windows 11.

Accounts

- None required. All work this session is offline and local.

Sample / starter files

- Instructor prepares a tiny file `demo/index.html` to show the "magic trick" in Section D (contents given in Section G). It contains one heading and one paragraph.
- Optional: a small image file such as `logo.png` to demonstrate that a webpage can show pictures.

Worksheets / printed materials

- Worksheet 1.1 — "The Journey of a URL" (one page, with four blank boxes to fill in).
- Worksheet 1.2 — "Label the House" (a simple house drawing with three blank labels for HTML, CSS, JavaScript).
- "Web Vocabulary Quick Card" (one per learner: browser, server, URL, DNS, HTML, CSS, JavaScript, file extension).

Internet

- Not required for this session — all work is done on local files. If the academy has Wi-Fi, the instructor may optionally open a real website (e.g. a bank or news site) at the end to show "View Page Source", but do not depend on it.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. If a learner has only a phone, pair them with a desktop user for the session; afterwards they can practise by viewing page source on the phone browser and taking photos of the code.

D. OPENING ACTIVITY (~6 minutes)

"The Magic Trick"

Instructor setup: On the projector, open the prepared `demo/index.html` file in Chrome. It should show a short heading and a paragraph (for example: *"Welcome to Adaeze Fashion"* and *"Beautiful clothes made in Port Harcourt."*). Do **not** show the code yet — show only the finished page.

What the instructor says:

> *"Good morning, everyone, and welcome to Web Design at Cyber Elias Academy. Before we type a single line, I want to show you a small magic trick.*
>
> *Look at this screen. What do you see? … A beautiful page with words on it. Adaeze Fashion — a clothing business right here in Port Harcourt. Now watch carefully."*
>
> *[Right-click on the page → **View Page Source** (or press `Ctrl + U`).]*
>
> *"Do you see this jumble of words with little marks like `<h1>`? **This** is what makes the page. Every website you have ever visited — Google, your bank, Jumia, your church's page — is built from text like this. It is not magic; it is a language, and it has rules.*
>
> *By the end of this course you will write this language yourself. Today we start by understanding how the whole web works — because you cannot build a house well if you do not know what a house is for."*

Expected learner response: Curiosity and some nervousness. Learners may say the code looks "hard" or "like a virus". Reassure them:

> *"It looks strange today. In four weeks it will read like plain English. I promise."*

Some learners may recognise the tags and say "that's HTML". Praise them and say, "Exactly — and today we find out what HTML really is and where it fits."

Why this opener works: It creates a small surprise (the page is "hiding" code), connects the work to a familiar local business, and removes the fear that coding is magic. It also gives the session a clear destination: by the end, they will do this themselves.

E. PREREQUISITE REVIEW (~7 minutes)

Run this as a quick oral round-robin. Put the questions on the projector. Every learner should answer at least one. These check the basic computer skills the course assumes.

Question 1: "How do you create a new folder on the Desktop?"

- Expected answer: Right-click an empty space → **New** → **Folder** → type a name → press Enter.

- Common gap: A learner may only know phones, not Windows. Show them physically on one machine.

Question 2: "What is a file extension? Give one example."

- Expected answer: The letters after the dot in a file name — for example `.html`, `.css`, `.jpg`, `.txt`. It tells the computer what kind of file it is.
- Common gap: Learners confuse the dot in a sentence with the dot in a file name, or think the extension is the whole name.

Question 3: "What is the difference between a file and a folder?"

- Expected answer: A file is a single piece of information (a document, an image). A folder is a container that holds files and other folders.
- Common gap: Some learners call everything a "document". Use the analogy below.

Question 4: "How do you save a file, and then how do you see changes in a browser?"

- Expected answer: `Ctrl + S` to save; then refresh the browser with `F5` (or `Ctrl + R`).
- Common gap: Learners change code but forget to save, then wonder why nothing changed. This is the number-one beginner habit to fix, so repeat it now.

Question 5: "Name one web browser you have used."

- Expected answer: Chrome, Firefox, Edge, Opera, Safari, or a phone browser. Any is fine.

Bridge to today (say this):

> *"Good. You can make folders, save files and use a browser. That is all you need to be a web designer. Think of a website as a **house you are building**:*
>
> - ***HTML** is the walls, doors and rooms — the structure and the contents.*
> - ***CSS** is the paint, tiles and curtains — the appearance.*
> - ***JavaScript** is the electricity and plumbing — the things that do something.*
>
> *Before we can build the house, we must understand the **village** it stands in. That village is the web. Let us begin."*

Gap handling: If more than a third of the class cannot create a folder or save a file, pause and do it physically with each struggling learner while the rest begin Section F quietly. Do not skip this — every later session depends on it. Pair strong learners with weak ones for the rest of the day.

F. FULL LECTURE NOTES (~40 minutes)

F.1 What the internet is, and how it differs from the web

The idea. The **internet** is a very large number of computers connected together so they can send information to one another. The **World Wide Web** (usually just "the web") is the collection of pages and websites that travel over that connection.

Plain-English definition.

- **Internet** = the network of cables, routers, satellites and computers that connect the world. It is the *infrastructure*.
- **World Wide Web** = the websites and web pages you look at *through* that infrastructure. It is the *content*.

Why they exist. People needed a way to send messages and share documents between computers far apart. The network (internet) came first. Later, someone invented an easy way to link documents together so they could be read with a click — that is the web.

Familiar analogy. Think of **roads and buildings**.

- The **internet** is the road network: streets, bridges, junctions. You drive on it but it is not a destination.
- The **web** is the buildings along the roads: shops, schools, offices. They are what you actually visit.
- You need the roads to reach the buildings, but the roads and the buildings are not the same thing.

A short, honest history. The internet grew out of research networks in the 1960s. The World Wide Web was invented in 1989 by **Tim Berners-Lee**, who wanted scientists to share documents easily. He created the first web browser and the first website. (Do not quote exact user numbers or dates you have not verified — the point is simply that the web is younger than the internet.)

Misconception to correct. "The internet and the web are the same thing." They are not. You can use the internet to send an email or make a WhatsApp call *without* visiting a web page at all. Email and video calls use the internet but are not "the web".

Limitation / honesty note. A learner does not need to know the deep technical details of cables and satellites to design websites. They need the difference between *getting to a page* and *the page itself*, and that is what the analogy gives them.

F.2 What happens when you type a URL — the journey, step by step

This is the heart of the session. Write these steps on the whiteboard as you explain them, and have learners fill in Worksheet 1.1.

First, what is a URL? URL stands for **Uniform Resource Locator**. In plain English it is a **web address** — the words you type to reach a page. Example: `https://www.adaezefashion.com/products.html`.

Step 1 — You type the address and press Enter. Your **browser** (Chrome) reads what you typed.

Step 2 — The browser asks DNS for directions. Computers do not really use names like `adaezefashion.com`; they use numbers called **IP addresses** (for example `192.0.2.10`). **DNS** stands for **Domain Name System**. It is like the **contacts app on your phone**: you tap a name, and the phone finds the number. DNS takes the name you typed and finds the number (IP address) of the computer that holds the website.

Step 3 — The browser sends a request to the server. The browser now travels across the internet to the correct **server** and asks: "Please give me the page `products.html`." This request is called an **HTTP request** (HTTP means **HyperText Transfer Protocol** — simply, the agreed set of rules computers use to ask for and send web pages).

Step 4 — The server responds. The **web server** is a computer that stores the website's files. It finds `products.html` (plus the CSS file, the images, and so on) and sends them back to your browser. This is called the **HTTP response**.

Step 5 — The browser renders the page. "Rendering" means **drawing the page on the screen**. The browser reads the HTML (structure), applies the CSS (appearance) and runs the JavaScript (behaviour), then paints the finished page for you. All of this usually takes a fraction of a second.

Analogy for the whole journey — ordering food.

1. You decide what you want and place the order (typing the URL).
2. The waiter checks the kitchen's address (DNS finding the IP).
3. The waiter carries your order to the kitchen (the request).
4. The kitchen cooks the food and sends it out (the response).
5. The waiter brings it to your table and you eat (the browser renders the page).

Worked example — follow `www.adaezefashion.com/index.html` together:

1. Browser reads `www.adaezefashion.com/index.html`.
2. DNS finds that `adaezefashion.com` lives at, say, `192.0.2.10`.
3. Browser asks that server for `index.html`.
4. Server sends back the HTML text, the CSS file and the images.
5. Browser draws Adaeze's home page — the logo, the headings, the colours.

Misconceptions.

- "The browser keeps the website inside it." No — the browser *asks* the server each time (though it may keep a **cache**, a saved copy, to load faster next time).
- "Typing a URL downloads the whole website at once." No — the browser asks for the HTML first, then requests the images and CSS it discovers inside.

Limitation. Real web servers, DNS and security (HTTPS) have many more details. For now, this five-step picture is accurate enough to build on, and every later session fits inside it.

F.3 Clients and servers — who asks and who answers

The idea. In every web conversation there are two roles:

- The **client** — the computer (and program, like the browser) that *asks* for information.
- The **server** — the computer that *holds* the information and *answers*.

Plain-English definition. A **client** requests; a **server** serves.

Why it matters. When you look at your own file in the browser, there is *no server involved yet*. When you publish your site, you put your files on a server so that any client in the world can ask for them. Understanding this stops learners being confused later when "it works on my computer but not online".

Analogy. A **customer and a shopkeeper**. You (the client) ask for a product; the shopkeeper (the server) has it in stock and hands it over. The shopkeeper must be open and stocked for the customer to succeed.

Worked example. Your friend in Lagos opens `www.adaezefashion.com` on their phone.

- Their phone is the **client**.
- The computer in the data centre holding Adaeze's site is the **server**.
- The client asks; the server answers with the page. Adaeze does not have to be online at that moment — the server works for her around the clock.

Misconception. "The server is the same as the internet." No — the internet is the road; the server is a building on the road where your files live.

F.4 The three building blocks: HTML, CSS and JavaScript

The idea. Every webpage is made from three languages, each with one clear job.

Language	Full name	Job	Plain sentence
HTML	HyperText Markup Language	Structure and content	*What is on the page*
CSS	Cascading Style Sheets	Presentation and appearance	*How it looks*
JavaScript	JavaScript	Behaviour and interactivity	*How it responds*

Plain-English definitions.

- **HTML** is a **markup language**: it "marks up" content with tags so the browser knows what each part is (this is a heading, this is a paragraph, this is an image). It is *not* a programming language — it does not calculate or decide.
- **CSS** is a **styling language**: it sets colours, fonts, spacing and layout. Without CSS, every page is plain black text on a white background.
- **JavaScript** is a **programming language**: it can make decisions and react to clicks and typing. It runs inside the browser. (We touch JavaScript only lightly in this course — full JavaScript is the separate Web Development course.)

Why three, not one. Fighting one language would be messy. If structure, appearance and behaviour are mixed together, changing a colour means hunting through the content, and one mistake can break everything. Separating the three jobs keeps a website clean, easy to update and easy to reuse. This idea is called **separation of concerns**.

The house analogy (learn this by heart).

- **HTML = structure:** the walls, rooms, doors, roof — the shape of the house and what is inside it.
- **CSS = appearance:** the paint, tiles, furniture and curtains.
- **JavaScript = behaviour:** the electricity, plumbing, lights and doorbell — the parts that *do* something.

A house with only structure and appearance is beautiful but dead. Add behaviour and the lights come on.

Worked example — the same page, three layers.

```
<!-- HTML: the structure and content -->
<h1>Adaeze Fashion</h1>
<p>Beautiful clothes made in Port Harcourt.</p>
```

```
/* CSS: the appearance */
h1 { color: #8B0000; text-align: center; }
p  { font-size: 18px; }
```

```
/* JavaScript: the behaviour (we only peek at this for now) */
button.onclick = function () { alert("Thank you for your order!"); };
```

Misconception. "HTML is the design." No — HTML is the *structure*. It says *what* the content is, not *how pretty* it looks. Many beginners think making text bigger in HTML is "design"; real design happens in CSS.

Limitation. Three building blocks are the foundation, but a modern site may also use frameworks and backend code. For a small business website (which is our goal), HTML and CSS alone can produce a professional result.

F.5 File types, extensions and what they mean

The idea. A **file extension** is the group of letters after the final dot in a file name. It tells the computer (and the browser) what kind of file it is and how to open it.

Common web file types:

Extension	Full name	What it is	Example
`.html`	HyperText Markup Language	A web page — the structure and content	`index.html`
`.css`	Cascading Style Sheets	The styling file	`style.css`
`.jpg` / `.jpeg`	JPEG image	A photograph	`model.jpg`
`.png`	Portable Network Graphics	An image, often with transparency	`logo.png`
`.gif`	Graphics Interchange Format	A simple image or animation	`banner.gif`
`.svg`	Scalable Vector Graphics	A sharp logo that never blurs	`logo.svg`

Why extensions matter. If you save a web page as `index.txt`, the computer thinks it is plain text. The browser will show the code instead of the page. Saving as `index.html` tells the browser "draw this as a web page".

Worked example. You type your first page in Notepad and press Save As. If you type the name `index` and Notepad adds `.txt`, your file becomes `index.txt` and it opens wrongly. The fix: name it `index.html`, or choose "All files" in the Save As dialog.

Misconception. "The extension is optional." It is not — it decides how the file behaves. Also watch out: Windows sometimes **hides** file extensions, so learners cannot see the `.html` at all. Show them how to turn on "File name extensions" in the View menu of File Explorer.

Naming rules (good habits).

- Use **lowercase**: `index.html`, not `Index.HTML`.
- **No spaces** — use a hyphen instead: `my-photo.jpg`, not `my photo.jpg`.
- Keep names short and meaningful.

F.6 Local files versus live websites

The idea.

- A **local file** lives on *your own computer* and only you can open it. It has an address like `C:\Users\Ada\Desktop\webdesign-practice\index.html`.
- A **live website** lives on a **server** and *anyone in the world* can open it. It has an address like `https://www.adaezefashion.com`.

Why it matters. In this course, for most sessions, you will work with **local files** — fast, free and private. At the end of the course you will **publish** (upload) your files to a server so the whole world can see Adaeze Fashion.

Analogy. A local file is a **cake in your kitchen** — only you can eat it. A live website is a **cake in a shop window** — anyone passing by can see it. To move from kitchen to shop window, you must carry the cake to the shop: that is *publishing*.

Worked example.

- You open `C:\Users\Ada\Desktop\webdesign-practice\index.html` in Chrome. Only Adaeze (the learner) sees it.
- After publishing, you open `https://adaezefashion.github.io`. Now a customer in Abuja, Lagos or London can see the same page.

Misconception. "If I finish my file on my computer, my website is already online." No — you must upload it to a server first. This is why the course ends with a publishing session.

F.7 Folders and file paths (relative versus absolute)

The idea. A **file path** is the route to a file — like directions to a house. Because a website has several files (HTML, CSS, images), the links between them use paths. If a path is wrong, the link or image breaks.

Two kinds of path:

Absolute path — the full address from the very top.

- On your computer: `C:\Users\Ada\Desktop\webdesign-practice\images\logo.png`
- On the web: `https://www.adaezefashion.com/images/logo.png`
- It always points to the same place, no matter where the file using it lives.

Relative path — the route *relative to the file you are in*.

- `images/logo.png` — "go into the `images` folder, then find `logo.png`".
- `logo.png` — "find `logo.png` in the same folder as this file".
- `../logo.png` — "go **up** one folder, then find `logo.png`" (the `..` means "up one level").

Why relative paths are used. They keep a website portable. If Adaeze moves her whole website folder from her laptop to a server, relative paths still work, because they describe the *arrangement* of files, not one machine's exact location. Absolute paths on your own computer (`C:\Users\Ada\...`) would break the moment the files moved to a server.

Analogy. Think of giving directions. An **absolute** direction is "24/26 Ebony Road, Off Rumuola Road, Port Harcourt". A **relative** direction is "walk to the shop next door, then turn left". The relative direction only makes sense from where you are standing — which is why we must get the *current location* right.

Worked example — the Adaeze project folder. Imagine this arrangement:

```
webdesign-practice/
├── index.html
├── about.html
├── css/
│   └── style.css
└── images/
    └── logo.png
```

- From `index.html`, to reach the logo you write `images/logo.png`.
- From `css/style.css`, to reach the same logo you must go up one level first: `../images/logo.png`.

Misconception. "A backslash and a forward slash are the same." On Windows file paths use backslashes (`\`), but **web paths always use forward slashes** (`/`). Writing `images\logo.png` in HTML can break on many servers. Always use `/` in HTML and CSS.

F.8 The browser as a tool (and View Page Source)

The idea. The **browser** is not just a viewer — it is your most important *testing tool*. It reads HTML, CSS and JavaScript and draws the result.

Popular browsers: Google Chrome (most widely used), Mozilla Firefox, Microsoft Edge, Safari (Apple devices), Opera. We use **Chrome** because it has excellent built-in **Developer Tools** (which we meet fully in a later session).

The browser is an interpreter, not a creator. It does not "understand" English; it follows the *rules* of HTML and CSS. If you make a mistake, the browser usually guesses and keeps going, which is why one small error may cause a strange result rather than a clear alarm.

View Page Source — reading any website.

1. Open a website in Chrome.
2. Right-click anywhere → **View page source** (or press `Ctrl + U`).
3. The code that builds the page appears.

This is a powerful learning habit: whenever you like how a page is built, view its source and read how it was done. (Look and learn — but write your own code; do not copy a whole site, which may be protected by copyright.)

Worked example. Open a news site, press `Ctrl + U`, and search (in the source, press `Ctrl + F`) for the headline you see. You will find the headline text wrapped in HTML tags — proof that the page is built from text.

Misconception. "Looking at page source is hacking." It is not. The browser always receives the HTML; viewing it is a normal feature. (Note that some sites hide their JavaScript and CSS, but the HTML is always visible.)

F.9 Introduction to Visual Studio Code — your workspace

The idea. A **text editor** is a program for writing code. The right editor colours and formats the code, helping you avoid mistakes. This course uses **Visual Studio Code (VS Code)**, which is free and one of the most popular editors in the world.

What a workspace is. A **workspace** is the set of tools and folders you use to build a website:

1. A **text editor** (VS Code) to write the code.
2. A **browser** (Chrome) to view the result.
3. A **project folder** (`webdesign-practice`) to hold all your files.

Why we open the whole folder, not single files. VS Code's strength is the **Explorer** panel on the left, which shows all your files at once. Opening the folder lets you see `index.html`, the `css` folder and the `images` folder together, and create new files in the right place.

Worked example — opening the workspace.

1. Open VS Code.
2. **File → Open Folder...** → choose `webdesign-practice` on the Desktop → **Select Folder**.
3. The Explorer shows your folder contents on the left.
4. Click the **New File** icon (a page with a small `+`) → type `index.html` → Enter.
5. You now have a blank `index.html` ready to write.

Our project folder layout (we will build toward this):

```
webdesign-practice/
├── index.html      ← home page
├── about.html      ← about page (later)
├── contact.html    ← contact page (later)
├── css/
│   └── style.css   ← styling (later)
└── images/
    └── logo.png
```

Misconception. "I must buy or install many programs." No — VS Code and Chrome are both free, and they are already on the academy machines.

Limitation / honesty. VS Code has hundreds of features we will not use. That is normal — professionals use a small set of features very well.

F.10 A complete first page — read it line by line

Here is the page we will create together. For now you do not need to understand every tag (that is Session 2); you need to recognise the *shape* of an HTML document and see it run.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>
  <h1>Welcome to Adaeze Fashion</h1>
  <p>Beautiful, well-made clothes designed in Port Harcourt.</p>
</body>
</html>
```

Line by line:

- `<!DOCTYPE html>` — a declaration telling the browser "this is a modern HTML5 document". Always the first line.

- `<html lang="en">` — the **root** element; everything else lives inside it. `lang="en"` means the page's language is English (good for accessibility and search engines).
- `<head>` — a hidden section holding *information about* the page (not shown on screen).
- `<meta charset="UTF-8">` — tells the browser which characters to expect, so symbols like the Naira sign (₦) show correctly.
- `<title>…</title>` — the words shown on the **browser tab**.
- `</head>` — ends the head section.
- `<body>` — everything **visible** on the page goes here.
- `<h1>…</h1>` — the main heading.
- `<p>…</p>` — a paragraph.
- `</body>` and `</html>` — close the body and the whole document.

Expected browser result: the browser tab reads *"Adaeze Fashion — Home"*; the page shows the big heading **Welcome to Adaeze Fashion** and below it the sentence *"Beautiful, well-made clothes designed in Port Harcourt."*

The planning note. This is the seed of our running project — Adaeze Fashion's home page. Over the course we will grow it into a 3–5 page responsive website.

F.11 Summary of the lecture

- The **internet** is the network; the **web** is the content on it.
- Typing a URL triggers a five-step journey: **type → DNS → request to server → response → render**.
- **Clients** ask; **servers** answer.
- Three languages: **HTML (structure), CSS (appearance), JavaScript (behaviour)**.
- **File extensions** decide how a file behaves — always save pages as `.html`.
- **Local files** are private; **live websites** are served to the world.
- **Relative paths** (`images/logo.png`) keep a website portable; use forward slashes `/` .
- The **browser** (Chrome) both displays and tests your work; **View Page Source** reveals any page's code.
- **VS Code** is your editor; a **workspace** = editor + browser + project folder.

G. INSTRUCTOR DEMONSTRATION (~15 minutes)

G.1 What the instructor prepares

- VS Code open on the projector, with **no folder open yet** (so learners can watch the full setup).
- Chrome open with a blank tab.
- The prepared `demo/index.html` (used in Section D) ready elsewhere in case it is needed.

G.2 Demonstration 1 — Setting up the workspace (5 minutes)

Say: "First I will set up my workspace exactly as you will set up yours."

Do it slowly, narrating each click:

1. Open VS Code. Say: "This is my code editor."
2. Click **File → Open Folder...**. Navigate to the **Desktop**. Say: "I need my project folder. If it is not there, I make it — right-click the Desktop → New → Folder → `webdesign-practice`."
3. Select `webdesign-practice` → **Select Folder**. Say: "Now VS Code is showing me the whole folder on the left. This is called the **Explorer**."
4. In the Explorer, click the **New Folder** icon → type `images` → Enter.
5. Click **New Folder** again → type `css` → Enter.
6. Click **New File** → type `index.html` → Enter.

Expected result: the Explorer shows `index.html` , an `images` folder and a `css` folder.

Question to ask: "Why do we put images in their own folder instead of alongside the HTML?" (Expected: to keep the project tidy; relative paths like `images/logo.png` stay clean, and the site is easier to maintain.)

G.3 Demonstration 2 — Creating and opening the first page (6 minutes)

Say: "Now I will write my first page and open it in Chrome."

1. Click `index.html` to open it in the editor.
2. Type the complete page from Section F.10, narrating the tags as you go (refer back to the line-by-line notes). Pause after `<head>` and ask: "Is the title shown on the page, or on the tab?" (Expected: on the **tab**.)
3. Press `Ctrl + S` and say loudly: "**Save!** If I forget this, nothing changes." Notice the dot beside the file name disappear when saved.
4. Open the file in Chrome: in VS Code's Explorer, right-click `index.html` → **Reveal in File Explorer** → double-click `index.html`. (Alternative: drag the file onto an open Chrome window.)
5. Point at the browser tab and the heading. Say: "The tab says *Adaeze Fashion — Home*; the page shows my heading. This is our home page."

Errors to inject, then correct (very important):

- **Inject 1 — wrong extension.** Save a copy as `index2.txt`, double-click it. Expected: it opens as plain text or asks to choose an app — **not** as a web page. Say: "The extension decides everything. This is the most common beginner mistake." Delete it. Correct form: `.html`.
- **Inject 2 — forgot to save.** Change the heading to "Welcome to Adaeze Fashion 2" but **do not** press Save, then refresh Chrome. Expected: the change does **not** appear. Say: "Save, then refresh. Always." Now press `Ctrl + S` and refresh — it appears.
- **Inject 3 — wrong path / typo in tag.** Type `<h1>` but forget the closing `</h1>`. Save, refresh. Expected: the heading still shows but following content may look off. Correct it live and explain that most tags need a closing tag.

Questions to ask during the demo:

- "What happens in the browser when I press `Ctrl + U`?" (It shows the page source — the same text we wrote.)
- "Is this page a local file or a live website right now?" (Local — only on this machine.)

G.4 Demonstration 3 — View Page Source on a real site (4 minutes, needs internet; otherwise use the local file)

If Wi-Fi is available: open a well-known site, press `Ctrl + U`, and use `Ctrl + F` to search for a word that is visible on the page. Show that the word appears inside tags. Say: "Every site in the world is built from text like this — and now you can read it."

If no internet: do the same on `index.html`. The lesson is identical.

H. GUIDED PRACTICAL EXERCISE (~18 minutes)

Objective: Each learner sets up the `webdesign-practice` workspace, creates `index.html` with the Adaeze Fashion home page, and opens it in Chrome.

Materials: VS Code and Chrome open on each machine; the projector showing the instructor's steps.

Procedure (instructor calls each numbered step; learners follow; instructor walks the room):

1. On the Desktop, create (or open) the folder `webdesign-practice`.
2. Open VS Code → **File** → **Open Folder...** → select `webdesign-practice`.
3. In the Explorer, create a folder `images` and a folder `css`.
4. Create a new file `index.html`.
5. Type this code (do **not** copy-paste — typing builds memory):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>
  <h1>Welcome to Adaeze Fashion</h1>
  <p>Beautiful, well-made clothes designed in Port Harcourt.</p>
```

```
</body>
</html>
```

6. Save with `Ctrl + S`.
7. In VS Code, right-click `index.html` → **Reveal in File Explorer**, then double-click `index.html`. It opens in Chrome.
8. Change the heading text to include **your own name** as the page author, e.g. `<h1>Welcome to Adaeze Fashion</h1>` stays, and add a new line `<p>Page by [your name].</p>`. Save and refresh.

Pause and predict (ask the whole class):

- After step 6, before opening: "Where will the words '*Adaeze Fashion — Home*' appear — on the page or on the tab?" (Expected: on the browser **tab**, because they are inside `<title>` in the `<head>`.)
- After step 8, before refreshing: "Will the new paragraph show before or after I refresh?" (Expected: only **after** saving and refreshing.)

Expected result: the Chrome tab reads "*Adaeze Fashion — Home*"; the page shows the heading, the sentence and the learner's name; no broken layout.

Troubleshooting while walking the room:

- Page shows the code as text → the file was saved as `.txt`. Rename to `.html`. (See K.1.)
- Nothing changed after editing → the file was not saved, or the browser was not refreshed. (See K.4.)
- Naira sign or strange characters → check `<meta charset="UTF-8">` is present and the file is saved as UTF-8. (See K.5.)
- Cannot find the file → confirm the folder is `webdesign-practice` on the Desktop and that VS Code has that folder open.

Completion checklist (learner ticks each):

- ☐ `webdesign-practice` folder open in VS Code
- ☐ `images` and `css` folders created
- ☐ `index.html` created and saved
- ☐ Complete document structure present (`DOCTYPE` , `html` , `head` , `body`)
- ☐ Title shows on the browser tab
- ☐ Heading and paragraph display in Chrome
- ☐ A paragraph with the learner's name added and visible after refresh

---

I. INDEPENDENT CLASS EXERCISE (~15 minutes)

"Web Detective"

Scenario. You are a web detective. You have just learned that every website's code can be seen. Now investigate on your own, without copying the instructor's steps, and prove what you understand.

Task (write down your answers on paper or in the workbook):

1. Open Chrome and go to any website you like (for example a news site, your bank, or your church's page). *(If there is no internet, use your own `index.html` instead.)*
2. Press `Ctrl + U` to view the page source.
3. Search the source with `Ctrl + F` for the first word that is visible on the page. Confirm it appears inside a tag.
4. In the source, find the `<head>` and the `<body>` sections. Write down one thing that lives in each.
5. Close the source. Now answer these three questions in your own words:
   - a) What does **DNS** do?
   - b) What is the difference between the **client** and the **server**?
   - c) Why do we say HTML is *structure* and CSS is *appearance*?
6. Open your `webdesign-practice/index.html`.
7. Add a **second paragraph** describing one product Adaeze Fashion might sell (for example: "We sew custom Ankara gowns for weddings and celebrations.").
8. Save, open in Chrome, and refresh to confirm the new paragraph appears.

9. Take your folder to the next level: create a **third** file called `notes.txt` in the same folder and write one sentence: "This is a text file — saving a page as .txt is a common beginner mistake." Save it, then double-click it and observe that it opens as plain text, not a web page.

Requirements / constraints:

- Work independently — only ask for help if truly stuck.
- Use forward slashes and lowercase names for all files.
- All 9 tasks must be attempted.

Time allowance: 15 minutes.

Success criteria:

- The three concept questions (5a–5c) are answered correctly *in the learner's own words*.
- The new paragraph is visible in the browser after refresh.
- The learner can explain why `notes.txt` did not open as a web page.

Hints if stuck:

- Hint 1: Page source is `Ctrl + U`; find is `Ctrl + F`.
- Hint 2: If your change does not appear, press `Ctrl + S` then `F5`.
- Hint 3: To add a paragraph, put text between `<p>` and `</p>`, just below the first paragraph.

Instructor answer key (do not show learners before they try):

1. A page opens with source visible via `Ctrl + U`.
2. Any visible word appears inside tags in the source (proves the page is built from text).
3. `<head>` holds information about the page (e.g. the `<title>`, meta information); `<body>` holds the visible content (headings, paragraphs, images).
4. **DNS:** translates a human-friendly website name (like `adaezefashion.com`) into the numeric IP address of the server, like a phone contacts app finding a number.
5. **Client vs server:** the client asks for information (your browser/computer); the server stores and sends it.
6. **HTML = structure, CSS = appearance:** HTML marks up *what* the content is (heading, paragraph, image); CSS controls *how* it looks (colour, size, spacing).
7. New paragraph added between `<p>…</p>`, e.g. `<p>We sew custom Ankara gowns for weddings and celebrations.</p>`.
8. After saving and refreshing, the second paragraph shows below the first.
9. `notes.txt` opens as plain text because its extension (`.txt`) tells Windows it is text, not a web page. Only `.html` is drawn as a page.

Marking guidance (out of 10): concept answers (4), correct page edit and display (3), correct explanation of the `.txt` result (2), independence and neatness (1).

J. REAL-WORLD CASE STUDY — "Adaeze's Online Presence"

Situation. Adaeze runs **Adaeze Fashion**, a small clothing business in Port Harcourt. She sews custom women's clothing. She has a Facebook page and an Instagram account with hundreds of followers, and she takes most orders on WhatsApp. She does not have a website.

Relevant facts.

- Her customers must scroll through many posts to see her work.
- She cannot arrange her collection in one neat place.
- She pays attention to social media, but the *platform* decides who sees her posts.
- She has heard that "websites are expensive" and is afraid to ask.

Decision point. Should Adaeze keep relying only on social media, or should she build her own simple website?

Possible approaches.

1. **Stay on social media only.** Free and immediate, but she does not control how her content is shown, and her business looks less established.

2. **Pay a professional for a big custom website.** Complete control, but possibly expensive and slow for a small business at this early stage.
3. **Build a simple 3–5 page website herself using HTML and CSS and free hosting.** Modest time investment, full control, low cost.
4. **Rent a shop.** Powerful for walk-in sales, but a website reaches people far beyond Port Harcourt at a tiny fraction of the cost.

Recommended solution (for this stage). Approach 3. Using the very skills in this course, Adaeze can build a Home page, an About page, a Collection page and a Contact page, then publish it for free. She then links that website from her social media bios so every follower has one clear place to go.

Example plan (this is exactly our running project):

- `index.html` — welcome, a strong call to action ("Order your custom outfit today").
- `about.html` — her story, her workshop in Port Harcourt, her values.
- `collection.html` — photos of her work with descriptions.
- `contact.html` — phone number, WhatsApp, a contact form, opening hours.

Trade-offs.

- Social media gives quick reach, but the platform owns the rules and the data.
- A paid professional gives polish now, but costs money and creates dependence.
- A self-built website takes some learning time, and the first version will be simple — but Adaeze *owns* it, it grows with her, and it makes her business look serious.
- A physical shop gives face-to-face trust but costs rent and reaches only those nearby.

Professional connection. This is real work that Nigerian freelancers do every day: build a small business a clean website for a fair fee. The exact skills in this session — understanding clients and servers, saving HTML files correctly, organising folders with images and CSS — are the foundation of every paid web design job.

Transfer question for the class: "Your cousin runs a small bakery. Give **three** benefits a simple website would give the bakery that a Facebook page alone does not." (Expected ideas: one organised place for menu and prices; a professional, owned presence; customers can find and contact the business directly without an algorithm deciding who sees it.)

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the session's live troubleshooting guide. When a learner hits a problem, do not give the answer at once — ask them to check the relevant row.

K.1 The page shows the code instead of the finished page

Symptoms: Chrome displays the HTML text (tags and all) rather than a drawn page. **Likely causes:** The file was saved with an extension like `.txt` instead of `.html`, or the browser treated it as text. **How to investigate:** Look at the full file name in File Explorer (turn on "File name extensions" if they are hidden). It must end in `.html`. **Corrective steps:** In VS Code, rename the file to `index.html` (or Save As with the correct extension). Reopen it in Chrome. **How to confirm resolved:** The browser now *draws* the heading and paragraph. **Prevention:** Always type `.html` and turn on file-extension display in File Explorer.

K.2 The file opens but looks completely blank

Symptoms: A white page with nothing on it. **Likely causes:** Content was typed outside `<body>`, a tag was mistyped, or the file is empty/unsaved. **How to investigate:** Open the code and check that the heading and paragraph sit between `<body>` and `</body>`. **Corrective steps:** Move the content inside the body; check for typos in `<h1>` and `<p>`; save. **How to confirm resolved:** The heading and paragraph now appear. **Prevention:** Follow the document shape in Section F.10; save often.

K.3 Strange characters instead of ₦ or accents

Symptoms: The Naira sign or other characters show as `?` or a box. **Likely cause:** The file was saved in the wrong text encoding. **How to investigate:** Check that `<meta charset="UTF-8">` is in the `<head>`. **Corrective steps:** In VS Code's bottom-right corner, click the encoding, choose **Save with Encoding → UTF-8**; save. **How to confirm resolved:** The ₦ sign displays correctly. **Prevention:** Always keep the `<meta charset="UTF-8">` line and save as UTF-8.

K.4 I changed the code but the browser still shows the old page

Symptoms: Edits are not visible. **Likely causes:** The file was not saved, or the browser was not refreshed. **How to investigate:** Look for the dot on the file tab in VS Code (means "unsaved"). Ask: "Did you save? Did you refresh?" **Corrective steps:** `Ctrl + S`, then refresh with `F5` (or `Ctrl + F5` for a hard refresh). **How to confirm resolved:** The new text appears. **Prevention:** Build the habit: **Save, then refresh.**

K.5 "Where is my folder?" / VS Code is showing the wrong place

Symptoms: Files are not where the learner expects; the Explorer looks empty. **Likely cause:** The wrong folder was opened, or the file was saved in a different location. **How to investigate:** Read the folder name at the top of the Explorer. **Corrective steps:** **File → Open Folder...** and select exactly `webdesign-practice`. **How to confirm resolved:** The Explorer shows `index.html`, `css` and `images`. **Prevention:** Always open the folder first, then create files inside it.

K.6 Confusing the internet with the web

Symptoms: The learner uses the two words as if identical. **Likely cause:** Everyday speech blurs the terms. **How to investigate:** Ask: "Can I send an email without visiting a page?" (Yes.) **Corrective steps:** Reuse the road/building analogy. **How to confirm resolved:** The learner explains the difference. **Prevention:** Use the analogy consistently all course.

Distinguish likely vs confirmed. In every case, the *likely* cause becomes a *confirmed* cause only after checking (seeing the exact file name, the missing charset line, the unsaved dot). Teach learners to **confirm before fixing**.

L. SESSION ASSIGNMENT

Assignment title: "My First Webpage — About My Business"

Purpose. Reinforce how the web works and the basic HTML document shape, in a context the learner owns, while starting the running Adaeze Fashion project.

Realistic scenario. Adaeze Fashion has asked you to prepare a first draft home page. Keep it simple — a heading, a welcoming paragraph, and a short note about the business — so that next session we can add more.

Detailed instructions.

1. In `webdesign-practice`, create a file named `index.html` (if not already done).
2. Write a complete document with the correct structure: `<!DOCTYPE html>`, `<html lang="en">`, `<head>`, `<body>`.
3. In the `<head>`, include `<meta charset="UTF-8">` and a `<title>` that reads `Adaeze Fashion — Home`.
4. In the `<body>`, include:
    - an `<h1>` heading: *Welcome to Adaeze Fashion*
    - a `<p>` paragraph of at least two sentences describing what the business does (write it in your own words)
    - a second `<p>` paragraph that mentions **Port Harcourt** and one type of clothing Adaeze might make.
5. Save the file and open it in Chrome.
6. Copy the page source (`Ctrl + U`, then `Ctrl + A` and `Ctrl + C`) or take a clear photo of the browser showing the page.

Required deliverables

- A saved `index.html` in `webdesign-practice`.
- The page displays correctly in Chrome.
- A copy/photo of either the page or its source in the workbook.

Submission format. Files remain in the `webdesign-practice` folder on the academy machine; show the working page to the instructor before leaving, and paste/photo the evidence into the workbook.

Deadline. Before Session 2.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Document structure	4	`DOCTYPE`, `html lang`, `head`, `body` all correct and closed

Criterion	Marks	Description of a strong answer
Title and charset	3	`<title>` present and descriptive; `<meta charset="UTF-8">` present
Heading	3	One clear `<h1>` about the business
Paragraphs	4	Two paragraphs, own words, mentions Port Harcourt and a clothing type
Saved and displayed	3	Saved as `.html`; page draws correctly in Chrome
Neatness / indentation	3	Code indented and readable

Common mistakes to avoid. Forgetting `<!DOCTYPE html>`; saving as `.txt`; missing closing tags; forgetting to save/refresh.

Instructor reference solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>
  <h1>Welcome to Adaeze Fashion</h1>
  <p>Adaeze Fashion designs and sews beautiful custom clothing for women and men.</p>
  <p>Based in Port Harcourt, we specialise in elegant Ankara gowns for weddings and special occasions.</p>
</body>
</html>
```

Expected display: tab reads "*Adaeze Fashion — Home*"; page shows the heading then the two paragraphs.

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Multiple choice. What does DNS do when you type a web address? a) It draws the page on your screen b) It translates a website name into the numeric address of the server c) It saves your files to the internet d) It styles the page with colour

Q2. Short answer. Name the three core languages of a webpage and say in one phrase what each does.

Q3. True or false. "The internet and the World Wide Web are exactly the same thing." Explain your answer in one sentence.

Q4. Fill in the blank. A _____ asks for information, and a _____ stores and sends it.

Q5. Explain in your own words. Why do we use a **relative** path like `images/logo.png` instead of the full path `C:\Users\Ada\Desktop\...`?

Q6. Practical identification. You save a page as `home.txt` and the browser shows the code as text. What went wrong, and how do you fix it?

Q7. Scenario. Your friend says, "I finished my website on my computer, so it is already online for the whole world." Is your friend correct? Explain.

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. (b) It translates a website name into the numeric address (IP address) of the server, like a contacts app finding a phone number.

A2. HTML = structure/content; CSS = appearance/styling; JavaScript = behaviour/interactivity.

A3. False. The internet is the network (the roads/cables); the web is the collection of websites and pages that travel over it. You can use the internet (email, calls) without visiting the web.

A4. A **client** asks; a **server** stores and sends.

A5. Because a relative path describes the file's position *within the project folder*, so the site keeps working when the whole folder is moved to a server. A full `C:\Users\Ada\...` path only works on one particular computer and breaks once uploaded.

A6. The file was saved with a `.txt` extension, so Windows treats it as plain text. Fix: rename or save it as `home.html`. (Turn on file-extension display in File Explorer to see the true ending.)

A7. No. A file on your own computer is a **local file** — only you can see it. To make it a **live website**, you must **publish** (upload) it to a server; only then can the world reach it.

N. REFLECTION AND RECAP

Plain-English summary. Today you learned how the web actually works. You followed the journey of a web address — typing it, DNS finding the server, the browser requesting the page, the server responding, and the browser rendering it. You met the idea of **clients** and **servers**, learned the three core languages (**HTML for structure, CSS for appearance, JavaScript for behaviour**), understood **file extensions** and the difference between **local files** and **live websites**, and learned to read a **file path**. Finally, you set up your workspace in **VS Code** and created, saved and opened your very first page for our running project, **Adaeze Fashion**.

Five important takeaways.

1. The **internet** is the network; the **web** is the content on it.
2. Typing a URL runs a five-step journey: **type → DNS → request → response → render**.
3. **HTML = structure, CSS = appearance, JavaScript = behaviour**.
4. Always save pages as `.html`, and remember: **save, then refresh**.
5. **Relative paths** with forward slashes keep your site portable — and VS Code + Chrome are your two main tools.

Self-check — can you now…

- Describe what happens when you type a URL?
- Say what a client and a server each do?
- Name the three building languages and their jobs?
- Explain the difference between a local file and a live website?
- Set up a workspace and open a page in Chrome without help?
- Explain why `home.txt` is not a web page?

Connection to the next session. Today you wrote a page, but you were mostly copying the *shape* of HTML. Next session, **Session 2 — HTML Basics**, we open the language properly: tags, elements and attributes, the full document structure, headings, paragraphs, lists, bold and italic, line breaks, and comments. By the end, you will write HTML from your own understanding — not from memory of a picture.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence.

- The learner explains the URL journey in the right order using their own words.
- The learner distinguishes client from server without prompting.
- The learner corrects someone who says "HTML is the design" — showing they hold the structure/appearance idea.
- The learner sets up the folder and file and opens the page in Chrome on the first or second try.
- The learner predicts that the `<title>` shows on the tab, not the page.
- The learner habitually saves and refreshes.

Common misconceptions to watch for and correct.

- "The internet and the web are the same." → Use the road/building analogy.
- "HTML is the styling." → Use the house analogy; HTML is the walls, CSS the paint.
- "A website on my computer is already online." → Explain local vs live; preview the publishing session.
- "The browser stores the website." → The browser requests; the server has the files (though a cache may keep a copy).
- "File extensions do not matter." → Show the `.txt` failure live.
- "You need to be good at maths to code." → Web design needs logic and care, not advanced maths.

Suggested remedial exercises.

1. Draw-and-label the URL journey (five boxes) and explain each arrow aloud to the instructor.
2. "Fix the broken page" drill: hand out a page saved as `.txt` with an unclosed `<h1>` and a missing charset line; ask the learner to find and fix all three.
3. Folder drill: give a picture of a project folder and ask the learner to write the relative path from two different files to `images/logo.png`.

Differentiation.

- **Level 1 (supported):** Provide a printed template of `index.html` with gaps to fill; pair with a confident learner; allow extra time; focus first on the three building languages and the URL journey.
- **Level 2 (standard):** Complete the exercises and assignment as written.
- **Level 3 (challenge):** Ask the learner to research what a **domain name** is and explain the difference between `http` and `https`; or to build a second page `about.html` that links to `index.html` using a relative path, and explain what the link does.

Timing guidance (105 minutes). Opening 6 · Prerequisite 7 · Lecture 40 · Demonstration 15 · Guided practical 18 · Independent exercise 15 · Case study 5 (compress if short on time) · Knowledge check 5 · Reflection/close 4. If the room is slow, protect the **guided practical** and the **case study** may be reduced to a 3-minute mention — doing the setup is the priority. If the room is fast, spend the spare time on Level 3 research tasks.

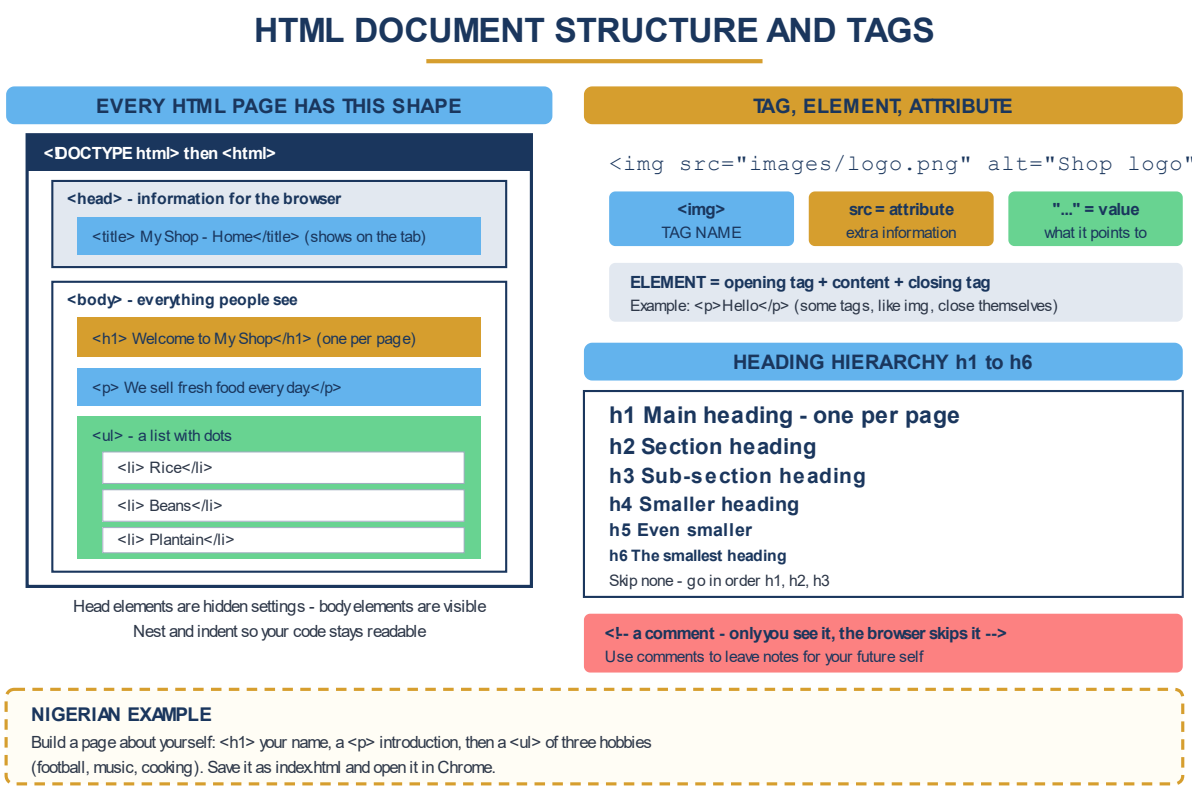
END OF SESSION 1

Next: Session 2 — HTML Basics: tags, elements, attributes, the full document structure, headings, paragraphs, lists, and comments.

SESSION 02 OF 12

HTML BASICS

CONCEPT DIAGRAM — SESSION 02



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 2 — HTML BASICS

A. SESSION IDENTIFICATION

Item	Detail
Session number	2 of 12
Session title	HTML Basics
Week / Module	Week 1 — Module 1: HTML Foundations
Recommended duration	105 minutes
Difficulty level	Beginner

Item	Detail
Prerequisite knowledge	Session 1 — How the Web Works. The learner can set up a `webdesign-practice` folder, create `index.html`, save it, and open it in Chrome.
Connection to previous session	In Session 1 learners saw the *shape* of an HTML file and opened a page. This session explains the language properly: tags, elements, attributes, document structure, text, lists and comments.
Preparation before class	Academy PCs on; VS Code and Chrome open; `webdesign-practice` folder present; a sample image `images/logo.png` (or `images/sample.jpg`) placed in each learner's folder; a demo file prepared for Section G; projector connected.
Running project	We continue building **Adaeze Fashion**. This session we replace the rough home page with a properly structured one using headings, paragraphs, lists, bold/italic and comments.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain what HTML is** and state clearly that it describes *structure and meaning*, not appearance.
2. **Write a complete HTML document** with the correct order: `<!DOCTYPE html>`, `<html>`, `<head>`, `<title>`, `<body>`.
3. **Define and distinguish** a *tag*, an *element* and an *attribute*, giving one example of each.
4. **Use headings** `h1` **–** `h6` in the correct hierarchy and explain why there should normally be one `h1` per page.
5. **Add paragraphs** with `<p>` and explain how the browser spaces them.
6. **Create unordered and ordered lists** using `<ul>`, `<ol>` and `<li>` correctly, and choose the right one for a given situation.
7. **Apply bold, italic and importance** using `<strong>`, `<em>`, `<b>` and `<i>`, and explain the difference between looks and meaning.
8. **Use line breaks** `<br>` **and horizontal rules** `<hr>` appropriately, including when *not* to use them.
9. **Write HTML comments** with `<!-- -->` and explain why they help.
10. **Nest and indent** elements correctly so the code is readable and valid.

C. REQUIRED MATERIALS

Hardware

- One computer per learner; instructor computer on a projector.
- Optional: one printed page per learner for the "tag game" opener (plain text on paper).

Software

- Visual Studio Code; Google Chrome.

Sample / starter files

- `webdesign-practice/index.html` from Session 1 (learners may continue it or create a fresh one).
- A small image in `webdesign-practice/images/` named `logo.png` or `sample.jpg` (used briefly to show structure; the full `<img>` lesson is in Session 3).
- Instructor demo file `demo/html-basics.html` (contents in Section G).

Worksheets / printed materials

- "HTML Tags Quick Reference" card (one per learner: `h1` – `h6`, `p`, `ul/ol/li`, `strong/em`, `br`, `hr`, comment).
- Worksheet 2.1 — "Tag or Element?" (match ten examples to the right word).
- Worksheet 2.2 — "Which List?" (five scenarios; tick unordered or ordered).

Internet

- Not required. All work is local.

Alternative for learners without VS Code

- Notepad works: type the code, then **File → Save As**, choose **All files**, and name it `index.html`. Warn that Notepad does not indent or colour the code.

D. OPENING ACTIVITY (~6 minutes)

"The Tag Game"

Instructor setup. On the whiteboard, write a plain sentence: `Welcome to Adaeze Fashion`. Have a marker ready. Optionally hand one learner a strip of paper with a tag printed on it (`<h1>`, `</h1>`, `<p>`, `<em>`).

What the instructor says:

> *"Look at this sentence on the board: Welcome to Adaeze Fashion. That is just text. Nothing tells a computer what it is. Is it a heading? A paragraph? A warning?*
>
> *In HTML, we wrap content in **tags** to say what it is. Watch."*
>
> *[Write around it:]* `<h1>Welcome to Adaeze Fashion</h1>`
>
> *"The `<h1>` at the front says: 'This is the main heading.' The `</h1>` at the end says: 'The heading stops here.' Tags are like **labels on jars**. The jar holds the content; the label says what the content is."*
>
> *[Now write:]* `This gown costs <strong>₦25,000</strong>`
>
> *"Here `<strong>` says: 'This part is important.' The browser will show it in bold. Every single thing on every website is content wrapped in tags like this."*

Expected learner response. Learners grasp that tags are *labels*, not decoration, and often try to guess other tags (e.g. "what tag makes it red?"). Use that: "Making it red is CSS — next week. Today we label what things *are*."

Why this opener works. It turns an abstract idea (markup) into a physical action (wrapping words on the board) and immediately corrects the biggest confusion — that tags are about structure, not looks.

E. PREREQUISITE REVIEW (~7 minutes)

Run as an oral round-robin; put the questions on the projector.

Question 1: "What is the difference between the internet and the web?"

- Expected answer: The internet is the network; the web is the content/pages on it.
- Gap: Learners reverse them. Reuse the road/building analogy in one sentence.

Question 2: "What do we call the complete set of a file's letters after the dot, and what extension do web pages use?"

- Expected answer: file extension; `.html`.
- Gap: Some forget the extension controls behaviour. Remind them of the `.txt` trap.

Question 3: "What are the three building blocks of a website, and what does each do?"

- Expected answer: HTML (structure), CSS (appearence), JavaScript (behaviour).
- Gap: If they say "HTML is the design", correct now with the house analogy.

Question 4: "In the file `webdesign-practice/images/logo.png`, what does the `/` do?"

- Expected answer: It shows a path — go into the `images` folder to find `logo.png`. Forward slash separates folders.
- Gap: Some write backslashes. Remind: **web paths use forward slashes**.

Question 5: "After you change code, what two things must you do to see the change?"

- Expected answer: Save (`Ctrl + S`) and refresh the browser (`F5`).
- Gap: This is the most common failure. Drill it now.

Bridge to today (say this):

> *"Last session you met the shape of an HTML page. Today we open it up. You will learn the words HTML uses for everything — headings, paragraphs, lists, bold and italic — and how to arrange them neatly. By the end, you will write a proper home page for Adaeze Fashion from your own understanding."*

Gap handling. If several learners cannot recall HTML/CSS/JS roles, spend two extra minutes with the house analogy before continuing. If many cannot save-and-refresh, pair them with confident learners for the whole session.

F. FULL LECTURE NOTES (~40 minutes)

F.1 What HTML is (and is not)

The idea. HTML stands for **HyperText Markup Language**. It is the language we use to describe the **structure and meaning** of a web page — what each piece of content *is*.

Plain-English definition. HTML wraps content in **tags** so the browser knows "this is a main heading", "this is a paragraph", "this is a list of items". It is a **markup language**, not a programming language: it does not calculate, decide or repeat. It simply labels.

Why it exists. A browser needs to know the *role* of each piece of content so it can display it sensibly, and so that other software — screen readers for blind users, search engines, and printing tools — can understand it too. Without structure, everything would be one grey blob of text.

Familiar analogy. Think of **labelling boxes in a storeroom**. You have boxes of "shoes", "shirts", "caps". The boxes hold the goods; the labels tell everyone what is inside. HTML is the labelling system for web content. CSS is the *paint on the shelves* (appearance). JavaScript is the *forklift* (behaviour).

Worked example.

```
<h1>Adaeze Fashion</h1>
<p>We sew beautiful custom clothing in Port Harcourt.</p>
```

The `<h1>` says "this is the most important heading on the page". The `<p>` says "this is a paragraph". The browser shows the heading large and bold and the paragraph in normal text — but that sizing is only a *default*; CSS can change it later.

Misconception. "HTML is a programming language." It is not. There is no logic, no maths, no decisions. Calling it a programming language confuses beginners into expecting it to "do" things. It only *describes*.

Limitation. HTML alone gives an ugly page (plain black text on white). It cannot make things pretty or interactive. That is fine — it is not its job.

F.2 Tags, elements and attributes — the vocabulary

These three words are used constantly. Learn them exactly.

Tag. A **tag** is a keyword inside angle brackets `< >`. There are two kinds:

- **Opening tag:** `<p>` — starts an element.
- **Closing tag:** `</p>` — ends an element. Notice the **forward slash** `/` before the name.

Some tags have **no closing tag** because they hold no content; these are called **void** (or self-closing) elements, for example `<br>`, `<hr>` and `<img>`.

Element. An **element** is the *whole package*: opening tag + content + closing tag.

```
<p>Hello, Port Harcourt!</p>
```

The element above is a paragraph element. "Element" is the thing; "tag" is the marker.

Attribute. An **attribute** gives *extra information* about an element. It goes **inside the opening tag** and almost always has a **name** and a **value** in quotes.

```
<a href="https://www.cea.ng">Visit Cyber Elias Academy</a>
```

- `href` is the attribute **name** (here: where the link goes).
- `"https://www.cea.ng"` is the attribute **value**, in quotes.

Why attributes exist. A tag like `<a>` says "this is a link", but it does not say *where* the link goes. Attributes supply that missing detail.

Worked example — one element with two attributes:

```
<img src="images/logo.png" alt="Adaeze Fashion logo">
```

- `src` = the file path of the image.
- `alt` = the alternative text (read aloud by screen readers, shown if the image fails to load).

Rules to remember.

1. Tags are not case-sensitive, but **write them in lowercase** — it is the standard and looks professional.
2. Every element that holds content needs a closing tag: `<p>` … `</p>`.
3. Attribute values go in **quotes** — use double quotes `"` and be consistent. (On a Nigerian keyboard, the double-quote key is usually **Shift + the apostrophe key** near the Enter key.)
4. Elements must be **properly nested** — the last one opened is the first one closed.

Misconception. "Any tag in angle brackets is fine." No — misspell a tag (e.g. `<paragrap>`), and the browser ignores it or treats it as an unknown element. Spelling matters.

F.3 The document structure — the skeleton of every page

Every HTML page follows the same shape. Learn it once and use it forever.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>
  <h1>Welcome to Adaeze Fashion</h1>
  <p>Beautiful custom clothing made in Port Harcourt.</p>
</body>
</html>
```

Line by line:

- `<!DOCTYPE html>` — the **DOCTYPE**. It is not a tag exactly; it is a *declaration* that tells the browser "this is an HTML5 document". It must be the **very first line**. Without it, browsers may switch to a slow, old-fashioned "quirks" mode.
- `<html lang="en">` — the **root element**. Everything lives inside it. `lang="en"` declares the page language as English, which helps screen readers pronounce words correctly.
- `<head>` — a **hidden** section holding *information about the page* — not shown on the screen.
  - `<meta charset="UTF-8">` — tells the browser which characters to use, so the Naira sign (₦) and accents show correctly. UTF-8 means "universal character set".
  - `<title>…</title>` — the words shown on the **browser tab** (and used by search engines as the clickable link).
- `</head>` — closes the head.
- `<body>` — **everything visible** on the page goes inside here: headings, paragraphs, lists, images, links.
- `</body>` — closes the body.
- `</html>` — closes the document.

Worked example — expected browser result.

- The **tab** shows: "*Adaeze Fashion — Home*".

- The **page** shows: a large bold heading *Welcome to Adaeze Fashion* and beneath it the sentence *Beautiful custom clothing made in Port Harcourt.*

Why the head is separate. Mixing page information (title, character set) with page content would be messy. The head is the page's "identity card"; the body is the page's "face".

Misconception. "Everything I write appears on the page." Not true — anything in the `<head>` (like the `<title>`) is *not* shown on the page itself, only on the tab.

F.4 Headings `h1` – `h6`

The idea. Headings give a page a **structure in levels**, like the chapters and sub-chapters of a book. HTML gives six levels: `<h1>` (most important) down to `<h6>` (least important).

Plain-English definition. A heading is a short piece of text that names a section of content.

Why they exist. Readers skim; headings help them find things fast. Screen readers let blind users jump between headings. Search engines use them to understand what a page is about.

Familiar analogy. A **table of contents** in a textbook:

- `<h1>` = the book title.
- `<h2>` = the chapter titles.
- `<h3>` = sub-sections inside a chapter. …and `h4` – `h6` go deeper still.

Worked example (runnable).

```
<h1>Adaeze Fashion</h1>
<h2>Our Services</h2>
<h3>Custom Gowns</h3>
<p>We design gowns for weddings and celebrations.</p>
<h3>Corporate Uniforms</h3>
<p>We supply neat uniforms for schools and offices.</p>
```

Expected result. *Adaeze Fashion* is the largest; *Our Services* is smaller; the two `<h3>` sub-headings are smaller still. The paragraphs are normal-sized text.

Rules and good habits.

- Use **one** `<h1>` **per page** — it names the whole page.
- Do **not skip levels** for looks (do not jump from `h1` straight to `h4`). The level expresses meaning, not size.
- Do **not** use headings just to make text big. That is CSS's job.

Misconception. "`<h4>` is just a smaller font, so I can use it whenever." No — the number is a *level of importance*. A page that jumps `h1` → `h5` confuses screen readers.

A note for the teacher: the browser's default sizes for `h1` – `h6` can be changed with CSS later; do not let learners think the size *is* the meaning.

F.5 Paragraphs `<p>`

The idea. A **paragraph** is a block of running text. Wrap it in `<p> … </p>`.

Worked example.

```
<p>Adaeze Fashion was founded in Port Harcourt in 2019.</p>
<p>We believe every customer deserves clothing that fits perfectly.</p>
```

Expected result. Two separate blocks of text, with a small gap between them. The browser automatically adds space **before and after** each paragraph.

Why paragraphs matter. They create natural reading blocks. If you write text *without* `<p>`, the browser glues lines together into one run-on block regardless of how you typed them in the file.

Worked example — the "glue" problem.

```
<p>
  First line in the file.
  Second line in the file.
  Third line in the file.
</p>
```

Expected result. All three lines appear as **one continuous paragraph**. The browser ignores the line breaks you typed in the code. (If you truly want a forced break, you use `<br>` — see F.8.)

Misconception. "Pressing Enter in my code creates a new line on the page." It does not. Only tags control the output.

F.6 Lists — `<ul>`, `<ol>` and `<li>`

The idea. Lists present a group of items. There are two kinds:

- **Unordered list** `<ul>` — items shown with **bullet points**; order does not matter.
- **Ordered list** `<ol>` — items shown with **numbers**; order matters.

Every item in either list is wrapped in `<li>` (**list item**).

Unordered example.

```
<ul>
  <li>Ankara gowns</li>
  <li>Corporate uniforms</li>
  <li>Bridal wear</li>
</ul>
```

Expected result: three bulleted lines:

- Ankara gowns
- Corporate uniforms
- Bridal wear

Ordered example.

```
<ol>
  <li>Choose your fabric</li>
  <li>Take your measurements</li>
  <li>We sew your outfit</li>
  <li>Collect your order</li>
</ol>
```

Expected result: the same four items, but numbered 1, 2, 3, 4.

How to choose. Ask: *does the order change the meaning?*

- A list of services → order does not matter → `<ul>`.
- Steps in a process → order matters → `<ol>`.

Nesting. An `<li>` can contain other elements, including another list:

```
<ul>
  <li>Women's wear
    <ul>
      <li>Gowns</li>
      <li>Skirts</li>
    </ul>
  </li>
  <li>Men's wear</li>
</ul>
```

Expected result: "Women's wear" and "Men's wear" as top bullets; "Gowns" and "Skirts" indented underneath.

Misconception. "I can put text directly inside `<ul>` without `<li>`." The rule: inside `<ul>` or `<ol>`, items **must** be wrapped in `<li>`. Text placed directly inside the list is invalid and shows unpredictably.

F.7 Bold, italic and importance — `<b>`, `<i>`, `<strong>`, `<em>`

The idea. Four tags control emphasis in text, but they are not all the same:

- `<strong>` — says "this is **important**". Shown bold.
- `<em>` — says "this is **emphasised**". Shown italic.
- `<b>` — makes text **bold** but carries no special meaning.
- `<i>` — makes text *italic* but carries no special meaning.

Plain-English difference. `<strong>` and `<em>` carry **meaning**; `<b>` and `<i>` are purely **visual**. Meaning matters to screen readers, which can change their tone for `<strong>` and `<em>`.

Rule of thumb. Prefer `<strong>` and `<em>`. Use `<b>` and `<i>` only when you want bold/italic with no extra meaning (for example a foreign word in `<i>`).

Worked example.

```
<p>Our <strong>opening hours</strong> are Monday to Saturday, <em>9am to 6pm</em>.</p>
```

Expected result: "opening hours" appears bold; "9am to 6pm" appears in italics.

Misconception. "`<strong>` and `<b>` are exactly the same." They *look* the same by default, but they mean different things, and a screen reader treats them differently. Prefer the meaningful one.

F.8 Line breaks `<br>` and horizontal rules `<hr>`

The idea.

- `<br>` forces a **line break** inside the current block (like pressing Enter once in a letter).
- `<hr>` draws a **horizontal rule** — a full-width line — used to mark a change of topic.

Both are **void elements**: they have no closing tag and no content.

Worked example.

```
<p>Adaeze Fashion<br>
24/26 Ebony Road<br>
Port Harcourt, Rivers State</p>

<hr>

<p>Thank you for visiting our website.</p>
```

Expected result: the address appears on three separate lines because of the `<br>` tags, then a horizontal line across the page, then the thank-you sentence.

When NOT to use them.

- Do **not** use many `<br>` tags to add space between paragraphs — use separate `<p>` elements, or CSS later.
- Do **not** use `<br>` to lay out a page in columns — that breaks on phones and is not real layout.

Misconception. "`<br>` is how I make a new paragraph." `<br>` makes a *line* break; `<p>` makes a new *paragraph block* with spacing. They are different tools.

F.9 Comments `<!-- -->`

The idea. A **comment** is a note in the code that the browser **ignores completely**. It is never displayed on the page.

Why they exist. To explain your code to yourself, your team-mates, or your future self. A comment like `<!-- Start of products section -->` saves minutes when you return to a file months later.

Worked example.

```
<!-- ===== Header ===== -->
<h1>Adaeze Fashion</h1>
```

```
<!-- The navigation links will go here in a later session -->

<!-- ===== Main content ===== -->
<p>Welcome to our shop.</p>
```

Expected result: the page shows only the heading and the paragraph. The comments are invisible.

Good and bad comments.

- Good: `<!-- Contact details for customers in Port Harcourt -->`
- Bad: `<!-- heading -->` above an `<h1>` — it states the obvious and adds noise.

Misconception. "Comments appear on the page." They never do. If you can see text on the page, it is not inside a comment.

F.10 Nesting and indentation — keeping code readable

The idea. Nesting means putting elements inside other elements. **Indentation** means using spaces or Tab to push inner elements to the right so you can *see* the structure.

Why nesting matters. HTML is a tree. The `<html>` contains `<body>`, which contains a `<ul>`, which contains `<li>` items. If you close the wrong tag, the tree breaks and the page misbehaves.

The nesting rule. The **last element opened is the first one closed** (like brackets in maths).

```
<p>This is <strong>very important</strong> news.</p>
```

Correct: `<strong>` opens and closes *inside* the `<p>`.

```
<p>This is <strong>very important.</p></strong>
```

Incorrect: the tags cross over each other. The browser will guess, producing unpredictable results.

Indentation example — hard to read:

```
<body><h1>Adaeze Fashion</h1><ul><li>Ankara gowns</li><li>Uniforms</li></ul></body>
```

Indented version — easy to read:

```
<body>
  <h1>Adaeze Fashion</h1>
  <ul>
    <li>Ankara gowns</li>
    <li>Uniforms</li>
  </ul>
</body>
```

The browser treats both the same, but **humans** find the second far easier to check and fix.

VS Code tip. Select the code and press **Shift + Alt + F** to auto-format (fix indentation) if the extension is available. Or press Tab to indent and Shift + Tab to un-indent. Set indentation to 2 spaces for this course.

Misconception. "Indentation changes how the page looks." It does not — indentation is for humans. But it prevents mistakes, which is why professionals always do it.

F.11 Putting it together — the Adaeze Fashion home page

Here is a full, runnable page using everything from today.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>
```

```
  <!-- Main heading -->
  <h1>Welcome to Adaeze Fashion</h1>

  <!-- A short introduction -->
  <p>Adaeze Fashion designs and sews <strong>beautiful custom clothing</strong>
     for women and men in Port Harcourt.</p>

  <!-- What we offer -->
  <h2>What We Do</h2>
  <ul>
    <li>Custom Ankara gowns</li>
    <li>Bridal wear</li>
    <li>Corporate uniforms</li>
  </ul>

  <!-- How it works -->
  <h2>How To Order</h2>
  <ol>
    <li>Choose your fabric</li>
    <li>Take your measurements</li>
    <li>We sew your outfit</li>
    <li>Collect your order</li>
  </ol>

  <hr>

  <!-- Contact -->
  <p>Visit us at <em>24/26 Ebony Road, Port Harcourt</em><br>
     or call <strong>0800 000 0000</strong>.</p>

</body>
</html>
```

Expected browser result.

- Tab: "*Adaeze Fashion — Home*".
- Page, top to bottom: the big heading; the intro paragraph with "beautiful custom clothing" in bold; the sub-heading *What We Do* with three bullets; the sub-heading *How To Order* with four numbered steps; a horizontal line; and a final paragraph with the address in italics on its own line and the phone number in bold.

Why this page is good. It has one `<h1>`, a logical heading hierarchy, a bulleted list (services — order does not matter) and a numbered list (steps — order matters), correct nesting and comments. This is a genuine, well-built business home page.

G. INSTRUCTOR DEMONSTRATION (~15 minutes)

G.1 What the instructor prepares

- VS Code open on the projector with the `webdesign-practice` folder open.
- Chrome open.
- The completed Adaeze home page from Section F.11 in a demo file, ready to reveal.
- A deliberate typo ready to inject (e.g. `<h1>Welcome` without `</h1>`).

G.2 Demonstration 1 — Building the page live (9 minutes)

Say: "Watch me build a real business home page, one piece at a time. I will explain every tag as I type."

Do and narrate:

1. Create/open `index.html`. Type the document skeleton (`DOCTYPE`, `html`, `head`, `title`, `body`). Ask: "Which of these is shown on the page — the head or the body?" (Body.)
2. Add the `<h1>`. Say: "One main heading. This is the page title the reader sees."
3. Add a `<p>` with a `<strong>` inside. Point out the nesting: "The `<strong>` opens and closes *inside* the `<p>`. Last opened, first closed."

4. Add `<h2>What We Do</h2>` and a `<ul>` with three `<li>` items. Ask: "Why bullets and not numbers here?" (Order does not matter.)
5. Add `<h2>How To Order</h2>` and an `<ol>` with four `<li>` items. Ask: "Why numbers now?" (The steps are in order.)
6. Add `<hr>` then a contact `<p>` with `<br>` and `<em>`.
7. Save (`Ctrl + S`) and open in Chrome. Walk the class through the result top to bottom.

Expected result: the page matches the "expected browser result" in F.11.

G.3 Demonstration 2 — Injecting and fixing errors (6 minutes)

Error 1 — unclosed heading. Delete the `</h1>`. Save, refresh.

- Expected: the heading may still show, but the paragraph after it can inherit the heading's large/bold style (the browser tried to guess). Say: "See how a missing closing tag leaks its style onto the next content? Many tags must be closed."
- Fix it live; refresh; the page is normal again.

Error 2 — text straight inside a list. Remove the `<li>` tags inside the `<ul>` so the words sit directly in `<ul>`. Save, refresh.

- Expected: the bullets disappear or the items look wrong. Say: "Inside `<ul>` and `<ol>`, every item must be wrapped in `<li>`."
- Restore the `<li>` tags.

Error 3 — skipping heading levels for looks. Change `<h2>What We Do</h2>` to `<h4>What We Do</h4>` and say: "It is smaller now — but is that *good*? No. It tells screen readers 'this is a level-4 heading' even though it is a main section. We choose levels by meaning, not by size."

- Restore to `<h2>`.

Error 4 — the "Enter key" mistake. Inside a paragraph, press Enter a few times in the code. Save, refresh. Expected: the text is still one block — the line breaks in the code do nothing. Say: "Only tags control the page. The Enter key in your code is only for *your* eyes." (Then, if wanted, add `<br>` to show a real break.)

Questions to ask during the demo:

- "What is the difference between a tag and an element?"
- "Why should there be only one `<h1>` ?"
- "When would I use `<ol>` instead of `<ul>` ?"
- "Do comments appear on the page?"

H. GUIDED PRACTICAL EXERCISE (~18 minutes)

Objective. Each learner builds a properly structured Adaeze Fashion home page using headings, paragraphs, lists, bold/italic, a line break, a horizontal rule and comments — opened and checked in Chrome.

Materials. VS Code and Chrome open; `webdesign-practice/index.html` available (continue the Session 1 file or create a new one).

Procedure (instructor calls each step; learners follow; instructor walks the room):

1. Open `webdesign-practice` in VS Code and open `index.html`.
2. Make sure the document starts with the correct structure: `<!DOCTYPE html>`, `<html lang="en">`, `<head>` with `<meta charset="UTF-8">` and a `<title>`, then `<body>`.
3. Inside `<body>`, add `<h1>Welcome to Adaeze Fashion</h1>`.
4. Add a paragraph: `<p>...</p>` describing the business in one or two sentences. Include one `<strong>` word (e.g. a price such as **₦25,000**) — the Naira sign tests your charset setting.
5. Add `<h2>What We Do</h2>` and an unordered list with **three** services.
6. Add `<h2>How To Order</h2>` and an ordered list with **four** steps.
7. Add `<hr>` and then a contact paragraph with the address split across two lines using `<br>`, and one `<em>` for the address.
8. Add at least **three comments**: one labelling the header, one labelling the services list, one labelling the contact area.
9. Indent the whole document neatly (Tab / Shift + Tab).
10. Save (`Ctrl + S`), open in Chrome, refresh.

Pause and predict (ask the class):

- After step 5, before previewing: "Will the services show with bullets or numbers?" (Bullets — `<ul>`.)
- After step 6: "Why numbers here?" (Because the order of the steps matters.)
- After step 7, before previewing: "Will the address be on one line or two?" (Two — the `<br>` forces a break.)
- After step 8: "Will the comments appear on the page?" (No.)

Expected result. A clean page: big heading; intro paragraph with a bold item; *What We Do* with three bullets; *How To Order* with four numbers; a horizontal line; a contact paragraph with the address on two lines in italics. The tab shows the page title.

Troubleshooting while walking the room:

- Naira sign shows as `?` → ensure `<meta charset="UTF-8">` is present and the file is saved as UTF-8.
- Bullets missing → the `<li>` tags are missing around the list text.
- Wrong list type → check `<ul>` vs `<ol>`.
- Heading too big/small → check the `<h1>` – `<h6>` levels, not CSS.
- Nothing changed → save, then refresh.
- Content leaking style → a closing tag is missing. (See K.1.)

Completion checklist (learner ticks each):

- ☐ Correct document structure (`DOCTYPE` , `html lang` , `head` , `title` , `body`)
- ☐ One `<h1>`
- ☐ At least two `<h2>` sub-headings
- ☐ At least two paragraphs
- ☐ One `<ul>` with three `<li>` items
- ☐ One `<ol>` with four `<li>` items
- ☐ One `<strong>` and one `<em>`
- ☐ One `<br>` and one `<hr>`
- ☐ At least three comments
- ☐ Code indented neatly
- ☐ Page displays correctly in Chrome after refresh

I. INDEPENDENT CLASS EXERCISE (~15 minutes)

"Build Adaeze's About Section"

Scenario. Adaeze has seen your home page and loves it. She now wants a small **"About Adaeze Fashion"** page section that tells her story and lists her achievements. You will build it on your own — no copying the home page.

Task. Create a new file `about.html` in `webdesign-practice` and build it from scratch with the following content:

1. A full document structure (`DOCTYPE` , `html` , `head` with a title "*About Adaeze Fashion*", `body`).
2. One `<h1>` that says: *About Adaeze Fashion*.
3. An introductory paragraph (2–3 sentences) telling the story: the business was founded in Port Harcourt, what it stands for (for example, quality and fit).
4. A `<h2>` heading *Our Journey* followed by an **ordered list** of **three** milestones in order (for example: 2019 — business founded; 2021 — first shop; 2024 — 500 happy customers).
5. A `<h2>` heading *Why Customers Choose Us* followed by an **unordered list** of **three** reasons.
6. One sentence containing a `<strong>` word (a key value such as **quality**) and one sentence containing an `<em>` word (for example the word *trusted*).
7. A horizontal rule `<hr>` and, below it, a short closing paragraph.
8. At least **two comments** explaining the purpose of sections.

Requirements / constraints:

- Exactly one `<h1>` ; use `<h2>` for the two sections.

- Each `<li>` must be inside a list; no text directly inside `<ul>` / `<ol>`.
- Correct nesting and neat indentation.
- Use lowercase tags and double-quoted attribute values (where attributes appear).
- Work independently; use your notes only for the tag list.

Time allowance: 15 minutes.

Success criteria:

- The page opens in Chrome and shows all sections in the right order.
- The ordered list shows **numbers**; the unordered list shows **bullets**.
- The document is valid (all tags closed) and neatly indented.
- The two comments do not appear on the page.

Hints if stuck:

- Hint 1: Start every page with the same skeleton from the lecture — the browser expects the head before the body.
- Hint 2: Ordered list = `<ol>`; unordered list = `<ul>`. Both need `<li>` items.
- Hint 3: A comment is `<!-- your note -->`. It never shows on the page.

Instructor answer key (do not show learners before they try):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Adaeze Fashion</title>
</head>
<body>

  <!-- Page heading -->
  <h1>About Adaeze Fashion</h1>

  <!-- Introduction -->
  <p>Adaeze Fashion was founded in Port Harcourt to bring <strong>quality</strong>
     and perfect fit to every customer. We are a <em>trusted</em> name for custom
     clothing in Rivers State.</p>

  <!-- Milestones -->
  <h2>Our Journey</h2>
  <ol>
    <li>2019 — Adaeze Fashion was founded</li>
    <li>2021 — We opened our first shop</li>
    <li>2024 — We celebrated 500 happy customers</li>
  </ol>

  <!-- Reasons to choose us -->
  <h2>Why Customers Choose Us</h2>
  <ul>
    <li>Perfect measurements every time</li>
    <li>Quality fabric and neat finishing</li>
    <li>Friendly, on-time delivery</li>
  </ul>

  <hr>

  <!-- Closing -->
  <p>Thank you for choosing Adaeze Fashion. We look forward to sewing for you.</p>

</body>
</html>
```

Expected display: tab reads "*About Adaeze Fashion*"; the page shows the heading, the intro paragraph with **quality** bold and *trusted* italic, *Our Journey* with numbered milestones, *Why Customers Choose Us* with three bullets, a line, and the closing paragraph.

Marking guidance (out of 12): document structure (2); one `<h1>` and two `<h2>` used correctly (2); ordered list of three, numbered (2); unordered list of three, bulleted (2); `<strong>` and `<em>` used (2); `<hr>` and at least two comments (1); nesting/indentation neat and page displays (1).

J. REAL-WORLD CASE STUDY — "The Broken-Looking Page"

Situation. Adaeze's niece, **Ngozi**, is learning web design and proudly shows Adaeze a page she built for the business. The text is all there, but the page looks strange: the heading is enormous and runs into the next sentence, one list has no bullets, and a paragraph is stuck to the text above it. Ngozi is upset and thinks HTML "does not work".

Relevant facts.

- Ngozi typed all the content correctly — the *words* are right.
- The problems are structural: a missing `</h1>`, a `<ul>` whose items were not wrapped in `<li>`, and text typed straight into the body without `<p>` tags.
- Ngozi does not yet understand that tags describe structure and must be closed and nested properly.

Decision point. How should Ngozi fix the page?

Possible approaches.

1. Give up on HTML and rebuild everything from scratch, hoping the problem disappears.
2. Randomly add and remove tags until it "looks right".
3. Read the code slowly, check the structure section by section, and fix the three specific structural mistakes.

Recommended solution (for this stage). Approach 3. Ngozi should:

- Find the missing `</h1>` and close the heading (this stops the "leak").
- Wrap each list item in `<li>` inside the `<ul>` (this brings the bullets back).
- Wrap the loose text in `<p> … </p>` (this restores proper paragraph spacing).
- Re-indent and re-check with the nested "last opened, first closed" rule.

Corrected snippet:

```
<h1>Welcome to Adaeze Fashion</h1>
<p>Custom clothing made in Port Harcourt.</p>
<ul>
  <li>Ankara gowns</li>
  <li>Bridal wear</li>
</ul>
```

Trade-offs.

- Approach 1 wastes time and throws away correct work; the content was never the problem.
- Approach 2 is guessing — it may "work" by luck but teaches nothing and will fail next time.
- Approach 3 takes a few focused minutes, teaches the learner to *read* structure, and fixes the root cause so the same mistakes are caught next time.

Professional connection. Every professional web developer spends time fixing structural HTML mistakes exactly like these. The habit of checking that tags are **closed, correctly nested and used in the right place** is a core professional skill — and it is the same skill the browser's own validation tools (learned in a later session) reward.

Transfer question for the class: "A friend's page has all the right words but looks broken. List **three** HTML structure problems you would check first." (Expected: unclosed tags; list items not in `<li>`; text not in `<p>`; wrong heading levels; tags nested incorrectly.)

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table for live troubleshooting. Do not answer at once — point the learner to the relevant row.

K.1 Missing closing tag — style leaks to later content

Symptoms: A heading or bold text seems to "bleed" onto the next paragraph; the layout looks stretched. **Likely cause:** An opening tag was never closed (for example `<h1>` without `</h1>`). **How to investigate:** Check each opening tag has a matching closing tag; look for a tag that has no `/` partner. **Corrective steps:** Add the missing closing tag. **How to confirm resolved:**

The next content returns to its normal size/style. **Prevention:** Type the opening tag, then immediately type the closing tag, then fill in the content between them.

K.2 List items have no bullets or numbers

Symptoms: A list shows as plain text lines, or the bullets vanish. **Likely cause:** The items are not wrapped in `<li>`, or the items were placed outside `<ul>` / `<ol>`. **How to investigate:** Check that every item sits between `<li>` and `</li>`, inside the list tag. **Corrective steps:** Wrap each item in `<li> … </li>`. **How to confirm resolved:** Bullets or numbers appear. **Prevention:** Type the list opening tag and one empty `<li></li>` first, then duplicate.

K.3 Text runs together / no paragraph spacing

Symptoms: Several sentences form one block with no gaps. **Likely cause:** Text was typed in the body without `<p>` tags, or line breaks in the code were expected to work. **How to investigate:** Check whether the text is wrapped in `<p> … </p>`. **Corrective steps:** Wrap each paragraph in `<p>` tags. Use `<br>` only for a forced line break inside a paragraph. **How to confirm resolved:** Clear gaps appear between paragraphs. **Prevention:** One idea = one `<p>`.

K.4 Wrong heading level / too many `h1`s

Symptoms: The page has several huge headings; sections are confusing. **Likely cause:** Using `<h1>` for everything, or picking levels by size rather than meaning. **How to investigate:** Count the `<h1>`s — there should normally be one. **Corrective steps:** Keep one `<h1>`; use `<h2>` for sections and `<h3>` for sub-sections. **How to confirm resolved:** A clear, single-level hierarchy. **Prevention:** Choose the level for *meaning*, not looks.

K.5 The document does not show correctly / page blank

Symptoms: Blank page or odd rendering. **Likely cause:** Content typed outside `<body>`, or a mis-typed tag name. **How to investigate:** Check the content sits between `<body>` and `</body>`; check tag spelling. **Corrective steps:** Move content into the body; fix spelling. **How to confirm resolved:** The page draws correctly. **Prevention:** Use the document skeleton every time.

K.6 Naira sign (₦) or accents show as `?`

Symptoms: The ₦ symbol shows as `?` or a box. **Likely cause:** Missing `<meta charset="UTF-8">`, or the file saved in the wrong encoding. **How to investigate:** Check the charset line; check VS Code's bottom-right encoding label. **Corrective steps:** Add the charset line; **Save with Encoding → UTF-8**. **How to confirm resolved:** ₦ displays. **Prevention:** Keep charset in every page from the start.

Distinguish likely vs confirmed. A likely cause becomes *confirmed* only after checking (seeing the missing tag, the absent `<li>`, the charset line). Teach learners to confirm before fixing.

L. SESSION ASSIGNMENT

Assignment title: "Adaeze Fashion — Full Home Page"

Purpose. Practise the complete range of basic HTML — document structure, headings, paragraphs, lists, emphasis, breaks, rules and comments — by building a realistic business home page.

Realistic scenario. Adaeze wants her home page to welcome customers, list her services, explain how to order, and show her contact details. You are the designer.

Detailed instructions.

1. Work in `webdesign-practice`, in `index.html`.
2. Start with the correct document structure and a `<title>` of "*Adaeze Fashion — Home*".
3. In the `<body>`, include:
   - one `<h1>`: *Welcome to Adaeze Fashion*
   - a welcome paragraph (2+ sentences), with one `<strong>` word (e.g. a price)
   - `<h2>Our Services</h2>` and an unordered list of at least **four** services
   - `<h2>How To Order</h2>` and an ordered list of at least **four** steps
   - `<h2>Contact Us</h2>` and a paragraph containing an address split with `<br>` and the phone number in `<strong>`

- one `<hr>` before the contact section
- at least **four** meaningful comments labelling sections

4. Indent the whole document neatly.
5. Save, open in Chrome and confirm it displays correctly.

Required deliverables

- `webdesign-practice/index.html` saved and displaying correctly.
- One `<h1>`, three `<h2>` sections, two lists, `<strong>`, `<em>`, `<br>`, `<hr>`, and four comments.

Submission format. File remains in the project folder; show the working page to the instructor before leaving; paste the code into the workbook.

Deadline. Before Session 3.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Document structure	3	`DOCTYPE`, `html lang`, `head`, `title`, `body` correct and closed
Headings	3	Exactly one `h1`; logical `h2` sections
Lists	4	One `<ul>` (4 items) and one `<ol>` (4 items), all items in `<li>`
Emphasis and breaks	3	`<strong>`, `<em>`, ` `, `<hr>` used correctly and purposefully
Comments	3	Four meaningful comments that do not appear on the page
Nesting and indentation	2	Correctly nested; neatly indented
Display and neatness	2	Page draws correctly in Chrome after refresh; no errors

Common mistakes to avoid. Missing closing tags; list items without `<li>`; multiple `<h1>`s; typing line breaks in code expecting them to show; forgetting to save/refresh.

Instructor reference solution. Use the Section F.11 page, extended with the Contact section:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>

  <!-- Header -->
  <h1>Welcome to Adaeze Fashion</h1>

  <!-- Introduction -->
  <p>Adaeze Fashion designs and sews <strong>custom clothing from ₦15,000</strong>
     for women and men in Port Harcourt.</p>

  <!-- Services -->
  <h2>Our Services</h2>
  <ul>
    <li>Custom Ankara gowns</li>
    <li>Bridal wear</li>
    <li>Corporate uniforms</li>
    <li>Children's clothing</li>
  </ul>

  <!-- Ordering process -->
  <h2>How To Order</h2>
  <ol>
    <li>Choose your fabric</li>
    <li>Take your measurements</li>
    <li>We sew your outfit</li>
    <li>Collect your order</li>
  </ol>
```

```
  <hr>

  <!-- Contact -->
  <h2>Contact Us</h2>
  <p>Visit us at <em>24/26 Ebony Road, Port Harcourt</em><br>
     or call <strong>0800 000 0000</strong>.</p>

</body>
</html>
```

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Multiple choice. Which is a **void** element (no closing tag)? a) `<p>` b) `<h1>` c) `<br>` d) `<ul>`

Q2. Short answer. Explain the difference between a **tag** and an **element** in one sentence each.

Q3. Error detection. Find the mistake and correct it: `<p>Come and see our <strong>latest designs.</p></strong>`

Q4. Multiple choice. For a numbered list of steps where order matters, which tag do you use? a) `<ul>` b) `<ol>` c) `<li>` d) `<p>`

Q5. Explain in your own words. Why should a page normally have only one `<h1>`?

Q6. Fill in the blanks. In the document structure, the _____ contains information about the page (like the title), and the _____ contains everything that is visible on the page.

Q7. Practical identification. Look at this line: `<a href="https://www.cea.ng">Cyber Elias Academy</a>`. Which part is the attribute, and what is its value?

Q8. Scenario. A learner writes a paragraph across three lines in the code, expecting three lines on the page — but the browser shows one block. Why, and what tag would force a line break?

INSTRUCTOR ANSWER KEY (Q1–Q8)

A1. (c) `<br>` — it is a void element. (`<img>` and `<hr>` are other examples.)

A2. A **tag** is the marker in angle brackets (e.g. `<p>` or `</p>`). An **element** is the whole package: opening tag + content + closing tag (e.g. `<p>Hello</p>`).

A3. The tags cross over (nesting error). Correct: `<p>Come and see our <strong>latest designs.</strong></p>` — the `<strong>` must open and close *inside* the `<p>`.

A4. (b) `<ol>`.

A5. Because the `<h1>` names the *whole page*; a page has one main topic. Multiple `<h1>`s confuse screen readers and search engines. Sections use `<h2>` and below.

A6. The **head** contains information about the page; the **body** contains the visible content.

A7. The attribute is `href`; its value is `https://www.cea.ng`. (The visible text "*Cyber Elias Academy*" is the content, not the attribute.)

A8. The browser ignores the line breaks you type in the code — they are for the human reader only. To force a visible line break inside a paragraph, use `<br>`. (For a new paragraph, use a new `<p>`.)

N. REFLECTION AND RECAP

Plain-English summary. Today you opened HTML properly. You learned that HTML is a **markup language** that describes the *structure and meaning* of content — not its looks. You met the three key words: **tag** (the marker), **element** (tag + content + closing tag) and **attribute** (extra information inside the opening tag). You learned the fixed document skeleton (`DOCTYPE`, `html`, `head`, `title`, `body`), and you practised the everyday elements: headings `h1` – `h6`, paragraphs `p`, unordered and ordered lists (`ul`, `ol`, `li`), bold and italic (`strong`, `em`, `b`, `i`), line breaks and horizontal rules (`br`, `hr`), and comments (`<!-- -->`).

Finally, you learned to **nest and indent** so your code stays correct and readable — and you built a real home page for Adaeze Fashion.

Five important takeaways.

1. HTML = **structure and meaning**; CSS = appearance. A tag is a *label*, not a decoration.
2. A **tag** is the marker; an **element** is tag + content + closing tag; an **attribute** gives extra information.
3. Use **one** `<h1>` per page and keep heading levels logical.
4. Use `<ul>` when order does not matter (bullets) and `<ol>` when it does (numbers); every item needs `<li>`.
5. **Close your tags and nest them correctly** (last opened, first closed), and **indent** for readability.

Self-check — can you now…

- Explain the difference between a tag, an element and an attribute?
- Write the full document structure from memory?
- Pick the right list type for a given situation?
- Correct a nesting error like `<strong>…</p></strong>`?
- Explain why one `<h1>` per page matters?
- Add a comment that will not appear on the page?

Connection to the next session. You can now structure *text*. Next session, **Session 3 — HTML Intermediate**, we add the things a real business page needs: **links** (to move between pages and to other sites), **images** with `alt` text, **tables** for price lists, **semantic elements** (`header` , `nav` , `main` , `section` , `footer`) that describe the parts of a page, and an introduction to **forms** (the contact form Adaeze needs).

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence.

- The learner writes the document skeleton without prompting.
- The learner distinguishes a tag from an element and an attribute and gives an example of each.
- The learner chooses `<ul>` vs `<ol>` by reasoning about order.
- The learner notices and fixes a nesting error (e.g. crossed tags).
- The learner uses one `<h1>` and logical `<h2>` sections by choice, not by size.
- The learner indents their code neatly.

Common misconceptions to watch for and correct.

- "HTML is a programming language." → It is a markup language; it describes, it does not compute.
- "`<strong>` and `<b>` are identical." → They look the same but carry different meaning.
- "I can put text straight inside `<ul>`." → Items must be inside `<li>`.
- "The Enter key in my code makes a new line on the page." → Only `<br>` or a new `<p>` does.
- "Comments show on the page." → They never do.
- "Big heading = important heading." → The level expresses meaning, not size.

Suggested remedial exercises.

1. "Close the tags" drill: hand out five lines each missing one closing tag; learners fix and check in the browser.
2. List sorting: give ten scenarios (a menu, a recipe, a shopping list, a to-do order) and ask `<ul>` or `<ol>` for each.
3. Nesting cards: write tags on paper strips (`<p>` , `<strong>` , `</p>` , `</strong>`) and ask learners to arrange them correctly, then type the result.

Differentiation.

- **Level 1 (supported):** Provide a skeleton file with the document structure and empty `<p>` , `<ul>` , `<ol>` already in place; learners fill in content. Pair with a confident learner and allow extra time.
- **Level 2 (standard):** Complete the exercises and assignment as written.
- **Level 3 (challenge):** Add a **nested list** (a services list with sub-items), use `h3` sub-sections under an `h2` , and research the `<abbr>` element (for abbreviations) and use it once with a `title` attribute.

Timing guidance (105 minutes). Opening 6 · Prerequisite 7 · Lecture 40 · Demonstration 15 · Guided practical 18 · Independent exercise 15 · Case study 4 · Knowledge check 4 · Reflection/close 3. If slow, shrink the case study to 2 minutes and protect the guided practical and independent exercise — writing valid HTML is the priority. If fast, use spare time for Level 3 tasks and peer review of each other's pages.

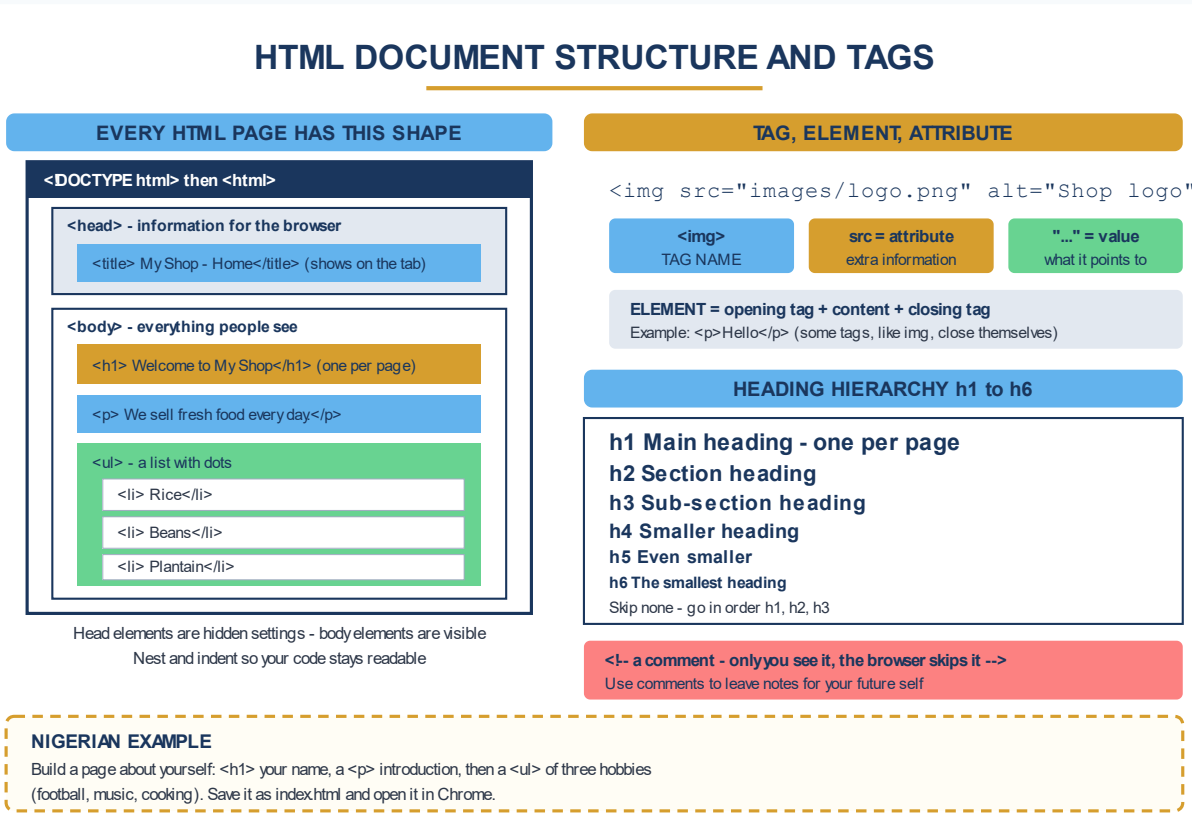
END OF SESSION 2

Next: Session 3 — HTML Intermediate: links, images, tables, semantic elements and an introduction to forms.

SESSION 03 OF 12

HTML INTERMEDIATE

CONCEPT DIAGRAM — SESSION 03



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 3 — HTML INTERMEDIATE

A. SESSION IDENTIFICATION

Item	Detail
Session number	3 of 12
Session title	HTML Intermediate
Week / Module	Week 1 — Module 1: HTML Foundations
Recommended duration	105 minutes
Difficulty level	Beginner

Item	Detail
Prerequisite knowledge	Sessions 1–2. The learner can write a valid HTML document with headings, paragraphs, lists, emphasis and comments.
Connection to previous session	Session 2 gave us text structure. This session adds the elements that make a page a *real website*: links to move between pages, images to show the products, tables for price lists, semantic elements that describe the parts of a page, and a first look at forms.
Preparation before class	Academy PCs on; VS Code and Chrome open; `webdesign-practice` open; at least one image placed in `webdesign-practice/images/` (for example `product1.jpg` or `logo.png`); a second HTML file `about.html` available for the link demo; projector connected.
Running project	Adaeze Fashion grows from a single page into a small **multi-page** site. We add navigation links to `about.html` and `contact.html`, a price-list table, semantic structure, and the beginnings of a contact form (finished in a later session).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Create hyperlinks** with the `<a>` tag and the `href` attribute.
2. **Distinguish** between an **internal link** (to another page on the same site) and an **external link** (to another website).
3. **Open a link in a new tab** using `target="_blank"`, and explain when this is appropriate.
4. **Link to a section of the same page** using an `id` and a `#` link.
5. **Add images** with `<img>`, correct `src` paths and meaningful `alt` text, and explain why `alt` is required.
6. **Build a simple table** with `<table>`, `<tr>`, `<th>` and `<td>`, and explain when a table is the correct tool.
7. **Use semantic elements** `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` and `<footer>` to describe the parts of a page.
8. **Explain the difference** between a generic `<div>` and semantic elements, and when a `<div>` is acceptable.
9. **Describe the parts of a form** — `<form>`, `<label>`, `<input>`, `<textarea>`, `<select>`, `<button>` — and name several input types.

C. REQUIRED MATERIALS

Hardware

- One computer per learner; instructor computer on a projector.
- Optional: a printed newspaper (for the semantic-opening analogy).

Software

- Visual Studio Code; Google Chrome.

Sample / starter files

- `webdesign-practice/index.html` from Session 2.
- At least one image in `webdesign-practice/images/` (e.g. `product1.jpg`, `product2.jpg`, `logo.png`). If no real images are available, the instructor supplies small placeholder images.
- Instructor demo files: `demo/links.html` and `demo/semantic.html` (contents in Section G).

Worksheets / printed materials

- "Images, Links and Semantics Quick Reference" card (one per learner: `a href`, `target`, `#id`, `img src/alt`, `table/tr/th/td`, `header/nav/main/section/article/aside/footer`).
- Worksheet 3.1 — "Link or Image?" (tick the correct element for eight scenarios).
- Worksheet 3.2 — "Div or Semantic?" (choose the best semantic tag for each part of a page).

Internet

- Helpful but **not required**. If Wi-Fi is available, the instructor may show a real site's navigation. *Note:* external links created today will not open anything without internet — the instructor demonstrates this honestly (see K.5).

Alternative for learners without sample images

- Use any picture found on the academy machine (copy it into `images/`). If none exists, the instructor can create a small coloured image in Paint and save it as `product1.png`.

D. OPENING ACTIVITY (~6 minutes)

"A Website You Cannot Navigation" (The Newspaper Analogy)

Instructor setup. Have the home page from Session 2 open in Chrome. It has headings, paragraphs and lists — but **no links and no images**.

What the instructor says:

> *"Look at our Adaeze Fashion page. Is it a website yet? … It has text. But can a customer click anything? Can they go to an About page? Can they see a single photo of Adaeze's beautiful gowns? No.*
>
> *A page with only text is like a **newspaper with no pictures and no page numbers** — you can read it, but you cannot move around it, and you cannot see the story."*
>
> *[Hold up a newspaper if available, or draw a simple page layout on the board: a title at the top, a row of section names, articles in the middle, a small side column, and a footer at the bottom.]*
>
> *"A newspaper is organised. It has a **masthead** (the newspaper name), a **menu** of sections, **articles**, and a **footer** with details. HTML has special tags with exactly these meanings: `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>`. And to move from one page to another, we need **links**. Today we turn our text page into a real website."*

Expected learner response. Learners quickly see that a page without links or images is not a usable site. Some will say "we need a button" or "we need the menu". Guide them: the clickable text is a **link**, and the menu is a **navigation** made of links.

Why this opener works. It creates a clear, felt gap ("this is not a website yet") and connects the new elements to something familiar (a newspaper's structure), so the semantic tags feel obvious rather than arbitrary.

E. PREREQUISITE REVIEW (~7 minutes)

Oral round-robin; questions on the projector.

Question 1: "What is the basic structure of an HTML document, in order?"

- Expected answer: `DOCTYPE`, `html`, `head` (with `title`), `body`.
- Gap: Missing `DOCTYPE` or content outside body. Remind with the skeleton.

Question 2: "What tag makes an ordered list, and what tag makes each item?"

- Expected answer: `<ol>` for the list; `<li>` for each item.
- Gap: Items not wrapped in `<li>`. Repeat the rule.

Question 3: "What is an attribute? Give one example."

- Expected answer: Extra information inside an opening tag with a name and a value, e.g. `href="..."` or `lang="en"`.
- Gap: Forgetting the quotes around the value.

Question 4: "Correct this: `<p>Visit our <em>shop.</p></em>`"

- Expected answer: `<p>Visit our <em>shop.</em></p>` — the `<em>` must open and close inside the `<p>`.
- Gap: Nesting. Re-state "last opened, first closed".

Question 5: "Why do we write relative paths like `images/logo.png`?"

- Expected answer: They describe the file's position in the project folder, so the site keeps working when moved to a server.
- Gap: Using backslashes. Remind: **web paths use forward slashes**.

Bridge to today (say this):

> *"You can already build text structure. Today we add the parts that make a real site: the links that connect pages, the images that show products, the table for a price list, and the semantic tags that label each region of the page. By the end, Adaeze Fashion will have a clickable menu — a real navigation."*

Gap handling. If several learners cannot recall the document skeleton or the nesting rule, spend two extra minutes re-writing the skeleton on the board together before continuing. Pair struggling learners with confident ones for the practicals.

F. FULL LECTURE NOTES (~40 minutes)

F.1 Links — the heart of the web

The idea. A **hyperlink** (or "link") is clickable text or an image that takes the user somewhere else — another page on the same site, another website, or a place further down the same page. Links are what make the web a *web*.

Plain-English definition. A link is created with the **anchor** element, `<a>`, and the `href` attribute, which holds the **destination**.

Why they exist. Without links, every page would be an island. Links let a visitor move from Home to About to Contact, or jump from a blog post to an external source, with one click.

Familiar analogy. A link is a **door**. The text between `<a>` and `</a>` is the door's label ("Enter the kitchen"); the `href` value is where the door leads.

The basic syntax.

```
<a href="about.html">About Us</a>
```

- `<a>` — the anchor (link) element.
- `href` — the attribute that holds the destination (think "hypertext reference").
- `"about.html"` — the value: where the link goes.
- `About Us` — the clickable text the user sees.

Expected result. The words *About Us* appear, usually in blue and underlined (the browser's default link style). Clicking them opens `about.html` in the same tab.

F.2 Internal versus external links

Internal link — points to a page **on the same website**.

```
<a href="about.html">About Us</a>
<a href="contact.html">Contact</a>
```

The value is a **relative path** (no `https://`), because the file lives in our own project folder.

External link — points to a page on **another website**.

```
<a href="https://www.cea.ng">Cyber Elias Academy</a>
```

The value is a full web address, starting with `https://`.

Why the difference matters. Internal links use relative paths so the whole site keeps working when moved to a server. External links need the full address because they live on someone else's server.

Worked example — the Adaeze navigation.

```
<nav>
  <a href="index.html">Home</a> |
  <a href="about.html">About</a> |
  <a href="contact.html">Contact</a>
</nav>
```

Expected result: three clickable words separated by `|` pipes; each opens its page in the same tab.

Misconception. "I must type the full address for my own pages too." No — for your *own* pages use the short relative path (`about.html`), not `C:\...` or a full web address.

F.3 Opening a link in a new tab

The idea. By default a link opens in the **same tab**. Adding `target="_blank"` opens it in a **new tab** instead.

```
<a href="https://www.cea.ng" target="_blank">Visit Cyber Elias Academy</a>
```

When to use it. Usually for **external** links — so that when a visitor clicks a link to another site, your own site stays open behind it. For your own internal pages, it is usually better to stay in the same tab.

A good habit (see the honesty note). Modern practice adds `rel="noopener"` alongside `target="_blank"` :

```
<a href="https://www.cea.ng" target="_blank" rel="noopener">Visit Cyber Elias Academy</a>
```

`rel="noopener"` is a small security measure that stops the new page from being able to control your page. It is not visible to the user but is good practice.

Misconception. " `target="_blank"` should be on every link." No — using it everywhere floods the user with tabs. Use it mainly for external links.

F.4 Linking to a section of the same page

The idea. You can link to a specific *part* of a page using an `id` . This is common on long pages with a "table of contents" or "back to top" link.

Step 1 — give the target element an `id` attribute (a unique name):

```
<h2 id="services">Our Services</h2>
```

Step 2 — link to it with a `#` and the same name:

```
<a href="#services">Go to Services</a>
```

Worked example.

```
<a href="#contact">Jump to Contact</a>

<h2>Our Services</h2>
<p>Custom gowns and uniforms.</p>

<h2 id="contact">Contact Us</h2>
<p>Call 0800 000 0000.</p>
```

Expected result: clicking *Jump to Contact* scrolls the page down to the *Contact Us* heading.

Rules. An `id` must be **unique** on the page (only one element per id). Names have no spaces — use lowercase and hyphens (`#our-services`).

Misconception. " `#` links always go to the top." `href="#"` alone goes to the top of the page, but `href="#contact"` goes to the element whose `id="contact"` . The name after `#` must match an `id` exactly.

F.5 Images — `<img>`, `src` and `alt`

The idea. The `<img>` element places a picture on the page. It is a **void** element — it has no closing tag and no content. Instead, everything it needs comes from **attributes**.

```
<img src="images/product1.jpg" alt="Adaeze Fashion Ankara gown in blue">
```

Line by line:

- `<img>` — the image element.
- `src` — the **source**: the file path of the image (here, inside the `images` folder — a relative path).
- `alt` — the **alternative text**: a short description of the image.

Why `alt` is required.

1. **Accessibility:** screen readers read the `alt` text aloud to blind users, so they know what the image shows.
2. **Broken images:** if the image cannot load (wrong path, no internet), the browser shows the `alt` text instead of an empty box.
3. **Search engines:** `alt` helps search engines understand the picture.

Good `alt` text. Describe the image's meaning briefly: `alt="Ankara gown in blue with gold trim"`. **Bad `alt` text.** `alt="image"` (useless) or `alt="gown.jpg"` (that is the file name, not a description).

Image paths.

- Relative: `images/product1.jpg` — the image is in the `images` folder of our project.
- Absolute (external): `https://www.example.com/photo.jpg` — the image lives on another website.

Sizing note (for now). An image displays at its own size by default. We will control size and layout with CSS in later sessions; do not use width/height attributes to fix big layout problems yet.

Worked example — a product on the Adaeze page.

```
<h2>Featured Product</h2>
<img src="images/product1.jpg" alt="Adaeze Fashion blue Ankara gown">
<p>Our latest Ankara gown — perfect for weddings and celebrations.</p>
```

Expected result: the *Featured Product* heading, then the picture, then the caption. If the file is missing, the browser shows the alt text "*Adaeze Fashion blue Ankara gown*" and a small broken-image icon.

Misconception. "If the image does not show, I forgot the `<img>` tag." Often the tag is correct but the **path** is wrong. Check `src` character by character. (See K.2.)

F.6 Tables — rows, columns and cells

The idea. A **table** displays data in a grid of **rows** and **columns**. Use it for *tabular data* — price lists, timetables, comparisons — where the rows and columns have meaning.

The tags.

- `<table>` — the whole table.
- `<tr>` — a **table row** (table row).
- `<th>` — a **table header** cell (bold and centred by default; names the column).
- `<td>` — a **table data** cell (the actual data).

Worked example — Adaeze's price list.

```
<table>
  <tr>
    <th>Item</th>
    <th>Price</th>
  </tr>
  <tr>
    <td>Ankara gown</td>
    <td>₦25,000</td>
  </tr>
```

```
  <tr>
    <td>Corporate uniform</td>
    <td>₦15,000</td>
  </tr>
  <tr>
    <td>Bridal wear</td>
    <td>₦80,000</td>
  </tr>
</table>
```

Expected result. A table with two columns. The first row (*Item*, *Price*) appears bold and centred (because those are `<th>`). The three data rows appear as normal cells.

Why not use tables for page layout. Long ago, people used tables to arrange a whole page (menus, columns). This is **wrong and outdated** because tables do not adapt to phones and confuse screen readers. Today, layout is CSS's job; tables are for data.

Misconception. "Tables are for layout." No — tables are for tabular data. If you are laying out a page, use CSS (later sessions).

F.7 Semantic HTML — describing the parts of a page

The idea. Semantic means "carrying meaning". There are HTML elements that *name the role* of a part of the page, so the browser, screen readers and search engines understand the structure.

The key semantic elements.

Element	Meaning	Where it goes
`<header>`	The top of a page (or a section); usually the logo and main heading	Top
`<nav>`	A group of navigation links	Usually inside the header
`<main>`	The single main content of the page	Middle
`<section>`	A themed grouping of related content, usually with a heading	Inside main
`<article>`	Self-contained content that makes sense on its own (a blog post, a product)	Inside main
`<aside>`	Related but secondary content (a sidebar, opening hours)	Beside or inside main
`<footer>`	The bottom of a page; copyright and contact details	Bottom

Why semantic HTML matters.

1. **Accessibility:** screen readers let users *jump* to the `<nav>`, `<main>` or `<footer>`, so blind users can skip around quickly.
2. **SEO:** search engines understand which content is important.
3. **Maintainability:** a developer reads `<header>` and instantly knows what it is; `<div>` says nothing.
4. **Consistency:** every page of a site uses the same pattern, so it is easy to keep consistent.

Worked example — the Adaeze Fashion page structure.

```
<header>
  <h1>Adaeze Fashion</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <section>
    <h2>Welcome</h2>
    <p>Custom clothing made in Port Harcourt.</p>
  </section>

  <section>
    <h2>Our Services</h2>
    <ul>
      <li>Ankara gowns</li>
```

```
      <li>Corporate uniforms</li>
    </ul>
  </section>
</main>

<footer>
  <p>&copy; 2026 Adaeze Fashion. All rights reserved.</p>
</footer>
```

Expected result. The page *looks* much like plain text at first (these elements have little default styling), but the structure is now meaningful. The `©` code prints the copyright symbol ©.

Note on `©`. `©` is an **HTML entity** — a short code that prints a special character. It prints ©. We will meet a few entities; they always start with `&` and end with `;`.

Misconception. "Semantic tags change how the page looks." Mostly they do not — they describe *meaning*. Their value is for accessibility, search engines and readability. Styling comes later with CSS.

F.8 `<div>` versus semantic elements

The idea. A `<div>` (short for "division") is a **generic container** with *no meaning*. It is useful for grouping things purely for styling or layout, but it tells us nothing about what it contains.

The rule of thumb. Use a **semantic** element when one fits (`header` , `nav` , `main` , `section` , `article` , `aside` , `footer`). Use a `<div>` only when **no** semantic element fits — usually as a styling wrapper for CSS.

Compare:

```
<!-- Unclear: what is this? -->
<div>
  <h1>Adaeze Fashion</h1>
</div>

<!-- Clear: this is the page header -->
<header>
  <h1>Adaeze Fashion</h1>
</header>
```

Why it matters. A page built entirely from `<div>` tags is like a storeroom where every box is labelled "stuff". A screen reader cannot announce "navigation", and a future developer cannot tell regions apart.

Worked example — acceptable use of a `<div>`.

```
<div class="hero">
  <h2>Custom clothing for every occasion</h2>
</div>
```

Here the `<div>` groups content that will later be styled as a "hero" banner (via the `class`). It is a reasonable use because no semantic tag describes a decorative banner.

Misconception. "Semantic tags are optional; `<div>` is simpler." For a small site they still matter: they make the page accessible and easy to maintain, and they are expected in professional work.

F.9 Forms — an introduction

The idea. A **form** collects information from the user — a name, an email, a message. At the end of this course, the contact form on Adaeze's Contact page is where customers will send enquiries.

Plain-English definition. A form is a group of **input controls** (fields) inside a `<form>` element, finished with a **submit button**.

The core tags.

Tag	Purpose
`<form>`	Wraps the whole form

Tag	Purpose
`<label>`	A text label that describes an input
`<input>`	A single field (its type changes its behaviour)
`<textarea>`	A larger, multi-line text box
`<select>` + `<option>`	A drop-down list of choices
`<button>`	The clickable button (usually "Submit")

Common `input` types.

- `type="text"` — single-line text.
- `type="email"` — a field that checks for an email format.
- `type="password"` — hides the characters as dots.
- `type="number"` — only numbers allowed.
- `type="date"` — a date picker.
- `type="checkbox"` — a tick box (can choose several).
- `type="radio"` — a round button (choose one from a group).
- `type="submit"` — a submit button (older style; we prefer `<button>`).

The `label` and `id` link — very important.

```
<label for="name">Name:</label>
<input type="text" id="name" name="name">
```

- The label's `for` attribute **must match** the input's `id`.
- This links them: clicking the *word* "Name:" moves the cursor into the field. Screen readers read the label aloud. Matching them is essential for accessibility.

Worked example — a simple contact form.

```
<form action="#" method="post">
  <p>
    <label for="name">Name:</label><br>
    <input type="text" id="name" name="name" required>
  </p>

  <p>
    <label for="email">Email:</label><br>
    <input type="email" id="email" name="email" required>
  </p>

  <p>
    <label for="message">Message:</label><br>
    <textarea id="message" name="message" rows="5"></textarea>
  </p>

  <p>
    <label for="service">Service:</label><br>
    <select id="service" name="service">
      <option>Ankara gown</option>
      <option>Corporate uniform</option>
      <option>Bridal wear</option>
    </select>
  </p>

  <button type="submit">Send Message</button>
</form>
```

Expected result. A form with a text field, an email field, a five-line message box, a drop-down list of services, and a *Send Message* button. The fields marked `required` cannot be left empty — if you press Send with them empty, the browser shows a small warning.

Honesty note (important). A plain HTML form does **not actually send** anything on its own. Without an `action` pointing to a server, or JavaScript, or a form service, clicking Submit just reloads (or does nothing useful). Real submission is a later topic. **Today, we are only building the form's *structure*** — the fields and labels. The contact form is completed later in the course.

Misconception. "Submitting will email Adaeze immediately." Not yet. A form needs a destination and a handler. Building the form correctly is the essential first step.

F.10 Putting it together — the Adaeze Fashion multi-page site (so far)

Here is a runnable `index.html` that uses links, an image, a table, semantic elements and the start of a form.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
</head>
<body>

  <header>
    <h1>Adaeze Fashion</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="about.html">About</a> |
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>

    <section>
      <h2>Welcome</h2>
      <p>Beautiful custom clothing designed in Port Harcourt.</p>
      <!-- The image lives in the images folder (relative path) -->
      <img src="images/product1.jpg" alt="Adaeze Fashion blue Ankara gown">
    </section>

    <section>
      <h2>Price List</h2>
      <table>
        <tr>
          <th>Item</th>
          <th>Price</th>
        </tr>
        <tr>
          <td>Ankara gown</td>
          <td>₦25,000</td>
        </tr>
        <tr>
          <td>Corporate uniform</td>
          <td>₦15,000</td>
        </tr>
      </table>
    </section>

    <aside>
      <h2>Opening Hours</h2>
      <p>Monday to Saturday, 9am – 6pm.</p>
    </aside>

  </main>

  <footer>
    <p>&copy; 2026 Adaeze Fashion. Visit us at 24/26 Ebony Road, Port Harcourt.</p>
  </footer>

</body>
</html>
```

Expected browser result.

- Top: **Adaeze Fashion** heading with a clickable menu: Home | About | Contact.
- *Welcome* section: a paragraph and the product image.
- *Price List*: a two-column table with bold headers *Item* and *Price* and two data rows.
- A sidebar block *Opening Hours* with one line of text.
- Footer: © 2026 Adaeze Fashion… address.
- Clicking *About* or *Contact* will fail until those files exist (that is the next step — create them!).

Why this page is good. It has semantic regions, a working navigation (once the other pages exist), a properly described image, a data table used correctly (not for layout), and a footer.

F.11 Summary of the lecture

- **Links** (`<a href="...">`) connect pages: **internal** (relative, your own pages) and **external** (full address, other sites).
- `target="_blank"` opens a link in a new tab — mainly for external links; add `rel="noopener"`.
- `id` + `#name` links to a section on the same page.
- **Images** (`<img src="" alt="">`) must have a correct **path** and meaningful **alt** text.
- **Tables** (`table/tr/th/td`) are for **data**, not layout.
- **Semantic elements** describe page regions; use them instead of `<div>` wherever one fits.
- **Forms** (`form/label/input/textarea/select/button`) collect input; the `label for` must match the `input id`. A plain HTML form does not send anything by itself yet.

G. INSTRUCTOR DEMONSTRATION (~15 minutes)

G.1 What the instructor prepares

- `webdesign-practice` open in VS Code with `index.html`, plus two new files to create in the demo (`about.html` , `contact.html`).
- An image in `images/` (e.g. `product1.jpg`).
- Chrome open.
- The demo files from Section F.10.

G.2 Demonstration 1 — Links and navigation (6 minutes)

Say: "Watch me connect our pages together."

1. Open `index.html` . Add the `<header>` with `<h1>` and a `<nav>` containing three links:

```
<nav>
  <a href="index.html">Home</a> |
  <a href="about.html">About</a> |
  <a href="contact.html">Contact</a>
</nav>
```

2. Create `about.html` and `contact.html` with a simple skeleton each (a heading and one paragraph).
3. Save all files. Refresh `index.html` . Click *About* — it opens `about.html` in the **same tab**. Click *Home* to return.
4. **Inject an error:** change `href="about.html"` to `href="abot.html"` (a typo). Save, refresh, click *About*.
   - Expected: a **404 / "file not found"** page. Say: "Broken links are the number-one website bug. One letter wrong and the customer is lost."
   - Fix it; click again; it works.
5. Add an external link with a new tab:

```
<a href="https://www.cea.ng" target="_blank" rel="noopener">Cyber Elias Academy</a>
```

   Show that it opens a new tab (if internet is available). If no internet, explain honestly that an external link needs internet.

Questions to ask:

- "Why does `about.html` not need `https://`?" (It is on our own site — a relative path.)
- "When should we use `target="_blank"`?" (Mainly for external links.)

G.3 Demonstration 2 — Images and a table (5 minutes)

Say: "Now let us show Adaeze's products and her prices."

1. Add an `<img>` with a correct relative path and a meaningful `alt`.
2. **Inject an error:** change `src="images/product1.jpg"` to `src="product1.jpg"` (missing folder). Save, refresh.
   - Expected: a broken-image icon showing the **alt** text. Say: "This is why `alt` matters — even when the image fails, the description remains."
   - Fix the path; the image appears.
3. Add the price-list `<table>` from F.6. Save, refresh. Point at the bold, centred header row.

Questions to ask:

- "Why must `alt` describe the image, not just say 'image'?" (Accessibility; screen readers.)
- "Could I use this table to lay out the whole page?" (No — tables are for data; layout is CSS.)

G.4 Demonstration 3 — Semantic structure and a form (4 minutes)

Say: "Finally, let me show you how to label the page's regions and start a contact form."

1. Reorganise the page into `<header>`, `<main>` (with two `<section>`s and an `<aside>`), and `<footer>`.
2. Add the contact form from F.9 to `contact.html`. Save, refresh.
3. Click the *Name:* label — the cursor jumps into the field. Say: "That only works because `for="name"` matches `id="name"`."
4. Try to submit with a `required` field empty — show the browser warning. **Explain honestly:** "It does not email yet; today we are only building the structure."

Inject an error: make the label `for="fullname"` while the input is `id="name"`. Click the label — nothing focuses. Fix the mismatch.

Questions to ask:

- "Why use `<header>` instead of `<div>`?" (Meaning: accessibility, SEO, readability.)
- "What must match the input's `id`?" (The label's `for`.)

H. GUIDED PRACTICAL EXERCISE (~18 minutes)

Objective. Each learner turns the Adaeze home page into a page with a working navigation, an image, a price-list table and semantic regions — and builds the skeleton of a contact page with a form.

Materials. VS Code and Chrome open; `webdesign-practice` with at least one image in `images/`.

Procedure (instructor calls each step; learners follow):

1. Open `index.html` in VS Code.
2. Wrap the top of the page in a `<header>` containing an `<h1>` and a `<nav>` with three links: `index.html`, `about.html`, `contact.html` (use the `|` between them).
3. Wrap the main content in `<main>`, using at least two `<section>` blocks with `<h2>` headings: *Welcome* and *Price List*.
4. Add an `<img>` inside the *Welcome* section: `<img src="images/product1.jpg" alt="Adaeze Fashion Ankara gown">` (use your actual image name).
5. Add a price-list `<table>` inside the *Price List* section with a header row (`<th>Item</th><th>Price</th>`) and **three** data rows.
6. Add an `<aside>` with an `<h2>Opening Hours</h2>` and one line of text.
7. Wrap the bottom in a `<footer>` with a `©` line.
8. Create a new file `contact.html` with a document skeleton, a `<header>` / `<nav>` (same menu) and a `<main>` containing a `<form>` with: a text field (name), an email field, a `<textarea>` (message), a `<select>` with three options, and a `<button type="submit">`.
9. Save both files. Open `index.html`, click the navigation links, and check the page renders.

Pause and predict (ask the class):

- After step 2, before clicking: "Will *About* open in the same tab or a new one?" (Same tab — no `target`.)
- After step 4, before previewing: "What shows if the image path is wrong?" (The alt text and a broken-image icon.)
- After step 5, before previewing: "Which row is bold and centred?" (The `<th>` header row.)
- After step 8: "Will clicking *Name:* put the cursor in the field?" (Only if `for` matches `id`.)

Expected result. A page with a clickable top menu, a welcome section with an image, a two-column price table, a sidebar of opening hours, and a footer — plus a contact page with a structured (non-sending) form.

Troubleshooting while walking the room:

- Broken image → check `src` path and that the file is inside `images/`.
- Link goes to 404 → check the `href` spelling and that the target file exists.
- Table columns collapsed → a missing `<tr>` / `<td>` / `<th>` tag.
- Label does not focus the field → `for` and `id` do not match.
- Nothing updates → save, then refresh.

Completion checklist (learner ticks each):

- [ ] `<header>` with `<h1>` and `<nav>`
- [ ] Three internal links working (Home, About, Contact)
- [ ] At least one external link with `target="_blank"`
- [ ] `<img>` with correct `src` and meaningful `alt`
- [ ] `<table>` with a `<th>` header row and three data rows
- [ ] `<main>` with two `<section>` blocks and one `<aside>`
- [ ] `<footer>` with a `©` line
- [ ] `contact.html` with a `<form>`, `<label>`, text input, email input, `<textarea>`, `<select>`, `<button>`
- [ ] Every `label for` matches its `input id`
- [ ] Both pages display correctly in Chrome

I. INDEPENDENT CLASS EXERCISE (~15 minutes)

"Adaeze's Collection Page"

Scenario. Adaeze wants a **Collection** page that shows three of her products with a picture and a small price table, all inside a proper semantic layout, with a menu linking back to the other pages. Build it yourself — do not copy the home page line for line, though you may reuse the menu idea.

Task. Create `collection.html` in `webdesign-practice` containing:

1. A full document structure with `<title>` "*Adaeze Fashion — Collection*".
2. A `<header>` with an `<h1>` (*Our Collection*) and a `<nav>` linking to `index.html`, `about.html`, `contact.html`.
3. A `<main>` with:
   - a `<section>` containing **one** `<img>` (a real image from your `images/` folder) with correct `src` and meaningful `alt`;
   - a `<section>` (heading *Prices*) containing a `<table>` with a header row (*Item*, *Price*) and **three** data rows of products and prices (use ₦);
   - an `<aside>` containing a "*Back to top*" link using `href="#"`, and one short note.
4. A `<footer>` with a `©` line and Adaeze's address.

Requirements / constraints:

- Use semantic tags (`header`, `nav`, `main`, `section`, `aside`, `footer`) — do **not** use `<div>` for the main regions.
- Every image has meaningful `alt` text.
- The table has one `<th>` header row and three `<td>` data rows (two cells each).
- Neat indentation; all tags closed.
- Work independently.

Time allowance: 15 minutes.

Success criteria:

- The page opens in Chrome; the menu links work (Home/About/Contact may lead to existing or placeholder pages).
- The image displays (or shows correct alt text if a path issue is present).
- The table shows a bold header row and three data rows.
- The "*Back to top*" link (`href="#"`) jumps to the top of the page.

Hints if stuck:

- Hint 1: Copy the **menu structure** idea from your home page but type it fresh in this file.
- Hint 2: `<img>` needs both `src` and `alt` . `<table>` uses `<tr>` , `<th>` (header) and `<td>` (data).
- Hint 3: A same-page link is `<a href="#top">` — and `href="#"` always goes to the top.

Instructor answer key (do not show learners before they try):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Collection</title>
</head>
<body>

  <header>
    <h1>Our Collection</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="about.html">About</a> |
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>

    <section>
      <h2>Featured Outfit</h2>
      <img src="images/product1.jpg" alt="Adaeze Fashion blue Ankara gown">
      <p>Our signature Ankara gown, tailored to your measurements.</p>
    </section>

    <section>
      <h2>Prices</h2>
      <table>
        <tr>
          <th>Item</th>
          <th>Price</th>
        </tr>
        <tr>
          <td>Ankara gown</td>
          <td>₦25,000</td>
        </tr>
        <tr>
          <td>Corporate uniform</td>
          <td>₦15,000</td>
        </tr>
        <tr>
          <td>Bridal wear</td>
          <td>₦80,000</td>
        </tr>
      </table>
    </section>

    <aside>
      <p><a href="#">Back to top</a></p>
      <p>Prices may change with fabric choice.</p>
    </aside>

  </main>

  <footer>
```

```
    <p>&copy; 2026 Adaeze Fashion, Port Harcourt.</p>
  </footer>

</body>
</html>
```

Marking guidance (out of 12): document structure (2); semantic regions used correctly (2); working navigation links (2); image with correct `src` and meaningful `alt` (2); table with `<th>` header and three data rows (2); `aside` with a working `href="#"` link and footer with `©` (1); neatness and page displays (1).

J. REAL-WORLD CASE STUDY — "The Inaccessible Government Page"

Situation. A public agency in Nigeria publishes a services page. The page looks fine on a normal screen, but **blind users complain** they cannot use it. The developers used `<div>` tags for everything and used tables to arrange the whole layout. Screen readers cannot tell where the navigation is, where the main content begins, or what the images show.

Relevant facts.

- The page has a menu, a main article and a footer, but all are `<div>`s with no meaning.
- Images have no `alt` text, so a screen reader says only "image".
- The layout uses a giant layout table, so a screen reader reads the content in a confusing order.
- The agency is legally and ethically expected to serve *all* citizens.

Decision point. How should the team fix the page so everyone can use it?

Possible approaches.

1. Leave it — most users have good eyesight.
2. Rebuild the page with semantic elements (`header` , `nav` , `main` , `article` , `footer`), add meaningful `alt` text to every image, and stop using tables for layout.
3. Add a separate "text-only" version of the site, as was done in the 1990s.

Recommended solution (for this stage). Approach 2. The team:

- replaces the top `<div>` with `<header>` and the menu `<div>` with `<nav>` ;
- marks the main content with `<main>` and the article with `<article>` ;
- wraps the bottom in `<footer>` ;
- adds descriptive `alt` text to every meaningful image (and empty `alt=""` for purely decorative ones);
- removes the layout table and lays the page out with CSS (learned later this course).

Trade-offs.

- Approach 1 excludes a whole group of users and fails public-service expectations.
- Approach 3 doubles the maintenance work and often gives disabled users a poorer, second-class experience — modern best practice is *one accessible site for everyone*.
- Approach 2 costs some rework now but gives one accessible, maintainable, professional site that also ranks better in search engines.

Professional connection. Accessibility is a mark of professional web design and is increasingly a legal requirement for public bodies. Real developers use semantic HTML and `alt` text as a matter of course. This is not "extra" work — it is part of doing the job properly.

Transfer question for the class: "Why is semantic HTML important for a website that serves the public?" (Expected: screen-reader users can navigate; the site is usable by everyone; search engines understand it; it is easier to maintain and legally sound.)

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 A link goes to the wrong place or shows "not found"

Symptoms: Clicking a link shows a 404 or "file not found" page. **Likely cause:** A spelling mistake in the `href`, a wrong file name, or a missing target file. **How to investigate:** Compare the `href` value, letter by letter, with the actual file name in the Explorer. **Corrective steps:** Correct the spelling; make sure the target file really exists. **How to confirm resolved:** The link opens the right page. **Prevention:** Use lowercase file names with no spaces; copy the exact file name into the `href`.

K.2 The image does not display (broken image icon)

Symptoms: A small broken-image icon and/or the alt text shows where the picture should be. **Likely cause:** The `src` path is wrong, the file is in the wrong folder, or the file name/case is wrong. **How to investigate:** Check whether the image is really inside the `images/` folder and whether the path matches exactly (case matters on servers). **Corrective steps:** Fix the path, e.g. `images/product1.jpg`, not `product1.jpg` or `images\product1.jpg`. **How to confirm resolved:** The picture appears. **Prevention:** Always keep images in `images/` and use forward slashes.

K.3 Label does not focus its input

Symptoms: Clicking the text label does nothing. **Likely cause:** The label's `for` does not match the input's `id`. **How to investigate:** Compare `for="..."` with `id="..."` character by character. **Corrective steps:** Make them identical. **How to confirm resolved:** Clicking the label moves the cursor into the field. **Prevention:** Write the input's `id` first, then copy it into the label's `for`.

K.4 Table looks broken / columns misaligned

Symptoms: Cells jump into the wrong row or the table collapses. **Likely cause:** A missing or mis-ordered `<tr>`, `<th>` or `<td>` tag, or a missing closing tag. **How to investigate:** Count the cells in each row; each row should have the same number of `<td>` / `<th>` cells. **Corrective steps:** Add the missing cell or close the tag. **How to confirm resolved:** Rows and columns line up. **Prevention:** Build one row fully, then duplicate it.

K.5 External link does nothing

Symptoms: Clicking an external link appears to do nothing. **Likely cause:** No internet connection, or a misspelled web address. **How to investigate:** Check the Wi-Fi icon; check the `href` starts with `https://` and is spelled correctly. **Corrective steps:** Connect to the internet; fix the address. If offline, this is expected — internal links still work. **How to confirm resolved:** The other site opens. **Prevention:** Test external links only when online; keep internal links relative.

K.6 Everything is built with `<div>`

Symptoms: No semantic elements used; future styling and accessibility suffer. **Likely cause:** Learner finds `<div>` familiar. **How to investigate:** Count the semantic tags; there should be several on a full page. **Corrective steps:** Replace the top region with `<header>`, the menu with `<nav>`, the content with `<main>`, sections with `<section>`, the bottom with `<footer>`. **How to confirm resolved:** The page uses meaningful regions. **Prevention:** Start each page from a semantic skeleton.

Distinguish likely vs confirmed. Confirm before fixing: look at the actual error, the actual path, the actual `for` / `id` pair. Do not guess.

L. SESSION ASSIGNMENT

Assignment title: "Adaeze Fashion — Three Linked Pages"

Purpose. Consolidate the whole of Week 1 (HTML Foundations) by building a small multi-page site with working navigation, an image, a table and a contact form skeleton — the foundation of the capstone.

Realistic scenario. Adaeze now needs three pages that fit together: **Home**, **About** and **Contact**, each with the same menu, so a customer can move around the site easily.

Detailed instructions.

1. In `webdesign-practice`, make sure you have three files: `index.html`, `about.html`, `contact.html`.
2. On **every** page, include the **same** `<header>` with an `<h1>` and a `<nav>` linking to all three pages (so the menu is consistent).
3. `index.html`: a `<section>` welcoming customers, one `<img>` with meaningful `alt`, and a `<section>` with a `<table>` price list (header row + three data rows).

4. `about.html` : an `<article>` telling Adaeze's story (2+ paragraphs) and an `<aside>` with a short "did you know" note.
5. `contact.html` : a `<form>` with a text input (name), an email input, a `<textarea>` (message), a `<select>` with three services, and a `<button type="submit">` . Every `label for` must match its input `id` .
6. Wrap each page's content in `<main>` and finish with a `<footer>` containing a `©` line.
7. Save all files and test by clicking through the menu from each page.

Required deliverables

- Three saved HTML files that link to each other.
- Semantic structure on every page; one image with `alt` ; one data table; one form with matched labels.

Submission format. Files remain in the project folder; demonstrate the working navigation to the instructor before leaving; paste the code into the workbook.

Deadline. Before Session 4.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Three pages linked	4	Menu on every page correctly links to all three; no broken links
Semantic structure	3	`header` , `nav` , `main` , `section` / `article` , `aside` , `footer` used
Image with alt	3	Correct relative `src` ; meaningful `alt`
Data table	3	`<th>` header row + three data rows, aligned
Contact form	4	All fields present; every `label for` matches an input `id`
Consistency and neatness	3	Same menu everywhere; indented; all tags closed

Common mistakes to avoid. Broken link spelling; missing image folder in path; labels not matching ids; different menus on each page; using `<div>` for main regions.

Instructor reference solution. Combine the Section F.10 page (Home) with an `about.html` and `contact.html` that reuse the same `<header>` / `<nav>` , and place the Section F.9 form inside `contact.html` 's `<main>` .

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Multiple choice. Which link opens in a **new tab**? a) `<a href="about.html">About</a>` b) `<a href="https://www.cea.ng" target="_blank">CEA</a>` c) `<a href="#contact">Contact</a>` d) `<a>About</a>`

Q2. Short answer. What is the difference between an **internal** link and an **external** link? Give one example of each.

Q3. Fill in the blanks. To add an image you write `<img _____="images/gown.jpg" _____="Ankara gown">` . The first attribute is the source; the second is the alternative text.

Q4. Explain in your own words. Why is `alt` text important? Give two reasons.

Q5. Multiple choice. Which tag creates a **table data cell**? a) `<tr>` b) `<th>` c) `<td>` d) `<table>`

Q6. Matching. Match each semantic element to its meaning: `header` , `nav` , `main` , `aside` , `footer` . a) navigation links b) top region of the page c) related side content d) the single main content e) bottom region of the page

Q7. Scenario. A learner builds a page entirely with `<div>` tags and asks, "Why should I bother with `<header>` , `<nav>` and `<main>` ?" Give one strong reason.

Q8. Error detection. This form label does not focus its input. Why?

```
<label for="fullname">Name:</label>
<input type="text" id="name" name="name">
```

INSTRUCTOR ANSWER KEY (Q1–Q8)

A1. (b) — it uses `target="_blank"`.

A2. An **internal** link points to a page on the *same* site and uses a relative path (e.g. `<a href="about.html">About</a>`). An **external** link points to *another* website and uses a full address (e.g. `<a href="https://www.cea.ng">CEA</a>`).

A3. `<img src="images/gown.jpg" alt="Ankara gown">` — `src` (source) and `alt` (alternative text).

A4. (1) Screen readers read `alt` aloud to blind users so they know what the image shows. (2) If the image fails to load, the browser shows the `alt` text instead of an empty box. (Also acceptable: helps search engines.)

A5. (c) `<td>`.

A6. `header` → (b); `nav` → (a); `main` → (d); `aside` → (c); `footer` → (e).

A7. Semantic tags describe the *meaning* of each region, which (i) lets screen-reader users jump to the navigation or main content, (ii) helps search engines understand the page, and (iii) makes the code readable and easy to maintain. Acceptance of any one strong reason is fine.

A8. The label's `for="fullname"` does not match the input's `id="name"`. Fix: change the label to `for="name"` (or the input to `id="fullname"`) so they match.

N. REFLECTION AND RECAP

Plain-English summary. Today you turned a text page into a real website. You learned to make **links** with `<a href="...">`, to tell **internal** links (your own pages, relative paths) from **external** links (other sites, full addresses), to open links in a **new tab** with `target="_blank"`, and to jump to a **section** with `id` and `#name`. You added **images** with `<img>`, correct `src` paths and meaningful `alt` text, and learned why `alt` is essential. You built a **data table** with `<table>`, `<tr>`, `<th>` and `<td>` — for data, not layout. You structured the page with **semantic elements** (`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`) and learned when a `<div>` is acceptable. Finally, you met **forms** — `<form>`, `<label>`, `<input>`, `<textarea>`, `<select>` and `<button>` — and the rule that `for` must match `id`, with the honest note that a plain form does not send anything yet.

Five important takeaways.

1. **Links** (`<a href="">`) are the web's doors: internal (relative) vs external (full address).
2. **Images** need a correct `src` **and** a meaningful `alt`.
3. **Tables** (`table/tr/th/td`) are for **data**, never for page layout.
4. **Semantic tags** describe page regions and beat `<div>` for accessibility, SEO and readability.
5. In **forms**, every label's `for` must match its input's `id`; a plain HTML form does not send data on its own yet.

Self-check — can you now…

- Create a working internal and external link?
- Explain when to use `target="_blank"`?
- Add an image with correct path and good `alt` text?
- Build a small data table with a header row?
- Replace `<div>`s with the right semantic elements?
- Build a form whose labels focus their fields?

Connection to the next session. You now have a well-structured, multi-page HTML site for Adaeze Fashion. But it still looks plain — black text on white. Next session, **Session 4 — CSS Basics**, we meet **CSS**, the language of appearance. You will learn selectors, colours, fonts, the three ways to add CSS, and the professional external stylesheet — and you will style the site you built this week.

(This session completes Week 1 — HTML Foundations.)

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence.

- The learner creates correct internal and external links and explains the difference.
- The learner adds an image with a correct relative path and *meaningful* `alt` text.
- The learner chooses semantic elements over `<div>` and can say why.
- The learner builds a table with a header row and aligned cells.
- The learner matches every label `for` to its input `id`.
- The learner tests links by clicking them rather than assuming they work.

Common misconceptions to watch for and correct.

- "My own pages need the full `https://` address." → Use relative paths for internal pages.
- "`alt` is optional." → It is essential for accessibility and broken-image handling.
- "Tables make good page layouts." → Tables are for data; layout is CSS's job.
- "`<div>` is fine for everything." → Prefer semantic elements; use `<div>` only as a last resort.
- "Submitting the form sends an email." → Not yet; a form needs a destination and a handler.
- "`target="_blank"` on every link." → Mainly for external links.

Suggested remedial exercises.

1. Link repair drill: a page with three misspelled `href` s; the learner finds and fixes each.
2. Image path drill: give five `src` values, some correct and some wrong; the learner identifies the working ones and explains why.
3. Semantic sorting: a plain-text layout description; the learner assigns `header` , `nav` , `main` , `section` , `aside` , `footer` to each part.
4. Label/id matching: five labels and five inputs jumbled; the learner pairs them.

Differentiation.

- **Level 1 (supported):** Provide a semantic skeleton file with the `<header>` , `<nav>` , `<main>` and `<footer>` already in place; learners fill in the image, table and form fields. Pair with a confident learner.
- **Level 2 (standard):** Complete the exercises and assignment as written.
- **Level 3 (challenge):** Add a **same-page table of contents** using `id` / `#` links; add a second `<img>` inside a link (so the image itself is clickable); add a checkbox and a radio group to the form, and a `<fieldset>` with a `<legend>` to group related inputs.

Timing guidance (105 minutes). Opening 6 · Prerequisite 7 · Lecture 40 · Demonstration 15 · Guided practical 18 · Independent exercise 15 · Case study 4 · Knowledge check 4 · Reflection/close 3. If slow, shorten the case study to 2 minutes and protect the guided and independent practicals — building the linked pages is the priority. If fast, spend spare time on Level 3 tasks and peer review of each other's site navigation.

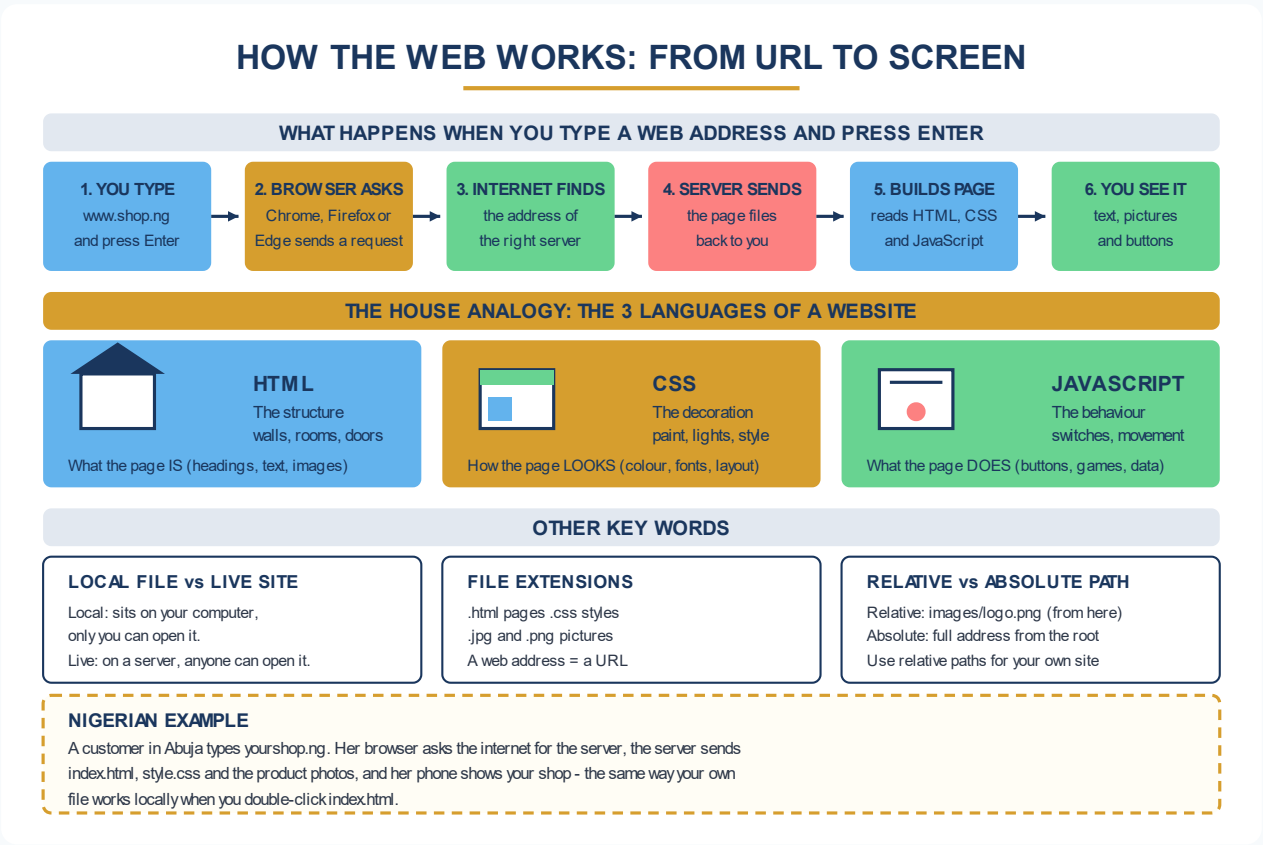
END OF SESSION 3

Next: Session 4 — CSS Basics: selectors, colours, fonts, the three ways to add CSS, and the external stylesheet.

SESSION 04 OF 12

CSS BASICS

CONCEPT DIAGRAM — SESSION 04



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 4 — CSS BASICS

A. SESSION IDENTIFICATION

Item	Detail
Session number	4 of 12
Session title	CSS Basics
Week / Module	Week 2 — Module 2: CSS Foundations
Recommended duration	105 minutes
Difficulty level	Beginner

Item	Detail
Prerequisite knowledge	Sessions 1–3. The learner can build a multi-page HTML site with semantic elements, links, images, a table and a form skeleton.
Connection to previous session	In Week 1 we built the **structure** of Adaeze Fashion — but it looks plain. This session adds the **appearance**: CSS. We take the existing HTML and style it without changing its content.
Preparation before class	Academy PCs on; VS Code and Chrome open; `webdesign-practice` open with `index.html`, `about.html`, `contact.html`; the `css` folder created; projector connected; a demo "before" and "after" page prepared (Section G).
Running project	Adaeze Fashion gets its first styling: colours, fonts and spacing, done in an **external stylesheet** (`css/style.css`), the professional standard.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain what CSS is** and why it exists, and state the split of duties: **HTML = structure, CSS = presentation**.
2. **Name the three ways** to add CSS — inline, internal and external — and explain why **external** is preferred.
3. **Write correct CSS syntax** — a selector, a property, a value, and a declaration ending in a semicolon.
4. **Use the main selectors** — element, class, id and grouped — and choose the right one for a task.
5. **Specify colours** using named colours, hex codes and rgb(), and comment on **text/background contrast**.
6. **Style text** using `color`, `font-family`, `font-size`, `font-weight`, `text-align` and `line-height`.
7. **Set a background colour and a border**, and explain the difference between **padding** and **margin** using the **box model**.
8. **Explain, at a beginner level, the "cascading" idea** — that later/more specific rules win.
9. **Create an external stylesheet**, link it to HTML with `<link>`, and confirm the styles appear in Chrome.

C. REQUIRED MATERIALS

Hardware

- One computer per learner; instructor computer on a projector.

Software

- Visual Studio Code; Google Chrome.

Sample / starter files

- `webdesign-practice/index.html` from Session 3 (the semantic Adaeze home page).
- A prepared "before/after" demo pair: `demo/before.html` (plain) and `demo/after.html` + `demo/after.css` (styled) — contents in Section G.
- The `css` folder inside `webdesign-practice`.

Worksheets / printed materials

- "CSS Quick Reference" card (one per learner: selector types, colour formats, common text properties, the box model diagram).
- Worksheet 4.1 — "Name the Part" (the three ways to add CSS and CSS syntax parts).
- Worksheet 4.2 — "Box Model Diagram" (label content, padding, border, margin).
- A simple printed **box model diagram** (concentric rectangles) if available.

Internet

- Not required. Optional: the instructor may show a free colour-picker site, but do not depend on it. (Colours can be chosen from the "named colours" list on the reference card.)

Alternative for learners without a colour picker

- Use named colours (`navy` , `firebrick`) or the printed hex list. No software needed.

D. OPENING ACTIVITY (~6 minutes)

"The Before and After"

Instructor setup. Display two versions of the same Adaeze Fashion page side by side on the projector (or one after the other): `demo/before.html` (HTML only — plain black text on white) and `demo/after.html` (the same content, styled with colour, fonts and spacing). Same words, same images, same structure — different *look*.

What the instructor says:

> *"Look at these two pages. Same text. Same picture. Same words about Adaeze's gowns. But one looks like a letter typed in the 1990s, and the other looks like a real business website. What changed?*
>
> *Nothing in the content. What changed is the **appearance**. And in web design, appearance has its own language: **CSS** — Cascading Style Sheets.*
>
> *Last week, HTML said: 'This is a heading, this is a price, this is a list.' It never said the heading should be dark red, or the list needs room to breathe. That was never HTML's job. Today, you learn to dress the page. By the end of the session, you will make Adaeze's site look like the page on the right."*

Expected learner response. Learners immediately prefer the styled page and become curious about *which* decisions made the difference. Many will guess "colour" or "font". Confirm: colour, font, spacing and background — all CSS. Some may ask "can I change the heading to blue?" — "Yes, and that is exactly today's work."

Why this opener works. It creates a strong, felt contrast and makes the *value* of CSS obvious before any syntax. It also reinforces the HTML/CSS split taught in Session 1, so the new language feels like a natural next layer rather than a new subject.

E. PREREQUISITE REVIEW (~7 minutes)

Oral round-robin; questions on the projector.

Question 1: "What are the three building blocks of a website, and what is each one's job?"

- Expected answer: HTML (structure), CSS (presentation/appearance), JavaScript (behaviour).
- Gap: If a learner says "CSS is the structure", correct immediately with the house analogy — the walls are HTML, the paint is CSS.

Question 2: "What are semantic HTML elements? Name two."

- Expected answer: tags that describe meaning — e.g. `<header>` , `<nav>` , `<main>` , `<footer>` .
- Gap: Learners list `<div>` . Remind that `<div>` has no meaning.

Question 3: "How do you link to another page on your own site?"

- Expected answer: `<a href="about.html">About</a>` (a relative path).
- Gap: Using full `https://` addresses for local pages.

Question 4: "What must a form label's `for` attribute match?"

- Expected answer: the input's `id` attribute.
- Gap: Confusing `id` with the `name` attribute. Re-state.

Question 5: "After saving, what do you do to see changes in the browser?"

- Expected answer: refresh (`F5`).
- Gap: This will be *crucial* today. Stress: **save, then refresh** — you will change CSS many times.

Bridge to today (say this):

> *"Your HTML is the skeleton — and it is solid. Today we add the skin, the clothes and the colour. But here is an important professional rule from the start:* ***we will not put the styling inside every HTML file.*** *We will keep it in one separate stylesheet, so one change updates the whole website. That is how professionals work, and that is what you will do today."*

Gap handling. If several learners cannot recall that CSS is for appearance, spend two extra minutes with the house analogy. If many struggle to save-and-refresh, pair them with confident learners, because CSS requires many quick save-refresh cycles.

F. FULL LECTURE NOTES (~40 minutes)

F.1 What CSS is and why it exists

The idea. CSS stands for **Cascading Style Sheets**. It is the language that controls how a web page **looks**: colours, fonts, spacing, borders, backgrounds and layout.

Plain-English definition. If HTML describes *what* each piece of content is, CSS describes *how* it should appear. HTML is the skeleton; CSS is the skin, hair and clothing.

Why it exists. Imagine a shop with 50 pages and a red heading on each. If the colour were written into every page's HTML, changing it later means editing 50 files. With CSS, you change it **once**, and all 50 pages update. CSS separates *content* from *presentation* — the second half of the "separation of concerns" idea from Session 1.

Familiar analogy. Think of a **person and their clothes**. The person's body (HTML) does not change, but different clothes (CSS) change how they look on different occasions. The same body can wear a work uniform or wedding attire.

Why "Cascading"? "Cascading" means rules can come from different places and *fall down* in priority. If two rules target the same element, the browser decides which one wins using the **cascade** (later rules and more specific rules win). We meet this gently in F.13.

Worked example.

```
h1 {
  color: navy;
}
```

This says: "for every `<h1>`, make the text colour navy." The HTML is unchanged; only appearance differs.

Misconception. "CSS is a programming language." It is a **styling language** — it describes appearance. It has no loops, no decisions, no calculations. Do not expect it to "do" anything; it only styles.

F.2 The three ways to add CSS

There are three ways to connect CSS to HTML. Learn all three, but **use the third**.

1. Inline CSS — written directly on an element with the `style` attribute.

```
<p style="color: red;">This text is red.</p>
```

- Pros: quick for a one-off test.
- Cons: it repeats the style on every element, mixes appearance into the HTML, and is hard to maintain. **Avoid it.**

2. Internal CSS — written in a `<style>` block inside the `<head>`.

```
<head>
  <style>
    p { color: red; }
  </style>
</head>
```

- Pros: keeps a single page's styles in one place.
- Cons: it only affects **this one page**. A 4-page site needs the styles copied into all 4 files. **Use only for quick experiments.**

3. External CSS — written in a separate `.css` file, linked from the HTML.

```
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

- Pros: **one file styles the whole website**; changing one file updates every page; the browser can cache it so pages load faster; the HTML stays clean.
- Cons: one extra file to manage — a trivial cost.
- This is the **professional standard**, and what we use for Adaeze Fashion.

Reading the link tag.

- `<link>` — a void element that connects a resource.
- `rel="stylesheet"` — "the relationship is: this is a stylesheet".
- `href="css/style.css"` — the relative path to our CSS file (inside the `css` folder).

Worked example — the Adaeze project layout.

```
webdesign-practice/
├── index.html
├── about.html
├── contact.html
├── css/
│   └── style.css      ← ALL styling lives here
└── images/
```

Every HTML page has the same `<link>` line in its `<head>`, so the whole site shares one style.

Misconception. "Inline CSS is fine — it is faster." It is faster to *type once*, but every future change becomes slow and error-prone. External CSS saves time over the life of a site, which is what matters professionally.

F.3 CSS syntax — selector, property, value, declaration

The idea. Every CSS rule has the same shape. Learn the parts by name.

```
h1 {
  color: navy;
  font-size: 32px;
}
```

The parts:

- `h1` — the **selector**: *which* elements to style.
- `{ }` — the **curly braces**: they hold the rule's declarations. (On a Nigerian keyboard, `{` is **Shift + [** and `}` is **Shift +]**.)
- `color` — a **property**: *what* to change.
- `navy` — a **value**: *how* to change it.
- `color: navy;` — the whole line is a **declaration** (property + value), and it **ends with a semicolon** `;`.

Why the semicolon matters. Each declaration acts like a sentence; the semicolon is its full stop. If you forget it, the browser may merge the declaration with the next one and ignore both. This is the single most common CSS beginner mistake. (See K.1.)

Whitespace and indentation. CSS ignores extra spaces and new lines. We indent the declarations inside the braces purely for readability, exactly as we indent HTML.

Worked example — one rule, three declarations:

```
p {
  color: #333333;      /* dark grey text */
  font-size: 16px;
  line-height: 1.6;
}
```


Expected result: every paragraph shows dark-grey text at 16 pixels with comfortable line spacing.

Misconception. "The `;` goes after the closing brace." No — the semicolon ends each *declaration inside* the braces, not the rule. The braces are closed with `}`.

F.4 Selectors — element, class, id, grouping

The idea. A **selector** tells CSS which element(s) to style. There are four you must know now.

1. Element selector — selects *all* elements of a type.

```
p {
  color: #333333;
}
```

Applies to every `<p>` on the page.

2. Class selector — selects elements that have a matching `class` attribute. It starts with a **dot** `.`.

```
.highlight {
  background-color: yellow;
}
```

```
<p class="highlight">This paragraph has a yellow background.</p>
<p>This one is normal.</p>
```

A class can be used on **many** elements, and an element can have more than one class (space-separated): `class="highlight big"`.

3. ID selector — selects the **one** element with a matching `id` attribute. It starts with a **hash** `#`.

```
#site-header {
  background-color: #1a3a5c;
}
```

```
<header id="site-header">...</header>
```

An `id` must be **unique** on the page — use it for a single specific element.

4. Grouped selector — applies the same rule to several selectors at once, separated by **commas**.

```
h1, h2, h3 {
  font-family: Georgia, serif;
}
```

This styles `h1`, `h2` and `h3` with one rule — saves repetition.

How to choose.

- Styling **all** headings → element selector.
- Styling several elements of the same *kind* (e.g. all "price" labels) → **class**.
- Styling one unique thing (e.g. the main header) → **id**.
- Same styles for several selectors → **group**.

Memory aid. *Dot for class, hash for id* (from Session 1's Web Dev course too — same rule everywhere).

Misconception. "I can use the same id many times." No — ids are unique. If you want to style several elements the same way, use a **class**, not repeated ids.

F.5 Colours — named, hex, rgb, and contrast

The idea. CSS can express a colour in several ways. All are valid; pick the one that suits the job.

1. Named colours — simple words.

```
color: navy;
color: firebrick;
color: seagreen;
```

There are around 140 named colours. Simple and readable, but limited.

2. Hex codes — a `#` followed by six characters (0–9 and A–F), representing Red, Green and Blue. Format: `#RRGGBB`.

```
color: #1a3a5c;   /* deep blue */
color: #ffffff;   /* white */
color: #000000;   /* black */
color: #e67e22;   /* orange */
```

- The first two characters are **red**, the next two **green**, the last two **blue**.
- `00` means none of that colour; `ff` means full.
- The most precise and widely used method.

3. rgb() — the same idea in decimal numbers from **0 to 255**.

```
color: rgb(230, 126, 34);   /* orange */
background-color: rgb(245, 245, 245);   /* light grey */
```

`rgb(255, 0, 0)` is pure red; `rgb(0, 0, 0)` is black.

Shortcut hex. When each pair is doubled (`#ff0000` , `#ffffff`), it can be shortened: `#f00` , `#fff` . Both mean the same. (Keep the long form until you are comfortable.)

Background colours.

```
body {
  background-color: #f5f5f5;
}
```

Sets the background of the whole page to a soft grey.

Contrast — critically important for a business site.

- **Contrast** means how clearly text stands out from its background.
- Dark text on a light background, or light text on a dark background, is readable. **Grey text on a grey background is not.**
- Many websites must be readable by everyone, including people with poor eyesight. Strong contrast is both good design and basic accessibility.
- A simple rule for beginners: **avoid light text on light backgrounds and dark text on dark backgrounds.** Test by squinting — if the words blur together, the contrast is too weak.

Worked example — a readable combination.

```
body {
  background-color: #ffffff;   /* white background */
  color: #222222;              /* near-black text */
}
```

Expected result: strong, easy-to-read contrast.

Misconception. "Hex codes are random." They are not — they are just Red/Green/Blue in base-16. `#ff0000` = full red, no green, no blue.

F.6 Text properties — fonts, size, weight, alignment

The idea. CSS controls every aspect of how text looks.

`font-family` — which typeface to use.

```
body {
  font-family: Arial, sans-serif;
```

```
}
```

- The browser tries the first font; if unavailable, it falls back to the next.
- Always end with a **generic family**: `serif`, `sans-serif` or `monospace`.

`font-size` — how large the text is.

```
p { font-size: 16px; }
h1 { font-size: 32px; }
```

Common units: **px** (pixels — fixed), **em** (relative to the parent), **rem** (relative to the root). For now, use **px** for clarity; we meet relative units later.

`font-weight` — how thick the text is.

```
strong { font-weight: bold; }
.normal { font-weight: normal; }
```

Values: `normal`, `bold`, or numbers like `400` (normal) and `700` (bold).

`text-align` — where the text sits horizontally.

```
h1 { text-align: center; }
p { text-align: left; }
```

Values: `left`, `center`, `right`, `justify`.

`line-height` — the space between lines (leading).

```
p { line-height: 1.6; }
```

A value like `1.6` means "1.6 times the font size". Comfortable reading is usually between `1.4` and `1.8`. A value of `1` feels cramped.

`text-decoration` — underlines and lines.

```
a { text-decoration: none; }        /* removes the underline on links */
.old-price { text-decoration: line-through; }
```

Often used to remove the default underline from navigation links.

Worked example.

```
h1 {
  font-family: Georgia, serif;
  font-size: 36px;
  font-weight: bold;
  text-align: center;
  color: #1a3a5c;
}
p {
  font-size: 16px;
  line-height: 1.6;
}
```

Expected result: a centred, bold, deep-blue serif heading; body paragraphs in comfortable spacing.

Misconception. "`em` and `px` are the same." They are not — `px` is fixed, `em` / `rem` are relative. Use `px` now; learn relative units later for responsive design.

F.7 Background and borders

The idea. Any element can have a **background colour** and a **border** (a line around it).

Background colour.

```
header {
  background-color: #1a3a5c;
}
```

Fills the header's area with colour → you would then usually make its text light for contrast.

Border. A border has three parts: **width**, **style**, **colour**.

```
table {
  border: 1px solid #cccccc;
}
th, td {
  border: 1px solid #cccccc;
  padding: 8px;
}
```

- `1px` — the border thickness.
- `solid` — the border style (others: `dashed`, `dotted`, `none`).
- `#cccccc` — the border colour.

Important note on HTML tables. By default, browser table borders leave small gaps between cells. CSS can collapse them:

```
table {
  border-collapse: collapse;
}
```

This makes the borders join into one clean grid — very common on price tables.

Worked example.

```
th, td {
  border: 1px solid #cccccc;
  padding: 8px;
  text-align: left;
}
```

Expected result: each table cell gets a thin grey line and a little breathing room inside.

Misconception. "A border and a background are the same." No — a background *fills*; a border is a *line around* the edge. You often use both together.

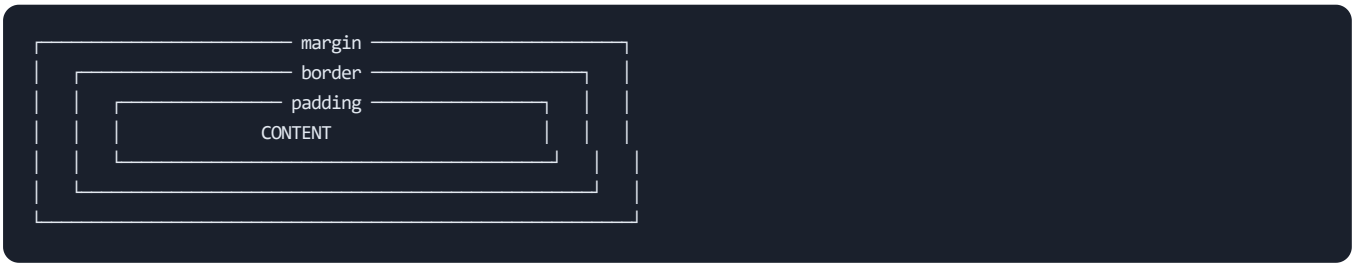
F.8 Padding and margin — the box model explained clearly

This is the most important layout idea for beginners. Take it slowly.

The idea. In CSS, every element is treated as a **box**. The box has, from the inside out:

1. **Content** — the text or image itself.
2. **Padding** — space *inside* the box, between the content and the border.
3. **Border** — the line around the box.
4. **Margin** — space *outside* the box, pushing other elements away.

The box model, pictured as concentric rectangles:



Padding versus margin — the plain-English rule.

- **Padding** = space *inside* the box (between the content and its own border). It pushes the border *out* from the content.

- **Margin** = space *outside* the box (between this box and neighbouring boxes).

Familiar analogy. Think of a **framed picture on a wall**.

- The **content** is the picture itself.
- **Padding** is the mat board *inside the frame*, between the picture and the frame.
- The **border** is the frame.
- **Margin** is the empty wall space *around the frame*, separating it from the next picture.

Worked example.

```
p {
  padding: 16px;      /* space inside the paragraph box */
  margin: 24px;       /* space outside it */
}
```

- With `padding: 16px`, the text sits 16 pixels away from the paragraph's edge (visible if the paragraph has a background or border).
- With `margin: 24px`, the paragraph is pushed 24 pixels away from the paragraphs around it.

Shorthand. One value sets all four sides. You can also give two values (`10px 20px` = 10px top/bottom, 20px left/right) or four values (top, right, bottom, left, clockwise). Start with one value.

Why it matters. Spacing is most of what makes a page look "professional" versus "cramped". Generous padding and margin make text easy to read, which is exactly the difference between the "before" and "after" pages in the opener.

Misconception. "Padding and margin are the same thing." They are the two halves of the box model: **padding is inside; margin is outside.** If you remember one thing today, remember this.

F.9 The cascading idea and specificity at a beginner level

The idea. "Cascading" and "specificity" explain what happens when **two rules disagree** about the same element.

Cascade (order). If two rules have the same specificity, the **later** rule wins — the one written further down the stylesheet.

```
p { color: blue; }
p { color: red; }   /* this one wins — it comes later */
```

Expected result: paragraphs are red.

Specificity (how specific a selector is). A **more specific** selector beats a less specific one, regardless of order. The rough ranking from weakest to strongest:

1. **Element** selector (`p`) — weakest.
2. **Class** selector (`.highlight`) — stronger.
3. **ID** selector (`#main`) — strongest.
4. **Inline** style (`style="..."`) — very strong (another reason to avoid it).

```
p { color: blue; }              /* element — weak */
.highlight { color: green; }    /* class — stronger */
#special { color: purple; }     /* id — strongest */
```

```
<p class="highlight" id="special">Which colour?</p>
```

Expected result: the text is **purple**, because the id rule is the most specific.

Why this matters. When a style "will not change", the usual cause is a more specific rule winning somewhere else. Knowing the ranking helps you debug quickly. (See K.5.)

Keep it simple. For now, **prefer classes**, keep specificity low, and avoid inline styles. That prevents most "why won't it change?" headaches.

Misconception. "The rule I want always wins." Not if a more specific rule disagrees. Specificity is the tie-breaker.

F.10 Putting it together — the Adaeze Fashion stylesheet

Here is the full external stylesheet we will build. It styles the Week 1 HTML **without changing a single word of content**.

The HTML `<head>` (on every page):

```
<head>
  <meta charset="UTF-8">
  <title>Adaeze Fashion — Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

`css/style.css`:

```
/* ===== Base styles ===== */
body {
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;   /* soft grey page background */
  color: #222222;              /* near-black, high contrast */
  margin: 0;                   /* remove the default page gap */
  padding: 20px;               /* breathing room inside the page edge */
}

/* ===== Header ===== */
header {
  background-color: #1a3a5c;   /* deep blue */
  color: #ffffff;              /* white text on dark blue (good contrast) */
  padding: 20px;
  text-align: center;
}

/* ===== Headings ===== */
h1 {
  font-family: Georgia, serif;
  font-size: 36px;
  margin-bottom: 8px;
}

h2 {
  color: #2c5f8a;              /* medium blue */
  font-size: 24px;
  margin-top: 32px;
}

/* ===== Paragraphs ===== */
p {
  font-size: 16px;
  line-height: 1.6;
}

/* ===== Links ===== */
a {
  color: #e67e22;              /* orange */
  text-decoration: none;
}
a:hover {
  text-decoration: underline;  /* underline appears when the mouse is over it */
}

/* ===== Table ===== */
table {
  border-collapse: collapse;
  background-color: #ffffff;
}
th, td {
  border: 1px solid #cccccc;
  padding: 8px 12px;
  text-align: left;
}
th {
  background-color: #2c5f8a;
  color: #ffffff;
}
```

```
/* ===== Footer ===== */
footer {
  margin-top: 32px;
  padding: 16px;
  text-align: center;
  color: #666666;              /* grey is fine here — footer text is secondary */
}
```

Line-by-line highlights:

- `body { font-family: Arial, sans-serif; }` — sets the default font for the whole page.
- `background-color: #f5f5f5;` and `color: #222222;` — light background, dark text → strong contrast.
- `header { background-color: #1a3a5c; color: #ffffff; }` — dark blue header with white text.
- `a:hover { text-decoration: underline; }` — the `:hover` **pseudo-class** applies styles only while the mouse is over the element. (A **pseudo-class** is a state; `:hover` means "while hovered".)
- `border-collapse: collapse;` — joins table cell borders into one clean grid.
- `th { background-color: #2c5f8a; color: #ffffff; }` — a coloured header row with white text (good contrast).

Expected browser result. A soft-grey page with a dark-blue header band, white centred title, orange links that underline on hover, a clean bordered price table with a blue header row, and a muted footer. Same words as before — completely different impression.

Note on `:hover`. The colon before `hover` makes it a *state*, not a class. Do not write `.hover`.

F.11 Summary of the lecture

- **CSS = presentation**; HTML = structure. Keep them in separate files.
- Three ways to add CSS — **inline, internal, external**; use **external**.
- CSS syntax: **selector { property: value; }** — end every declaration with `;`.
- Selectors: **element**, **class** (`.`), **id** (`#`), **grouped** (commas).
- Colours: **named**, **hex** (`#RRGGBB`), **rgb()**. Watch **contrast**.
- Text: `font-family`, `font-size`, `font-weight`, `text-align`, `line-height`, `text-decoration`.
- **Background** and **borders** style boxes; `border-collapse` fixes table gaps.
- **Box model: padding is inside; margin is outside**.
- **Cascade/specificity:** later rules win; more specific (id > class > element) rules win.

G. INSTRUCTOR DEMONSTRATION (~15 minutes)

G.1 What the instructor prepares

- `webdesign-practice` open in VS Code with `index.html` from Session 3.
- The `css` folder ready.
- `demo/before.html` (plain) and `demo/after.html` + `demo/after.css` prepared for the opener, if used again here.
- Chrome open.

G.2 Demonstration 1 — Creating and linking the stylesheet (4 minutes)

Say: "Watch me connect one CSS file to a whole website."

1. In the Explorer, right-click the `css` folder → **New File** → type `style.css`.
2. Open `index.html`. In the `<head>`, add:

   ```
   <link rel="stylesheet" href="css/style.css">
   ```

   Say: "One line connects the page to the stylesheet. `rel` says it is a stylesheet; `href` is the path."
3. Add a visible test rule to `style.css`:

```
body { background-color: #f5f5f5; }
```

4. Save both files. Refresh Chrome. The page background turns soft grey.
5. **Inject an error:** change `href="css/style.css"` to `href="style.css"` (missing folder). Save, refresh — the background disappears. Say: "Wrong path, no styles. The file must be found exactly where `href` says." Fix it.

Questions to ask:

- "Where does the `<link>` go — the head or the body?" (Head.)
- "If I have 4 pages, how many CSS files do I need?" (One — linked from all four.)

G.3 Demonstration 2 — Colours, text and the header (6 minutes)

Say: "Now let us dress the page: colours, fonts and spacing."

1. Add the `body` font and colour rules. Refresh.
2. Add the `header` rule (deep-blue background, white text, padding, centred). Refresh — the header becomes a coloured band.
3. Add `h1` (Georgia serif, 36px) and `h2` (medium blue, 24px). Refresh.
4. Add `a` and `a:hover` (orange; underline on hover). Hover over a link to show the change.
5. **Inject an error:** delete the semicolon after `color: #ffffff` in the header rule. Save, refresh. Expected: the white-text declaration (and possibly the next one) is ignored — the header text may vanish against the blue or look wrong. Say: "This is the number-one CSS mistake: a missing semicolon. It can kill the declaration after it too." Fix it.

Questions to ask:

- "Is this text easy to read on that background?" (Check contrast aloud.)
- "Which selector am I using for the header?" (Element selector `header`.)

G.4 Demonstration 3 — Table, box model and the cascade (5 minutes)

Say: "Let us make the price table clean, see the box model, and prove how the cascade works."

1. Add the `table`, `th, td` and `th` rules (border, padding, collapsed borders, blue header). Refresh — the table now has a clean grid.
2. **Box model demo:** temporarily add `padding: 30px;` and then `margin: 30px;` to the table and point at the difference — padding pushes the *cells* in from the edge; margin pushes the *table* away from surrounding content. Remove the temporary rules.
3. **Cascade demo:** add to `style.css`:

   ```
   h2 { color: green; }
   #skills-title { color: purple; }
   ```

   and add `class="section-title" id="skills-title"` to an `<h2>`. Then add a class rule `.section-title { color: red; }`. Save, refresh. Ask the class to predict the colour. Expected: **purple** — the id rule wins over class and element. Say: "More specific wins. That is specificity."

Questions to ask:

- "Padding or margin — which is *inside* the box?" (Padding.)
- "If two rules are equally specific, which wins?" (The later one.)

H. GUIDED PRACTICAL EXERCISE (~18 minutes)

Objective. Each learner creates `css/style.css`, links it from `index.html`, and styles the Adaeze home page with colours, fonts, backgrounds, borders, padding/margin, a hover effect and at least one class — then confirms it in Chrome.

Materials. VS Code and Chrome open; `webdesign-practice/index.html` from Session 3; the `css` folder.

Procedure (instructor calls each step; learners follow; instructor walks the room):

1. In the `css` folder, create `style.css`.
2. In the `<head>` of `index.html`, add `<link rel="stylesheet" href="css/style.css">`.
3. Add the `body` rule: Arial/sans-serif font, a light background (`#f5f5f5`), dark text (`#222222`), `margin: 0`.
4. Add a `header` rule: deep-blue background (`#1a3a5c`), white text (`#ffffff`), `padding: 20px`, `text-align: center`.
5. Add an `h1` rule (larger size, e.g. 36px) and an `h2` rule (medium blue, 24px).
6. Add a `p` rule (`font-size: 16px`, `line-height: 1.6`).
7. Add an `a` rule (orange `#e67e22`, `text-decoration: none`) and an `a:hover` rule (`text-decoration: underline`).
8. Add a `table` rule (`border-collapse: collapse`) and a `th, td` rule (thin grey border, `padding: 8px 12px`).
9. Create a **class** `.highlight` (e.g. `background-color: #fff3cd;` and `padding: 8px;`) and apply it to one paragraph (`class="highlight"`).
10. Save both files and refresh Chrome. Walk through the page top to bottom to confirm each style.

Pause and predict (ask the class):

- After step 3, before refreshing: "Will the background of the *whole page* change, or just one paragraph?" (The whole page — `body` selects everything inside it.)
- After step 7: "Will the link be underlined *all* the time, or only when hovered?" (Only when hovered — `:hover` is a state.)
- After step 9, before refreshing: "Which paragraphs turn yellow — all of them, or only the one with `class="highlight"`?" (Only the one with the class.)
- After step 4, before checking: "Is the white text readable on the blue background?" (Yes — good contrast.)

Expected result. A soft-grey page; a dark-blue header band with white text; dark-blue headings; orange links that underline on hover; a clean bordered price table; one highlighted paragraph with a pale yellow background and padding. Same content, professional look.

Troubleshooting while walking the room:

- No styles appear → check the `<link>` path and that `style.css` is saved. (See K.2.)
- One rule ignored → check for a missing semicolon in the rule just before it. (See K.1.)
- Text hard to read → adjust colour/background for contrast. (See K.4.)
- A style "will not change" → a more specific rule is winning. (See K.5.)
- Changes not visible → save, then refresh (and try `Ctrl + F5`).

Completion checklist (learner ticks each):

- ☐ `css/style.css` created
- ☐ `<link>` in the `<head>` with the correct path
- ☐ `body` styled (font, background, colour, margin)
- ☐ `header` styled (background, colour, padding, alignment)
- ☐ `h1` and `h2` styled
- ☐ `p` styled (size, line-height)
- ☐ `a` and `a:hover` styled
- ☐ `table`, `th`, `td` styled (border, padding, collapse)
- ☐ At least one `.highlight` class used on an element
- ☐ Page displays the new look in Chrome after refresh

I. INDEPENDENT CLASS EXERCISE (~15 minutes)

"Style the About Page"

Scenario. Adaeze's **About** page (`about.html`) is still unstyled. Style it on your own using the *same* external stylesheet, so both pages look consistent. You may add rules to `style.css`, but do **not** edit the HTML's content — only add the `<link>` if it is missing and add `class` / `id` attributes where needed.

Task.

1. Ensure `about.html` has the `<link rel="stylesheet" href="css/style.css">` line in its `<head>` (same as the home page).

2. In `style.css`, or in a clearly marked new section of it, add the following rules:
    - a rule for `h1` that sets a **Georgia/serif** font and a colour of your choice (check contrast!);
    - a rule for `p` that sets `font-size: 16px` and `line-height: 1.7`;
    - a **class** selector `.intro` that gives a paragraph a light background (choose a colour with strong contrast for the text);
    - an **id** selector `#journey` that styles the *Our Journey* heading (e.g. a different colour or a bottom border);
    - an `aside` rule that gives the sidebar a light border and some padding;
    - a `footer` rule that centres the text and makes it a muted grey.
3. Apply `class="intro"` to the introduction paragraph and `id="journey"` to the *Our Journey* heading in `about.html`.
4. Save both files, open `about.html` in Chrome, and refresh.

Requirements / constraints:

- Use **one external stylesheet** shared with the home page — no `<style>` blocks and no inline `style="..."`.
- Every declaration ends with a **semicolon**.
- Check **contrast** on every colour choice (text must be clearly readable on its background).
- Do not change the words of the content.

Time allowance: 15 minutes.

Success criteria:

- The About page shows the new styling in Chrome.
- The page looks **consistent** with the home page (same font family, similar colours).
- The `.intro` class and `#journey` id both visibly apply.
- No inline or internal CSS is used.

Hints if stuck:

- Hint 1: The `<link>` line must be identical to the one on the home page, inside `<head>`.
- Hint 2: A class selector starts with a dot (`.intro`); an id selector starts with a hash (`#journey`).
- Hint 3: If a rule seems ignored, check the semicolon on the line above, and check whether a more specific rule is overriding it.

Instructor answer key (do not show learners before they try):

Add this section to `css/style.css`:

```
/* ===== About page ===== */
h1 {
  font-family: Georgia, serif;
  color: #1a3a5c;
}
p {
  font-size: 16px;
  line-height: 1.7;
}
.intro {
  background-color: #eef4fb;   /* pale blue */
  padding: 12px;
  color: #222222;              /* dark text — good contrast on pale blue */
}
#journey {
  color: #2c5f8a;
  border-bottom: 2px solid #e67e22;
  padding-bottom: 4px;
}
aside {
  border: 1px solid #cccccc;
  padding: 12px;
  background-color: #ffffff;
}
footer {
  text-align: center;
  color: #666666;
  margin-top: 24px;
}
```

And in `about.html`, the introduction paragraph becomes `<p class="intro">...</p>` and the journey heading becomes `<h2 id="journey">Our Journey</h2>`.

Expected result: the About page matches the site's look: Georgia serif heading in deep blue, comfortable paragraphs, a pale-blue intro box, an underlined *Our Journey* heading, a bordered sidebar and a centred grey footer.

Marking guidance (out of 12): external stylesheet linked (2); `h1` font and colour (2); `p` size and line-height (2); `.intro` class applied and visible (2); `#journey` id applied and visible (2); `aside` border/padding and `footer` styling (1); contrast checked and no inline/internal CSS (1).

J. REAL-WORLD CASE STUDY — "The Website Nobody Trusts"

Situation. Emeka runs a small logistics business in Port Harcourt. He built his own website using the HTML he learned — correct content, a working menu, a clear price list. But customers keep calling to *ask if the business is real*. The site has plain black Times New Roman text on a white background, cramped lines, and no colours. It looks like an old school assignment, and customers do not trust it enough to place orders.

Relevant facts.

- The content and structure are fine — the problem is purely **presentation**.
- Competitors have clean, colourful, readable sites.
- Emeka does not want to pay a designer yet, and he already knows HTML.
- He has learned that "appearance" is a separate skill.

Decision point. How should Emeka make his site look trustworthy without rebuilding it?

Possible approaches.

1. Add lots of flashy colours and many fonts to look "modern".
2. Learn and apply **basic CSS** — one colour scheme, two fonts, generous spacing and strong contrast — using an external stylesheet.
3. Pay a professional for a full redesign.
4. Give up on the website and rely only on WhatsApp again.

Recommended solution (for this stage). Approach 2. Emeka should:

- create one `style.css` and link it from every page;
- choose a small, consistent colour scheme (e.g. a dark blue for the header, orange for calls to action, dark text on light backgrounds);
- set a clean sans-serif body font and a comfortable `line-height`;
- add generous padding and margins so nothing is cramped;
- style the price table cleanly and give links a clear hover state. This is exactly the skill learned today — a small amount of CSS transforms trust.

Trade-offs.

- Approach 1 (many colours and fonts) usually makes a site look messy and *less* trustworthy, not more. Restraint is professional.
- Approach 3 gives a polished result but costs money and creates dependence.
- Approach 4 abandons a working asset.
- Approach 2 is free, fast, keeps Emeka in control, and — because it uses an external stylesheet — one change updates the whole site.

Professional connection. In real web work, "the client does not trust the site" is almost always a design and presentation problem, not a content problem. Knowing how to fix it with a clean external stylesheet is a core freelance skill, and it is the same skill that makes a small Nigerian business look credible online.

Transfer question for the class: "Your friend has a website with good content but no styling. Name **three** CSS things you would fix first to make it look professional." (Expected: establish a consistent colour scheme with strong contrast; set a clean readable font with comfortable line-height; add generous padding and margins so the layout is not cramped. Also acceptable: style the links and the table.)

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 A rule does not work (or the one after it fails too)

Symptoms: A style is ignored; sometimes the *next* style also fails. **Likely cause:** A missing semicolon at the end of a declaration. **How to investigate:** Look at the declaration just above the failing one; check for a missing `;`. **Corrective steps:** Add the semicolon. **How to confirm resolved:** Both rules now apply. **Prevention:** End **every** declaration with `;` before writing the next line.

K.2 None of the CSS appears

Symptoms: The page has no styling at all. **Likely cause:** The `<link>` path is wrong, the file is not saved, or the link is outside `<head>`. **How to investigate:** Compare `href` with the real file location; check the file is saved. **Corrective steps:** Fix the path (e.g. `css/style.css`), move the `<link>` into `<head>`, save. **How to confirm resolved:** Styles appear after refresh. **Prevention:** Create the CSS file first, then link to its exact path.

K.3 The page still shows the old look

Symptoms: You changed the CSS but nothing moved. **Likely cause:** The file was not saved, or the browser was not refreshed (or is showing a cached copy). **How to investigate:** Check the unsaved dot in VS Code; do a **hard refresh** with `Ctrl + F5`. **Corrective steps:** Save, then hard-refresh. **How to confirm resolved:** The new look appears. **Prevention:** Save, then refresh — every single time.

K.4 Text is hard to read

Symptoms: Text blends into the background. **Likely cause:** Poor **contrast** (light text on a light background, or dark on dark). **How to investigate:** Squint — if the words blur, contrast is too weak. **Corrective steps:** Darken the text or lighten the background (or invert both). **How to confirm resolved:** Text is clearly readable. **Prevention:** Keep dark text on light backgrounds or light text on dark backgrounds.

K.5 A style will not change no matter what

Symptoms: You edit a rule but an old value stays. **Likely cause:** A **more specific** rule (id, or inline style) is winning. **How to investigate:** Use Chrome DevTools: right-click the element → **Inspect** → look at the **Styles** panel, which shows which rule applies and which are crossed out. **Corrective steps:** Increase this rule's specificity (use the same or a stronger selector) or remove the conflicting rule. Avoid inline styles. **How to confirm resolved:** Your value now applies. **Prevention:** Prefer **classes**, keep specificity low, and do not use inline styles.

K.6 Padding and margin confused

Symptoms: Space appears on the wrong side of an element (inside vs outside). **Likely cause:** Mixing up padding (inside) and margin (outside). **How to investigate:** Temporarily add a background colour; padding shows *inside* the coloured area, margin is *outside* it. **Corrective steps:** Swap the property as needed. **How to confirm resolved:** The space is where you intended. **Prevention:** Remember the phrase: **padding is inside; margin is outside.**

K.7 `:hover` written as `.hover`

Symptoms: The hover effect never appears. **Likely cause:** You wrote `.hover` (a class) instead of `:hover` (a state). **How to investigate:** Check the colon. **Corrective steps:** Use `a:hover { ... }`. **How to confirm resolved:** The effect appears on mouse-over. **Prevention:** Remember: a leading dot is a class; a leading colon is a state.

Distinguish likely vs confirmed. Confirm before fixing: open DevTools and *see* which rule applies, check the actual semicolon, check the actual path. Do not guess.

L. SESSION ASSIGNMENT

Assignment title: "Style Adaeze Fashion — External Stylesheet"

Purpose. Apply the full range of CSS Basics — external stylesheet, selectors, colours, text, backgrounds, borders, the box model and the cascade — to a real business site.

Realistic scenario. Adaeze is delighted with the structure you built. Now she wants her site to look professional and consistent across all pages, using one stylesheet.

Detailed instructions.

1. Ensure you have `css/style.css` and that **all** your HTML pages (`index.html` , `about.html` , `contact.html`) link to it in their `<head>` .
2. In `style.css` , include:
   - a `body` rule (font-family with a generic fallback, a light `background-color` , a dark `color` , `margin` and `padding`);
   - a `header` rule (background colour, text colour with good contrast, padding, centred text);
   - `h1` and `h2` rules (font-family, `font-size` , `color`);
   - a `p` rule (`font-size` , `line-height`);
   - an `a` rule and an `a:hover` rule;
   - a `table` , `th, td` and `th` set of rules (border, `padding` , `border-collapse`);
   - at least **one class** selector used on a real element;
   - at least **one id** selector used on a real element.
3. Save everything and open each page in Chrome to confirm the consistent look.
4. Write a short note (3–4 lines) in your workbook explaining **one** design choice — for example why you chose a particular contrast or colour scheme.

Required deliverables

- `css/style.css` with the rules above.
- All HTML pages linking to the stylesheet.
- Every declaration ending in a semicolon; no inline or internal CSS.
- A 3–4 line reflection on one design choice.

Submission format. Files remain in the project folder; demonstrate the styled site to the instructor before leaving; paste the CSS and the reflection into the workbook.

Deadline. Before Session 5.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
External stylesheet linked on all pages	4	Correct `href` ; no inline/internal CSS present
Body, header and text styling	4	Font, colours, background, padding all set with good contrast
Colours and contrast	3	Consistent palette; text clearly readable everywhere
Table styling	3	Borders, padding and collapse applied cleanly
Class and id selectors	3	Both defined and used on real elements
Syntax and neatness	2	Every declaration has a semicolon; CSS is indented and grouped
Reflection note	1	A clear, honest reason for one design choice

Common mistakes to avoid. Missing semicolons; wrong `<link>` path; inline styles left behind; poor contrast; using `.hover` instead of `:hover` ; using the same id twice.

Instructor reference solution. Use the Section F.10 stylesheet, adding one `.highlight` -style class and one id such as `#site-header` on the `<header>` and styling it, plus the reflection note.

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Multiple choice. Which way of adding CSS is the professional standard for a multi-page site? a) inline b) internal c) external d) none of these

Q2. Short answer. Name the three parts of a CSS rule and give an example.

Q3. Multiple choice. Which selector targets the element with `class="price"`? a) `#price` b) `.price` c) `price` d) `*price`

Q4. Fill in the blanks. In the hex code `#1a3a5c`, the first pair of characters represents _____, the second pair _____, and the last pair _____.

Q5. Explain in your own words. What is the difference between **padding** and **margin**? Use the framed-picture idea.

Q6. True or false. "Internal CSS styles every page of a website automatically." Explain.

Q7. Error detection. Why does this rule fail to apply its colour?

```
h1 {
  color: navy
  font-size: 36px;
}
```

Q8. Scenario. A learner sets `p { color: red; }` but the paragraphs stay blue. What are two likely reasons, and how would you investigate?

INSTRUCTOR ANSWER KEY (Q1–Q8)

A1. (c) external — one file styles every page and changes are centralised.

A2. Selector (which element), **property** (what to change), **value** (how). Example: `h1 { color: navy; }` — selector `h1`, property `color`, value `navy`.

A3. (b) `.price` (dot = class).

A4. The first pair (`1a`) is the **red** component, the second (`3a`) is **green**, the last (`5c`) is **blue**.

A5. Padding is the space *inside* the box, between the content and its border (like the mat board inside a picture frame). **Margin** is the space *outside* the box, separating it from other elements (like the empty wall around the frame).

A6. False. Internal CSS (inside a `<style>` block in one page's `<head>`) affects **only that page**. For multiple pages you need an **external** stylesheet linked from every page.

A7. The declaration `color: navy` is missing its **semicolon**. Without it, the browser may merge it with the next line and ignore it. Fix: `color: navy;`.

A8. Two likely reasons: (1) another rule is **more specific** (an id or inline style) and is overriding the `p` rule; (2) the `p` rule itself has an **error** (missing semicolon, wrong selector, or the stylesheet is not linked). Investigate with Chrome **DevTools** → right-click the paragraph → **Inspect** → **Styles** panel to see which rule wins and which are crossed out.

N. REFLECTION AND RECAP

Plain-English summary. Today you gave Adaeze Fashion its look. You learned that **CSS controls presentation** while HTML controls structure, and why they should live in separate files. You met the **three ways** to add CSS — inline, internal and external — and why **external** is the professional choice. You learned the shape of a CSS rule (**selector, property, value, declaration**, each declaration ending in `;`), the main **selectors** (element, `.class`, `#id`, grouped), and how to express **colours** (named, hex, rgb) with attention to **contrast**. You styled **text** (font, size, weight, alignment, line-height, decoration), set **backgrounds and borders**, and learned the **box model** — with the key rule that **padding is inside and margin is outside**. Finally, you learned the beginner's picture of the **cascade and specificity** — later and more specific rules win — and you built a real external stylesheet for the whole site.

Five important takeaways.

1. **HTML = structure, CSS = presentation.** Keep your CSS in an **external** stylesheet.
2. A rule is **selector { property: value; }** — and every declaration needs a **semicolon**.

3. Selectors: element (`p`), class (`.price`), id (`#main`), grouped (`h1, h2`). Prefer **classes**.
4. **Padding is inside; margin is outside** — spacing is most of what makes a page look professional.
5. Colours can be **named, hex (`#RRGGBB`), or rgb()** — always check **contrast**.

Self-check — can you now…

- Explain why external CSS beats inline CSS?
- Write a CSS rule for a class and an id from memory?
- Set a page's font, background and text colour with good contrast?
- Explain padding versus margin using the picture-frame idea?
- Predict which rule wins when two rules disagree?
- Link an external stylesheet and see the styling appear?

Connection to the next session. You can now make a page look good. Next session, **Session 5 — CSS Layout**, we go deeper into controlling *where* things sit: the **box model** in more detail, the **display** property (`block` , `inline` , `inline-block`), **positioning**, and **Flexbox** — the tool that arranges menus, columns and cards neatly. That is the step between "styled" and "well laid out".

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence.

- The learner creates and links an external stylesheet without prompting and uses it on more than one page.
- The learner writes correct syntax, including semicolons, and can read a rule aloud (selector, property, value).
- The learner chooses the right selector (element vs class vs id) and prefers classes.
- The learner checks contrast and can justify a colour choice.
- The learner distinguishes padding from margin and uses them purposefully.
- The learner uses DevTools to investigate "why won't this style change?".

Common misconceptions to watch for and correct.

- "CSS is a programming language." → It is a styling language; it describes, it does not compute.
- "Inline CSS is fine." → External is the professional standard; inline is hard to maintain.
- "Padding and margin are the same." → Padding inside; margin outside.
- "Hex codes are random." → They are Red/Green/Blue in base-16 (`#RRGGBB`).
- "Internal CSS styles the whole site." → It styles only one page.
- " `.hover` gives a hover effect." → Use `:hover` (a state, with a colon).
- "A missing semicolon only affects that one line." → It can break the declaration *after* it too.

Suggested remedial exercises.

1. Semicolon hunt: hand out a stylesheet with five missing semicolons; the learner finds and fixes them and checks in the browser.
2. Selector sorting: give ten selectors and ten tasks; the learner matches the best selector type to each task and explains why.
3. Box model labelling: print a blank concentric-box diagram; the learner labels content, padding, border and margin, then sets each in CSS and observes the effect.
4. Colour contrast fix: give three "hard to read" colour pairs; the learner adjusts the text or background to improve contrast and explains the change.

Differentiation.

- **Level 1 (supported):** Provide a `style.css` starter with the `body` , `header` and `h1` rules written and commented; learners complete the `p` , `a` and table rules. Pair with a confident learner.
- **Level 2 (standard):** Complete the exercises and assignment as written.
- **Level 3 (challenge):** Add `border-radius` (rounded corners), a `box-shadow` , `text-transform: uppercase` for a heading, and a Google-style font stack; introduce a second colour used consistently as an accent, and explain the palette choice.

Timing guidance (105 minutes). Opening 6 · Prerequisite 7 · Lecture 40 · Demonstration 15 · Guided practical 18 · Independent exercise 15 · Case study 4 · Knowledge check 4 · Reflection/close 3. If slow, shorten the case study to 2 minutes and protect the

guided practical — writing and linking the stylesheet is the priority. If fast, use spare time for Level 3 styling and a peer "design review" where learners justify their colour and font choices.

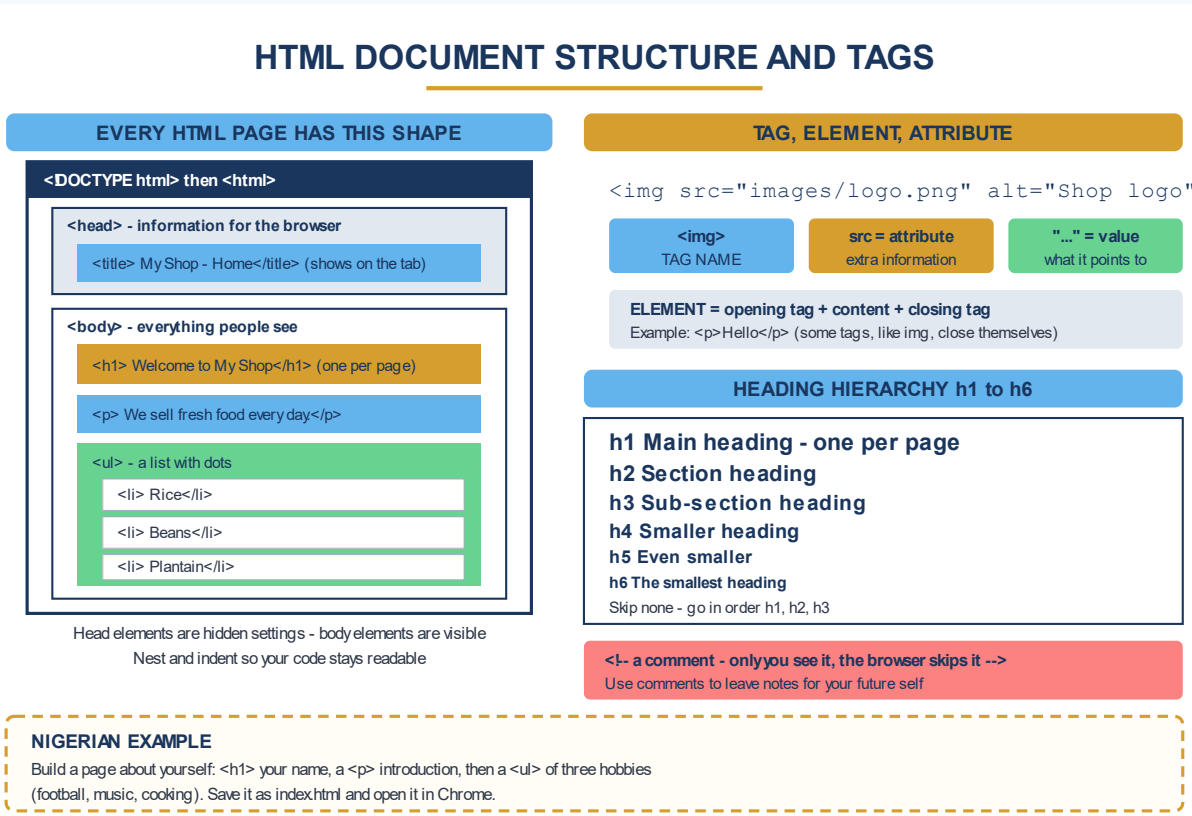
END OF SESSION 4

Next: Session 5 — CSS Layout: the box model in depth, display, positioning and Flexbox.

SESSION 05 OF 12

CSS LAYOUT

CONCEPT DIAGRAM — SESSION 05



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 5 — CSS LAYOUT

A. SESSION IDENTIFICATION

Item	Detail
Session number	5 of 12
Session title	CSS Layout
Week / Module	Week 2 — Module 2: CSS Foundations
Recommended duration	105 minutes
Difficulty level	Beginner (comfortable with HTML and basic CSS)

Item	Detail
Prerequisite knowledge	Sessions 1–4. The learner must be able to write an HTML page with headings, paragraphs, links, images and sections, and link an external CSS file with `<link rel="stylesheet" href="css/style.css">`. If a learner cannot link a CSS file and change a text colour, run the 15-minute refresher in Appendix 5A before this session.
Connection to previous work	In Session 4 learners learned to style colour, fonts and spacing. The styles worked, but the page still stacked everything in one dull column, top to bottom. This session teaches how to control *where* things sit on the page — the difference between a plain document and a real website layout.
Preparation before class	Academy PCs on and logged in; VS Code and Chrome installed; a practice folder per learner named `adaeze-fashion`; a copy of each learner's Week 1 profile page and Session 4 styled page; the projector set to show the instructor's Chrome and VS Code; print the "Box Model Diagram" handout (one per learner) and the "Flexbox Quick Reference" card (one per learner).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in their own words** how elements sit on a page in normal flow, and name the difference between a block element and an inline element.
2. **Apply the four `display` values** — `block`, `inline`, `inline-block` and `none` — to make an element behave in a chosen way, and state the visible effect of each.
3. **Use `width`, `height` and `max-width`** to size an element, and explain why `max-width` is safer than a fixed `width`.
4. **Explain and apply `box-sizing: border-box`** so that padding and border are included inside the width, and describe what happens to the box when it is missing.
5. **Explain why `float` is a legacy technique** and recognise it in old code, naming at least one situation where it still appears (wrapping text around an image).
6. **Build a Flexbox row** using `display: flex`, and control it with `flex-direction`, `justify-content`, `align-items`, `gap` and `flex-wrap`.
7. **Build a basic two-column CSS Grid** using `display: grid`, `grid-template-columns` and `gap`.
8. **Choose between Flexbox and Grid** for a given layout and give a reason for the choice.
9. **Centre an element horizontally and vertically** using Flexbox.
10. **Reproduce two common real layouts** — a navigation bar using Flexbox and a card grid — and describe the expected on-screen result.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). A shared computer between two learners is acceptable; pair them so one types and one reads the screen, then swap halfway through the practical.
- Instructor computer connected to a projector or large screen.

Software

- Google Chrome (any recent version). The DevTools "box model" diagram referred to in Section G exists in all modern Chrome versions.
- Visual Studio Code (VS Code). Any version from 2022 onwards works.
- Windows 10 or Windows 11.

Accounts

- None required. All work this session is offline.

Sample files

- The instructor prepares and shares a starter folder `adaeze-fashion` containing `index.html`, a `css/` folder with `style.css`, and an `images/` folder. The contents are given in Section G. Learners may also type from scratch — recommended for muscle memory.

Worksheets

- Worksheet 5.1 — "Block or Inline?" (printed, 1 page).
- Worksheet 5.2 — "Box Model Labelling" (in the learner workbook, Part 3).

Tools

- Chrome DevTools (built into Chrome — press `F12`). This session uses the **Elements** tab and its coloured box-model diagram at the bottom of the Styles pane.

Internet

- Not required for this session. If the academy has internet, the instructor may optionally show the MDN page for Flexbox at the very end, but do not depend on it.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. If a learner has only a phone, pair them with a desktop user for the session and ask them to photograph the code and examples for later practice.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open two versions of the same business page for *Adaeze's Fashion Design*. The first version has all the content (logo, links, a welcome message, three product boxes, a footer) but no layout — everything is left-aligned and stacked in one long, boring column, all in the default font. The second version is the same page with a simple layout: the logo and menu sit side by side along the top, the three products sit in a neat row, and the footer is centred.

What the instructor says:

> *"Look at these two pages. Same words. Same colours. Same pictures. But the first one looks like a school assignment, and the second one looks like a shop you would actually buy from. What is the difference?*
>
> *The difference is **layout** — the arrangement of things on the page: what sits beside what, what is on top, how wide each part is. Last week you learned how to style one element. Today you learn how to arrange many elements. That is the skill that turns a document into a real website."*

Expected learner response: Learners may say "one has columns", "the other is more organised", or "the second looks nicer". Accept all answers and confirm: the *content* never changed — only the *arrangement*.

Then demonstrate the hook: In the projector's Chrome, right-click the second (nice) page, choose **Inspect**, and click the **Elements** tab. Hover over the three product boxes — Chrome outlines each one with coloured bars showing content (blue), padding (green), border (yellow) and margin (orange). Say:

> *"Every single thing on a webpage is actually a rectangle — a box — even when it does not look like one. Today you will learn to see the boxes and to move them wherever you want."*

Why this opener works: It creates a clear gap between "I can style one element" and "I can arrange a whole page", and it immediately gives learners a tool (the box diagram) they will use for the rest of the course.

E. PREREQUISITE REVIEW (7 minutes)

Run this quickly as an oral round-robin. Put the five questions on the projector.

Question 1: "What are the three parts of a CSS rule?"

- Expected answer: a selector, a property and a value (e.g. `p { color: red; }`).
- Common gap: learners name the property and value but forget the selector. Remind them: the selector is *what* you are styling.

Question 2: "What does the `display` property do?" (Even though we cover it in depth today, check what they already think.)

- Expected answer: it controls how an element behaves in the page flow. Accept "I am not sure yet" — this is the first thing we teach today.

Question 3: "How do you link an external CSS file to an HTML page, and where does the line go?"

- Expected answer: `<link rel="stylesheet" href="css/style.css">` inside the `<head>`.
- Common gap: writing it in the `<body>` or misspelling `stylesheet`. Re-state the exact spelling.

Question 4: "What is the difference between `.price` and `#price`?"

- Expected answer: `.price` is a class selector and can appear on many elements; `#price` is an id selector and must be unique on the page.
- Common gap: swapping them. Restate: dot = class, hash = id.

Question 5: "If a CSS change does not appear in the browser, what is the first thing to check?"

- Expected answer: save the file (`Ctrl + S`) and refresh the page (`F5`). Then check the file path is correct.
- Common gap: learners forget to save before refreshing.

Bridge to today (say this):

> *"Last session you dressed one person. Today you arrange the whole room — you decide who stands where. In CSS, the two modern tools for arranging things are **Flexbox** and **Grid**. By the end of today you will be able to build the top of any business website: a logo on the left, a menu on the right, and a row of product cards below."*

If more than a third of the class cannot answer Question 3, spend 10 extra minutes on the Appendix 5A CSS refresher before continuing.

F. FULL LECTURE NOTES

F.1 Normal flow — how elements sit by default

The idea. Before you arrange anything, you must understand what happens *without* your help. This default behaviour is called **normal flow** (or "the normal document flow").

Plain-English definition. Normal flow is the way the browser lays out elements when you have not told it to do anything special. The browser reads your HTML from top to bottom and places each element on the page in the order it appears.

Why it exists. HTML was designed for documents — letters, reports, articles. A document naturally flows top to bottom, line by line, just like text in a book. That is why the first version of any HTML page is a single column: the browser is being a good reading machine.

What it does:

- Block elements stack **vertically** — one underneath the other.
- Inline elements sit **side by side** on the same line, like words in a sentence.
- Everything starts at the top-left and moves down and right.

Analogy. Think of normal flow as **writing on a fresh sheet of paper**. You start at the top-left. You write across the line; when the line is full, you go down to the next line. You never jump around unless you deliberately do something. Layout in CSS is the art of deliberately jumping around.

Worked example — see normal flow for yourself:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <title>Normal Flow</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Adaeze's Fashion Design</h1>
  <p>Custom clothing made in Port Harcourt.</p>
  <a href="shop.html">Shop</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</body>
</html>
```

Expected browser result: The `<h1>` appears on its own line. Below it, the paragraph appears on its own line. Below that, the three links appear **on one line, side by side**, separated only by the browser's default word spacing: `Shop About Contact`. The headings and paragraph each start on a new line because they are block elements; the links stay together because they are inline elements.

Misconception. Learners often think the browser "knows" that a heading should be on its own line because of its *meaning*. It does not — it is on its own line because headings are **block** elements. Change the display and you change the behaviour.

Limitation. Normal flow alone cannot make two things sit side by side as columns, cannot centre a box in the middle of the screen, and cannot make a menu spread out along a bar. For that we need to change how elements display, and then use Flexbox or Grid.

F.2 The `display` property — block, inline, inline-block, none

The idea. The `display` property tells the browser *what kind of box* an element is. Changing it is like telling a person "you may now behave as a column" or "you may now behave as a word".

Plain-English definition. `display` controls how an element participates in the layout: does it take a whole line, sit inline with text, or disappear entirely?

Why it exists. Different content needs different behaviour. A heading should own its line; a link should sit within a sentence; a whole section you can hide with JavaScript later needs a way to be switched off. `display` gives you that control.

The four values we use this week:

Value	Behaviour	Starts on a new line?	Can you set width/height?	Common elements
`block`	Takes the full width available, one after another	Yes	Yes	`<div>`, `<p>`, `<h1>` – `<h6>`, `<header>`, `<section>`, `<footer>`, `<main>`
`inline`	Sits in the text flow, only as wide as its content	No	No	`<span>`, `<a>`, `<strong>`, `<em>`
`inline-block`	Sits in the text flow like inline, but accepts width/height like block	No	Yes	Often used for buttons or small badges
`none`	Removed from the page completely	—	—	Used to hide elements (often switched with JavaScript later)

Worked example — compare block, inline and inline-block:

```
/* css/style.css */
.blocky {
  display: block;
  background-color: #f3b6c8;
  padding: 8px;
}
.inliney {
  display: inline;
  background-color: #b6d0f3;
  padding: 8px;
}
.card {
  display: inline-block;
```

```
  width: 120px;
  height: 60px;
  background-color: #b6f3c8;
  margin: 6px;
  text-align: center;
  padding-top: 20px;
}
.hidden {
  display: none;
}
```

```
<body>
  <div class="blocky">Box A (block)</div>
  <div class="blocky">Box B (block)</div>

  <p>Here are links: <span class="inliney">Home</span>
     <span class="inliney">Shop</span> and they sit inline.</p>

  <div class="card">Card 1</div>
  <div class="card">Card 2</div>

  <p class="hidden">You should not see me at all.</p>
</body>
```

Expected browser result: Box A and Box B each take a full-width pink strip on their own line. The two blue "Home" and "Shop" words sit inside the sentence, side by side, with blue backgrounds. The green cards sit side by side in a row and hold their fixed size (120×60). The hidden paragraph takes up no space at all — there is a gap in the text flow where it would have been, but nothing is shown. **Note:** an `inline` element ignores `width` and `height`, so the blue spans are only as wide as their words. This surprises beginners every time — it is the difference between `inline` and `inline-block`.

Important note on `padding` for inline elements. An inline element accepts left/right padding (as shown) but top/bottom padding does not push other lines apart the way learners expect. If a learner wants a clickable "button" out of a link, they should usually set `display: inline-block` so width, height and padding all behave predictably.

Misconception. "If I set `display: none`, the element is invisible but still pushes things." False — `display: none` removes it from the layout entirely. (This is different from `visibility: hidden`, which hides it but keeps its space; we do not need that this week.)

Limitation. `display: block` gives you stacking; `display: inline-block` gives you small side-by-side pieces. Neither can easily spread items apart, wrap them onto new rows, or align them vertically. That is the job of Flexbox and Grid (F.6 and F.7).

F.3 Sizing — `width`, `height` and `max-width`

The idea. You can set how wide and how tall a box is.

Plain-English definition. `width` is how wide the element is; `height` is how tall. `max-width` means "never get wider than this, but you may be narrower".

Why it exists. By default, a block element is as wide as its parent and as tall as its content. Sometimes you want a fixed card size; sometimes you want a box to grow but stop before it becomes too wide to read.

Common units:

Unit	Meaning	Beginner example
`px`	Fixed pixels	`width: 300px;`
`%`	Percentage of the parent's width	`width: 50%;`
`rem`	Times the root (page) font size	`width: 20rem;`
`vw` / `vh`	Percent of viewport width / height	`height: 100vh;` (full screen height)

Worked example — fixed width versus max-width:

```
.fixed {
  width: 600px;
  background-color: #ffe6a7;
```

```
}
.fluid {
  width: 90%;
  max-width: 600px;
  background-color: #a7e6ff;
}
```

Expected browser result: Both boxes look similar on a wide desktop. But if you drag the browser window narrower to phone width, the yellow `.fixed` box **stays 600px wide and overflows** off the right side of the screen (causing a horizontal scrollbar), while the blue `.fluid` box **shrinks to 90% of the window** and never overflows. That is exactly why `max-width` is the safer choice for a responsive website — a theme we expand fully in Session 6.

Rule of thumb for this course: prefer `max-width` with a percentage or fixed upper limit for content containers; use fixed `width` / `height` only for small, predictable pieces such as icons or cards.

Misconception. "Width includes everything." By default it does **not** — it applies only to the content box. The next sub-topic fixes that.

Limitation. Setting a height on a box that contains text is risky: on a narrow screen the text may become taller than the fixed height and spill out. Prefer `min-height` (a floor, not a ceiling) when you must set a height at all.

F.4 The box model and `box-sizing`

The idea. Every element is a rectangle made of four layers. This is called the **CSS box model**.

Plain-English definition. The box model says each element has, from the inside out:

1. **Content** — the text or image itself.
2. **Padding** — space *inside* the box, between the content and the border.
3. **Border** — the edge line around the padding.
4. **Margin** — space *outside* the box, pushing other elements away.

Why it exists. Separating "space inside" (padding) from "space outside" (margin) lets you create breathing room around text and control the gaps between boxes independently.

Analogy. Imagine each element is a **framed picture on a wall**. The picture itself is the content. The white card around the picture is the padding. The wooden frame is the border. The gap you leave between this framed picture and the next one is the margin. (This is the same "room" idea some learners met in earlier handouts — content is the furniture, padding is the floor space to the walls, border is the walls, margin is the corridor between rooms.)

Worked example — the classic surprise:

```
.box {
  width: 300px;
  padding: 20px;
  border: 5px solid #333;
  margin: 10px;
  background-color: #f3b6c8;
}
```

Expected browser result: The content area is 300px wide. But the *total* width you see is: 300 (content) + 20 (left padding) + 20 (right padding) + 5 (left border) + 5 (right border) = **350px**, plus 10px of margin on each side that pushes neighbours away. Beginners expect 300px total and are shocked when the box measures 350px. This is the single most confusing thing in CSS layout — and `box-sizing` fixes it.

The fix — `box-sizing: border-box`:

```
* {
  box-sizing: border-box;
}

.box {
  width: 300px;          /* now 300px INCLUDING padding and border */
  padding: 20px;
  border: 5px solid #333;
```

```
  background-color: #f3b6c8;
}
```

Expected browser result: The box now measures exactly 300px total. The padding and border are counted *inside* the 300px, so the content area shrinks to 300 − 40 − 10 = 250px, but the visible box is a clean 300px. Widths become predictable, which is why nearly every professional stylesheet starts with `* { box-sizing: border-box; }`.

Say this to the class:

> *"The `*` selector means 'every element on the page'. So this one little rule makes every box include its padding and border in its width. From today, we put it at the top of every stylesheet. It will save you hours of confusion."*

How to see it in the browser: Press `F12`, click **Elements**, select a box, and look at the coloured diagram at the bottom of the right pane — blue content, green padding, yellow border, orange margin. The instructor will do this live in Section G.

Misconception. "Padding and margin are the same." No: padding is *inside* the border (it adds to the box and takes the element's own background colour); margin is *outside* (it is always transparent and pushes other elements away).

Limitation. `box-sizing` changes how width is *calculated*; it does not stop an element from overflowing if its *content* (such as a large un-resized image) is bigger than the box. Images need their own care (Session 6).

F.5 Floats — and why they are now legacy

The idea. Long before Flexbox and Grid existed, designers used the `float` property to place boxes side by side.

Plain-English definition. `float: left` or `float: right` pulls an element out of the normal vertical flow and pushes it to one side, letting the following content wrap around it.

Why it existed. In the early 2000s there was no Flexbox or Grid. To build a two-column layout, designers floated the columns left and right and then "cleared" the float afterwards. It worked, but it was fragile.

What it does today. Its *original* and still-useful job is **wrapping text around an image** — exactly how a magazine wraps an article around a photo:

```
.article-img {
  float: left;
  width: 200px;
  margin: 0 15px 10px 0;
}
```

```
<img class="article-img" src="images/adaeze.jpg" alt="Adaeze at her sewing machine">
<p>Adaeze started her fashion business in 2019 with one sewing machine...</p>
<p>The paragraph text wraps neatly around the right side of the photo.</p>
```

Expected browser result: The photo sits on the left; both paragraphs flow down beside it on the right, wrapping under the picture once the text passes the bottom of the image. This is the one place float is still the natural tool.

Why floats are legacy for layout:

- A floated element is removed from normal flow, so its parent can "collapse" and lose its height — a bug that required `clearfix` hacks.
- Vertical centring with floats was painful.
- Responsive behaviour (wrapping, gap, changing direction on mobile) had to be hand-built.
- Flexbox and Grid do all of this cleanly and predictably.

What the learner should do about floats. *Recognise* them in older code and know what they mean. *Do not* use them to build page layout in new code — use Flexbox or Grid. If you see a wrapper with `float: left` and a mysterious `clear: both` block, you are looking at an old-fashioned layout.

Misconception. "Float is wrong and I should never use it." Float is not wrong; it is simply no longer the best tool for layout. It is still correct for text-wrapping an image, and understanding it helps you read existing websites.

Limitation. Even with clearfix tricks, float layouts have no built-in `gap`, no easy vertical alignment and no clean wrapping — exactly the gaps Flexbox and Grid were created to fill.

F.6 Flexbox — arranging items in one direction

The idea. Flexbox (short for "Flexible Box Layout") arranges a group of items along a single line — either a row or a column — and gives you powerful control over spacing and alignment.

Plain-English definition. When you set `display: flex` on a parent element, that parent becomes a **flex container**, and its direct children become **flex items**. The browser then arranges those items in a flexible line you can control.

Why it exists. Before Flexbox, arranging items in a row and spreading them out, centring them, or making them shrink on small screens required hacks. Flexbox was designed (and is now supported in every modern browser) to make these common jobs simple and reliable.

The two-axis idea (diagrams in words). Flexbox works on two axes:

- The **main axis** is the direction the items flow. By default it runs **left → right** (a row).
- The **cross axis** runs **perpendicular** to the main axis — for a row, the cross axis runs **top → bottom**.

Picture a bookshelf. The **main axis** is the shelf you place books along (left to right). The **cross axis** is how each book stands tall (top to bottom). `justify-content` moves books *along* the shelf; `align-items` moves them *up and down* on the shelf.

If you change `flex-direction: column`, the whole thing rotates: the main axis now runs **top → bottom**, and the cross axis runs **left → right**. The two properties keep their meaning relative to the axes, which is why they can feel confusing at first — always ask "which way is my main axis pointing?"

Turning on Flexbox:

```
.nav {
  display: flex;            /* parent becomes a flex container */
}
```

The container properties you need this week:

Property	Values we use	What it does
`display`	`flex`	Turns the element into a flex container; children become flex items.
`flex-direction`	`row` (default), `column`	Sets which way the main axis points. `row` = left→right; `column` = top→bottom.
`justify-content`	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`	Spaces items **along the main axis**.
`align-items`	`stretch` (default), `flex-start`, `center`, `flex-end`	Aligns items **across the cross axis**.
`gap`	e.g. `20px`, `1rem`	Adds even spacing **between** items (not on the outer edges).
`flex-wrap`	`nowrap` (default), `wrap`	Whether items may move onto a new line when there is not enough room.

Worked example A — a row with even spacing:

```
.menu {
  display: flex;
  justify-content: space-between;
  gap: 20px;
  background-color: #1a3a5c;
  padding: 16px;
}
.menu a {
  color: white;
  text-decoration: none;
}
```

```
<nav class="menu">
  <a href="index.html">Home</a>
  <a href="shop.html">Shop</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

Expected browser result: A dark navy bar. The four links sit in one horizontal row; the first link is flush left, the last is flush right, and the middle ones are spread evenly between them (that is `space-between`). There are at least 20px gaps. `gap` adds nothing outside the row — the outer edges stay flush with the padding.

Worked example B — wrapping cards onto new rows:

```
.cards {
  display: flex;
  flex-wrap: wrap;     /* allow items to drop to the next line */
  gap: 20px;
}
.card {
  flex: 1 1 220px;     /* grow, shrink, but ideally 220px wide */
  padding: 16px;
  border: 1px solid #ddd;
  border-radius: 8px;
}
```

Expected browser result: On a wide desktop, three or four cards sit in one row, each about 220px wide. On a narrow window, the cards **wrap** onto a second row and then a third, rather than squashing into unreadable slivers. The `flex: 1 1 220px` shorthand means "grow if there is space, shrink if needed, but aim for 220px".

The `flex` shorthand explained: `flex: 1 1 220px` is three values in one:

- first `1` = **grow** (may grow to fill extra space),
- second `1` = **shrink** (may shrink when space is tight),
- `220px` = **basis** (the starting ideal width).

Misconception. "I need Flexbox on the children too." No — you set `display: flex` on the **parent**. The children become flex items automatically; you only add properties to the children (like `flex: 1`) when you want to fine-tune them individually.

Limitation. Flexbox is excellent for **one line of items** (a row or a column). When you want to control items in **two directions at once** — rows *and* columns forming a real grid — Grid is the better tool (next topic).

F.7 CSS Grid basics — laying out rows and columns

The idea. CSS Grid arranges items into a true two-dimensional grid of rows and columns.

Plain-English definition. When you set `display: grid` on a parent, that parent becomes a **grid container**, and its children become **grid items** that you place into columns and rows you define.

Why it exists. Some layouts are naturally a grid: a photo gallery, a product listing, a page with a sidebar and a main area. Flexbox can approximate these, but Grid describes them directly and precisely.

The two core properties this week:

Property	What it does
`display: grid`	Turns the element into a grid container.
`grid-template-columns`	Defines the columns of the grid, e.g. `1fr 1fr 1fr` for three equal columns.
`gap`	Adds even space between rows and columns (same idea as Flexbox).

The `fr` unit. `fr` means "fraction of the available space". Three columns of `1fr 1fr 1fr` each take one-third of the width. Two columns of `2fr 1fr` make the first column twice as wide as the second.

Worked example — a three-column card grid:

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);  /* three equal columns */
  gap: 20px;
}
.gallery img {
  width: 100%;
  display: block;
}
```

```
<section class="gallery">
  <img src="images/dress1.jpg" alt="Ankara dress">
  <img src="images/dress2.jpg" alt="Lace gown">
  <img src="images/dress3.jpg" alt="Aso ebi outfit">
  <img src="images/dress4.jpg" alt="Office wear">
  <img src="images/dress5.jpg" alt="Traditional attire">
  <img src="images/dress6.jpg" alt="Bridal gown">
</section>
```

Expected browser result: Six images arranged in **two rows of three**, with 20px gaps between every image. Each image is one-third of the container width. If you reduced the columns to `repeat(2, 1fr)`, you would get three rows of two.

`repeat(3, 1fr)` **explained:** it is shorthand for writing `1fr 1fr 1fr`. The first number is how many columns; the second is the size each should be.

Misconception. "Grid and Flexbox are competitors — I must pick one forever." Not at all. They are partners: use Grid for the overall page grid, and Flexbox inside individual cells for aligning the small pieces. Real websites mix both constantly.

Limitation. Grid is slightly newer than Flexbox. It is supported in every modern browser (Chrome, Firefox, Edge, Safari), but very old browsers (Internet Explorer 10 and earlier) do not support it. For our course this is a non-issue — we target modern browsers.

F.8 When to use Flexbox, when to use Grid

This is a decision learners ask about constantly. Use this rule of thumb:

Situation	Choose	Why
A row of navigation links	**Flexbox**	Only one direction (a row); need `space-between`.
Centring one thing in the middle of a box	**Flexbox**	Two lines of CSS with `justify-content` and `align-items`.
A row of cards that should wrap	**Flexbox**	`flex-wrap: wrap` plus `flex: 1 1 220px` handles wrapping.
A photo gallery with fixed columns and rows	**Grid**	Needs control in two directions.
A page with a sidebar and a main content area	**Grid**	Two columns of different sizes side by side.

One-line memory aid: *Flexbox is for one direction; Grid is for two.* If you can describe the layout as "a line of things", reach for Flexbox. If you must talk about both rows *and* columns at once, reach for Grid.

F.9 Centring things — the classic problem, solved

The idea. Centring was once notoriously hard in CSS. Flexbox makes it easy.

Horizontal centring only:

```
.title {
  text-align: center;   /* centres inline content such as text */
}
```

This centres the *text inside* a box. It does not centre the box itself. To centre a block box horizontally, you can also use `margin: 0 auto;` (auto left and right margins split the leftover space equally) — but this requires the box to have a set width.

Perfect centring (both directions) — the modern one-liner trio:

```
.hero {
  display: flex;
  justify-content: center;  /* centre along the main axis (horizontal) */
  align-items: center;      /* centre along the cross axis (vertical) */
  min-height: 400px;        /* give the box some height to centre within */
}
```

```
<section class="hero">
  <div>
    <h1>Custom Fashion for Every Occasion</h1>
    <p>Beautiful, affordable clothing made just for you.</p>
  </div>
</section>
```

Expected browser result: The heading and paragraph sit dead centre of a 400px-tall hero area, both horizontally and vertically. Before Flexbox, this took multiple hacks; now it is three lines.

Important condition. Vertical centring only *works* when the container has height to spare. If the container is exactly the height of its content, there is nothing to centre within. That is why `min-height` is included.

F.10 Putting it together — two practical layouts

Layout A — the navigation bar. Build the top of *Adaeze's Fashion Design* with a logo on the left and the menu on the right.

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #4b2e83;   /* deep purple — Adaeze's brand colour */
  padding: 14px 24px;
}
.logo {
  color: #f4c542;              /* gold */
  font-size: 22px;
  font-weight: bold;
  text-decoration: none;
}
.topbar nav {
  display: flex;
  gap: 24px;
}
.topbar nav a {
  color: white;
  text-decoration: none;
}
```

```
<header class="topbar">
  <a href="index.html" class="logo">Adaeze's Fashion</a>
  <nav>
    <a href="index.html">Home</a>
    <a href="shop.html">Shop</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

Expected browser result: A purple bar across the top. On the left, the gold business name. On the right, four white links evenly spaced with 24px between them, all vertically centred with the logo. This single header can be copied onto every page — exactly what we do in Session 8.

Layout B — the product card grid. Show six dresses in a neat grid.

```
.products {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
```

```
  max-width: 1000px;
  margin: 40px auto;
  padding: 0 16px;
}
.product {
  border: 1px solid #eee;
  border-radius: 10px;
  overflow: hidden;
}
.product img {
  width: 100%;
  display: block;
}
.product-info {
  padding: 12px 16px;
}
.product-info h3 {
  margin: 0 0 6px;
  font-size: 18px;
}
.price {
  color: #b8860b;
  font-weight: bold;
}
```

```
<section class="products">
  <article class="product">
    <img src="images/dress1.jpg" alt="Ankara dress">
    <div class="product-info">
      <h3>Ankara Dress</h3>
      <p class="price">₦18,500</p>
    </div>
  </article>
  <!-- repeat for five more products -->
</section>
```

Expected browser result: A centred container (max 1000px wide, so it does not stretch across a very large monitor) holds a grid of product cards, three per row. Each card has a photo on top and the name and naira price below, with a thin border and rounded corners. The page background on either side of the 1000px container is empty space, keeping the content readable.

Why `overflow: hidden` on the card: it makes the image's square corners follow the card's rounded corners, so the photo does not stick out past the rounded border.

F.11 Summary of the session

- **Normal flow** stacks block elements vertically and inline elements side by side.
- `display` decides an element's box type: `block`, `inline`, `inline-block` or `none`.
- `width` / `height` size a box; `max-width` is safer because it never overflows.
- **The box model** = content + padding + border + margin. Use `* { box-sizing: border-box; }` so width includes padding and border.
- **Float** is legacy for layout; still fine for wrapping text around an image.
- **Flexbox** arranges one row or one column: `display: flex` plus `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap`.
- **Grid** arranges rows and columns at once: `display: grid`, `grid-template-columns`, `gap`.
- **Rule of thumb:** Flexbox for one direction, Grid for two.
- **Centring** horizontally and vertically is three lines of Flexbox.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A folder on the Desktop: `demo-webdesign/week2/` containing:
  - `index.html`
  - `css/style.css`
  - `images/` (any three or four small JPGs, or use placeholder coloured boxes if no images are available)
- Both files open in VS Code; Chrome open to the page.
- The projector showing Chrome DevTools (press `F12`) with the **Elements** tab.

G.2 Demonstration 1 — the box model in DevTools (5 minutes)

Say: "Before I arrange anything, I want you to *see* that every element is a box."

1. Open `index.html`, which contains a single `<div class="box">Adaeze's Fashion</div>` and the CSS from F.4.
2. Press `F12`, click **Elements**, and click the `<div class="box">` in the HTML panel.
3. Point to the coloured diagram at the bottom of the right pane: blue = content, green = padding, yellow = border, orange = margin.
4. Change `width: 300px;` to `width: 500px;` in the DevTools Styles pane (not in the file) and watch the blue box grow live. Say: "I am changing this without saving — it is only for experimenting, and it disappears when I refresh."
5. Now uncheck the `box-sizing` line by clicking the little box next to it. Watch the total width jump by 40px + 10px, because padding and border are suddenly added *outside* the width again.

Ask: "Why did the box get wider when I turned off box-sizing?" → Because without `border-box`, the width applies only to the content, so padding and border are added on top.

Deliberately inject an error: Remove the `* { box-sizing: border-box; }` rule from the top of `style.css` and refresh. The two boxes now look slightly different sizes even though both say `width: 300px` — because one has more padding. Say: "This is the trap. From now on, box-sizing is at the top of every stylesheet I write." Then restore it.

G.3 Demonstration 2 — build the navigation bar with Flexbox (5 minutes)

Say: "Watch how two lines of Flexbox turn a vertical list of links into a proper menu bar."

1. In `index.html`, add the `<header class="topbar">` structure from F.10 Layout A.
2. Save and refresh. Point out that right now the links stack vertically in a plain list, because they are inside a block `<header>` and the `<nav>` is a block too.
3. Add to `style.css`:

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.topbar nav {
  display: flex;
  gap: 24px;
}
.topbar a { color: #f4c542; text-decoration: none; }
```

4. Save and refresh. The links jump into a horizontal row, spread across the bar.

Ask: "Which property pushed the logo to the left and the menu to the right?" → `justify-content: space-between`.

Deliberately inject an error: Change `justify-content: space-between;` to `justify-content: spacebetween;` (missing hyphen) and refresh. Nothing changes — the rule is ignored because the value is invalid. Say: "An invalid value is simply ignored; the browser does not always shout at you. Always double-check spelling." Restore it.

Deliberately inject a second error: Add `flex-direction: column;` to `.topbar` and refresh. The logo and menu now stack on top of each other. Ask the class to predict what will happen before you refresh. Remove it afterwards.

G.4 Demonstration 3 — build the card grid with Grid (5 minutes)

Say: "Now the same idea, but for a grid of products."

1. Add the `<section class="products">` structure from F.10 Layout B.
2. Add the CSS. Save and refresh.
3. The six cards arrange into three columns.
4. Live in DevTools, change `repeat(3, 1fr)` to `repeat(2, 1fr)`, then to `repeat(4, 1fr)`, and let the class watch the grid rearrange. Then undo.
5. Turn on the DevTools **grid inspector**: in the Elements panel, a small grid badge appears next to the `.products` element; clicking it shows the grid lines overlaid on the page. Say: "Chrome draws the grid lines for you — useful when a layout gets complicated."

Questions to ask during the demos:

- "Which would you use for the navigation bar, Flexbox or Grid? Why?" (Flexbox — one row.)
- "Which would you use for the product listing? Why?" (Grid — rows *and* columns.)
- "What does `gap: 24px` do that `margin` would not?" (It adds space only *between* items, not around the outside, and it does not add space after the last item.)

G.5 Common errors to inject and correct

Injected error	Expected result	How to correct
`display: flexbox;` (wrong keyword)	No flex layout; links stay stacked	It must be `display: flex;` (the value is `flex`, not `flexbox`).
Forgetting `gap` and using `margin` on only some items	Uneven spacing, extra space at the ends	Use `gap` on the container.
`grid-template-columns: 1fr 1fr 1fr` without `display: grid`	No columns at all	`grid-template-columns` only works once the parent is a grid container.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners build a small layout for *Adaeze's Fashion Design*: a Flexbox navigation bar and a Flexbox card row, using the box model and box-sizing.

Starting materials: A folder on each learner's Desktop called `adaeze-fashion`, containing an empty `index.html` and an empty `css/style.css`. The instructor shows both.

Procedure (instructor calls out each numbered step; learners follow on their machines; the instructor walks the room):

1. In `css/style.css`, add the global box-sizing rule:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #222;
}
```

2. In `index.html`, add the page structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="topbar">
```

```
    <a href="index.html" class="logo">Adaeze's Fashion</a>
    <nav>
      <a href="index.html">Home</a>
      <a href="shop.html">Shop</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <h1>Welcome to Adaeze's Fashion</h1>
    <div class="cards">
      <div class="card"><h3>Ankara Dress</h3><p>₦18,500</p></div>
      <div class="card"><h3>Lace Gown</h3><p>₦32,000</p></div>
      <div class="card"><h3>Aso Ebi Set</h3><p>₦27,500</p></div>
    </div>
  </main>
</body>
</html>
```

3. Style the top bar with Flexbox. Add to `style.css`:

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo {
  color: #f4c542;
  font-size: 22px;
  font-weight: bold;
  text-decoration: none;
}
.topbar nav {
  display: flex;
  gap: 24px;
}
.topbar nav a {
  color: white;
  text-decoration: none;
}
```

4. Style the cards. Add:

```
main {
  max-width: 1000px;
  margin: 0 auto;
  padding: 0 16px;
}
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
.card {
  flex: 1 1 220px;
  padding: 18px;
  border: 1px solid #ddd;
  border-radius: 10px;
  background-color: #faf7ff;
}
```

5. Save both files (`Ctrl + S`).

6. Open `index.html` in Chrome. Refresh.

7. **Pause and predict:** "Before I refresh, what will the top of the page look like?" Have learners write a one-sentence prediction. Then refresh and compare.

Expected result: A purple bar at the top with the gold name *Adaeze's Fashion* on the left and four white menu links on the right, spaced out. Below, the heading *Welcome to Adaeze's Fashion*, then three card boxes side by side in a row, each showing a product name and a naira price. If the window is narrowed, the three cards wrap onto two lines.

Points to pause on and predict:

- After step 3: "*Which property moves the menu to the right?*" → `justify-content: space-between`.
- After step 4: "*What happens to the cards if I drag the window very narrow?*" → They wrap and stack, because of `flex-wrap: wrap`.
- Near the end: "*What colour is the logo text?*" → Gold (`#f4c542`), because the `.logo` class sets it.

Troubleshooting while walking the room:

- Menu still stacked vertically → check `.topbar nav { display: flex; }` is present and saved.
- Cards overlapping or touching → check `gap: 20px;` is on `.cards`, not on `.card`.
- Naira sign shows as `?` → save the HTML as UTF-8 (see Session K.6 pattern; VS Code bottom-right encoding).
- Blue default link colour on the menu → add `color: white;` to `.topbar nav a`.
- Whole page has a white fringe around it → add `body { margin: 0; }`.

Completion checklist (learner ticks each):

- [ ] `* { box-sizing: border-box; }` at the top of the stylesheet
- [ ] `.topbar` uses `display: flex` with `justify-content: space-between`
- [ ] `.topbar nav` uses `display: flex` with a `gap`
- [ ] Logo styled gold on a purple bar
- [ ] Menu links are white with no underline
- [ ] `.cards` uses `display: flex`, `flex-wrap: wrap` and `gap`
- [ ] Each card uses `flex: 1 1 220px`
- [ ] Page saved, refreshed and shows the layout described above

---

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: On your own, build a **services page layout** for *Adaeze's Fashion Design* using Flexbox for the header row and **Grid** for the services.

Scenario: Adaeze also trains new tailors. She wants a simple page listing three training services: *Beginner Sewing Class*, *Ankara Design Workshop* and *Business of Fashion*. Each service is a card with a title and a short sentence. The page also needs the same top bar from the practical exercise.

Starting information (given):

- Business name: `Adaeze's Fashion`.
- Three services (use these exact titles and descriptions):
  1. **Beginner Sewing Class** — "Learn to cut, sew and finish a simple garment in 6 weeks."
  2. **Ankara Design Workshop** — "Turn beautiful Ankara fabric into modern styles."
  3. **Business of Fashion** — "Price your work, find customers and grow your brand."
- Brand colours: deep purple `#4b2e83` and gold `#f4c542`.

Requirements:

1. Create a new file `services.html` and link the same `css/style.css`.
2. Reuse the `.topbar` header from the practical (copy it exactly).
3. Create a `<section class="services">` containing three `<div class="service">` cards, each with an `<h3>` title and a `<p>` description.
4. Use **Grid** (`display: grid`, `grid-template-columns: repeat(3, 1fr)`, `gap`) for `.services`.
5. Give each card padding, a border, a light background and rounded corners.
6. Centre the whole section with `max-width: 1000px; margin: 0 auto;`.
7. Save, open in Chrome and confirm three equal cards side by side.

Constraints:

- Flexbox for the top bar; Grid for the services. Do not mix them up.
- No copying the instructor's exact CSS for the cards — write your own values (colours, padding, radius).
- Keep the code indented so it is easy to read.

Time allowance: 15 minutes.

Success criteria:

- The top bar shows the logo left and menu right.
- Three service cards sit in one row, equal width, with even gaps.
- No red errors, and the page looks tidy at a normal window size.

Hints available if stuck:

- Hint 1: A Grid container needs `display: grid;` before `grid-template-columns` does anything.
- Hint 2: `repeat(3, 1fr)` is exactly the same as `1fr 1fr 1fr`.
- Hint 3: `gap: 20px;` goes on the grid container `.services`, not on each `.service` card.

Instructor answer key (do not show learners):

```
.services {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  max-width: 1000px;
  margin: 40px auto;
  padding: 0 16px;
}
.service {
  padding: 20px;
  border: 1px solid #e0d7f0;
  border-radius: 10px;
  background-color: #faf7ff;
}
.service h3 {
  margin-top: 0;
  color: #4b2e83;
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Services</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="topbar">
    <a href="index.html" class="logo">Adaeze's Fashion</a>
    <nav>
      <a href="index.html">Home</a>
      <a href="shop.html">Shop</a>
      <a href="about.html">About</a>
      <a href="services.html">Services</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <h1>Training Services</h1>
    <section class="services">
      <div class="service">
        <h3>Beginner Sewing Class</h3>
        <p>Learn to cut, sew and finish a simple garment in 6 weeks.</p>
      </div>
      <div class="service">
        <h3>Ankara Design Workshop</h3>
        <p>Turn beautiful Ankara fabric into modern styles.</p>
```

```
      </div>
      <div class="service">
        <h3>Business of Fashion</h3>
        <p>Price your work, find customers and grow your brand.</p>
      </div>
    </section>
  </main>
</body>
</html>
```

Marking guidance (out of 10):

- Top bar uses Flexbox correctly (2)
- Services section uses Grid with three equal columns (3)
- Cards have padding, border, radius and background (2)
- Section is centred with `max-width` and `margin: 0 auto` (1)
- Code is tidy and readable (1)
- No errors in the browser (1)

J. REAL-WORLD CASE STUDY

Situation. *Adaeze* sells her clothes mostly through WhatsApp. A friend builds her a simple website, but the first draft has every product listed in one long vertical column — a photo, then a name, then a price, then the next photo, all the way down the page. On a desktop screen, customers must scroll and scroll to see ten products. Adaeze notices that people open the page and leave quickly.

Relevant facts.

- The website has correct content and correct colours; only the *arrangement* is poor.
- Adaeze's customers often compare several outfits before deciding.
- Adaeze is not a programmer, but she has just learned Flexbox and Grid.

Decision point. How can layout alone change how the shop feels to a customer?

Possible approaches.

1. Leave the long column and hope customers scroll.
2. Use a **Grid** to show products in neat rows of three, so many outfits are visible at once.
3. Pay a company to rebuild the site from scratch.

Recommended solution (for this stage). Approach 2. Converting the single column into a `grid-template-columns: repeat(3, 1fr)` gallery lets a customer see three outfits at a glance and compare them side by side. The content does not change at all — only the layout — yet the page suddenly feels like a real shop.

Example solution:

```
.shop {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  max-width: 1000px;
  margin: 0 auto;
  padding: 0 16px;
}
```

Trade-offs.

- Approach 1 costs nothing but loses customers — the very problem.
- Approach 2 takes about 20 minutes and needs no new content.
- Approach 3 is expensive and unnecessary; the fix is a handful of CSS lines.

Professional connection. Junior web developers earn money fixing exactly this kind of problem for small businesses: the content is fine, the layout is the issue. Knowing Flexbox and Grid is one of the most employable front-end skills because almost

every real page needs some arrangement of rows and columns.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the session's live troubleshooting guide. When a learner hits an issue, do not give the answer immediately — ask them to check the relevant row.

K.1 Flexbox "not working" — items still stacked

Symptoms: You wrote `display: flex;` but everything still sits in one vertical column. **Likely causes:** (a) you put `display: flex` on the *child* instead of the *parent*; (b) the rule is not saved; (c) a selector does not match any element; (d) a typo such as `display: flexbox;`. **How to investigate:** In DevTools → Elements, click the parent element and check the Styles pane for your rule. If it is greyed out or missing, the selector or spelling is wrong. **Safe corrective steps:** Put `display: flex` on the container that holds the items; fix the value to exactly `flex`; save and refresh. **How to confirm resolved:** The children line up in a row. **How to prevent recurrence:** Always ask "am I styling the container or the item?"

K.2 Widths are bigger than expected

Symptoms: You set `width: 300px` but the box measures 350px or more. **Likely cause:** `box-sizing` is not set, so padding and border are added on top of the width. **How to investigate:** Check the box-model diagram in DevTools — the numbers add up. **Safe corrective steps:** Add `* { box-sizing: border-box; }` at the top of the stylesheet. **How to confirm resolved:** The total width now matches the value you set. **How to prevent recurrence:** Put the box-sizing rule at the top of *every* stylesheet you write.

K.3 Cards or columns overflow the page (horizontal scrollbar)

Symptoms: A horizontal scrollbar appears and content runs off the right side. **Likely causes:** A fixed `width` bigger than the screen, an image that is too wide, or `flex-wrap` missing so items refuse to drop to the next line. **How to investigate:** Drag the window narrow and watch which element runs off the edge. **Safe corrective steps:** Replace fixed widths with `max-width`; add `flex-wrap: wrap;`; add `img { max-width: 100%; }`. **How to confirm resolved:** The scrollbar disappears and content stays inside the window. **How to prevent recurrence:** Prefer `max-width` and percentages; avoid fixed widths on layout containers.

K.4 `gap` adds no space

Symptoms: You set `gap` but the items touch each other. **Likely cause:** You put `gap` on an item or on a non-flex/grid element. **How to investigate:** Check that the element with `gap` also has `display: flex` or `display: grid`. **Safe corrective steps:** Move `gap` to the flex/grid container. **How to confirm resolved:** Even space appears between items. **How to prevent recurrence:** Remember: spacing *between* items belongs on the container.

K.5 Grid created only one column

Symptoms: You expected three columns but got one, or items stack. **Likely cause:** `grid-template-columns` is present but `display: grid` is missing, or a typo in the value. **How to investigate:** In DevTools, look for the grid badge next to the element; if there is none, it is not a grid. **Safe corrective steps:** Add `display: grid;` above `grid-template-columns`. **How to confirm resolved:** The grid badge appears and columns form.

K.6 The whole page has a white border around it

Symptoms: Unwanted blank space around the entire page. **Likely cause:** The browser's default `body` margin (usually 8px). **Safe corrective steps:** Add `body { margin: 0; }`. **How to confirm resolved:** The layout touches the edges of the window. **How to prevent recurrence:** Include `body { margin: 0; }` in your base styles.

Distinguish likely vs confirmed: In every case above, the *likely* cause becomes *confirmed* only after you check it in DevTools — for example, seeing that your rule is missing, or that the box-model numbers add up wrong.

L. SESSION ASSIGNMENT

Assignment title: "My Flexbox and Grid Layout Page"

Purpose: Practise arranging elements with Flexbox and Grid, applying the box model and box-sizing, and using `display` values deliberately.

Realistic scenario: You are the web designer for *Adaeze's Fashion Design* in Port Harcourt. Adaeze wants a simple "Lookbook" page that shows her top six outfits in a neat grid, with the same purple-and-gold top bar the site will use everywhere.

Detailed instructions:

1. Create a folder `assignment5` containing `index.html` and `css/style.css`.
2. In `index.html`:
   - Add a `<header class="topbar">` with the business name on the left and links (Home, Shop, About, Contact) on the right.
   - Add an `<h1>`: "Our Lookbook".
   - Add a `<section class="lookbook">` containing **six** `<article class="item">` cards. Each card has an `<h3>` (outfit name) and a `<p class="price">` (a naira price). Invent realistic names and prices.
3. In `css/style.css`:
   - Start with `* { box-sizing: border-box; }` and `body { margin: 0; }`.
   - Use **Flexbox** for `.topbar` with `justify-content: space-between` and `align-items: center`.
   - Use **Grid** for `.lookbook` with `grid-template-columns: repeat(3, 1fr)` and a `gap`.
   - Give each `.item` padding, a border, rounded corners and a light background.
   - Centre the lookbook with `max-width: 1000px; margin: 0 auto;`.
4. Save, open in Chrome and check the layout.

Required deliverables:

- `assignment5/index.html`
- `assignment5/css/style.css`
- A short note (3–4 sentences) in your workbook explaining why you used Flexbox for the top bar and Grid for the lookbook.

Submission format: Files zipped or placed in the class shared folder; the note written or photographed into the workbook.

Deadline: Before Session 6.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
File structure & linking	2	`index.html` in root, `style.css` in `css/`, linked correctly
Box sizing applied	2	`* { box-sizing: border-box; }` present and works
Flexbox top bar	4	Logo left, menu right, vertically aligned, even gaps
Grid lookbook	4	Six cards in a 3-column grid with even gaps
Card styling	3	Padding, border, radius and background set sensibly
Centring & max-width	2	Lookbook centred, content max 1000px
Reasoning note	2	Clear, correct explanation of Flexbox vs Grid choice
Code quality	1	Indented, readable, no stray errors

Common mistakes to avoid: missing `display: flex` / `display: grid`; putting `gap` on the wrong element; forgetting box-sizing; fixed widths that overflow.

Instructor reference solution:

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; color: #222; }

.topbar {
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo { color: #f4c542; font-size: 22px; font-weight: bold; text-decoration: none; }
.topbar nav { display: flex; gap: 24px; }
.topbar nav a { color: white; text-decoration: none; }

main { max-width: 1000px; margin: 0 auto; padding: 0 16px; }
h1 { color: #4b2e83; }

.lookbook {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  margin-bottom: 40px;
}
.item {
  padding: 18px;
  border: 1px solid #e0d7f0;
  border-radius: 10px;
  background-color: #faf7ff;
}
.item h3 { margin-top: 0; color: #4b2e83; }
.price { color: #b8860b; font-weight: bold; margin-bottom: 0; }
```

The note should say something like: "Flexbox is one-directional, so it is perfect for the top bar where the logo and menu sit in a single row. Grid is two-directional, so it is perfect for the lookbook, where I need rows and columns of equal cards at once."

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. Name the four layers of the CSS box model, from the inside out.

Q2. Multiple choice. Which value makes an element sit in the text flow but still accept a width and height? a) `display: block;` b) `display: inline;` c) `display: inline-block;` d) `display: none;`

Q3. Explain in your own words. What does `box-sizing: border-box;` change, and why do we put it at the top of every stylesheet?

Q4. Draw/describe. You set `display: flex;` on a container. In words, where is the main axis and where is the cross axis by default? Which property spaces items along the main axis?

Q5. Practical. Write the two CSS declaration lines that would centre a single item both horizontally and vertically inside a flex container.

Q6. Choose the tool. For each job, write "Flexbox" or "Grid" and give a one-line reason: a) A row of navigation links. b) A three-column photo gallery. c) Centring a business name in a hero banner.

Q7. Error detection. This CSS does not produce columns. What is missing?

```
.gallery {
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. Content, padding, border, margin.

A2. (c) `display: inline-block;` — it sits in the text flow like inline but accepts width and height like block.

A3. It makes the element's `width` include its padding and border, instead of adding them on top. We put it at the top of every stylesheet so all boxes are sized predictably and widths mean what they say.

A4. For a row (the default), the **main axis runs left to right** and the **cross axis runs top to bottom**. `justify-content` spaces items along the main axis.

A5.

```
justify-content: center;
align-items: center;
```

(Accept also a full flex container example using `display: flex;`.)

A6.

- a) Flexbox — a single row (one direction).
- b) Grid — needs rows *and* columns (two directions).
- c) Flexbox — centring on two axes with `justify-content` / `align-items`.

A7. It is missing `display: grid;`. Without it, `grid-template-columns` has no grid to apply to.

N. REFLECTION AND RECAP

Plain-English summary. Today you learned to control *where* elements sit on a page. You met **normal flow** (the browser's default stacking), the `display` property (block, inline, inline-block, none), and sizing with `width`, `height` and `max-width`. You learned the **box model** — content, padding, border, margin — and the crucial fix `box-sizing: border-box;`. You saw why **float** is now legacy for layout, then learned **Flexbox** (one direction, with `justify-content`, `align-items`, `gap`, `flex-wrap`) and the basics of **Grid** (rows and columns at once). Finally you built two real layouts: a navigation bar and a product grid.

Five important takeaways:

1. Block elements stack; inline elements sit side by side; `inline-block` does both.
2. Every element is a box: content + padding + border + margin.
3. `* { box-sizing: border-box; }` should start every stylesheet.
4. Flexbox for one direction, Grid for two.
5. Centring both ways is just `display: flex; justify-content: center; align-items: center;`.

Questions to help you recognise what you can now do:

- Can I make two boxes sit side by side without a table or a float?
- Can I explain why my box is wider than the width I set?
- Can I build a top bar with the logo left and menu right, from memory?
- Can I choose correctly between Flexbox and Grid and say why?

Connection to the next session: Right now these layouts look great on a big desktop screen. But what happens on a small phone? Next session we make the layout **responsive** — adapting automatically to phone, tablet and desktop using relative units and media queries.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- The learner puts `display: flex` / `grid` on the **parent** container without being reminded.
- The learner can explain, without notes, why a box measured 350px when set to 300px.
- The learner chooses Flexbox or Grid with a reason ("this is one row" vs "this needs rows and columns").
- The learner uses `gap` on the container rather than margins on splayed items.
- The learner uses the DevTools box diagram to diagnose a sizing problem.

Common misconceptions to watch for and correct:

- "Flexbox goes on the children." → It goes on the parent; children become items automatically.
- "Padding and margin are the same." → Padding is inside, margin is outside.
- "The value is `display: flexbox`." → The correct value is `display: flex`.
- "Grid and Flexbox are rivals; pick one." → They are partners; most sites use both.

- "Float is evil." → Float is just no longer the right tool for layout; it still wraps text around images.

Suggested remedial exercises for learners who struggle:

1. **Box-model tracing:** give three elements with the same `width` but different padding and border; ask learners to predict the total width, then check in DevTools.
2. **Flex/Grid sorting:** show six one-line layouts and ask learners to sort them into "Flexbox" and "Grid".
3. **Fix the stacked menu:** hand out a header where `display: flex` is on the `<a>` instead of the `<nav>`; ask learners to find and fix it.

Differentiation:

- **Level 1 (supported):** provide the completed `index.html` and a `style.css` with the top-bar CSS already written but the card CSS missing; guide them to finish it.
- **Level 2 (standard):** the practical and independent exercises as written.
- **Level 3 (challenge):** add a second row using `align-self` on one card to test vertical alignment, then rebuild the same lookbook with Grid `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` and observe how it reflows.

Timing guidance: If the room is slow, shorten the case study (Section J) to a 3-minute mention and protect the guided practical — the hands-on layout work is the priority. If the room is fast, extend the DevTools grid-inspector time in Section G.4.

APPENDIX 5A — 15-MINUTE CSS / HTML REFRESHER (only if needed)

If learners cannot link a stylesheet or write a basic rule, run this before Section D.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Refresher</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Adaeze's Fashion</h1>
  <p class="intro">Custom clothing in Port Harcourt.</p>
  <a href="shop.html">Shop now</a>
</body>
</html>
```

```
h1 { color: #4b2e83; }
.intro { color: #555; font-size: 18px; }
a { color: #b8860b; }
```

Key points to confirm: the `.css` file is linked in the `<head>` with the correct path; a rule has a selector, property and value; the `.intro` class selector uses a dot; save and refresh before checking. Only proceed once every learner can change the `<h1>` colour.

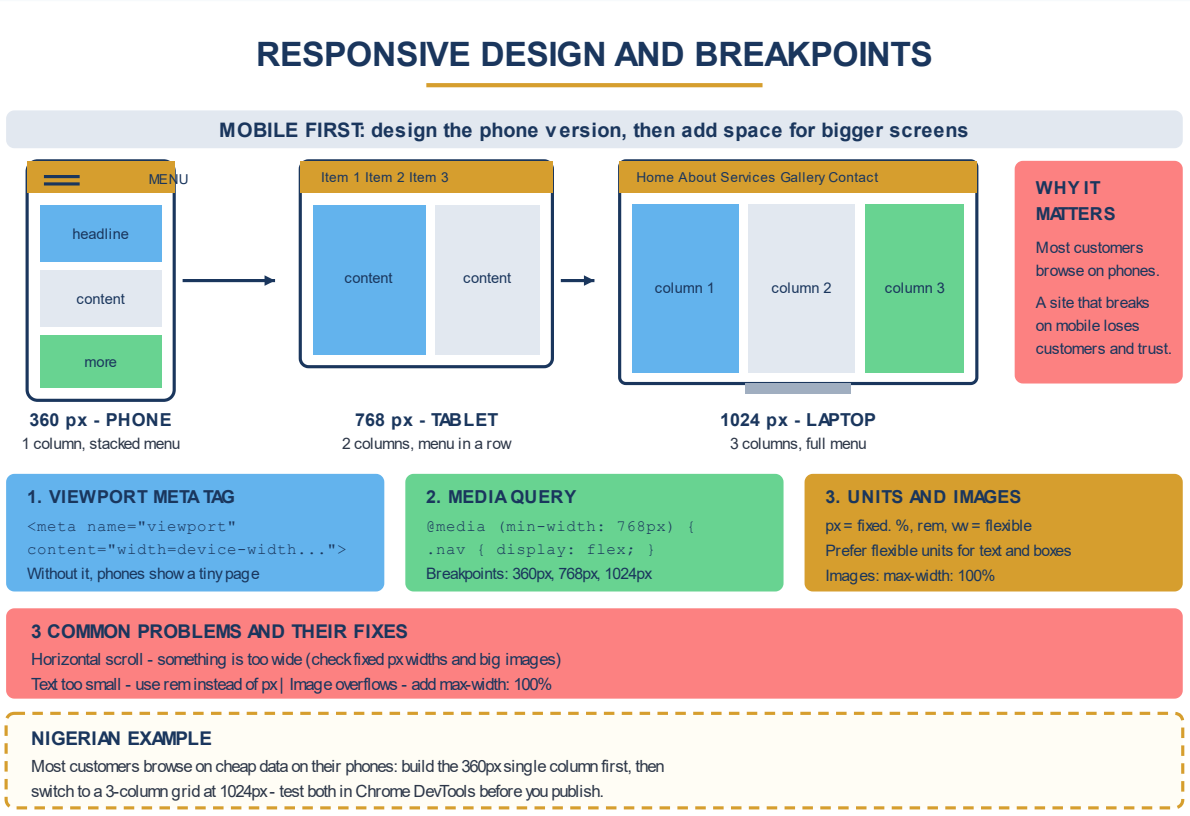
END OF SESSION 5

Next: Session 6 — CSS Responsive Design.

SESSION 06 OF 12

CSS RESPONSIVE DESIGN

CONCEPT DIAGRAM — SESSION 06



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 6 — CSS RESPONSIVE DESIGN

A. SESSION IDENTIFICATION

Item	Detail
Session number	6 of 12
Session title	CSS Responsive Design
Week / Module	Week 2 — Module 2: CSS Foundations
Recommended duration	105 minutes
Difficulty level	Beginner (comfortable with Flexbox and Grid from Session 5)

Item	Detail
Prerequisite knowledge	Sessions 1–5. The learner must be able to build a page with a Flexbox navigation bar and a Grid or Flexbox card row, and apply `box-sizing: border-box`. If a learner cannot make two boxes sit side by side, run the 15-minute refresher in Appendix 6A before this session.
Connection to previous work	In Session 5 learners arranged elements into rows and columns. Those layouts looked good in a wide desktop window — but on a phone they break. This session makes every layout adapt automatically to phones, tablets and desktops.
Preparation before class	Academy PCs on and logged in; VS Code and Chrome installed; the `adaeze-fashion` folder from Session 5 available on each machine; the projector set to show the instructor's Chrome with the DevTools **device toolbar** ready; print the "Responsive Design Checklist" (one per learner). If the academy has one smartphone or tablet, keep it handy for the real-device test.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain in their own words** what responsive design means and why it matters for a Nigerian business whose customers mostly browse on phones.
2. **Add the viewport meta tag** to an HTML page and describe what changes on a phone without it.
3. **Explain mobile-first thinking** and describe the difference between starting with mobile styles and starting with desktop styles.
4. **Choose correctly between fixed units (`px`) and relative units** (`%`, `rem`, `em`, `vw`, `vh`) for a given property, and justify the choice.
5. **Make images and containers fluid** using `max-width: 100%`, `width: 90%` and `max-width` on wrappers.
6. **Write a media query** using `min-width` and `max-width`, and name the common breakpoints (360px, 768px, 1024px).
7. **Build a responsive navigation bar** that changes from a row on desktop to a stacked menu on a phone.
8. **Build a responsive card grid** that goes from three columns on desktop to one column on a phone.
9. **Test a layout** using the Chrome DevTools device toolbar and interpret what they see.
10. **Diagnose three common responsive problems** — horizontal scroll, text too small, images overflowing — and describe the fix for each.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). A shared computer between two learners is acceptable; pair them so one types and one checks the screen, then swap halfway.
- Instructor computer connected to a projector or large screen.
- Optional: one smartphone or tablet for the real-device test at the end.

Software

- Google Chrome (any recent version). The **device toolbar** (also called device mode) described here is present in all modern Chrome versions.
- Visual Studio Code (VS Code). Any version from 2022 onwards works.
- Windows 10 or Windows 11.

Accounts

- None required. All work this session is offline.

Sample files

- The `adaeze-fashion` folder from Session 5, containing `index.html`, `css/style.css` and an `images/` folder. The instructor prepares a copy of the Session 5 result in case a learner did not complete it.

Worksheets

- Worksheet 6.1 — "Mobile or Desktop First?" (printed, 1 page).
- Worksheet 6.2 — "Breakpoint Planner" (in the learner workbook, Part 3).

Tools

- Chrome DevTools → device toolbar (`Ctrl + Shift + M`). This session depends on it.

Internet

- Not required for this session. If the academy has internet, the instructor may optionally show one well-known responsive site (for example a bank or e-commerce site) resized in the device toolbar, but do not depend on it.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. Learners with only a phone can pair with a desktop user and later test their own work on the academy smartphone.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the *Adaeze's Fashion* homepage from Session 5 — the one with a purple top bar and a three-column card grid. Then slowly drag the browser window narrower, as if it were a phone screen. The top bar's menu wraps awkwardly, the three cards squash into unreadable slivers, and a horizontal scrollbar may appear at the bottom.

What the instructor says:

> *"Look at this. On the big screen it was beautiful. But watch what happens when I make the window narrow — like a phone. [Drag it narrow.] The menu breaks. The cards flatten. You cannot read it. And remember: most people in Nigeria who visit a business website are on a phone, not a laptop. So a website that only looks good on a laptop is a website that loses most of its customers.*
>
> *Today we fix this. Today we learn **responsive design** — building one website that looks good on every screen, big or small."*

Expected learner response: Learners will visibly see the layout breaking. Some may suggest "make two separate websites" — accept the idea and gently explain that responsive design lets **one** website serve everyone, which is cheaper and easier to update.

Then demonstrate the hook: Press `Ctrl + Shift + M` in Chrome to open the **device toolbar**, and choose "iPhone SE" (or set a custom width of 375px). Say:

> *"This little button turns Chrome into a phone simulator. From now on, every page you build, you will check here — on a phone view — before you call it finished. This is how professionals test. Let us begin."*

Why this opener works: It turns an abstract idea ("responsive") into a visible, painful problem the learner can see in ten seconds, and it immediately gives them the tool (device toolbar) they will use all session.

E. PREREQUISITE REVIEW (7 minutes)

Run this as an oral round-robin. Put the six questions on the projector.

Question 1: "What two CSS properties would I put on a container to make its children sit in a row?"

- Expected answer: `display: flex;` (and optionally `gap`). Accept Grid with `display: grid; grid-template-columns: ...`.
- Common gap: learners write `flex: 1` on the parent. Restate: `display: flex` goes on the parent.

Question 2: "What does `justify-content: space-between` do?"

- Expected answer: it spaces flex items along the main axis, pushing the first item to the start and the last to the end.
- Common gap: confusing it with `align-items`. Remind them: main axis vs cross axis.

Question 3: "Why do we put `* { box-sizing: border-box; }` at the top of our CSS?"

- Expected answer: so that width includes padding and border, making box sizes predictable.
- Common gap: "I forgot." Reassure them it is easy to add and it will matter today when sizing changes by screen.

Question 4: "What is `max-width` and how is it different from `width`?"

- Expected answer: `max-width` sets an upper limit — the element can be narrower but never wider. `width` sets a fixed size.
- Common gap: thinking they do the same thing. Clarify: `max-width` shrinks, `width` does not.

Question 5: "What unit makes an element a percentage of its parent's width?"

- Expected answer: the `%` (percentage) unit.
- Common gap: writing `%` after a space, or forgetting that `%` is relative to the parent, not the screen.

Question 6: "If I want an element to be as tall as the whole browser window, what unit might I use?"

- Expected answer: `vh` (viewport height), e.g. `height: 100vh;`.
- Common gap: confusing `vh` with `%`. Remind them: viewport units relate to the screen; percentages relate to the parent.

Bridge to today (say this):

> *"Every layout we built last session used fixed ideas — a row here, three columns there — as if everyone had the same screen. Today we make those layouts think. We tell the browser: if the screen is small, do this; if it is big, do that. That way one website welcomes the person on a ₦20,000 phone and the person on a big office monitor."*

If more than a third of the class cannot answer Question 1, spend 10 extra minutes on Appendix 6A before continuing.

F. FULL LECTURE NOTES

F.1 What responsive design is, and why it matters

The idea. Responsive design means building one website that automatically adapts its layout to the size of the screen it is viewed on.

Plain-English definition. Instead of making a separate "phone website" and a "laptop website", you build **one** website whose layout rearranges itself: three product cards side by side on a big screen, one card per row on a small phone.

Why it exists. People browse the web on many different devices — small phones, large phones, tablets, laptops, big desktop monitors. If a site is built for one size only, it looks broken on the others. Responsive design solves this once for everyone.

Why it matters in Nigeria specifically. For many small businesses and their customers, the phone is the main way people get online. A customer in Port Harcourt might search for a fashion designer on a phone while on the go. If Adaeze's website is hard to read or navigate on that phone, the customer simply leaves. Responsive design is therefore not a "nice to have" — it is how a small business keeps the customers it already has.

What the learner recognises. A website is responsive if, when you drag the browser window narrower:

- the text rewraps instead of being cut off,
- the layout changes shape (columns become rows, menus stack),
- images shrink instead of overflowing,
- no horizontal scrollbar appears.

Analogy. Think of a **locust (okro) that adjusts to the pot**. The ingredient is the same, but it spreads to fill whatever pot you cook it in. A responsive website is the same content spread to fit any screen.

Misconception. "Responsive design is only for phones." It is for **all** sizes — big monitors included. It is about *adapting*, not about *mobile only*.

Limitation. Responsive design cannot fix bad content. If an image is too large a file, or the text is confusing, responsiveness only makes those problems fit the screen neatly — it does not solve them.

F.2 The viewport meta tag — the one line that changes everything

The idea. Before any media query can work properly, the browser on a phone needs to be told to use the phone's real screen width.

Plain-English definition. The **viewport** is the visible area of a webpage on a device. The **viewport meta tag** is a line in the `<head>` that tells the phone browser: "use the actual device width as the page width, and do not zoom out."

Why it exists. Early phones tried to show full desktop pages by pretending to be about 980px wide and shrinking everything. The result was tiny unreadable text that the user had to pinch-zoom. The viewport meta tag tells the phone to stop pretending.

The line to add (always, in the `<head>`):

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

What each part means:

- `width=device-width` — make the page as wide as the device's real screen.
- `initial-scale=1.0` — start at 1× zoom (100%), not zoomed out.

Worked example — before and after:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <!-- without the viewport line, phones zoom the page out and text looks tiny -->
  <title>Adaeze's Fashion</title>
</head>
<body>
  <h1>Adaeze's Fashion</h1>
  <p>Custom clothing in Port Harcourt.</p>
</body>
</html>
```

Expected browser result on a phone (or the device toolbar): The page looks like a shrunken desktop — the heading is very small and there is empty space on the right, because the phone assumed a ~980px page and shrank it to fit.

Now add the viewport line:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Expected result after adding it: The heading fills the screen width at a comfortable, readable size. Now media queries will fire at the correct widths, because the page width equals the device width.

Misconception. "The viewport tag makes my site responsive by itself." It does not — it is the *prerequisite*. Without it, your media queries do not behave as expected on phones. With it, your layout is *ready* to be made responsive.

Limitation. The viewport tag does nothing to the layout by itself. It only sets the stage. The actual adapting is done by fluid sizing (F.4) and media queries (F.6).

F.3 Mobile-first thinking

The idea. Write the styles for the small screen **first**, then add styles for larger screens as the screen grows.

Plain-English definition. Mobile-first means your base CSS targets phones (or, more precisely, the smallest layout), and then you layer on extra rules for tablets and desktops using `min-width` media queries.

Why it exists. If you design for desktop first, you end up writing complicated overrides to *undo* desktop styles for phones — often fighting your own CSS. Starting small and adding up is simpler, and it forces you to prioritise what really matters (content), because a phone screen has no room for clutter.

Two ways to think about it:

Approach	Base styles describe…	Media queries use…	Tendency
Mobile-first (recommended)	the phone layout (simple, one column)	`min-width` — "when the screen is at least this wide, add…"	grows outward

Approach	Base styles describe…	Media queries use…	Tendency
Desktop-first (older)	the desktop layout (wide, multi-column)	`max-width` — "when the screen is this narrow, change…"	shrinks inward

Say this to the class:

> *"Mobile-first is like packing for a small bag first: you take only what you truly need. Then, if you get a bigger bag, you add more. If you pack for the biggest bag first and then must fit everything into a small bag, you are forced to throw things out — that is what writing desktop-first CSS feels like."*

Worked example — mobile-first in practice:

```
/* Base = phone layout: cards stack in one column */
.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

/* Tablet and up: two columns */
@media (min-width: 768px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Desktop and up: three columns */
@media (min-width: 1024px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Expected browser result: On a phone, one card per row. Widen the window past 768px and the cards jump to two per row. Widen past 1024px and they become three per row. The code only ever *adds*; it never undoes.

Misconception. "Mobile-first means the site only targets phones." No — the name describes the *order* in which you write CSS, not a limit on devices. The site still looks great on desktop; you simply built up to it.

Limitation. Mobile-first works best when your design genuinely starts from content. For a course this is ideal; but be aware that some very complex older projects are still maintained desktop-first, and you will meet both styles in the wild.

F.4 Relative units versus fixed pixels

The idea. Instead of hard-coding sizes in pixels, use units that scale with the screen, the parent, or the root font size.

Plain-English definition. A **fixed unit** (`px`) always means the same size. A **relative unit** is measured in relation to something else — the parent, the screen, or the base font.

Why it exists. A `600px` container is fine on a laptop and overflowing on a phone. Relative units let the same rule shrink gracefully.

The units you need this week:

Unit	Relative to…	Example	When to use it
`px`	Nothing — fixed	`border: 2px solid;`	Small fixed details: borders, thin lines
`%`	The parent element's width	`width: 90%;`	Container widths, columns
`rem`	The **root** font size (usually 16px)	`font-size: 1.5rem;`	Font sizes and spacing — predictable and accessible
`em`	The **parent's** font size	`padding: 1.5em;`	Spacing relative to the current text size
`vw`	1% of the viewport (screen) width	`width: 100vw;`	Full-width sections (use carefully)

Unit	Relative to...	Example	When to use it
`vh`	1% of the viewport height	`min-height: 100vh;`	Full-screen hero areas

The base font size trick: the browser default root font size is **16px**. So `1rem = 16px`, `1.5rem = 24px`, `2rem = 32px`. If a user has set a larger default font for accessibility, `rem` respects that — another reason `rem` is kinder than fixed `px` for text.

Worked example — the same layout, fixed versus fluid:

```
/* Fixed — breaks on small screens */
.fixed-hero {
  width: 960px;
  padding: 40px;
  font-size: 48px;
}

/* Fluid — adapts everywhere */
.fluid-hero {
  width: 90%;
  max-width: 960px;
  padding: 2rem;
  font-size: clamp(1.6rem, 5vw, 3rem);
}
```

Expected browser result: On a 320px phone, `.fixed-hero` is 960px wide and overflows the screen badly (horizontal scrollbar), with huge text. `.fluid-hero` is 90% of the screen (about 288px), has comfortable padding, and its heading scales to a readable size between roughly 26px and 48px. On a wide desktop, both end up around 960px — but only the fluid one behaves at both extremes.

A note on `clamp()`. `clamp(minimum, preferred, maximum)` is a modern way to make a value fluid. `clamp(1.6rem, 5vw, 3rem)` means "never smaller than 1.6rem, ideally 5% of the screen width, never bigger than 3rem". It is a neat one-line solution for responsive font sizes. If it feels advanced, remember it, but you can also simply override font sizes inside media queries.

Misconception. "Percentages are always better than pixels." Not always. Borders, thin dividers and small icons are fine in `px` because you do *not* want them changing size. The skill is **choosing**, not avoiding pixels.

Limitation. `vw` can cause overflow if you also add padding or borders, because `100vw` includes the scrollbar width on some systems and ignores padding. Prefer `width: 100%` for full-width elements; use viewport units mainly for heights and fluid font sizes.

F.5 Fluid images and containers

The idea. Images are the most common cause of a broken mobile layout. One simple rule tames them.

Plain-English definition. A **fluid image** is one that never grows wider than its container but keeps its proportions.

Why it exists. Photos are often saved at desktop size (say 1200px wide). On a 360px phone, a 1200px photo overflows and creates a horizontal scrollbar — the classic "broken mobile site" symptom.

The two-line fix (put it near the top of every stylesheet):

```
img {
  max-width: 100%;
  height: auto;
}
```

What each line does:

- `max-width: 100%` — the image may be smaller than its container but never wider. So it shrinks to fit a phone.
- `height: auto` — keep the height proportional as the width shrinks, so the image is not stretched or squashed.

Worked example — image overflow, fixed:

```
<div class="product">
  <img src="images/ankara-dress.jpg" alt="Ankara dress made by Adaeze's Fashion">
```

```
  <h3>Ankara Dress</h3>
</div>
```

```
/* Without this rule the image may overflow */
.product img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

Expected browser result: On a wide screen the dress photo is large; on a narrow phone it shrinks to fit the card width, keeping its shape. No horizontal scrollbar. The `display: block;` also removes the small gap that inline images leave underneath (because they sit on the text baseline).

Fluid containers (wrappers). The same idea applies to the whole content area:

```
.container {
  width: 90%;            /* 90% of the parent, leaves breathing space */
  max-width: 1100px;     /* but never wider than 1100px on huge screens */
  margin: 0 auto;        /* centres the container */
  padding: 0 16px;       /* small side padding on phones */
}
```

Expected browser result: On a phone, the container is 90% of the screen with 16px inner padding, so text does not touch the edges. On a large monitor, it stops growing at 1100px and sits centred, so lines do not become uncomfortably long to read.

Misconception. "`max-width: 100%` makes the image blurry." It does not. It scales the same image down; blurriness comes from scaling a *small* image *up*, which this rule never does.

Limitation. `max-width: 100%` fixes overflow but not file size. A 5 MB photo is still slow to load on a phone even if it fits. Optimising image file size is covered in Session 11.

F.6 Media queries — telling the browser "when screen is this wide, do this"

The idea. A **media query** is a block of CSS that only applies when a condition is true — usually a condition about the screen width.

Plain-English definition. A media query is an "if" statement for CSS. It says: *if* the screen is at least (or at most) a certain width, apply these rules.

Why it exists. Some changes cannot be done with fluid units alone. For example, "when there is room, show the menu as a row; when there is not, stack it" needs an explicit condition. Media queries provide that condition.

The syntax:

```
@media (min-width: 768px) {
  /* rules in here apply only when the screen is 768px wide OR WIDER */
}

@media (max-width: 767px) {
  /* rules in here apply only when the screen is 767px wide OR NARROWER */
}
```

Reading it aloud: "at media, minimum width 768 pixels" = "when the viewport is at least 768px wide". "at media, maximum width 767 pixels" = "when the viewport is at most 767px wide".

Common breakpoints and what they represent:

Breakpoint	Typical device	Meaning
0 – 479px	Older/small phones	Narrowest layouts
480 – 767px	Most modern phones (e.g. ~360–430px wide)	One column, stacked menu
768 – 1023px	Tablets	Two columns, roomier

Breakpoint	Typical device	Meaning
1024px and up	Laptops and desktops	Three columns, horizontal menu

Why 360px, 768px and 1024px? These are widely used device sizes: many Android phones report a CSS width near 360px, tablets near 768px, and laptops/desktops from 1024px upward. They are **guidelines**, not strict laws — the best breakpoint is wherever *your* layout starts to look cramped.

Say this to the class:

> *"Do not create a media query for every phone on earth. Choose two or three breakpoints where your layout needs to change, and use those. A phone, a tablet and a desktop is enough for our project."*

Worked example — a responsive card grid (mobile-first):

```
.cards {
  display: grid;
  grid-template-columns: 1fr;      /* phone: one column */
  gap: 16px;
}

@media (min-width: 768px) {         /* tablet and up */
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1024px) {        /* desktop and up */
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Expected browser result: Resize the window slowly and watch the cards reflow: one per row, then two, then three. This is the exact behaviour of professional e-commerce sites.

Misconception. "Media queries must be in a separate file." They can be, but most beginners keep them at the **bottom of the same stylesheet**, after the base styles. There they override earlier rules of equal specificity, which is exactly what we want.

Limitation. Media queries only respond to the *viewport* size, not to the size of a parent container. (Advanced "container queries" exist but are beyond this course.) For our 3–5 page business site, viewport-based media queries are more than enough.

F.7 Building a responsive navigation bar

The idea. The navigation bar is the most important responsive element — if the menu is broken, the whole site feels broken.

The pattern: On desktop, show the logo and the menu links in one horizontal row. On a phone, stack the links vertically (or centre them) so they are easy to tap with a thumb.

Worked example:

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;                  /* let the menu drop below the logo if needed */
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo { color: #f4c542; font-size: 22px; font-weight: bold; text-decoration: none; }
.topbar nav {
  display: flex;
  gap: 24px;
}
.topbar nav a { color: white; text-decoration: none; padding: 6px 4px; }

/* Phone: stack the menu, centre it, and give links room to tap */
```

```
@media (max-width: 767px) {
  .topbar {
    flex-direction: column;
    align-items: center;
    gap: 12px;
  }
  .topbar nav {
    flex-wrap: wrap;
    justify-content: center;
    gap: 14px;
  }
}
```

Expected browser result: On a wide desktop, the gold logo sits left and the four white links sit right. On a phone (device toolbar at 375px), the top bar becomes a centred column: the logo on top, then the four links wrapped and centred beneath it, each with comfortable padding so a thumb can tap them. Nothing overflows.

Why `padding` on the links matters on mobile: fingers are less precise than a mouse. Adding padding increases the tap area, which is one of the simplest accessibility wins available.

A note on hamburger menus. A "hamburger menu" (the three-line icon) that opens and closes needs either JavaScript (Session 10) or a CSS-only checkbox trick (advanced). For this course, a **stacked, centred menu** is a perfectly professional mobile solution and needs no JavaScript. We revisit the hamburger in Session 10.

Misconception. "If I just use Flexbox with `flex-wrap`, I do not need a media query." Sometimes true — but a media query gives you deliberate control (like centring the stacked menu) rather than leaving the wrap to chance.

F.8 Testing with the Chrome DevTools device toolbar

The idea. Chrome can pretend to be a phone so you can test without owning one.

How to open it:

1. Open your page in Chrome.
2. Press `F12` (or right-click → **Inspect**).
3. Click the **Toggle device toolbar** icon (a small phone/tablet icon at the top-left of the DevTools panel), or press `Ctrl + Shift + M`.
4. The page view becomes a device simulator. A dropdown at the top lets you choose a device (e.g. iPhone SE, iPhone 12 Pro, Pixel, iPad) or **Responsive** (where you can drag the width freely).
5. Drag the handle on the right edge of the page to change the width, or type a width in the box.

What to look for while testing:

- A **horizontal scrollbar** at the bottom → something is too wide (often a fixed-width element or an image without `max-width: 100%`).
- **Text that is too small** to read → likely a missing viewport meta tag or hard-coded tiny fonts.
- A menu that **overlaps** the logo or wraps badly → navigation needs a media query.
- Cards that **squash into thin slivers** instead of stacking → the grid needs a mobile column rule.

Worked example — the test routine (do this for every page):

1. Open the page in Chrome.
2. `Ctrl + Shift + M` to open device mode.
3. Set the width to **360px** (a common small phone). Check: no horizontal scroll; text readable; menu stacked.
4. Set the width to **768px** (tablet). Check: layout has more room; two columns if expected.
5. Set the width to **1280px** (desktop). Check: three columns; horizontal menu.
6. If the academy has a real phone, open the page on it as the final confirmation.

Expected browser result: By the end, the page looks correct and intentional at 360px, 768px and 1280px, with no horizontal scrollbar at any width.

Misconception. "If it looks good in the device toolbar, it is definitely fine on real phones." The toolbar is an excellent estimate, but real devices differ in fonts, pixel density and browser behaviour. Use both when possible.

Limitation. The device toolbar changes the *rendered viewport*, not the network speed or the actual touch input. It cannot tell you whether the site is slow to load, and it cannot truly test finger taps.

F.9 Putting it together — a fully responsive small page

Bring everything from this session into one place.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Shop</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="topbar">
    <a href="index.html" class="logo">Adaeze's Fashion</a>
    <nav>
      <a href="index.html">Home</a>
      <a href="shop.html">Shop</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main class="container">
    <h1>Our Collection</h1>
    <section class="cards">
      <article class="card"><img src="images/dress1.jpg" alt="Ankara dress"><h3>Ankara Dress</h3><p>₦18,500</p></article>
      <article class="card"><img src="images/dress2.jpg" alt="Lace gown"><h3>Lace Gown</h3><p>₦32,000</p></article>
      <article class="card"><img src="images/dress3.jpg" alt="Aso ebi set"><h3>Aso Ebi Set</h3><p>₦27,500</p></article>
    </section>
  </main>
</body>
</html>
```

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; color: #222; }

img { max-width: 100%; height: auto; display: block; }

.container { width: 90%; max-width: 1100px; margin: 0 auto; padding: 0 16px; }

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo { color: #f4c542; font-size: 22px; font-weight: bold; text-decoration: none; }
.topbar nav { display: flex; gap: 24px; }
.topbar nav a { color: white; text-decoration: none; padding: 6px 4px; }

.cards {
  display: grid;
  grid-template-columns: 1fr;         /* phone default */
  gap: 16px;
  margin-bottom: 40px;
}
.card { border: 1px solid #e0d7f0; border-radius: 10px; overflow: hidden; }
.card h3 { margin: 12px 16px 4px; color: #4b2e83; }
.card p  { margin: 0 16px 16px; color: #b8860b; font-weight: bold; }

@media (min-width: 768px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}
```

```
@media (min-width: 1024px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}

@media (max-width: 767px) {
  .topbar { flex-direction: column; align-items: center; gap: 12px; }
  .topbar nav { flex-wrap: wrap; justify-content: center; gap: 14px; }
  h1 { font-size: 1.6rem; }
}
```

Expected browser result:

- **Phone (360px):** purple bar stacked (logo on top, centred menu below), heading of a comfortable size, one product card per row, no horizontal scrollbar.
- **Tablet (768px):** menu returns to a row, cards two per row.
- **Desktop (1280px):** full horizontal bar, cards three per row, container centred at max 1100px.

Misconception. "I need to write everything twice, once for mobile and once for desktop." No — most rules are written **once** (the base), and only the things that must change are repeated inside media queries.

F.10 Summary of the session

- **Responsive design** = one website that adapts to every screen size.
- Always add the **viewport meta tag**: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.
- **Mobile-first** = write the small-screen styles first, then add larger-screen styles with `min-width` media queries.
- Prefer **relative units** (`%` , `rem` , `em` , `vw` , `vh`) for sizes that should adapt; keep `px` for small fixed details.
- Make images fluid with `img { max-width: 100%; height: auto; }` .
- **Media queries** apply CSS conditionally: `@media (min-width: 768px) { ... }` .
- Common breakpoints: **360px, 768px, 1024px** — guidelines, not laws.
- Test every page with the **Chrome device toolbar** (`Ctrl + Shift + M`) at 360px, 768px and 1280px.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A folder: `demo-webdesign/week2-responsive/` with `index.html` , `css/style.css` and an `images/` folder.
- Chrome open to the page, DevTools closed at the start.
- The projector showing Chrome.

G.2 Demonstration 1 — the viewport tag and the "tiny text" problem (5 minutes)

Say: "I will show you the single most common mobile mistake."

1. Open `index.html` **without** a viewport meta tag. Press `Ctrl + Shift + M` and set the width to 375px.
2. Point out that the text is tiny and the content is squeezed into the left portion, with blank space to the right — because the phone thinks the page is ~980px.
3. Add the viewport line to the `<head>` :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

4. Save, and click the device-toolbar refresh. The text jumps to a normal, readable size filling the width.

Ask: "What changed?" → The page now uses the device's real width, so the layout and media queries behave correctly.

Deliberately inject an error: Change `width=device-width` to `width=960` . Refresh in device mode. The text shrinks again. Say: "If you get this value wrong, you are back to a tiny desktop page on a phone." Restore it.

G.3 Demonstration 2 — making the navigation responsive (5 minutes)

Say: "Now let us make the menu behave on a phone."

1. Show the Session 5 top bar on desktop (logo left, four links right).
2. Open device mode at 375px. Point out that the links may crowd or wrap badly.
3. Add the mobile media query from F.7 (stack and centre the menu).
4. Save. In device mode, the menu is now a neat centred column; switch back to desktop width and the row returns.

Ask: "Why did we add padding to the menu links?" → To make them easier to tap with a thumb.

Deliberately inject an error: Add `flex-direction: column;` to `.topbar` outside the media query (so it also affects desktop) and refresh at desktop width. The whole bar stacks on desktop too — wrong. Ask the class how to fix it: move it *inside* the `@media (max-width: 767px)` block. Do so.

G.4 Demonstration 3 — the responsive card grid and device testing (5 minutes)

1. Show the `.cards` grid with `grid-template-columns: 1fr;` as the base.
2. Add the two `min-width` media queries (768px → 2 columns, 1024px → 3 columns).
3. Save. In device mode, slowly drag the width handle from 320px up to 1300px and let the class watch the cards reflow 1 → 2 → 3.
4. Point to the **horizontal scrollbar** if one appears at any width; then add `img { max-width: 100%; height: auto; }` to remove the overflow, and show the scrollbar disappear.

Questions to ask during the demos:

- "What unit did we use for the base card column? Why not `300px`?" → `1fr`; a fixed width would overflow on a phone.
- "Which breakpoint turns the cards into three columns?" → 1024px.
- "What is the fastest way to check a page on a phone size?" → Device toolbar, `Ctrl + Shift + M`.

G.5 Common errors to inject and correct

Injected error	Expected result	How to correct
Missing viewport meta tag	Tiny zoomed-out page on phones	Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.
`max-width: 100%` missing on an image	Horizontal scrollbar on mobile	Add `img { max-width: 100%; height: auto; }`.
Media query `@media screen and (min-width 768px)` (missing colon/ `and`)	The whole block is ignored	Correct syntax: `@media (min-width: 768px)`.
Container `width: 960px;`	Overflow on phones	Replace with `width: 90%; max-width: 960px;`.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners make the Session 5 *Adaeze's Fashion* page responsive: add the viewport tag, fluid images, a mobile-first card grid, and a responsive navigation bar.

Starting materials: Each learner's `adaeze-fashion` folder from Session 5 (with `index.html` and `css/style.css`). The instructor shares a Session 5 result for anyone who needs a starting point.

Procedure (instructor calls out each numbered step; learners follow; the instructor walks the room):

1. Open `index.html`. Confirm the `<head>` contains the viewport meta tag. If not, add it directly under the `<meta charset="UTF-8">` line:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2. In `css/style.css`, make sure the boxes and images are fluid. Add near the top (below the box-sizing rule):

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

3. Ensure the `.cards` container uses a mobile-first single-column base:

```
.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}
```

4. Add the growing breakpoints at the **bottom** of the stylesheet:

```
@media (min-width: 768px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1024px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

5. Add a responsive navigation rule at the bottom, after the desktop base styles:

```
@media (max-width: 767px) {
  .topbar {
    flex-direction: column;
    align-items: center;
    gap: 12px;
  }
  .topbar nav {
    flex-wrap: wrap;
    justify-content: center;
    gap: 14px;
  }
}
```

6. Save (`Ctrl + S`) and refresh the browser.

7. **Pause and predict:** "Before I open the device toolbar — what will change at 375px compared to the full window?" Have learners write a one-line prediction. Then test.

Expected result: At full desktop width, the page shows a horizontal top bar and three cards per row. In the device toolbar at 375px, the top bar stacks (logo on top, menu centred below), the cards appear one per row, the text is readable and there is **no** horizontal scrollbar. At 768px the cards are two per row.

Points to pause on and predict:

- After step 4: *"At exactly 768px, how many cards per row?"* → Two (the `min-width: 768px` query applies at 768 and above).
- After step 5: *"Will the desktop layout be affected by the `max-width: 767px` rule?"* → No; it only applies below 768px.
- At the end: *"If a horizontal scrollbar appears, what is the first thing to check?"* → An image missing `max-width: 100%`, or a fixed-width element.

Troubleshooting while walking the room:

- Text still tiny in device mode → viewport tag missing or misspelled.
- Cards stay one per row on desktop → the `min-width: 1024px` query has a typo, or it is above the base rule but the base rule is more specific.

- Menu still crowded on mobile → the `@media (max-width: 767px)` block is missing or placed above the desktop `.topbar` rule so it is overridden.
- Horizontal scrollbar → add `img { max-width: 100%; }`, and check for any `width: 960px`-style fixed values.

Completion checklist (learner ticks each):

- [ ] Viewport meta tag present in the `<head>`
- [ ] `img { max-width: 100%; height: auto; }` present
- [ ] `.cards` base is `grid-template-columns: 1fr;`
- [ ] `min-width: 768px` query gives two columns
- [ ] `min-width: 1024px` query gives three columns
- [ ] `max-width: 767px` query stacks and centres the menu
- [ ] Tested in the device toolbar at 360px, 768px and 1280px
- [ ] No horizontal scrollbar at any of those widths

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: On your own, build a **responsive "About Adaeze" page** with a hero banner, a short story, and a two-column "Values" section that becomes one column on a phone.

Scenario: Adaeze wants an About page with:

- A hero section containing the heading "*Our Story*" and a one-line tagline.
- A paragraph telling her story (write 3–4 sentences yourself).
- A **Values** section with three value cards: *Quality*, *Affordability* and *Style*, each with a one-line description.
- The same purple top bar with a responsive menu.

Starting information (given):

- Brand colours: deep purple `#4b2e83`, gold `#f4c542`.
- Tagline: *"Handmade in Port Harcourt, designed for you."*
- Values descriptions:
  1. **Quality** — "Every stitch is checked before it leaves our shop."
  2. **Affordability** — "Beautiful clothing that fits a real budget."
  3. **Style** — "Modern designs that celebrate Nigerian fabrics."

Requirements:

1. Create `about.html` and link `css/style.css`.
2. Add the viewport meta tag.
3. Build the responsive top bar (same as the practical).
4. Build a hero section with an `<h1>`, a tagline `<p>`, comfortable padding, and a background colour.
5. Build a `.values` section using **Grid**. It must show **three columns on desktop, two on tablet, and one on a phone** using mobile-first `min-width` media queries.
6. Make sure any images (use one placeholder or your own) are fluid with `max-width: 100%`.
7. Test in the device toolbar at 360px, 768px and 1280px.

Constraints:

- Mobile-first: the base `.values` rule must be **one column**; add columns upward with `min-width`.
- Use relative units for widths and font sizes where sensible; do not fix the container at `1000px`.
- No horizontal scrollbar at any width.

Time allowance: 15 minutes.

Success criteria:

- One column at 360px, two at 768px, three at 1280px.
- The menu stacks and centres on the phone view.

- Text is readable at every width; no overflow.

Hints available if stuck:

- Hint 1: Start with `.values { grid-template-columns: 1fr; }`, then a `min-width: 600px` query for two columns and a `min-width: 1024px` query for three.
- Hint 2: The viewport tag goes in the `<head>`, not the `<body>`.
- Hint 3: If the menu does not stack, your `@media (max-width: 767px)` block may be *above* the desktop `.topbar` rule — move it to the bottom.

Instructor answer key (do not show learners):

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; color: #222; }
img { max-width: 100%; height: auto; display: block; }

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo { color: #f4c542; font-size: 22px; font-weight: bold; text-decoration: none; }
.topbar nav { display: flex; gap: 24px; }
.topbar nav a { color: white; text-decoration: none; padding: 6px 4px; }

.hero {
  background-color: #f3ecfb;
  padding: 3rem 1rem;
  text-align: center;
}
.hero h1 { color: #4b2e83; margin: 0 0 0.5rem; font-size: clamp(1.8rem, 5vw, 3rem); }
.hero p { margin: 0; color: #555; }

.container { width: 90%; max-width: 1100px; margin: 0 auto; padding: 2rem 0; }

.values {
  display: grid;
  grid-template-columns: 1fr;   /* phone default */
  gap: 20px;
}
.value-card {
  padding: 20px;
  border: 1px solid #e0d7f0;
  border-radius: 10px;
  background-color: #faf7ff;
}
.value-card h3 { color: #4b2e83; margin-top: 0; }

@media (min-width: 600px) {
  .values { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1024px) {
  .values { grid-template-columns: repeat(3, 1fr); }
}
@media (max-width: 767px) {
  .topbar { flex-direction: column; align-items: center; gap: 12px; }
  .topbar nav { flex-wrap: wrap; justify-content: center; gap: 14px; }
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — About</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
```

```
  <header class="topbar">
    <a href="index.html" class="logo">Adaeze's Fashion</a>
    <nav>
      <a href="index.html">Home</a>
      <a href="shop.html">Shop</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <section class="hero">
    <h1>Our Story</h1>
    <p>Handmade in Port Harcourt, designed for you.</p>
  </section>

  <main class="container">
    <p>Adaeze started with one sewing machine and a big dream. Today her small
    workshop dresses customers across Rivers State. Every outfit is cut,
    sewn and finished by hand, with care for the person who will wear it.</p>

    <section class="values">
      <div class="value-card">
        <h3>Quality</h3>
        <p>Every stitch is checked before it leaves our shop.</p>
      </div>
      <div class="value-card">
        <h3>Affordability</h3>
        <p>Beautiful clothing that fits a real budget.</p>
      </div>
      <div class="value-card">
        <h3>Style</h3>
        <p>Modern designs that celebrate Nigerian fabrics.</p>
      </div>
    </section>
  </main>
</body>
</html>
```

Marking guidance (out of 10):

- Viewport meta tag present (1)
- Responsive top bar works (2)
- `.values` base is one column, mobile-first (2)
- Two-column query at ≥600px, three-column at ≥1024px (2)
- Fluid images / no overflow (1)
- Readable hero with sensible relative sizing (1)
- Tested at three widths, no horizontal scroll (1)

J. REAL-WORLD CASE STUDY

Situation. *Chinedu* runs a small phones-and-accessories shop in Port Harcourt. He pays a developer ₦180,000 for a website. It looks impressive on the developer's big laptop. But almost all of Chinedu's customers find him through a phone. When they open the site on their phones, the page loads as a tiny desktop page, the "Buy" button is impossible to tap, and the product photos spill off the side. Customers message him saying "your site is not working on phone", and sales through the site stay near zero.

Relevant facts.

- The website has correct content and looks fine on a desktop.
- The customers almost always use phones.
- The developer never added a viewport meta tag and used fixed `width: 1000px` layouts with full-size images.

Decision point. What are the highest-value fixes, in order?

Possible approaches.

1. Tell customers to use a laptop. (Not realistic.)
2. Pay for a whole new website.
3. Fix the existing site to be responsive: add the viewport tag, fluid images and a mobile-first layout with media queries.

Recommended solution (for this stage). Approach 3. The content is fine; only the layout needs to adapt. Adding the viewport meta tag, `img { max-width: 100%; }`, mobile-first media queries and a stacked menu can transform the experience for phone users, usually for a fraction of a rebuild.

Trade-offs.

- Approach 1 ignores how customers actually behave.
- Approach 2 is expensive and unnecessary; the content is already good.
- Approach 3 takes a short, focused effort and directly targets the customers who matter most.

Professional connection. Freelance web designers in Nigeria win repeat clients by fixing exactly this problem. A designer who can take a broken desktop-only site and make it shine on a ₦20,000 Android phone is delivering measurable business value — more orders, fewer complaints.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the session's live troubleshooting guide. When a learner hits an issue, ask them to check the relevant row before giving the answer.

K.1 Horizontal scrollbar appears on mobile

Symptoms: A scrollbar at the bottom of the page; content runs off the right edge. **Likely causes:** (a) a fixed-width element (e.g. `width: 1000px`); (b) an image without `max-width: 100%`; (c) a very long unbroken word or URL; (d) a negative margin. **How to investigate:** In device mode, scroll right to find which element is cut off. In DevTools → Elements, hover elements to see whose box extends past the viewport. **Safe corrective steps:** Replace fixed widths with `max-width` + `%`; add `img { max-width: 100%; }`; test again. **How to confirm resolved:** The scrollbar disappears at 360px. **How to prevent recurrence:** Prefer fluid units for containers and always make images fluid.

K.2 Text looks tiny on a phone

Symptoms: The page appears as a shrunken desktop page; text is hard to read. **Likely cause:** Missing or incorrect viewport meta tag. **How to investigate:** Look in the `<head>` for `<meta name="viewport" ...>`. **Safe corrective steps:** Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">`. **How to confirm resolved:** Text fills the phone width at a readable size. **How to prevent recurrence:** Put the viewport tag in every page's `<head>` from the start.

K.3 Media query seems to do nothing

Symptoms: A `@media` block has no visible effect. **Likely causes:** (a) syntax error (missing colon, missing `and`); (b) the query is placed *above* the rule it should override, so it loses the cascade; (c) the values inside are less specific or overridden. **How to investigate:** In DevTools, check whether the rule appears (and is not greyed out) when you resize. Inspect the computed styles. **Safe corrective steps:** Fix the syntax; move media queries to the **bottom** of the stylesheet; use consistent selectors. **How to confirm resolved:** The rule visibly applies at the intended width. **How to prevent recurrence:** Keep all media queries together at the end of the file.

K.4 Page overflows even though `max-width: 100%` is set

Symptoms: An image or box still overflows on mobile. **Likely cause:** A child element (or padding) is wider than the parent, or the image is inside a fixed-width wrapper. **How to investigate:** In DevTools, drag the width and hover to find the offender; add a temporary `outline: 1px solid red;` to suspect elements. **Safe corrective steps:** Make the wrapper fluid too; check for padding that pushes past 100% (remember `box-sizing: border-box` helps here). **How to confirm resolved:** The overflow and scrollbar are gone.

K.5 Layout is great on desktop but ugly on tablet

Symptoms: At ~768px the layout is neither the phone one nor the desktop one; it looks awkward. **Likely cause:** A missing tablet breakpoint. **How to investigate:** Test at 768px in device mode and note what is cramped. **Safe corrective steps:** Add a `min-`

`width: 768px` (or `600px`) query with an intermediate layout. **How to confirm resolved:** The layout looks intentional at 768px. **How to prevent recurrence:** Plan three widths from the start: phone, tablet, desktop.

K.6 Everything works but the menu overlaps the logo

Symptoms: Menu links sit on top of the logo on narrow screens. **Likely cause:** The top bar has no wrapping or stacking rule for small screens. **Safe corrective steps:** Add `flex-wrap: wrap;` and a mobile `@media (max-width: 767px)` rule that sets `flex-direction: column`. **How to confirm resolved:** On a phone the logo is on its own line with the menu beneath.

Distinguish likely vs confirmed: In every case, confirm the cause before fixing — check the actual element in DevTools (its computed width, its position) rather than guessing.

L. SESSION ASSIGNMENT

Assignment title: "My Responsive Business Page"

Purpose: Practise making a page adapt from phone to desktop using the viewport tag, relative units, fluid images and media queries.

Realistic scenario: You are building the *Adaeze's Fashion* shop page. Adaeze insists that it must look correct on a phone first, because most of her customers shop from their phones.

Detailed instructions:

1. In your `adaeze-fashion` folder, create `shop.html` and link `css/style.css`.
2. Add the viewport meta tag.
3. Build the responsive purple top bar (stacked and centred on phones).
4. Build a **Shop** section with at least **six** product cards (image, name and naira price). Use your own placeholder images or simple coloured boxes if images are not ready.
5. Style the cards mobile-first:
   - base: **one column**,
   - `min-width: 768px`: **two columns**,
   - `min-width: 1024px`: **three columns**.
6. Make all images fluid with `max-width: 100%`.
7. Add at least one fluid font-size using `rem` (for example, your headings should use `rem`, not a fixed `px`).
8. Test in the Chrome device toolbar at **360px, 768px and 1280px**.

Required deliverables:

- `adaeze-fashion/shop.html`
- Updated `adaeze-fashion/css/style.css`
- A short testing note (3–4 sentences or a small table) recording what you saw at each of the three widths.

Submission format: Files in the class shared folder; the testing note written or photographed into the workbook.

Deadline: Before Session 7.

Marking rubric (20 marks):

Criterion	Marks	Description of a strong answer
Viewport meta tag	2	Correctly present in the `<head>`
Responsive navigation	3	Horizontal on desktop, stacked and centred on phones
Mobile-first grid	4	One column base, two at 768px, three at 1024px
Fluid images	3	`max-width: 100%`; no overflow at 360px
Relative units used	3	Widths and at least one font size use `%` / `rem`
Testing note	2	Clear record of the three widths tested

Criterion	Marks	Description of a strong answer
No horizontal scroll	2	Clean at 360px, 768px and 1280px
Code quality	1	Indented, readable, media queries grouped at the end

Common mistakes to avoid: forgetting the viewport tag; compiling desktop-first and then fighting overrides; leaving a fixed `width` on the container; forgetting to make images fluid; putting media queries above the rules they should override.

Instructor reference solution:

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; color: #222; }
img { max-width: 100%; height: auto; display: block; }

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  background-color: #4b2e83;
  padding: 14px 24px;
}
.logo { color: #f4c542; font-size: 1.4rem; font-weight: bold; text-decoration: none; }
.topbar nav { display: flex; gap: 24px; }
.topbar nav a { color: white; text-decoration: none; padding: 6px 4px; }

.container { width: 90%; max-width: 1100px; margin: 0 auto; padding: 0 16px; }
h1 { color: #4b2e83; font-size: 2rem; }

.shop {
  display: grid;
  grid-template-columns: 1fr;      /* phone */
  gap: 16px;
  margin: 24px 0 40px;
}
.product { border: 1px solid #e0d7f0; border-radius: 10px; overflow: hidden; }
.product h3 { margin: 12px 16px 4px; color: #4b2e83; font-size: 1.1rem; }
.product .price { margin: 0 16px 16px; color: #b8860b; font-weight: bold; }

@media (min-width: 768px) {
  .shop { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1024px) {
  .shop { grid-template-columns: repeat(3, 1fr); }
}
@media (max-width: 767px) {
  .topbar { flex-direction: column; align-items: center; gap: 12px; }
  .topbar nav { flex-wrap: wrap; justify-content: center; gap: 14px; }
}
```

The testing note should look like: "At 360px — one card per row, stacked menu, no scrollbar, text readable. At 768px — two cards per row, menu in a row. At 1280px — three cards per row, full menu, centred container."

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence, what is responsive design?

Q2. Fill in the blank. The line to add to the `<head>` for phones is `<meta name="_____" content="width=device-width, initial-scale=1.0">`.

Q3. Multiple choice. Which media query applies when the screen is 768px wide or wider? a) `@media (max-width: 768px) { }` b) `@media (min-width: 768px) { }` c) `@media (width: 768px) { }` d) `@media (min-device: 768px) { }`

Q4. Explain in your own words. What is "mobile-first", and why can it be easier than designing desktop-first?

Q5. Practical. Write the CSS rule that makes every image shrink to fit its container while keeping its proportions.

Q6. Units. Name the unit that is relative to the root font size, and say what `1rem` equals by default in most browsers.

Q7. Error detection. A learner's page shows a horizontal scrollbar on a phone. List two likely causes and the fix for each.

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. Responsive design means building one website whose layout adapts automatically to different screen sizes so it looks and works well on phones, tablets and desktops.

A2. `viewport` — the full line is `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.

A3. (b) `@media (min-width: 768px) { }`.

A4. Mobile-first means writing the small-screen (phone) styles first as the base, then adding larger-screen styles with `min-width` media queries. It is easier because you only ever *add* styles as the screen grows; desktop-first forces you to *undo* desktop styles for phones, which means writing more overrides and creating more conflicts.

A5.

```
img {
  max-width: 100%;
  height: auto;
}
```

A6. `rem` is relative to the **root** font size. By default in most browsers `1rem = 16px`.

A7. Two likely causes: (a) an image without `max-width: 100%` → add `img { max-width: 100%; height: auto; }`; (b) a fixed-width element such as `width: 1000px` → replace with `width: 90%; max-width: 1000px;`. (Also accept: a long unbroken word/URL, or a negative margin.)

N. REFLECTION AND RECAP

Plain-English summary. Today you turned a desktop-only page into one that adapts to every screen. You learned why responsive design matters for a Nigerian business whose customers mostly use phones, added the **viewport meta tag**, practised **mobile-first** thinking, chose between fixed and **relative units**, made **images and containers fluid**, and wrote **media queries** at the common breakpoints of 360px, 768px and 1024px. You built a responsive navigation bar and a card grid, and tested them in the **Chrome device toolbar**.

Five important takeaways:

1. Responsive design = one website that adapts to every screen size.
2. Always include the viewport meta tag.
3. Mobile-first: base styles for phones, then `min-width` queries for bigger screens.
4. `img { max-width: 100%; height: auto; }` prevents the classic overflow.
5. Test at 360px, 768px and 1280px — every single time.

Questions to help you recognise what you can now do:

- Can I make a page go from one column on a phone to three on a desktop?
- Can I explain why `1000px` width is a bad idea for a container?
- Can I test a page at three widths without help?
- Can I fix a horizontal scrollbar on a phone?

Connection to the next session: This session ends Week 2. You can now build and lay out responsive pages. Next week we start a real project: planning the full *Adaeze's Fashion* website — its purpose, pages, wireframes, content, colours and file structure — before writing a single line of code. This is the professional planning process that saves hours later.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- The learner adds the viewport tag without being told.
- The learner writes the small-screen layout first (mobile-first) rather than fighting desktop rules.
- The learner uses `%` or `max-width` for containers and reserves `px` for small fixed details.
- The learner tests at three widths and can explain what they saw.
- The learner diagnoses a horizontal scrollbar and names the likely cause before fixing it.
- The learner keeps media queries at the bottom of the stylesheet and explains why.

Common misconceptions to watch for and correct:

- "Responsive is only for mobile." → It is for all screen sizes.
- "I need a breakpoint for every device." → Two or three well-chosen breakpoints are enough.
- "The viewport tag makes my site responsive." → It only sets the stage; the layout still needs fluid units and media queries.
- "Percentages are always better than pixels." → Use the right unit for the job; borders and thin details are fine in `px`.
- "If it looks fine in device mode, real phones are definitely fine." → Device mode is a strong estimate, not a guarantee; test a real phone when possible.

Suggested remedial exercises for learners who struggle:

1. **Resize-and-describe:** give three pages and ask learners to test each at 360px and note the first thing that breaks.
2. **Unit sorting:** list ten CSS values and ask learners to sort them into "fixed" and "relative", with a reason.
3. **Fix the query:** give a broken media query (`@media max-width 767px`) and ask learners to correct the syntax.

Differentiation:

- **Level 1 (supported):** provide `about.html` already written, with the base `.values { grid-template-columns: 1fr; }` and one media query filled in; learners add the remaining query and test.
- **Level 2 (standard):** the exercises as written.
- **Level 3 (challenge):** implement a CSS-only "hamburger" menu using a hidden checkbox and the `:checked` pseudo-class, and/or use `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` to make the grid reflow without explicit breakpoints. Observe and compare with the media-query approach.

Timing guidance: The device-toolbar testing is the heart of this session — protect the time for it. If the room is slow, shorten the case study (Section J) to a 3-minute mention and reduce the practical checklist, but always leave at least 8 minutes for learners to test at three widths themselves. If the room is fast, extend the challenge in the differentiation notes.

APPENDIX 6A — 15-MINUTE LAYOUT REFRESHER (only if needed)

If learners cannot produce a Flexbox row or a Grid, run this before Section D.

```
<div class="row">
  <div class="box">One</div>
  <div class="box">Two</div>
  <div class="box">Three</div>
</div>
```

```
.row {
  display: flex;      /* children sit in a row */
  gap: 16px;
}
.box {
  flex: 1;            /* each box shares the space equally */
  padding: 16px;
  background-color: #f3ecfb;
  border: 1px solid #4b2e83;
}
```

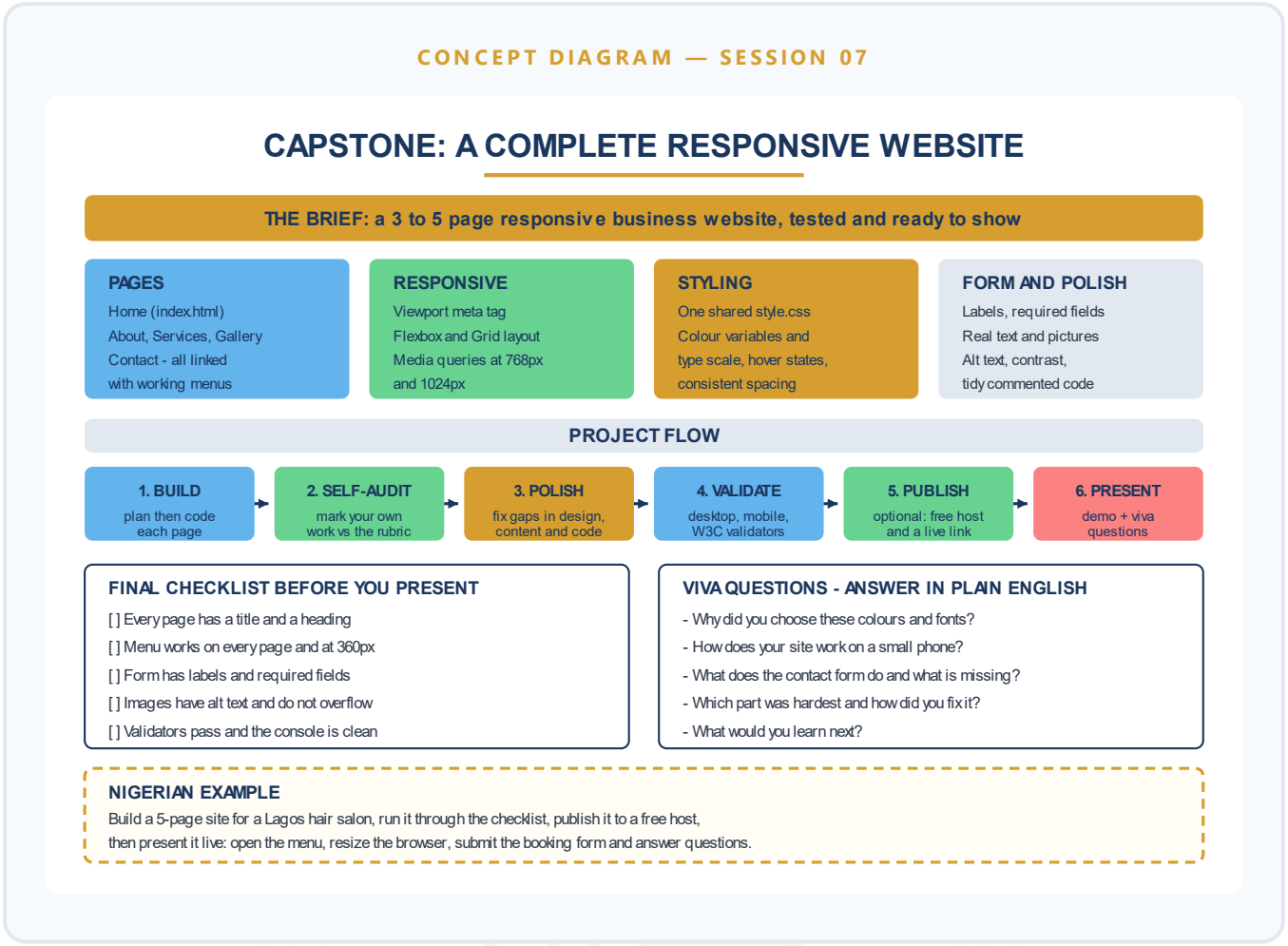
Confirm that every learner can (a) make three boxes sit in a row, and (b) explain that `display: flex` goes on the parent. Only proceed once this is solid — this session builds directly on it.

END OF SESSION 6 — END OF WEEK 2 (CSS FOUNDATIONS)

Next: Session 7 — Planning a Website.

SESSION 07 OF 12

PLANNING A WEBSITE



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 7 — PLANNING A WEBSITE

A. SESSION IDENTIFICATION

Item	Detail
Session number	7 of 12
Session title	Planning a Website
Week / Module	Week 3 — Module 3: Building a Website
Recommended duration	105 minutes
Difficulty level	Beginner (mostly thinking and drawing — no coding this session)

Item	Detail
Prerequisite knowledge	Sessions 1–6. The learner should be able to build and style a responsive HTML page. This session has no coding, but it prepares every page the learner will code in Session 8. Note: no laptop is strictly needed; paper, pen and a printed plan template are the main tools.
Connection to previous work	For six sessions learners have built small pages on request. This session introduces the professional habit that comes *before* code: planning. It mirrors how real designers work — understand the client, plan the pages, sketch the layout, gather content, then build.
Preparation before class	Print a wireframe template (one per learner) and a site-plan template (one per learner); provide plain A4 paper and pens or pencils (two per learner); print a one-page "Adaeze's Fashion brief" (see Section F.1); have the projector ready to show example wireframes and a colour-contrast demo; optionally prepare one or two real business cards or flyers as colour/font examples.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Read a project brief** and extract its key requirements: purpose, audience, pages and content.
2. **Define the purpose and goals** of a website in one or two clear sentences.
3. **Describe the target audience** of a small Nigerian business and explain how that audience shapes design decisions.
4. **Create a sitemap** that shows every page of the website and how the pages relate.
5. **Draw a wireframe** (by hand) for a page, showing header, navigation, main content and footer as labelled boxes.
6. **Plan page content** — decide what text and images each page needs before coding.
7. **Choose a simple colour palette and two fonts**, and explain why contrast between text and background matters for accessibility.
8. **Set up a clean file and folder structure** for a multi-page site and apply sensible file-naming conventions.
9. **Explain why** planning before coding saves time and prevents rework.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops) — used only for looking up nothing this session; it is fine if a learner works entirely on paper at their desk.
- Instructor computer connected to a projector or large screen.

Software

- Visual Studio Code and Google Chrome are **not required** this session, but the instructor may open a sample project folder in VS Code at the end to show the planned file structure.

Printed materials

- "Wireframe Template" (one per learner) — a blank page-sized rectangle with a faint grid.
- "Site Plan Template" (one per learner) — a form with fields for purpose, audience, pages, content, colours, fonts and file structure.
- "Adaeze's Fashion — Project Brief" (one per learner).
- "Colour Palette & Contrast Handout" (one per learner).

Tools

- A4 paper (at least two sheets per learner), pens and pencils.
- Optional: a ruler for drawing straight boxes; coloured pencils if available.
- Optional: one smartphone to show how a real phone screen looks while sketching.

Internet

- Not required. If available, the instructor may show an online colour-contrast checker at the very end, but do not depend on it. A simple paper method (described in F.7) works without internet.

Alternative for learners who lack a facility

- This session is naturally paper-based, so no special accommodation is needed. If a learner cannot draw, they may lay out boxes with sticky notes or describe the wireframe verbally while the instructor or a partner sketches it.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, show a photograph of a building under construction (or describe one) — or, if none is available, draw a quick sketch of a house with foundation lines and say it is a plan. Then show a crumpled, poorly built structure beside a neat finished building.

What the instructor says:

> *"Imagine you are building a house for your family. Would you start laying blocks on any open ground and hope it turns out well? No. First you sit with the builder and agree what the house is for, how many rooms you need, where the kitchen goes, where the doors are. You draw a plan. Then you build.*
>
> *A website is the same. For six sessions you have been learning the tools — the blocks and cement of the web. Today we stop and draw the plan for a real business website: Adaeze's Fashion in Port Harcourt. We will decide what pages it needs, what goes on each page, what it looks like, and how the files will be arranged. Tomorrow, and for the rest of the course, we build from this plan."*

Expected learner response: Learners may be impatient to code. Address this directly: "A one-day plan today saves a week of rework later. Professional designers never skip this step."

Then demonstrate the hook: Show a before/after. On the projector, display two wireframes: one messy, with boxes randomly placed and unlabelled; and one neat, labelled sketch of the same page (header bar at the top, hero band, three product boxes, footer at the bottom). Say:

> *"See? This is not art. It is a simple map. You can draw it in ten minutes with a pen. And once you have it, building the page in HTML becomes almost automatic."*

Why this opener works: It reframes planning from "boring paperwork" to "the thing that makes building easy", which is exactly the professional truth this session teaches.

E. PREREQUISITE REVIEW (7 minutes)

Run this as an oral round-robin. Put the six questions on the projector. This session has no coding, so this quick review keeps earlier skills alive.

Question 1: "What are the three building blocks of every website?"

- Expected answer: HTML (structure), CSS (appearance) and JavaScript (behaviour).
- Common gap: forgetting JavaScript. Remind them it comes in Session 10.

Question 2: "Name three semantic HTML elements we use for page structure."

- Expected answer: `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`.
- Common gap: naming `<div>` as semantic. Clarify: `<div>` is a generic box; semantic tags describe meaning.

Question 3: "How do you link a CSS file from `index.html`?"

- Expected answer: `<link rel="stylesheet" href="css/style.css">` in the `<head>`.
- Common gap: wrong path. Point out we will *plan* that folder today.

Question 4: "What is a media query, and what two words can start one?"

- Expected answer: CSS that applies at certain screen widths; `min-width` or `max-width`.
- Common gap: confusing the two. Remind them: `min-width` = "this wide or wider".

Question 5: "What does the viewport meta tag do?"

- Expected answer: makes the page use the device's real width so mobile layouts work correctly.
- Common gap: "I forgot it." Good reminder to include it on every page we build.

Question 6: "What is a relative path? Give one example."

- Expected answer: a path relative to the current file, e.g. `about.html` or `images/logo.png`.
- Common gap: using full Windows paths like `C:\Users\...`. Remind them we never do this.

Bridge to today (say this):

> *"You now have the tools — HTML, CSS, responsive design. But a builder with all the tools and no plan builds a mess. Today we become the designer and the architect. Client first, pages second, sketches third, content fourth, colours and fonts fifth, file structure last."*

If more than a third of the class cannot answer Question 1, spend a few minutes re-capping the three languages before continuing (but keep it brief — this session must protect drawing time).

F. FULL LECTURE NOTES

F.1 Reading the project brief — what does the client actually want?

The idea. Before you design anything, you must understand the client's goal. In professional work this comes from a **project brief** — a short document that states what the client needs.

Plain-English definition. A project brief answers, in plain language, four questions: *What is the website for? Who will use it? What pages does it need? What content goes on those pages?*

Why it exists. A website with no clear purpose drifts. If you do not know what the client is trying to achieve, you will build the wrong thing — beautifully. The brief keeps everyone anchored.

The brief for our project (read aloud):

> ***Client:*** *Adaeze Okafor, owner of Adaeze's Fashion, Port Harcourt.* ***Business:*** *A small fashion business that designs and sews custom clothing — Ankara dresses, lace gowns, aso ebi sets and office wear — and also runs short sewing classes.* ***Problem:*** *Adaeze sells mostly through WhatsApp and Instagram. She cannot show her full collection neatly, and customers keep asking the same questions about prices, location and delivery.* ***Goal:*** *A simple, professional website where customers can see her work, learn about her, and contact her to place an order.* ***Budget:*** *Small. The website must be built with free tools and free hosting.* ***Devices:*** *Most customers use phones; the website must work perfectly on a phone first.*

The four questions, answered for Adaeze:

Question	Answer
Purpose	Showcase the collection and let customers make contact to order.
Audience	Adults in Port Harcourt and Rivers State who buy or commission clothing; plus people interested in sewing classes.
Pages needed	Home, Shop (Products), About, Contact.
Content needed	Photos of outfits, prices in naira, Adaeze's story, address, phone, social links, a contact form.

How a learner recognises a good purpose statement. A strong purpose statement is short and specific: "To showcase Adaeze's clothing and let customers contact her to order." A weak one is vague: "To have a website." If the purpose could apply to any business, it is too vague.

Misconception. "The client always knows what they want." Often they know the *outcome* they want (more orders) but not the *pages* or *features*. Part of your job is to translate their goal into a plan and confirm it with them.

Limitation. A brief can change. Real projects often have the client add a page later. Good planning makes such changes cheap, because the structure is clear and consistent.

F.2 Defining purpose and goals

The idea. Write down the website's purpose (why it exists) and its **goals** (what success looks like).

Plain-English definition. Purpose is the single reason the site exists. Goals are the measurable outcomes that tell you if it worked.

Why it exists. Purpose keeps design decisions honest. Every element on the page should serve the purpose. If it does not, it is clutter.

Adaeze's purpose and goals:

> ***Purpose:*** *To show Adaeze's clothing clearly and make it easy for a customer to start an order.*
>
> ***Goals:***
>
> 1. *A customer can see at least six outfits without scrolling forever.*
> 2. *A customer can find Adaeze's phone number and address in under ten seconds.*
> 3. *A customer can send a message through the contact form.*

How purpose shapes design. Because the first goal is "see outfits clearly", the Shop page must use a tidy grid of photos with prices (the Grid skill from Session 5). Because the second goal is "find contact details fast", the phone number and address must appear in the footer of every page. Because most customers use phones, every page must be responsive.

Say this to the class:

> *"Every time you are unsure whether to add something to a page, ask: does this help the purpose? If not, leave it out. A clean, focused website beats a busy one."*

Misconception. "More features make a better website." Usually the opposite is true for a small business. A clear, fast, phone-friendly site with the essentials beats a crowded one with a broken contact form.

F.3 Understanding the audience

The idea. Design *for the people who will use the site*, not for yourself.

Plain-English definition. The **audience** (or target users) are the people the website is meant to serve. Knowing them shapes the language, colours, images and even the page structure.

Why it exists. The same business can have different audiences. A clothing website for teenagers would look and sound different from one for adults buying aso ebi for a family event. Matching the audience makes the site feel right to them.

Adaeze's audience (described plainly):

Audience group	What they want	Design implication
Individual customers	To see outfits and prices, and to contact Adaeze	Clear photos, visible prices, prominent contact button
Event/party buyers (aso ebi)	To order matching outfits for a group	A note about group orders, a contact form for details
Sewing-class students	To know about training and fees	A Services/Training section on the site
Phone users (the majority)	Speed and easy tapping	Responsive layout, large tap targets, light images

How to talk to an audience. Use simple, warm English — the language of the customers, not technical jargon. Adaeze's customers do not want to read "leveraging synergies"; they want to know if the dress comes in their size and how much delivery costs. Write *for the reader*.

Misconception. "Everyone is my audience." Attempting to please everyone usually pleases no one. Picking a primary audience (here: phone-using clothing customers in Port Harcourt) gives every design decision a clear reference point.

F.4 Planning the pages — the sitemap

The idea. A **sitemap** is a simple diagram (like a family tree) showing every page on the website and how they relate.

Plain-English definition. A sitemap is the list and shape of your pages, usually drawn as boxes with lines. The Home page sits at the top; other pages branch off it.

Why it exists. A sitemap lets you see the whole website at a glance before building it, so you do not discover halfway through that you forgot a page or gave two pages the same job.

Adaeze's sitemap:



Every page is reachable from the Home page through the top navigation. Four to five pages is the right size for a small business site — enough to cover the essentials without overwhelming the visitor.

Navigation rules that follow from the sitemap:

1. Keep the main menu to **5 items or fewer**.
2. Use the **same menu on every page** (consistency builds trust).
3. Use **clear labels**: "Shop", not "Stuff"; "About", not "Info".
4. The **logo always links back to Home**.
5. Mark the **current page** in the menu (we will do this with CSS in Session 9).

How a learner recognises a good sitemap. It is shallow (not many levels deep) and every page has one clear job. If two pages do the same thing, merge them. If a page has no home in the structure, question whether it is needed.

Misconception. "More pages look more professional." A four-page site with clear content beats a twelve-page site with empty pages. Depth is not quality.

Limitation. A sitemap shows *what* pages exist, not *what they look like*. That is the job of the wireframe (next).

F.5 Wireframing — sketching the layout by hand

The idea. A **wireframe** is a rough sketch of a page that shows where each block of content goes, using only boxes and labels — no colours, no fonts, no real text.

Plain-English definition. A wireframe is the "skeleton" of a page drawn on paper. It answers: what is at the top, what is in the middle, what is at the bottom, and how wide is each part?

Why it exists. It is far faster to move a box on paper than to move it in code. Wireframes let you try five layouts in twenty minutes and pick the best before writing a single line of HTML.

Why we draw on paper (not in software). Paper is fast, free and cannot distract you with colours and effects. You can scribble, erase and try again. Professional designers still do this first.

The standard blocks of a wireframe (draw these as rectangles):

- **Header** — top bar with logo (left) and navigation (right).
- **Hero** — a big banner area with a headline and a call-to-action button.
- **Main content** — the main sections (e.g. featured products, text).
- **Sidebar** — optional narrower column beside the main content.

- **Footer** — bottom bar with contact details and social links.

Conventions for drawing:

- Draw a rectangle for a box; label it (e.g. "LOGO", "MENU").
- Draw an **X through a box** to mean "an image goes here".
- Draw **wavy lines** to mean "text goes here".
- Write the page name at the top of the sheet.

Worked example — a wireframe of Adaeze's Home page (described in words, drawn on paper):

```
+-------------------------------------------------+
|  LOGO (Adaeze's Fashion)          HOME SHOP ... |  <- header + nav bar
+-------------------------------------------------+
|                                                 |
|        "Custom Fashion for Every Occasion"      |
|        [ View Our Collection  (button) ]        |
|                                                 |  <- hero (full width)
+-------------------------------------------------+
|   [X]        [X]        [X]                     |
|  dress1     dress2     dress3                   |
|  ₦18,500    ₦32,000    ₦27,500                  |  <- featured products (3 boxes)
+-------------------------------------------------+
|  Our Story         |   Get in Touch             |
|  ~~~~~ text ~~~~   |   Phone: 0801 234 5678     |  <- two columns: story + contact
|  ~~~~~ text ~~~~   |   Address: Aba Road, PH    |
+-------------------------------------------------+
|  (c) 2026 Adaeze's Fashion   Instagram  Facebook |  <- footer
+-------------------------------------------------+
```

Expected result of drawing it: On paper, you can now see the page from top to bottom. Anyone — even a non-designer — can look at this and understand what the Home page will contain. That clarity is the point.

How a learner recognises a good wireframe. It has clear blocks, the labels make each block's purpose obvious, and the most important content (here, the outfits and the contact details) is high on the page.

Misconception. "A wireframe must look nice." No — neatness is optional; clarity is essential. A messy sketch that communicates the layout is more useful than a beautiful drawing that hides it.

Limitation. A wireframe is only one page. You need one wireframe **per page** (Home, Shop, About, Contact). They should share the same header and footer, which is why the navigation feels consistent to the visitor.

F.6 Gathering content — text and images

The idea. Decide exactly what words and pictures each page will contain *before* you build it.

Plain-English definition. Content is everything the visitor reads and sees: headings, paragraphs, prices, photos, contact details. Content is the reason the site exists.

Why it exists. Many beginners build an empty layout and then scramble for content, which leads to lorem-ipsum placeholders and a half-finished site. Planning content first means the layout can be designed to fit the real material.

Content plan for Adaeze's Fashion:

Home page

- Headline: "*Custom Fashion for Every Occasion*"
- Subheadline: "*Beautiful, affordable clothing, handmade in Port Harcourt.*"
- Featured products: 3 photos with names and naira prices.
- Call to action button: "*View Our Collection*" (links to Shop).

Shop page

- Heading: "*Our Collection*"
- Six to nine products, each with a photo, a name and a price (e.g. *Ankara Dress — ₦18,500*).
- Note: "*To order, contact us or send a message using the form.*" (No online cart in this course.)

About page

- Heading: "*Our Story*"
- Three to four short paragraphs about Adaeze, the business and its values.
- One photo of Adaeze or the workshop.
- The three values: Quality, Affordability, Style.

Services page (if included)

- Heading: "*Training Services*"
- Three cards: Beginner Sewing Class, Ankara Design Workshop, Business of Fashion.

Contact page

- Heading: "*Get in Touch*"
- A contact form: Name, Email, Message, and a Send button.
- Address: *Aba Road, Port Harcourt, Rivers State*.
- Phone: *0801 234 5678* (a made-up example number).
- Social links: Instagram, Facebook.

Where to get images for practice. Use your own photographs, or free stock sites, or (for practice) simple coloured boxes. For a real client, always ask the client for photos of their actual products — real photos sell far better than stock images.

Writing tips for small-business content:

- Short paragraphs (2–3 sentences).
- Friendly, plain English.
- Put the most important information first.
- Spell-check carefully; typos reduce trust.

Misconception. "I will add the text later." If you leave content for later, you will design a layout that does not fit the real content and end up rebuilding. Plan the words now, even in rough form.

Limitation. Writing good content takes time and, honestly, it is often the hardest part of a website. In this course we write short, honest content; a real client may hire a copywriter or write it themselves.

F.7 Choosing colours and fonts (and why contrast matters)

The idea. Pick a small, consistent **colour palette** and one or two **fonts** that match the business, and make sure text is easy to read against its background.

Plain-English definition. A **colour palette** is the small set of colours used across the whole site (usually two or three plus neutrals). **Fonts** (typefaces) are the letter styles used for headings and body text.

Why it exists. A consistent palette and font make a website feel professional and trustworthy. Random colours and many fonts make it look amateurish. Consistency also signals reliability — important when customers are deciding whether to spend money with you.

Adaeze's palette (a simple, on-brand example):

Role	Colour	Hex	Used for
Primary	Deep purple	`#4b2e83`	Header, headings, buttons
Accent	Gold	`#f4c542`	Logo text, highlights, prices
Background	Very light purple	`#faf7ff`	Card and section backgrounds
Text	Near-black	`#222222`	Body text
Muted text	Grey	`#555555`	Secondary text

Why purple and gold? They feel elegant and creative, match a fashion brand, and — importantly — purple and gold have strong contrast against white, so text stays readable.

Fonts: choose one font for headings and (for simplicity) the same or a complementary one for body text. Adaeze's choice:

- **Headings:** Montserrat (a clean, bold, modern sans-serif).

- **Body:** Open Sans or simply Arial (a plain, highly readable sans-serif).

Fallback fonts matter. You should always end a `font-family` list with a generic fallback, e.g. `font-family: 'Montserrat', Arial, sans-serif;`. If the first font is not available, the browser tries the next. This keeps the site looking good even without an internet connection.

Contrast and accessibility. Contrast is how different the text colour is from the background colour. Low contrast (light grey text on white) is hard to read, especially in bright Nigerian sunlight or for older users. High contrast (near-black text on white) is easy to read for almost everyone.

A simple way to check contrast without any tool:

1. Imagine the page printed in greyscale (black-and-white photocopy).
2. If you can still clearly read the text, the contrast is probably fine.
3. For buttons, the button text must stand out from the button colour too.

Why this matters — the accessibility idea. Some of Adaeze's customers may have weaker eyesight, may be using an old phone with a dull screen, or may be reading in bright sun. Good contrast is not a small detail — it decides whether your customer can use your site at all.

Worked example — good and bad contrast:

```
/* BAD: light grey on white — hard to read */
.old-text {
  color: #cccccc;
  background-color: #ffffff;
}

/* GOOD: near-black on white — easy to read */
.new-text {
  color: #222222;
  background-color: #ffffff;
}

/* GOOD button: white text on deep purple */
.btn {
  color: #ffffff;
  background-color: #4b2e83;
  padding: 12px 24px;
  text-decoration: none;
}
```

Expected browser result: The `.old-text` paragraph is barely legible — you have to squint. The `.new-text` paragraph is crisp and comfortable. The `.btn` is a clear purple button with white text that jumps out. This is exactly the difference good contrast makes.

How a learner recognises a good palette. It has a small number of colours, they appear consistently (the same purple everywhere), and every piece of text is easy to read against its background.

Misconception. "I need many colours to look creative." A small, well-chosen palette looks *more* professional than a rainbow. Pick two brand colours plus black, white and a grey, and stick to them.

Limitation. We are planning colours now, but applying them happens in Session 9 (Styling the Website). Today you only decide the palette and write down the hex codes.

F.8 File and folder structure and naming conventions

The idea. Plan where every file will live before you create it.

Plain-English definition. File structure is the way you organise your files into folders. **Naming conventions** are the rules for naming those files.

Why it exists. A website with scattered, badly named files is a nightmare to update — you will lose track of which `style` file is the real one, and links will break when you publish online. A clean structure makes a project easy to build, easy to maintain and easier to hand over to a client.

Adaeze's planned file structure:

```
adaeze-fashion/
├── index.html          (Home page — must be named index.html)
├── about.html          (About page)
├── shop.html           (Shop / Products page)
├── contact.html        (Contact page)
├── css/
│   └── style.css       (One shared stylesheet for the whole site)
├── images/
│   ├── logo.png
│   ├── hero.jpg
│   ├── dress1.jpg
│   ├── dress2.jpg
│   └── ...
└── js/
    └── script.js       (JavaScript — added in Session 10)
```

Naming rules (learn these by heart):

1. **All lowercase** — `about.html`, never `About.html`.
2. **Hyphens instead of spaces** — `ankara-dress.jpg`, never `ankara dress.jpg` or `AnkaraDress.jpg`.
3. **Short but descriptive** — `dress1.jpg` is fine; `my-lovely-new-dress-photo-final-v2.jpg` is not.
4. **The Home page is always `index.html`** — the web server looks for this name automatically when someone visits your site's main address.
5. **No special characters** — avoid `&`, `#`, `?`, spaces; these can break links.

Why lowercase and hyphens matter so much. Windows does not care about letter case (`About.html` and `about.html` are treated the same), but most web servers run on Linux, which *does* care. So a link to `About.html` works perfectly on your computer and breaks the moment you publish it. Using all lowercase everywhere avoids this trap completely. Spaces in file names become `%20` in URLs and are error-prone; hyphens avoid the problem.

Worked example — a correct link versus a broken one:

```
<!-- CORRECT: matches the real lowercase file name -->
<a href="about.html">About</a>

<!-- BROKEN once published: capital A does not match about.html on Linux servers -->
<a href="About.html">About</a>
```

Expected result: The first link opens the About page everywhere, on your laptop and online. The second works on your Windows laptop but gives a "404 Not Found" on the live website. That 404 is one of the most common beginner mistakes — and it is entirely avoidable by following the naming rules.

How a learner recognises a clean project. All HTML files sit together in the root folder; CSS is in `css/`; images are in `images/`; JavaScript is in `js/`; and every file name is lowercase with hyphens.

Misconception. "The folder name does not matter." It does. Keep the project folder simple too (e.g. `adaeze-fashion`), because it may later become part of your website's address.

Limitation. This flat structure is perfect for a 3–5 page site. Very large sites with hundreds of pages need deeper organisation, but that is beyond this course.

F.9 Summary of the session

- Start with the **brief**: purpose, audience, pages, content.
- Write a short, specific **purpose statement** and a few **goals**.
- Know your **audience** — Adaeze's customers are mostly on phones.
- Draw a **sitemap**: Home at the top, four pages branching off.
- Draw a **wireframe** for every page: labelled boxes, X for images, wavy lines for text.
- Plan the **content** — the actual words and photos — before building.
- Choose a small **colour palette** and one or two **fonts**; make sure contrast is high.
- Plan the **file structure** and follow **naming rules**: lowercase, hyphens, `index.html` for Home.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- The blank wireframe template and site-plan template (projected or copied onto the board so everyone can see).
- A one-page printed "Adaeze's Fashion brief" (Section F.1).
- A pen and paper (or the board) to draw live.

G.2 Demonstration 1 — building the sitemap from the brief (5 minutes)

Say: "Watch how I turn the brief into a map, in under five minutes."

1. Read the client's goal aloud: "See her work, learn about her, and contact her."
2. Ask the class: "Which pages does that suggest?" Collect answers: Home, Shop, About, Contact (and possibly Services).
3. Draw the sitemap on the board: a box labelled HOME at the top, with lines down to About, Shop, Services and Contact.
4. Say: "Notice I keep it shallow. Every page is one step from Home. That is good for a small business — the visitor never gets lost."

Ask: "What is the job of the Home page?" → To make a strong first impression and point visitors to the other pages.

Deliberately inject a mistake: Draw a page called "Products" and another called "Shop" both under Home. Ask the class what is wrong. → Two pages doing the same job; merge them. Erase one.

G.3 Demonstration 2 — drawing the Home-page wireframe (6 minutes)

Say: "Now the sketch. I will talk through it as I draw."

1. Draw a page-sized rectangle. Write "HOME" at the top.
2. Draw the header band at the very top; label the left "LOGO" and the right "MENU".
3. Draw a tall band below for the hero; write the headline in the middle and a small rectangle labelled "BUTTON".
4. Draw three equal boxes in a row; put an X in each and label them "dress1 / ₦", "dress2 / ₦", "dress3 / ₦".
5. Draw a two-column band: left = wavy lines labelled "Our Story", right = "Phone / Address".
6. Draw the footer band; label "© 2026" and "Instagram / Facebook".

Ask at each step: "Why does the header come first?" → Consistency and navigation; it is the same on every page.

Deliberately inject a mistake: Put the phone number and address only at the very bottom, tiny. Ask: "Will a customer find this in ten seconds?" → No. Move contact details up into the footer where they are visible on every page.

G.4 Demonstration 3 — palette and file structure (4 minutes)

1. On the board, write the palette: primary `#4b2e83`, accent `#f4c542`, background `#faf7ff`, text `#222222`.
2. Show a quick contrast comparison: write one sentence in light grey on white, another in near-black on white. Ask which is easier to read in bright light. → The near-black one.
3. Open a sample project folder in VS Code (`adaeze-fashion/`) with `index.html`, `css/style.css`, `images/`, and `js/`. Point out each file's place.
4. Rename one file from `About.html` to `about.html` live, and explain the Windows-vs-Linux case-sensitivity trap.

Questions to ask during the demos:

- "What does each box in the wireframe represent in HTML?" → `<header>`, `<section>`, `<footer>`, etc.
- "Why do we keep the menu to five items or fewer?" → So it stays easy to read and tap on a phone.
- "Which folder holds the stylesheet?" → `css/`.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners produce a **sitemap** and a **home-page wireframe** for *Adaeze's Fashion*, and start their site-plan document.

Starting materials: Two sheets of A4 paper, a pen or pencil, the printed "Adaeze's Fashion brief" and the site-plan template.

Procedure (instructor calls out each numbered step; learners work on paper; the instructor walks the room):

1. At the top of the first sheet, write the project name: **Adaeze's Fashion — Sitemap**.
2. Draw a box in the centre and write **HOME (index.html)** inside it.
3. Below Home, draw four boxes in a row, connected to Home with lines. Label them:
   - **About (about.html)**
   - **Shop (shop.html)**
   - **Services (services.html)**
   - **Contact (contact.html)**
4. On the second sheet, write the project name: **Adaeze's Fashion — Home Wireframe**.
5. Draw a page-sized rectangle. At the top edge write the page name, "HOME".
6. Draw the **header band** across the top. Write "LOGO" on the left and "MENU" on the right.
7. Draw a **hero band** below the header. In the middle, write the headline "Custom Fashion for Every Occasion". Draw a small rectangle under it labelled "BUTTON: View Our Collection". Put an X in the band to show a background image goes here.
8. Draw a row of **three equal boxes** for featured products. Put an X in each box (image) and two short wavy lines underneath (name and price). Label the prices "₦18,500", "₦32,000", "₦27,500".
9. Draw a **two-column band**: on the left, three wavy lines labelled "Our Story"; on the right, write "Phone: 0801 234 5678" and "Address: Aba Road, PH".
10. Draw the **footer band** at the bottom. Write "© 2026 Adaeze's Fashion" on the left and "Instagram Facebook" on the right.
11. **Pause and predict:** "Look at your sketch. If I showed this to a customer, could they tell where to click to see the clothes?" Have learners point to the button and explain.

Expected result: Each learner holds one sitemap sheet with five connected boxes, and one Home-page wireframe showing header, hero, three product boxes, a story-plus-contact band and a footer, all labelled. Nothing needs to be artistic — only clear.

Points to pause on and predict:

- After step 3: *"Why is every page one line from Home?"* → Keeps navigation shallow and simple.
- After step 8: *"Where will the prices appear on the finished page?"* → Under each product photo.
- After step 11: *"Which block do you predict will be the most important to get right on a phone?"* → The header/menu and the product boxes (so they are tappable and clear).

Troubleshooting while walking the room:

- Learner's boxes all different sizes and confusing → suggest a ruler or fold the paper into equal parts.
- Learner drew colours and fine detail → gently remind them a wireframe is boxes and labels only; save colours for later.
- Learner cannot fit everything → that is useful information: it means the page is trying to do too much. Simplify.
- Learner copies a neighbour → ask them to explain their own choices; they should own the plan.

Completion checklist (learner ticks each):

- ☐ Sitemap drawn with Home at the top and four pages below
- ☐ Every page box labelled with its file name
- ☐ Home wireframe drawn with header, hero, products, story/contact, footer
- ☐ Images marked with an X
- ☐ Text marked with wavy lines
- ☐ The call-to-action button shown
- ☐ Sketch is clear enough for someone else to understand

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: On your own, plan the **Shop page** of Adaeze's Fashion: draw its wireframe and write its content plan. Then complete a small site-plan document for the whole site.

Scenario: Adaeze's Shop page must let customers browse her collection and know how to order. Customers mostly use phones.

Starting information (given):

- Business: *Adaeze's Fashion*.
- Products to list (invent prices if you wish): Ankara Dress, Lace Gown, Aso Ebi Set, Office Wear, Traditional Attire, Bridal Gown.
- There is **no online checkout** in this project; ordering happens by contacting Adaeze.

Requirements:

1. On a fresh sheet, draw a **wireframe of the Shop page**. It must include:
   - the same header band (logo left, menu right),
   - a page heading band ("Our Collection"),
   - a **grid of six product boxes** (two rows of three), each with an X (image), a name line and a price line,
   - a short "how to order" note,
   - the same footer band.
2. Write a **content plan** for the Shop page: the heading, the six product names with prices, and the "how to order" sentence.
3. Complete a **site-plan document** for the whole site with the following fields:

```
PROJECT: Adaeze's Fashion
PURPOSE (1–2 sentences): _____
PRIMARY AUDIENCE: _____
PAGES: _____
COLOURS: primary ____  accent ____  text ____  background ____
FONTS: headings _____   body _____
FILE STRUCTURE:
  adaeze-fashion/
    index.html
    about.html
    shop.html
    services.html
    contact.html
    css/style.css
    images/
    js/
```

Constraints:

- Wireframe uses boxes and labels only — no colours.
- File names are lowercase with hyphens; Home is `index.html`.
- At least two colours and two fonts specified, with a reason.

Time allowance: 15 minutes.

Success criteria:

- A clear Shop wireframe with a six-item product grid and a "how to order" note.
- A completed content plan.
- A filled site-plan document with purpose, audience, pages, palette, fonts and file structure.

Hints available if stuck:

- Hint 1: The Shop page reuses the exact same header and footer as the Home page — copy them from your Home wireframe.
- Hint 2: Six products in two rows of three = six equal boxes.
- Hint 3: For colours, choose one main brand colour and one accent, plus near-black text on white.

Instructor answer key (do not show learners):

Shop wireframe (described): Top header band (LOGO left, MENU right); heading band "Our Collection"; a 3×2 grid of product boxes, each with an X, a name line and a price line; a one-line note "To order, call 0801 234 5678 or use the contact form"; footer band with copyright and social links.

Content plan:

```
Heading: Our Collection
Products:
```

```
  Ankara Dress — ₦18,500
  Lace Gown — ₦32,000
  Aso Ebi Set — ₦27,500
  Office Wear — ₦15,000
  Traditional Attire — ₦24,000
  Bridal Gown — ₦75,000
How to order: To order, contact us by phone or use the contact form. We deliver within Port Harcourt.
```

Site-plan document (model answer):

```
PROJECT: Adaeze's Fashion
PURPOSE: To showcase Adaeze's clothing and make it easy for customers to contact her to order.
PRIMARY AUDIENCE: Adults in Port Harcourt who buy or commission clothing, mostly browsing on phones.
PAGES: Home, About, Shop, Services, Contact.
COLOURS: primary #4b2e83, accent #f4c542, text #222222, background #faf7ff.
FONTS: headings Montserrat, body Open Sans (fallback Arial).
FILE STRUCTURE: as given (index.html, about.html, shop.html, services.html, contact.html, css/style.css, images/, js/).
```

Marking guidance (out of 10):

- Shop wireframe complete with header, heading, six-product grid and footer (3)
- Content plan with six products and prices (3)
- Site plan purpose and audience (2)
- Palette and fonts specified with a reason (1)
- File structure and naming correct (1)

J. REAL-WORLD CASE STUDY

Situation. Two freelance designers each take on a small business website for a bakery in Port Harcourt. *Designer A* starts writing code immediately, excited to show off. *Designer B* spends half a day on planning: a short brief chat with the owner, a sitemap, paper wireframes, a list of content, a colour palette and fonts, and a folder structure.

Relevant facts.

- The bakery owner wants customers to see cakes and order by phone.
- Both designers are equally skilled at HTML and CSS.
- The owner changes their mind partway through ("actually, I want a page for our delivery areas").

What happens to Designer A. Because there is no structure, the new "delivery areas" request forces changes across every page. The navigation must change everywhere, the folder grows messy, and the colour choices drift page by page. The project runs long and looks inconsistent.

What happens to Designer B. The sitemap makes the new page easy to slot in: add one box to the map, one wireframe, one HTML file. The consistent template and folder structure mean the change costs an hour, not a week. The final site looks unified and professional.

Decision point. Why did planning give Designer B such an advantage?

Possible approaches (for any project).

1. Skip planning and code immediately.
2. Plan fully (brief, sitemap, wireframes, content, palette, structure) before coding.
3. Plan a little and adjust as you go.

Recommended solution. Approach 2 for a project of any real size. A short, focused planning session makes later changes cheap and keeps the site consistent.

Trade-offs.

- Approach 1 gives a fast first page but a slow, painful project overall.
- Approach 2 costs a few hours up front and saves days later.
- Approach 3 is a reasonable middle path for very small projects, but even then a sitemap and wireframe are worth the time.

Professional connection. In real agencies, planning documents (brief, sitemap, wireframes) are deliverables the client sometimes signs off before development begins. Being able to produce them is a genuine professional skill, and it is often what separates a paid designer from an amateur.

K. COMMON MISTAKES AND TROUBLESHOOTING

Because this session is about planning, the "troubleshooting" here is about planning mistakes rather than code bugs.

K.1 Skipping the plan and jumping straight to code

Symptoms: The learner wants to open VS Code immediately and starts typing. **Why it hurts:** Layout, navigation and structure will be inconsistent; changes become expensive. **Fix:** Insist on a sitemap and one wireframe before coding. Ten minutes of drawing saves an hour of rework. **Prevention:** Treat planning as the first step of every project, not an optional extra.

K.2 Vague purpose ("to have a website")

Symptoms: The purpose statement could apply to any business. **Why it hurts:** Without a clear purpose, there is no way to decide what belongs on the page. **Fix:** Rewrite it specifically: "To showcase Adaeze's clothing and let customers contact her to order." **Prevention:** Always name the outcome (what the visitor should be able to do).

K.3 Too many pages or too few

Symptoms: A twelve-page site with empty pages, or a one-page site hiding everything. **Why it hurts:** Empty pages look unfinished; cramming everything onto one page overwhelms visitors. **Fix:** Aim for 3–5 clear pages, each with one job. **Prevention:** Let the sitemap reveal overlap; merge pages that do the same thing.

K.4 Over-detailed wireframe (colours, fonts, shading)

Symptoms: The learner spends a long time making the sketch "pretty". **Why it hurts:** It wastes time and distracts from the point — the layout. **Fix:** Boxes, labels, X for images, wavy lines for text. Nothing more. **Prevention:** Remind learners that colours come in Session 9.

K.5 Inconsistent page structure

Symptoms: Different pages have different headers or menus. **Why it hurts:** Visitors feel lost; the site feels untrustworthy. **Fix:** Draw the same header and footer on every wireframe. **Prevention:** Plan a single shared template for header and footer that every page reuses.

K.6 Bad file names

Symptoms: Files like `About Page.html`, `dress photo 1.jpg`, `Shop.html`. **Why it hurts:** Spaces and capitals break links once published to a Linux server. **Fix:** Rename using lowercase and hyphens: `about.html`, `dress-photo-1.jpg`, `shop.html`. **Prevention:** Agree the naming rules before creating any files.

K.7 Low-contrast colour choices

Symptoms: Light grey text on white; yellow text on white. **Why it hurts:** Text is hard or impossible to read, especially in bright light or on older screens. **Fix:** Use near-black text on light backgrounds, and white text on dark buttons. **Prevention:** Use the "greyscale photocopy" test described in F.7.

L. SESSION ASSIGNMENT

Assignment title: "My Website Plan"

Purpose: Produce a complete, professional plan for the *Adaeze's Fashion* website before any coding begins.

Realistic scenario: Adaeze has hired you to build her website. Before you touch any code, she wants to see a plan so she can approve it. This mirrors real client work.

Detailed instructions:

1. **Brief summary:** write the purpose of the website in one or two sentences and list the primary audience.
2. **Sitemap:** draw a clear sitemap showing Home at the top and all other pages below, each labelled with its file name.
3. **Wireframes:** draw a wireframe for **each** of the following pages:
    - Home
    - Shop
    - About
    - Contact Each wireframe must show the header (logo + menu), the main content blocks for that page, and the footer.
4. **Content plan:** write the actual text and list the images each page needs (headings, paragraphs, product names and naira prices, contact details).
5. **Colour palette and fonts:** choose a primary colour, an accent colour, a text colour and a background colour (give hex codes), and choose one font for headings and one for body text. Write one sentence explaining your choice and confirming good contrast.
6. **File structure:** draw the project folder tree, using correct lowercase, hyphenated names and `index.html` for the Home page.

Required deliverables:

- A single, tidy "My Website Plan" document (handwritten on paper is perfectly acceptable, or typed).
- Sitemap, four wireframes, content plan, palette/fonts and file structure.

Submission format: Hand in the paper plan (photographed into the workbook) or a typed document. Keep the original for reference when coding in Session 8.

Deadline: Before Session 8.

Marking rubric (25 marks):

Criterion	Marks	Description of a strong answer
Purpose & audience	3	Specific, believable, audience-aware
Sitemap	4	Home plus all pages, labelled with file names, shallow structure
Wireframes (all four)	8	Two per page clearly drawn; header, content, footer present on each (2 each)
Content plan	4	Real text and image needs listed for every page
Palette & fonts	3	Sensible colours with hex codes, two fonts, contrast considered
File structure	3	Correct names, folders, `index.html` for Home

Common mistakes to avoid: skipping a page's wireframe; using capitals or spaces in file names; vague purpose; low-contrast colours; forgetting the footer on a page.

Instructor reference solution (summary):

- Purpose: "To showcase Adaeze's clothing and make it easy for customers to contact her to order." Audience: adults in Port Harcourt, mostly on phones.
- Sitemap: Home → About, Shop, Services, Contact (each labelled with its file name).
- Wireframes: each page shares the same header/footer; Home has hero + three products; Shop has a six-item grid; About has story text + photo + values; Contact has a form and contact details.
- Content: as in the Section I key.
- Palette: `#4b2e83`, `#f4c542`, `#222222`, `#faf7ff`; fonts Montserrat + Open Sans; reason: elegant, on-brand, high contrast.
- File structure: as in the Section H/I key.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. In one sentence each, what is a *sitemap* and what is a *wireframe*?

Q2. Multiple choice. What should the Home page file be named? a) `home.html` b) `index.html` c) `Home Page.html` d) `main.html`

Q3. Explain in your own words. Why is it better to plan a website before writing any code?

Q4. Practical. List the colour palette and fonts you chose for Adaeze's Fashion, and give one reason for your choice.

Q5. File naming. Which of these file names are correct, and why? a) `About.html` b) `shop.html` c) `ankara dress.jpg` d) `contact.html`

Q6. Contrast. A learner puts light grey text (`#cccccc`) on a white background. Why is this a problem, and how would you fix it?

Q7. Scenario. Adaeze asks you to add a "Testimonials" page. Explain three steps you would take in your plan (not in code) to add it.

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. A sitemap is a diagram showing all the pages of a website and how they relate (usually Home at the top with pages branching off). A wireframe is a rough, labelled sketch of a single page showing where each block of content goes, using boxes and labels rather than colours or fonts.

A2. (b) `index.html` .

A3. Because planning defines the purpose, pages, content, colours and structure in advance, so you can make changes cheaply (move a box on paper) before the expensive work of coding. It prevents inconsistency, broken links and rework, and mirrors how professional designers work.

A4. Any sensible answer. Example: primary `#4b2e83` (deep purple), accent `#f4c542` (gold), text `#222222` , background `#faf7ff` ; headings Montserrat, body Open Sans. Reason: elegant and on-brand for fashion, with strong contrast for readability.

A5. Correct: (b) `shop.html` and (d) `contact.html` — lowercase, hyphen-free, no spaces. Incorrect: (a) starts with a capital letter (breaks on Linux servers); (c) contains a space (becomes `%20` and is error-prone). Correct versions: `about.html` , `ankara-dress.jpg` .

A6. Light grey on white has very low contrast, so the text is hard to read — especially in bright light or for people with weaker eyesight. Fix by using a dark text colour such as `#222222` on the white background (high contrast), and reserve light grey for large, secondary elements only.

A7. Example steps: (1) add a "Testimonials" box to the sitemap under Home; (2) draw a wireframe for the new page (header, heading, a few testimonial cards, footer); (3) plan its content (real quotes with names) and pick a file name such as `testimonials.html` , then add it to the navigation. (Also accept: update the site plan document and check the menu still has five or fewer items.)

N. REFLECTION AND RECAP

Plain-English summary. Today you learned the professional habit that comes before code: planning. You turned a client brief into a purpose and goals, described the audience, drew a **sitemap** of the pages, sketched **wireframes** for each page, planned the **content**, chose a simple **colour palette and fonts** with good **contrast**, and agreed on a clean **file and folder structure** with sensible **naming conventions**. You produced a paper plan for *Adaeze's Fashion* that you will build from starting next session.

Five important takeaways:

1. Always start with the brief: purpose, audience, pages, content.
2. Draw a sitemap and a wireframe for every page before coding.
3. Aim for 3–5 clear pages, each with one job.
4. Choose a small palette and one or two fonts; keep contrast high.
5. File names: lowercase, hyphens, `index.html` for Home.

Questions to help you recognise what you can now do:

- Can I turn a client's goal into a sitemap?
- Can I sketch a page layout in ten minutes on paper?
- Can I explain why `About.html` will break online?
- Can I tell whether text on a background is readable?

Connection to the next session: You now hold the blueprint for Adaeze's Fashion. Next session, Session 8, we turn that plan into real HTML — creating `index.html`, `about.html`, `shop.html`, `services.html` and `contact.html`, each with the same header, navigation and footer, all correctly linked together. Bring your plan to class; you will build directly from it.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- The learner reads the brief and states the purpose specifically, not vaguely.
- The learner's sitemap is shallow and every page has a clear, distinct job.
- The learner draws wireframes that are clean and labelled, not over-decorated.
- The learner reuses the same header and footer across every page's wireframe.
- The learner chooses a small palette with strong contrast and explains why.
- The learner uses correct file names (lowercase, hyphens, `index.html`) without prompting.

Common misconceptions to watch for and correct:

- "Planning is a waste of time; let me just code." → Planning is what makes coding fast and consistent.
- "A wireframe should look beautiful." → It only needs to be clear; details come later.
- "More pages = more professional." → Fewer, focused pages beat many empty ones.
- "Colours do not matter much." → Contrast decides whether people can read your site.
- "File name capitalisation does not matter." → It matters the moment you publish to a Linux server.

Suggested remedial exercises for learners who struggle:

1. **Brief-to-sitemap drill:** give a one-paragraph brief and ask for a four-page sitemap in five minutes.
2. **Wireframe labelling:** give a blank rectangle and a list of blocks (header, hero, products, footer) and ask the learner to place and label them.
3. **Naming fix-up:** give a list of ten bad file names and ask the learner to correct each.

Differentiation:

- **Level 1 (supported):** provide a partially completed wireframe (header and footer already drawn) and a partially filled site plan; the learner completes them.
- **Level 2 (standard):** the practical and independent exercises as written.
- **Level 3 (challenge):** plan a more ambitious site (six or seven pages, adding a Gallery and a Testimonials page), and design a mobile-specific wireframe (a phone-shaped sketch) alongside the desktop one.

Timing guidance: The drawing is the heart of this session. If the room is slow, reduce the lecture explanations (Section F) to the essentials and protect at least 15 minutes for the guided wireframe. If the room is fast, let learners begin a second wireframe (the Shop page) live in class, so they arrive at Session 8 with more of the plan done.

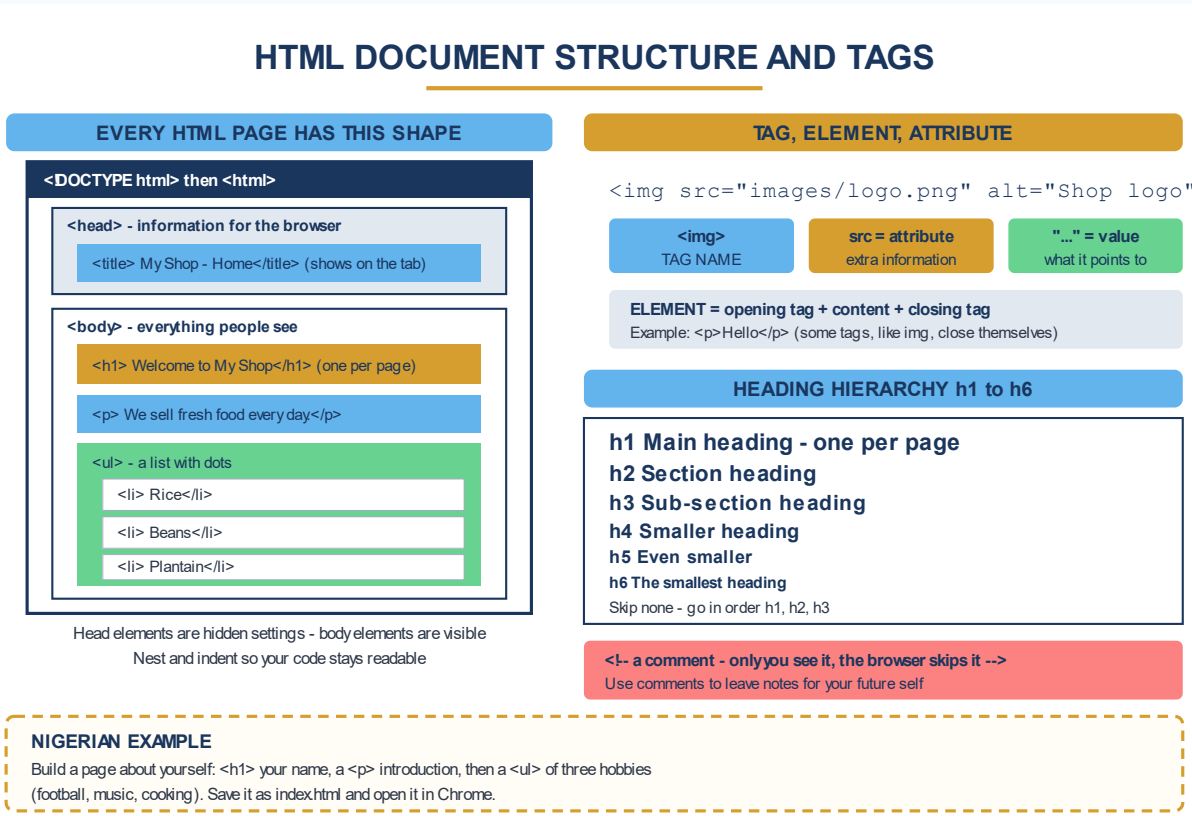
END OF SESSION 7

Next: Session 8 — Building the Structure.

SESSION 08 OF 12

BUILDING THE STRUCTURE

CONCEPT DIAGRAM — SESSION 08



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 8 — BUILDING THE STRUCTURE

A. SESSION IDENTIFICATION

Item	Detail
Session number	8 of 12
Session title	Building the Structure
Week / Module	Week 3 — Module 3: Building a Website
Recommended duration	105 minutes
Difficulty level	Beginner (comfortable with HTML from Sessions 1–3 and structure from Session 7)

Item	Detail
Prerequisite knowledge	Sessions 1–7. The learner must be able to write a valid HTML page with headings, paragraphs, links, images, lists and semantic elements, and must have produced a website plan (sitemap, wireframes and content) in Session 7. If a learner has no plan, give them a printed copy of the instructor's plan at the start of class.
Connection to previous work	In Session 7 learners drew the blueprint for *Adaeze's Fashion*: the sitemap, wireframes, content and file structure. This session turns that blueprint into the real HTML skeleton — the five pages, their shared header and footer, and the content placeholders — ready to be styled in Session 9.
Preparation before class	Academy PCs on and logged in; VS Code and Chrome installed; the `adaeze-fashion` folder from Session 7 prepared on each machine (or created fresh this session); small placeholder images copied into `images/` (or learners use simple coloured boxes); the projector set to show VS Code and Chrome side by side; print the "Multi-Page Website Checklist" (one per learner) and the "Adaeze's Fashion content sheet" (from Session 7).

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Create multiple HTML files** for a multi-page website using correct file names, including `index.html` for the Home page.
2. **Build a consistent header and navigation** (logo plus menu) that is identical on every page.
3. **Build a consistent footer** (copyright, contact details, social links) that is identical on every page.
4. **Use semantic HTML elements** — `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>` — correctly across a real project.
5. **Link pages together** using correct relative paths, and verify every link works.
6. **Add images with descriptive `alt` text** and correct relative image paths.
7. **Build a contact form** with proper `<label>` elements, input types and the `required` attribute (markup only — styling comes in Session 9).
8. **Organise the project folder** so that HTML, CSS, images and JavaScript each live in the right place.
9. **Explain why** a consistent structure across pages matters for maintainability and user trust.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). A shared computer between two learners is acceptable; pair them so one types and one checks links, then swap halfway.
- Instructor computer connected to a projector or large screen.

Software

- Visual Studio Code (VS Code). Any version from 2022 onwards works.
- Google Chrome (any recent version).

Accounts

- None required. All work this session is offline.

Sample files

- The instructor prepares and shares a `starter/adaeze-fashion/` folder containing:
  - an empty `css/` folder (the stylesheet is written in Session 9),
  - an `images/` folder with a few small placeholder images (`logo.png`, `hero.jpg`, `dress1.jpg`, `dress2.jpg`, `dress3.jpg`, `about.jpg`),
  - an empty `js/` folder for later.

- If images are unavailable, learners can use a dark grey box later; but for the image tags this session, reference the intended file names so the paths are ready.

Printed materials

- "Multi-Page Website Checklist" (one per learner).
- The learner's own plan from Session 7 (or the instructor's plan if missing).

Tools

- Chrome DevTools (built into Chrome — press F12). Used briefly to confirm links.
- Optional: a smartphone or tablet for checking the structure later (Session 9 focuses on styling).

Internet

- Not required for this session.

Alternative for learners who lack a facility

- Any laptop with VS Code and Chrome works. Learners with only a phone can pair with a desktop user and, afterwards, view the file structure using a free phone code editor if available.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, display the *Adaeze's Fashion* Home page wireframe from Session 7 beside an empty VS Code window. Then, from a hidden tab, show the finished HTML skeleton (unstyled) of the same page — the header, hero text, three product blocks and footer, with the browser's default plain look.

What the instructor says:

> *"Last session you drew this blueprint. Today we build the house. You may notice the finished structure looks plain — black text on white, everything stacked. That is exactly right. Today we build the **structure**, the skeleton. Painting and decorating come next session. If you skip the skeleton and try to make it pretty first, you will spend the whole day fighting a broken structure.*
>
> *By the end of today, you will have five real pages of the Adaeze's Fashion website — Home, Shop, Services, About and Contact — all linked together, all sharing the same header and footer, all ready for CSS."*

Expected learner response: Learners may be impatient that it looks "ugly". Reassure them: an unstyled but well-structured page is a professional milestone; styling is quick once the structure is solid.

Then demonstrate the hook: In the projector browser, click the links in the starter page to move between Home, About, Shop and Contact — showing that the same menu works everywhere. Say:

> *"Watch how the menu follows me to every page. That is the magic of a consistent structure. By the end of today, your own menu will do this."*

Why this opener works: It connects directly to the learner's own plan, frames "plain-looking" as intentional and professional, and shows the payoff (working navigation) before the work begins.

E. PREREQUISITE REVIEW (7 minutes)

Run this as an oral round-robin. Put the six questions on the projector. Keep it quick — the practical work needs the time.

Question 1: "What are the five pages planned for Adaeze's Fashion?"

- Expected answer: Home (index.html), About, Shop, Services, Contact.
- Common gap: forgetting Services. Remind them it is optional but planned.

Question 2: "What file name must the Home page always have, and why?"

- Expected answer: `index.html`, because web servers look for that name automatically.
- Common gap: naming it `home.html`. Restate the reason.

Question 3: "Name the semantic elements we use for page structure, from top to bottom."

- Expected answer: `<header>`, `<nav>`, `<main>` (with `<section>` / `<article>`), `<footer>`.
- Common gap: using `<div>` everywhere. Clarify: `<div>` is fine for wrapping, but semantic tags describe meaning.

Question 4: "What is a relative path? Give an example linking to the About page and to an image."

- Expected answer: a path relative to the current file: `href="about.html"` and `src="images/dress1.jpg"`.
- Common gap: full Windows paths. Remind them we never use `C:\Users\...`.

Question 5: "What attribute makes an image accessible when it cannot be seen?"

- Expected answer: `alt` (alternative text).
- Common gap: leaving `alt` empty or writing "image". Remind them it should *describe* the image.

Question 6: "Which folder holds the stylesheet, and which holds the photos?"

- Expected answer: `css/style.css` and `images/`.
- Common gap: everything in the root. Remind them of the planned structure.

Bridge to today (say this):

> *"Your plan is the map. Today we follow it and build five pages that all share one header and one footer, and that link to each other correctly. Consistency is the whole game: if the menu works on one page, it must work on all of them."*

If more than a third of the class did not produce a Session 7 plan, hand out the instructor's plan and give a 3-minute recap of the sitemap before continuing.

F. FULL LECTURE NOTES

F.1 Creating multiple HTML files — one file per page

The idea. A multi-page website is simply several HTML files in the same folder, each representing one page.

Plain-English definition. Each page of the site is its own `.html` file. When a visitor clicks a link, the browser loads a *different file* and shows it. There is no "one giant file" containing the whole website.

Why it exists. Splitting a site into separate files keeps each file small and manageable, lets several people work on different pages, and lets the server send only the page the visitor asked for (faster loading).

The file names for our project:

Page	File name	Why this name
Home	`index.html`	The default page a server shows first. Always this exact name.
About	`about.html`	Lowercase, descriptive.
Shop	`shop.html`	Lowercase, descriptive. (We do not call it `products.html` to match our menu label.)
Services	`services.html`	Lowercase, descriptive.
Contact	`contact.html`	Lowercase, descriptive.

Naming rules (repeated from Session 7 — they are that important):

1. All lowercase.
2. Hyphens instead of spaces.
3. Short and descriptive.
4. No special characters (`&`, `#`, `?`, spaces).

5. The Home page is always `index.html`.

Worked example — the same page shell used for every file. Every `.html` file begins with the same first block:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <!-- header, main content and footer go here -->
</body>
</html>
```

Line by line:

- `<!DOCTYPE html>` — tells the browser this is modern HTML5.
- `<html lang="en">` — the root element; `lang="en"` says the page is in English (helps screen readers and search engines).
- `<meta charset="UTF-8">` — lets the page show the ₦ symbol and other characters correctly.
- `<meta name="viewport" ...>` — makes the page work on phones (Session 6).
- `<title>` — the text shown on the browser tab; each page has its own title.
- `<link rel="stylesheet" href="css/style.css">` — connects the shared stylesheet (empty until Session 9).
- `<body>` — everything the visitor sees.

Expected browser result: If you open any of these files now, you see a blank white page with the correct title in the browser tab. That is correct — the content comes next.

Misconception. "I need a server to make multi-page files link." No. You can open `index.html` by double-clicking it, and links to `about.html` will work locally.

Limitation. With separate files, if you change the header you must change it on every page. (There are ways to share it automatically, but they need tools beyond this course. For five pages, we simply copy the shared blocks carefully.)

F.2 Building a consistent header and navigation

The idea. The **header** is the top of every page. It contains the site's identity (the logo or business name) and the **navigation** (nav) — the menu that moves the visitor between pages.

Plain-English definition. The header is a shared bar at the top; the navigation is the set of links inside it. Because they are identical on every page, the visitor always knows where they are and how to move around.

Why it exists. Consistency builds trust and reduces confusion. If the menu moved or changed on each page, visitors would get lost and leave. A shared header is the single most important consistency feature of a multi-page site.

The header markup for Adaeze's Fashion:

```
<header class="topbar">
  <a href="index.html" class="logo">Adaeze's Fashion</a>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="shop.html">Shop</a></li>
      <li><a href="services.html">Services</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```

Why each part is there:

- `<header>` — the semantic container for the top area.
- `<a href="index.html" class="logo">` — the business name, wrapped in a link so clicking it always returns Home.

- `<nav>` — the semantic container for navigation links.
- `<ul>` and `<li>` — an unordered list is the standard, accessible way to mark up a menu (screen readers announce "list of 5 items", which helps users).
- Each `<a href="...">` — a relative link to another page's file.

Copy this block exactly onto every page. The only thing that changes from page to page is the `<title>` and the `<main>` content — never the header.

Expected browser result (unstyled): A plain list of five links stacked vertically, with the business name above them: *Adaeze's Fashion*, then *Home*, *Shop*, *Services*, *About*, *Contact*. It looks plain now, but in Session 9 the Flexbox CSS from Session 5 will turn it into a purple bar with the logo on the left and the menu on the right.

Misconception. "Lists are only for bullet points of ordinary text." Lists are the correct markup for menus too, because a menu *is* a list of links. Screen readers and search engines understand it better this way.

Limitation. The plain `<ul>` shows bullet points until CSS removes them. That is expected — styling is next session.

F.3 Building the main content — Home, Shop, About, Contact

The idea. The **main content** is the part of the page that is *different* on every page. It lives inside `<main>`.

Plain-English definition. `<main>` holds the unique content of this page (the reason the visitor came). By convention, a page has exactly **one** `<main>`.

Why it exists. Marking the main content helps search engines and assistive technology skip straight to the important part of the page, ignoring the repeated header and footer.

Home page main content:

```
<main>
  <section class="hero">
    <h1>Custom Fashion for Every Occasion</h1>
    <p>Beautiful, affordable clothing, handmade in Port Harcourt.</p>
    <a href="shop.html" class="button">View Our Collection</a>
  </section>

  <section class="featured">
    <h2>Featured Products</h2>
    <div class="grid">
      <article class="product">
        <img src="images/dress1.jpg" alt="Ankara dress in bright colours">
        <h3>Ankara Dress</h3>
        <p class="price">₦18,500</p>
      </article>
      <article class="product">
        <img src="images/dress2.jpg" alt="Lace gown with long sleeves">
        <h3>Lace Gown</h3>
        <p class="price">₦32,000</p>
      </article>
      <article class="product">
        <img src="images/dress3.jpg" alt="Aso ebi set in matching fabric">
        <h3>Aso Ebi Set</h3>
        <p class="price">₦27,500</p>
      </article>
    </div>
  </section>
</main>
```

Line by line:

- `<section>` groups related content; each section usually has its own heading.
- The `.hero` section is the big welcome banner.
- `<h1>` is the main page heading — use **one** `<h1>` **per page**.
- The `.button` link points to the Shop page — a call to action.
- `.featured` lists three products.
- Each product is an `<article class="product">` — an independent, self-contained piece of content (a card).
- `<img>` has a relative `src` and a meaningful `alt`.

- `.price` shows the naira amount.

Shop page main content:

```
<main>
  <section class="page-head">
    <h1>Our Collection</h1>
  </section>

  <section class="shop">
    <div class="grid">
      <article class="product">
        <img src="images/dress1.jpg" alt="Ankara dress in bright colours">
        <h3>Ankara Dress</h3>
        <p class="price">₦18,500</p>
      </article>
      <!-- ...five more product articles... -->
    </div>
    <p class="order-note">To order, call 0801 234 5678 or use our contact form.</p>
  </section>
</main>
```

Expected browser result (unstyled): The heading *Our Collection*, then six product blocks stacked vertically (each with its image, name and price), and finally the order note. In Session 9, this becomes a responsive grid.

About page main content:

```
<main>
  <section class="about">
    <h1>Our Story</h1>
    <img src="images/about.jpg" alt="Adaeze at her sewing machine in the workshop">
    <p>Adaeze Okafor started Adaeze's Fashion with a single sewing machine in 2019.
       What began as a hobby soon grew into a trusted name in Port Harcourt.</p>
    <p>Today the workshop creates Ankara dresses, lace gowns, aso ebi sets and office
       wear for customers across Rivers State — all sewn by hand, with care.</p>

    <h2>What We Value</h2>
    <ul>
      <li><strong>Quality</strong> — every stitch is checked before it leaves our shop.</li>
      <li><strong>Affordability</strong> — beautiful clothing that fits a real budget.</li>
      <li><strong>Style</strong> — modern designs that celebrate Nigerian fabrics.</li>
    </ul>
  </section>
</main>
```

Expected browser result (unstyled): The story heading, the workshop photo, two paragraphs, then a sub-heading *What We Value* with a bulleted list. (This is the same Values content from Session 6's exercise, now in real HTML.)

Contact page main content (with the form):

```
<main>
  <section class="contact">
    <h1>Get in Touch</h1>

    <form action="#" method="post">
      <label for="name">Your Name</label>
      <input type="text" id="name" name="name" required>

      <label for="email">Email Address</label>
      <input type="email" id="email" name="email" required>

      <label for="message">Message</label>
      <textarea id="message" name="message" rows="5" required></textarea>

      <button type="submit">Send Message</button>
    </form>

    <div class="contact-info">
      <h2>Visit or Call Us</h2>
      <p>Aba Road, Port Harcourt, Rivers State</p>
      <p>Phone: 0801 234 5678</p>
```

```
      <p>Instagram: @adaezefashion</p>
    </div>
  </section>
</main>
```

Line by line:

- `<form action="#" method="post">` — a form that will later send to the given address; `#` is a placeholder for now.
- `<label for="name">` — a label connected to the input with the matching `id="name"`. Clicking the label focuses the input; screen readers read it aloud. **This is an accessibility requirement, not optional.**
- `<input type="text" id="name" name="name" required>` — a single-line text box. `required` means the browser will not submit the form if it is empty.
- `<input type="email" ...>` — the browser also checks that the text looks like an email address.
- `<textarea rows="5">` — a multi-line message box.
- `<button type="submit">` — sends the form.

Expected browser result (unstyled): A simple form with each label on its own line above its input box, a large message box, and a "Send Message" button. Crucially, if you click "Send Message" with the fields empty, the browser shows a small bubble saying the field is required — that comes from the `required` attribute, with no CSS or JavaScript.

Why the form has no working "send" yet. Sending a form to an email needs a server or a form service. This session builds the **markup** only. The point today is correct, accessible, semantic form structure. (Styles arrive in Session 9; interactivity in Session 10; understanding the limits honestly is part of being professional.)

Misconception. "The `placeholder` attribute can replace a `<label>`." No. A placeholder disappears as soon as the user types, and screen readers handle it inconsistently. Always use a real `<label>`. (You may *also* add a placeholder for users' convenience.)

Limitation. A basic HTML form cannot send email by itself. We are honest about this: the markup is correct, but a real working form needs a service such as a form-handling tool or a server — beyond this course, though worth knowing.

F.4 Building the footer

The idea. The **footer** is the bar at the bottom of every page. It usually holds the copyright, contact details and social links.

Plain-English definition. A footer is a shared closing section, repeated on every page, giving the visitor quick access to essential information.

Why it exists. Good footers put contact details on every page so a customer can reach the business without hunting. They also carry the copyright notice and social links.

The footer markup for Adaeze's Fashion:

```
<footer class="site-footer">
  <p>&copy; 2026 Adaeze's Fashion. All rights reserved.</p>
  <div class="social">
    <a href="https://instagram.com">Instagram</a>
    <a href="https://facebook.com">Facebook</a>
  </div>
  <p>Aba Road, Port Harcourt &middot; 0801 234 5678</p>
</footer>
```

Line by line:

- `<footer>` — the semantic footer container.
- `©` — a special code that displays the © symbol.
- `.social` — a group of external links (these are full URLs because they point outside our site).
- `·` — displays a small centred dot (·) between the address and phone.

Copy this exact footer onto every page, just before `</body>`.

Expected browser result (unstyled): Three lines at the bottom: the copyright, the two social links side by side, and the address/phone. Plain now; styled later.

Misconception. "The footer is only for copyright." A footer is prime real estate: contact details, opening hours, social links and quick links all belong there, on every page.

F.5 Linking pages together with relative paths

The idea. Pages are connected by `<a href="...">` links, and the path inside `href` tells the browser where to find the other file.

Plain-English definition. A **relative path** describes where a file is *relative to the file you are in*. A link to `about.html` (no folder) means "a file named `about.html` sitting beside me".

Why it exists. Relative paths keep the site portable: the same links work whether the files sit on your Desktop, on a USB stick, or on a web server. Absolute paths like `C:\Users\Adaeze\...` break the moment the site moves.

The paths we use:

Link target	`href` value	Meaning
Another page in the same folder	`href="about.html"`	A sibling file
An image in a subfolder	`src="images/dress1.jpg"`	A file inside the `images/` folder
The stylesheet in a subfolder	`href="css/style.css"`	A file inside the `css/` folder
An external website	`href="https://instagram.com"`	A full URL (necessary because it is outside our site)

Worked example — link the menu to all pages:

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="shop.html">Shop</a></li>
    <li><a href="services.html">Services</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>
```

Expected browser result: From any page, clicking a menu link loads the matching page. Clicking the logo (`index.html`) always returns Home.

The case-sensitivity trap. On Windows, `About.html` and `about.html` are the same file. On most web servers (Linux), they are *different* files. If your HTML says `href="About.html"` but the file is `about.html` , the link works on your laptop and breaks online. **Always match the case exactly, and always use lowercase.**

Misconception. "The file must be in the exact place the link says only on my computer." No — the structure on your computer *becomes* the structure on the web. Get it right locally and it works everywhere.

Limitation. Relative links only find files that actually exist at that path. A typo in either the link or the file name produces a "file not found" / 404. Always test every link (see Section K).

F.6 Images and `alt` text

The idea. Images are added with `<img src="..." alt="...">` , and the `alt` text describes the image for people who cannot see it.

Plain-English definition. `src` is the path to the image file; `alt` is the alternative text shown if the image fails to load and read aloud by screen readers.

Why it exists. `alt` text makes the site accessible to blind and low-vision users, helps if the image is slow or broken, and gives search engines useful information. It is a basic professional and ethical requirement.

Rules for writing good `alt` text:

1. Describe what the image *shows*, briefly. Example: `alt="Ankara dress in bright colours"` , not `alt="image1"` .
2. Do **not** start with "Image of" or "Picture of" — screen readers already say it is an image.
3. If an image is purely decorative (adds no information), use `alt=""` (empty) so screen readers skip it.
4. Keep it short and useful.

Worked example:

```
<img src="images/dress1.jpg" alt="Ankara dress in bright colours">
```

Expected browser result: The dress photo appears. If the file were missing, the browser would show the `alt` text *"Ankara dress in bright colours"* in its place — proof that the alt text is doing its job.

Misconception. "`alt` is optional." It is effectively required for a professional site. Leaving it out harms accessibility and looks careless.

F.7 Organising the project folder

The idea. Keep every file in its planned place.

Plain-English definition. The project folder (our root) contains the HTML files, the `css/` folder, the `images/` folder and the `js/` folder.

The final structure this session produces:

```
adaeze-fashion/
├── index.html
├── about.html
├── shop.html
├── services.html
├── contact.html
├── css/
│   └── style.css        (empty for now — written in Session 9)
├── images/
│   ├── logo.png
│   ├── hero.jpg
│   ├── dress1.jpg
│   ├── dress2.jpg
│   ├── dress3.jpg
│   └── about.jpg
└── js/
    └── script.js        (empty for now — written in Session 10)
```

Why it matters at this stage: every `href` and `src` in your HTML points into this structure. If you put an image in the wrong folder, the browser cannot find it, and you get a broken-image icon.

Expected result: In VS Code's Explorer panel, you can see all five HTML files in the root, with the `css`, `images` and `js` folders below. You can find any file in two seconds.

Misconception. "The folder layout on my desktop does not matter as long as it opens." It matters because the links are relative to it. A clean local structure equals a clean published structure.

F.8 Summary of the session

- A multi-page website = several `.html` files; Home is always `index.html`.
- Every page shares an **identical header** (logo + `<nav>` list) and **identical footer**.
- `<main>` holds the unique content; use semantic `<section>` and `<article>`.
- Link pages with **relative paths** like `href="about.html"`.
- Images use `<img src="..." alt="...">` with **descriptive alt text**.
- The **contact form** uses `<label>`, `<input>`, `<textarea>` and `required`; markup only for now.
- Keep files in the planned **folder structure**: HTML in root, CSS in `css/`, images in `images/`, JS in `js/`.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- A fresh folder `demo-webdesign/week3/adaeze-fashion/` with the `css/`, `images/` and `js/` subfolders and placeholder images.
- VS Code open on that folder.

- Chrome open to a blank tab, ready to open files.
- The Session 7 Home-page wireframe displayed (printed or on a second screen) for reference.

G.2 Demonstration 1 — building the Home page shell (6 minutes)

Say: "I will build `index.html` from the top down, exactly matching our wireframe."

1. Create `index.html`. Type the document shell (DOCTYPE, html, head with charset, viewport, title, stylesheet link, body).
2. Add the shared `<header class="topbar">` block from F.2. Save and open in Chrome. Point out the plain vertical list of links — "ugly is fine; it is the skeleton."
3. Add `<main>` with the `.hero` section (heading, paragraph, button linking to `shop.html`) and the `.featured` section with three product `<article>` cards.
4. Add the shared `<footer>` block from F.4.
5. Save and refresh. The page now reads top to bottom: menu, welcome message, three products, footer.

Ask: "Which part of this page will look the same on the next page?" → The header and the footer.

Deliberately inject an error: Change the stylesheet link to `href="styles/style.css"` (wrong folder). Refresh, and note that the page still shows (CSS is empty anyway), but in DevTools → Console there is a red error: `Failed to load resource: ... style.css`. Say: "A wrong path fails silently visually here, but the Console tells the truth. Always check it." Fix it back to `css/style.css`.

Deliberately inject a second error: Change the button link to `href="shop.htm"` (wrong extension). Click it — the browser says it cannot find the file. Ask the class to spot the mistake. Fix to `shop.html` after we create that file in the next demo.

G.3 Demonstration 2 — cloning the structure to make other pages (5 minutes)

Say: "Now the professional shortcut: copy the shared blocks, change only the unique part."

1. Save `index.html`.
2. Create `about.html`. Copy the entire `<head>` shell, the header, and the footer from `index.html`. Change the `<title>` to "Adaeze's Fashion — About". Replace the `<main>` content with the story and values from F.3.
3. Create `shop.html`, `services.html` and `contact.html` the same way (quickly, for demonstration), each with its own `<title>` and unique `<main>`.
4. Change the menu link to `shop.html` (fixing the earlier error).

Ask: "Why do we copy rather than retype the header?" → Because it must be identical; copying guarantees consistency and saves time.

Deliberately inject an error: In `about.html`, forget to update the `<title>` (still says "Home"). Note that the browser tab then mislabels the page. Fix it.

G.4 Demonstration 3 — the contact form and testing links (4 minutes)

1. Open `contact.html` and add the form markup from F.3 (labels, inputs, textarea, button, `required`).
2. Save and open in Chrome. Click "Send Message" with empty fields — the browser shows the "required field" bubble. Say: "No CSS, no JavaScript — the browser does that for free, because we marked the fields `required`."
3. Now test the navigation: from every page, click every menu link, then click the logo. Confirm all work.
4. Show DevTools → Console, and note that there are no red errors.

Questions to ask during the demos:

- "What does the `alt` attribute do if the image loads fine?" → Nothing visible, but it is read by screen readers and shown if the image fails.
- "Why does clicking the logo always go Home?" → Because the logo is a link with `href="index.html"`.
- "What does `required` do?" → Prevents submitting the form with an empty field.

G.5 Common errors to inject and correct

Injected error	Expected result	How to correct
`href="About.html"` (capital)	Works on Windows, breaks online	Use lowercase `about.html` to match the file.

Injected error	Expected result	How to correct
Missing `</a>` or `</li>`	Layout/menu breaks below the error	Close every tag.
Image path `src="image/dress1.jpg"` (wrong folder)	Broken-image icon	Correct to `images/dress1.jpg`.
Two `<h1>` elements on one page	Confusing heading hierarchy	Keep one `<h1>` per page; use `<h2>` / `<h3>` below.
`<label>` with no matching `id`	Clicking the label does nothing	Match `for="email"` to `id="email"`.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners build the full HTML skeleton of the *Adaeze's Fashion* website: `index.html`, `about.html`, `shop.html`, `services.html` and `contact.html`, each with a shared header and footer, plus the contact form.

Starting materials: A folder `adaeze-fashion` on the Desktop containing `css/` (with an empty `style.css`), `images/` (with placeholder images) and `js/` folders.

Procedure (instructor calls out each numbered step; learners follow on their machines; the instructor walks the room):

1. Open VS Code and use **File → Open Folder** to open `adaeze-fashion`.
2. Create `index.html` with the full document shell:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <!-- content will go here -->
</body>
</html>
```

3. Inside `<body>`, add the shared header and navigation:

```
<header class="topbar">
  <a href="index.html" class="logo">Adaeze's Fashion</a>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="shop.html">Shop</a></li>
      <li><a href="services.html">Services</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```

4. Add the Home `<main>` content (hero + three featured products) from F.3.
5. Add the shared footer before `</body>`:

```
<footer class="site-footer">
  <p>&copy; 2026 Adaeze's Fashion. All rights reserved.</p>
  <div class="social">
    <a href="https://instagram.com">Instagram</a>
    <a href="https://facebook.com">Facebook</a>
  </div>
```

```
  <p>Aba Road, Port Harcourt &middot; 0801 234 5678</p>
</footer>
```

6. Save `index.html`. Open it in Chrome (double-click or right-click → Open with Chrome). You should see the plain menu, welcome text, three products and the footer.
7. Create `about.html`: copy the shell, header and footer from `index.html`; change the `<title>` to "Adaeze's Fashion — About"; and set the `<main>` to the story + values from F.3.
8. Create `shop.html` the same way: change the title and use the six-product grid `<main>` from F.3. *(Tip: copy the Home page and swap the hero/featured for the shop grid.)*
9. Create `services.html`: change the title and add a `<main>` with an `<h1>` "Training Services" and three `<article>` cards (Beginner Sewing Class, Ankara Design Workshop, Business of Fashion) using the descriptions from Session 7.
10. Create `contact.html`: change the title and add the `<main>` with the **form** (labels, `required`) and the contact details from F.3.
11. Save all five files.
12. **Pause and predict:** "Before I click, in which order will the menu links take me, and what will each page's heading be?" Have learners write their prediction, then test.

Expected result: Five pages. Each has the same purple-in-Session-9 header (currently a plain list), the same footer, and its own heading and content. Clicking any menu link moves correctly between pages; clicking the logo returns Home. On `contact.html`, submitting the empty form triggers the browser's "required" message.

Points to pause on and predict:

- After step 3: *"Will this exact header appear on every page?"* → Yes; we copy it onto each.
- After step 8: *"How many products should `shop.html` show?"* → Six.
- After step 10: *"What happens if I click Send with empty fields?"* → The browser blocks it and asks for the required fields.

Troubleshooting while walking the room:

- A link gives a "file not found" → check the `href` matches the real file name exactly (lowercase).
- Broken-image icon → check the `src` path and that the image exists in `images/`.
- Menu identical except one page → the learner edited the header on one page only; re-copy the correct header everywhere.
- Naira sign shows as `?` → save the file as UTF-8 (see Section K).
- Two `<h1>` on a page → keep one; make the rest `<h2>` / `<h3>`.

Completion checklist (learner ticks each):

- ☐ `index.html` created with header, hero, featured products and footer
- ☐ `about.html` created with the same header/footer and story + values
- ☐ `shop.html` created with the same header/footer and six products
- ☐ `services.html` created with the same header/footer and three service cards
- ☐ `contact.html` created with the same header/footer and the contact form
- ☐ Every page has the identical `<header>` and `<footer>`
- ☐ Menu links work from every page; the logo returns Home
- ☐ All images have meaningful `alt` text
- ☐ The form has matching `<label>` / `id` pairs and `required` fields
- ☐ Files saved in the correct folder structure

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: On your own, build a sixth page — `gallery.html` — and add it to the site's navigation on every page.

Scenario: Adaeze wants a **Gallery** page to show photos of finished outfits and happy customers (for now, just photo placeholders). She also wants it reachable from the main menu.

Starting information (given):

- The Gallery page should show **eight** photos in a grid structure, each with a short `alt` description.

- Page heading: "*Gallery*".
- Same header and footer as all other pages.
- Brand unchanged.

Requirements:

1. Create `gallery.html`. Use the same document shell, header and footer as the other pages.
2. In `<main>`, add an `<h1>` "Gallery" and a `<section class="gallery">` containing **eight** `<img>` elements inside `<article>` or `<figure>` blocks. Each image needs a descriptive `alt` (for example, `alt="Customer wearing a blue Ankara gown"`).
3. Add a **Services/Home/Shop/About/Gallery/Contact** menu item: update the `<nav>` list on **every** page so Gallery appears between About and Contact. (Yes — you must edit all six files; this is the consistency lesson.)
4. Save and test: from every page, confirm the Gallery link works and the Gallery page shows the eight images (or placeholder text if images are missing).

Constraints:

- File name exactly `gallery.html` (lowercase).
- Every `alt` must describe the image, not say "image".
- Use relative paths (`images/photo1.jpg`, etc.).
- Keep the header and footer identical to the other pages.

Time allowance: 15 minutes.

Success criteria:

- `gallery.html` exists with eight images and a heading.
- All six pages now show a Gallery link in the menu, and the link works.
- Image `alt` text is descriptive.
- No broken links or missing-image icons (or placeholders clearly intended).

Hints available if stuck:

- Hint 1: To add Gallery to the menu, copy this line and paste it before the Contact `<li>` on every page:

```
<li><a href="gallery.html">Gallery</a></li>
```

- Hint 2: An image tag looks like `<img src="images/photo1.jpg" alt="Short description">`.
- Hint 3: If the image does not show, check the exact file name and folder.

Instructor answer key (do not show learners):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze's Fashion — Gallery</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="topbar">
    <a href="index.html" class="logo">Adaeze's Fashion</a>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="shop.html">Shop</a></li>
        <li><a href="services.html">Services</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="gallery.html">Gallery</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section class="gallery">
      <h1>Gallery</h1>
```

```
      <div class="grid">
        <img src="images/photo1.jpg" alt="Customer wearing a blue Ankara gown">
        <img src="images/photo2.jpg" alt="Bridal party in matching aso ebi outfits">
        <img src="images/photo3.jpg" alt="White lace gown with beaded sleeves">
        <img src="images/photo4.jpg" alt="Office wear blazer and skirt set">
        <img src="images/photo5.jpg" alt="Traditional attire in green and gold">
        <img src="images/photo6.jpg" alt="Child's outfit in bright patterned fabric">
        <img src="images/photo7.jpg" alt="Adaeze's workshop with sewing machines">
        <img src="images/photo8.jpg" alt="Two customers collecting finished dresses">
      </div>
    </section>
  </main>

  <footer class="site-footer">
    <p>&copy; 2026 Adaeze's Fashion. All rights reserved.</p>
    <div class="social">
      <a href="https://instagram.com">Instagram</a>
      <a href="https://facebook.com">Facebook</a>
    </div>
    <p>Aba Road, Port Harcourt &middot; 0801 234 5678</p>
  </footer>
</body>
</html>
```

Marking guidance (out of 10):

- `gallery.html` created with correct shell, header and footer (3)
- Eight images with descriptive `alt` text (3)
- Gallery link added to the menu on all six pages (2)
- All links and images work (1)
- Code clean and consistent (1)

J. REAL-WORLD CASE STUDY

Situation. A small bakery in Port Harcourt, *Sweet Crumb*, hires a junior web designer to build its first site. On the Home page, everything is perfect. But on the About page, the menu has a different colour, the footer is missing, and the "Order" link points to a page that does not exist. On the Contact page, the form has no labels — just empty boxes — and older customers complain they cannot tell what to type where.

Relevant facts.

- The designer built each page by hand, from scratch, without reusing a consistent structure.
- The header, menu and footer were typed differently on each page, so small differences crept in.
- The form was copied from a random example online that used placeholders instead of labels.

Decision point. What single habit would have prevented almost every one of these problems?

Possible approaches.

1. Fix each page separately and hope no new differences appear.
2. Rebuild using a **consistent structure**: one header, one footer, copied identically onto every page, and proper labelled forms.
3. Give up and use a website builder.

Recommended solution (for this stage). Approach 2. Consistency is the professional fix. Build one correct header and footer, copy them exactly onto every page, link every page with correct relative paths, and use accessible labels on forms. This is precisely the method taught today.

Trade-offs.

- Approach 1 is whack-a-mole: fixing one page often introduces differences on another.
- Approach 2 costs a little discipline up front and pays off for the life of the site.
- Approach 3 abandons the skills the business paid for and reduces control.

Professional connection. Consistency and accessibility are two of the fastest ways a junior developer stands out. A site where the menu, footer and forms behave the same on every page feels trustworthy. A site with a missing footer and unlabelled inputs feels careless — and customers notice, even if they cannot name the problem.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this table as the session's live troubleshooting guide. When a learner hits an issue, ask them to check the relevant row before giving the answer.

K.1 A menu link says "file not found" (404)

Symptoms: Clicking a link shows a page that cannot be found. **Likely causes:** the `href` is misspelled; the case does not match (`About.html` vs `about.html`); the target file is in a different folder. **How to investigate:** Compare the `href` value with the exact file name in VS Code's Explorer. **Safe corrective steps:** Correct the spelling and case; ensure the file is in the same folder (for pages). **How to confirm resolved:** The link opens the correct page. **How to prevent recurrence:** Always use lowercase; confirm files exist before linking.

K.2 Broken-image icon

Symptoms: A small broken-image icon appears where a photo should be. **Likely causes:** wrong `src` path; image in the wrong folder; wrong file extension (`.png` vs `.jpg`). **How to investigate:** Check the exact file name and folder in Explorer; compare with the `src` . **Safe corrective steps:** Fix the path to `images/actualfilename.jpg` . **How to confirm resolved:** The image shows (or the `alt` text shows if the file is truly missing — then add the file).

K.3 The header or footer differs on one page

Symptoms: One page's menu or footer looks or reads differently. **Likely cause:** The page was typed separately instead of copying the shared block. **How to investigate:** Compare the header block across pages. **Safe corrective steps:** Copy the correct header/footer from a good page and paste it in, replacing the inconsistent one. **How to confirm resolved:** All pages have an identical menu and footer.

K.4 Tags not closed

Symptoms: Content after a certain point looks wrong; the layout collapses. **Likely cause:** A missing closing tag such as `</li>` , `</a>` or `</div>` . **How to investigate:** Remember that browsers repair what they can; check the *first* line above where things break. VS Code colour-coding helps — note where the colours change. **Safe corrective steps:** Close every element in the correct order (last opened, first closed). **How to confirm resolved:** The structure renders as expected.

K.5 The form does not do anything when submitted

Symptoms: Clicking "Send Message" either does nothing or reloads the page. **Likely cause:** The form has no destination `action` (it is `#`), because sending requires a server or a form service. **How to investigate:** Check the `<form action="...">` . **Safe corrective steps:** For this course, this is expected. The markup is correct; connecting it to a real send service is beyond this session. Confirm the `required` validation still works. **How to confirm resolved:** The browser blocks empty submissions (validation works) and the labels are correctly connected.

K.6 Naira sign shows as `?` or a box

Symptoms: `₦` appears as a strange symbol. **Likely cause:** The file was saved in a non-UTF-8 encoding. **Safe corrective steps:** In VS Code bottom-right, click the encoding, choose **Save with Encoding → UTF-8**, and ensure `<meta charset="UTF-8">` is in the head. **How to confirm resolved:** The naira sign displays correctly.

K.7 Label does not focus its input

Symptoms: Clicking a form label does nothing. **Likely cause:** The label's `for` does not match the input's `id` . **How to investigate:** Compare `for="email"` with `id="email"` . **Safe corrective steps:** Make them match exactly. **How to confirm resolved:** Clicking the label places the cursor in the input.

Distinguish likely vs confirmed: In every case, confirm by checking the actual file names, paths and tag pairs rather than guessing — VS Code's Explorer and colour-coding are your evidence.

L. SESSION ASSIGNMENT

Assignment title: "My Multi-Page Website Structure"

Purpose: Build the complete, well-structured HTML skeleton of the *Adaeze's Fashion* website — five pages with a consistent header, footer and navigation, correct links, images with alt text, and a proper contact form.

Realistic scenario: Adaeze has approved your plan from Session 7. Now she wants to see the website's structure before you style it. The content should be real, not placeholder gibberish.

Detailed instructions:

1. In your `adaeze-fashion` folder, create these five files: `index.html`, `shop.html`, `services.html`, `about.html`, `contact.html`.
2. Every page must have:
   - the correct document shell (DOCTYPE, `lang`, charset, viewport, a unique `<title>`, and a link to `css/style.css`),
   - the **identical** `<header class="topbar">` with the logo and a `<nav>` list of all page links,
   - a `<main>` with that page's unique content,
   - the **identical** `<footer class="site-footer">` with copyright, social links and contact details.
3. Page content:
   - **Home:** a hero section (heading, short paragraph, "View Our Collection" button linking to `shop.html`) and three featured products (image, name, naira price).
   - **Shop:** a heading and six products (image, name, naira price), plus a "how to order" note.
   - **Services:** a heading and three service cards (Beginner Sewing Class, Ankara Design Workshop, Business of Fashion) with short descriptions.
   - **About:** the story (two or three paragraphs), one image, and a "What We Value" list (Quality, Affordability, Style).
   - **Contact:** a form with `<label>` + `<input>` for Name and Email, a `<textarea>` for Message (all `required`), and a submit button; below the form, the address, phone and Instagram.
4. All images must use relative paths and have meaningful `alt` text.
5. All internal links must use correct relative paths and match file names exactly (lowercase).
6. Indent your code consistently and add brief HTML comments above each major section (e.g. `<!-- header -->`, `<!-- products -->`).

Required deliverables:

- Five HTML files in the `adaeze-fashion` folder.
- The `css/style.css` (empty for now) and `images/` folder in place.

Submission format: Files placed in the class shared folder (zip the `adaeze-fashion` folder) or saved on the academy machine for review.

Deadline: Before Session 9.

Marking rubric (25 marks):

Criterion	Marks	Description of a strong answer
Five files correct	4	All five present with correct lowercase names; Home is `index.html`
Consistent header/nav	5	Identical on every page; all five links present and correct
Consistent footer	3	Identical on every page with copyright and contact details
Semantic HTML	3	`<header>`, `<nav>`, `<main>`, `<section>` / `<article>`, `<footer>` used correctly; one `<h1>` per page
Content complete	3	Each page has its planned content with real text and naira prices
Images & alt text	3	Relative paths correct; every `alt` descriptive
Contact form	2	Labels matched to inputs; `required` used; textarea and submit button present

Criterion	Marks	Description of a strong answer
Linking & testing	1	All menu and logo links work with no 404s
Code quality	1	Indented, commented, no unclosed tags

Common mistakes to avoid: inconsistent headers/footers; wrong or capitalised file names; missing `alt` text; unlabelled form inputs; forgetting the viewport meta tag on some pages.

Instructor reference solution: The full markup for each page follows the code in Section F (F.2 header, F.3 page mains, F.3 contact form, F.4 footer). During marking, open the folder, click through all five pages from the menu, right-click an image to confirm its path, and view source on the Contact page to confirm labels match inputs. A complete solution is the accumulation of every code block in Section F, applied consistently to all five files.

M. KNOWLEDGE CHECK

(Learner version — answers only in the Instructor Key.)

Q1. Short answer. What file name must the Home page have, and why?

Q2. Multiple choice. Which is the best `alt` text for a photo of a blue Ankara dress on the Shop page? a) `alt="image"` b) `alt="dress1.jpg"` c) `alt="Blue Ankara dress with short sleeves"` d) (no alt attribute)

Q3. Explain in your own words. Why is it important that the header and footer are identical on every page?

Q4. Practical. Write the HTML for a navigation item that links to the About page (`about.html`), using the correct list markup.

Q5. Form markup. Write the two HTML lines that create a labelled input where clicking the label focuses the text box, for a field named "email".

Q6. Semantic structure. Put these tags in the order they appear on a page: `<main>` , `<footer>` , `<header>` .

Q7. Error detection. This link works on the learner's Windows laptop but fails once the site is published. Why?

```
<a href="Shop.html">Shop</a>
```

(Assume the file is actually named `shop.html` .)

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. `index.html` — because web servers are configured to show `index.html` by default when a visitor opens the site's main address.

A2. (c) `alt="Blue Ankara dress with short sleeves"` — it describes the image clearly. "image" and the file name are not useful, and omitting `alt` harms accessibility.

A3. Because consistency helps visitors always know where the menu is and how to reach the business; it builds trust. If the header or footer changed between pages, users would feel lost, and the site would look unprofessional. It also makes the code easier to maintain.

A4.

```
<li><a href="about.html">About</a></li>
```

A5.

```
<label for="email">Email Address</label>
<input type="email" id="email" name="email" required>
```

(The key point is that `for="email"` matches `id="email"` .)

A6. `<header>` , `<main>` , `<footer>` .

A7. Because the link uses a capital "S" (`Shop.html`) while the actual file is lowercase (`shop.html`). Windows is case-insensitive so it works locally, but most web servers (Linux) are case-sensitive, so the link breaks online. Fix: use `href="shop.html"` .

N. REFLECTION AND RECAP

Plain-English summary. Today you turned your Session 7 plan into real HTML. You created five pages — `index.html` , `shop.html` , `services.html` , `about.html` and `contact.html` — each with the **same** header, navigation and footer and its own unique content inside `<main>` . You linked pages with correct **relative paths**, added images with meaningful `alt` **text**, built an accessible **contact form** with labels and `required` fields, and kept everything neatly in the planned **folder structure**.

Five important takeaways:

1. Home is always `index.html` ; each page is its own file.
2. The header, navigation and footer must be **identical** on every page.
3. Use relative paths (`about.html` , `images/dress1.jpg`) — never full Windows paths.
4. Every image needs descriptive `alt` text; every form field needs a matching `<label>` .
5. Consistent structure on every page makes the site trustworthy and easy to maintain.

Questions to help you recognise what you can now do:

- Can I create a new page by copying the shared header and footer?
- Can I link all pages so the menu works from anywhere?
- Can I write a labelled, required form?
- Can I find and fix a broken image or link?

Connection to the next session: Your website now has a complete, working structure — but it looks plain, because we have not written a single line of CSS yet. Next session, Session 9, we write the shared `style.css` that turns this skeleton into a polished, on-brand, responsive website: the purple-and-gold top bar, the product card grid, and the styled contact form.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- The learner creates the header once and copies it accurately onto every page.
- The learner uses lowercase file names and correct relative paths without reminders.
- The learner writes meaningful `alt` text rather than "image1".
- The learner connects every label to its input with matching `for` / `id` .
- The learner clicks through every menu link and the logo to test before declaring finished.
- The learner keeps one `<h1>` per page and uses `<h2>` / `<h3>` below it.

Common misconceptions to watch for and correct:

- "The header can differ a little on each page." → It must be identical; copy, do not retype.
- " `alt` text is optional." → It is a basic accessibility requirement.
- "Placeholders can replace labels in forms." → Labels are required for accessibility; placeholders are optional extras.
- "Capital letters in file names are fine." → They break on Linux servers; always use lowercase.
- "The form should send email now." → Markup is correct, but sending needs a service beyond this session; `required` validation still works.

Suggested remedial exercises for learners who struggle:

1. **Link-hunt:** give a site with three broken links and one broken image; ask the learner to find and fix each using the file names.
2. **Copy-the-header drill:** give one correct header block and two pages with slightly different headers; ask the learner to make them identical.
3. **Form-pairing drill:** give a form with unmatched `for` / `id` values and ask the learner to fix every pair so labels focus their inputs.

Differentiation:

- **Level 1 (supported):** provide `index.html` fully written and a partially written `about.html`; the learner completes the About page and creates the Gallery page.
- **Level 2 (standard):** the practical and independent exercises as written.
- **Level 3 (challenge):** add a `gallery.html` with eight images and a `faq.html` with a definition-style list; also add a `<meta name="description">` to each page's head and explain its purpose.

Timing guidance: The practical work is the priority — protect it. If the room is slow, reduce the demonstrated page count and let learners finish `contact.html` at home, but ensure the shared header/footer consistency is done in class. If the room is fast, let learners begin the Gallery extension live. The single most important thing to check before the session ends is that **all five pages link correctly** — this is the foundation for styling next session.

APPENDIX 8A — 10-MINUTE HTML REFRESHER (only if needed)

If learners cannot write a basic page with a heading, paragraph and link, run this before Section D.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Refresher</title>
</head>
<body>
  <h1>Adaeze's Fashion</h1>
  <p>Custom clothing in Port Harcourt.</p>
  <a href="shop.html">Shop</a>
  <img src="images/dress1.jpg" alt="Ankara dress">
</body>
</html>
```

Key points to confirm: every tag opens and closes; visible content lives in `<body>`; links use `<a href="...">`; images use `<img src="..." alt="...">`. Only proceed once every learner can save this and open it in Chrome.

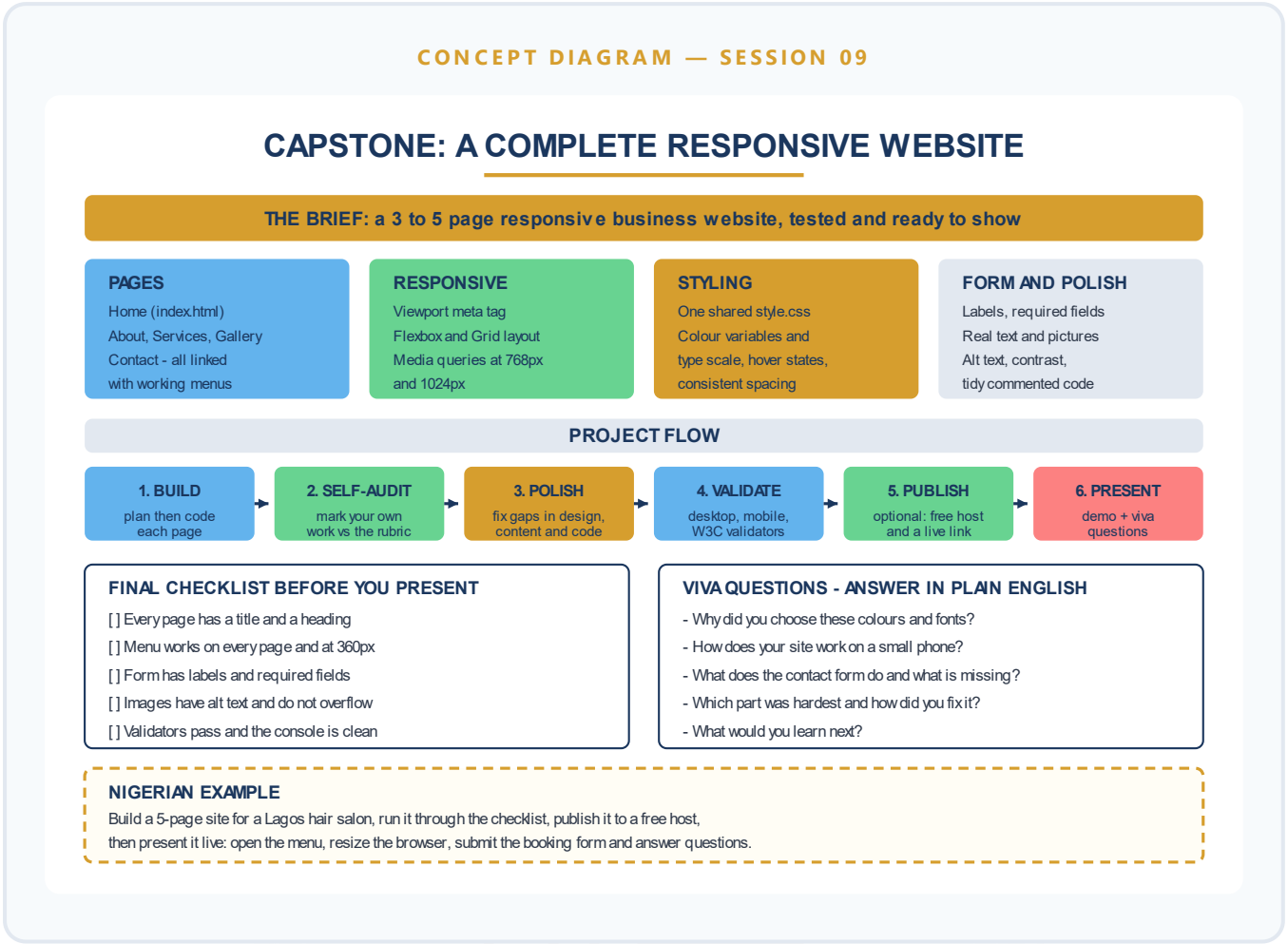
END OF SESSION 8

Next: Session 9 — Styling the Website.

SESSION 09 OF 12

STYLING THE WEBSITE

CONCEPT DIAGRAM — SESSION 09



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 9 — STYLING THE WEBSITE

A. SESSION IDENTIFICATION

Item	Detail
Session number	9 of 12
Session title	Styling the Website
Week / Module	Week 3 — Module: Building a Website
Recommended duration	105 minutes
Difficulty level	Beginner (builds directly on Sessions 4–8)

Item	Detail
Prerequisite knowledge	Sessions 1–8. Learners can write semantic HTML for a multi-page site, link pages with relative paths, and know CSS selectors, the box model, flexbox, the `link` tag and media queries.
Connection to previous work	In Session 8 the learner built the *skeleton* of the Adaeze Fashion website: four HTML pages with a header, navigation, main content and footer. It works, but it is plain black-on-white text — an unstyled document. This session turns that skeleton into a designed, branded website using one shared external stylesheet.
Preparation before class	Academy PCs on and logged in; VS Code and Chrome installed; each learner's Session 8 project folder open and complete; a projector that shares the instructor's screen; a printed "CSS Styling Checklist" per learner; the instructor's own finished Adaeze Fashion site ready to show as a target.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Create one shared external stylesheet** and link it correctly from every page using `<link rel="stylesheet" href="css/style.css">` in the `<head>`.
2. **Define and apply a colour palette using CSS custom properties** (variables) so the whole site changes from one place.
3. **Define and apply a typography scale** — a set of heading and body sizes that are reused consistently.
4. **Style the header and navigation** with flexbox, including a hover state on the links.
5. **Style the hero section, product cards and buttons** using flexbox and CSS Grid.
6. **Apply a consistent spacing system** (repeated padding and margin values) so nothing looks cramped or random.
7. **Add borders, shadows and rounded corners** to give elements depth.
8. **Style a contact form** — labels, inputs, textarea and a clear focus state.
9. **Organise a stylesheet** into commented sections in a logical order, and reuse classes instead of repeating rules.
10. **Style all pages of a multi-page website consistently** and verify the result in the browser.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Sharing between two learners is acceptable — one types, one calls out what looks wrong, then swap.
- Instructor computer connected to a projector or large screen.

Software

- Visual Studio Code (any 2022-or-later version).
- Google Chrome (any recent version). Its DevTools will be used again in Session 11.
- Windows 10 or Windows 11.

Accounts

- None required. Everything this session is done on the machine.

Sample files

- The learner's own Session 8 project: `adaeze-fashion/` containing `index.html`, `shop.html`, `about.html`, `contact.html`, an `images/` folder and an empty `css/` folder.
- Instructor's finished reference site and a complete `style.css` (contents shown in Sections F, G and L) for demonstrating the target.

Worksheets

- "CSS Styling Checklist" (printed, 1 page — the same list appears in Section H).

- "My Colour Palette" planning sheet (in the workbook) — learners copy the palette they chose in Session 7.

Tools

- Chrome DevTools (press `F12`) — optional today, essential in Session 11.

Internet

- Not required for this session. (Internet returns in Session 10 for map and video embeds.)

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. The whole session is offline and file-based.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, first open the learner's *unstyled* Session 8 page (plain text, default serif font, blue underlined links, no colour). Then, in a second tab, open the instructor's fully styled Adaeze Fashion site.

What the instructor says:

> *"Look carefully. Same words. Same pictures. Same four pages. But which one would make you trust Adaeze with your money?*
>
> *The plain one looks like a document someone typed in a hurry. The styled one looks like a shop. The only difference between them is one file — a stylesheet. By the end of today, your plain site will look like that one. And the good news is you already know all the pieces; today we simply bring them together for a real business."*

Flip between the two tabs three or four times so the contrast lands. Then tell the class the plan for the session: colours, fonts, spacing, header, hero, products, buttons, footer, form — all in one organised file.

Expected learner response: Learners will nod, laugh at how plain the first one looks, and some will ask "Is that my site?" Accept that. Point out it is *their* site from last week and it is completely normal to look plain at this stage.

Why this opener works: It sells the value of CSS in ten seconds (trust = money for a business) and reduces the learner's fear by reminding them they already learned each individual piece. It frames today as assembly, not new magic.

E. PREREQUISITE REVIEW (7 minutes)

Run this as a fast oral round-robin. Put the questions on the projector.

Question 1: "What are the three parts of every CSS rule?"

- Expected answer: a **selector**, a **property** and a **value**. Example `.price { color: #d4a574; }`.
- Common gap: learners forget the colon between property and value, or the semicolon at the end. Remind them: `property: value;` — colon inside, semicolon at the end.

Question 2: "What are the four layers of the box model, from the content outward?"

- Expected answer: **content, padding, border, margin.**
- Common gap: mixing up padding (space *inside* the border) and margin (space *outside* the border). Say: "Padding is the bubble wrap inside the box; margin is the gap between boxes on the shelf."

Question 3: "What does `display: flex` let you do, and give me one place we used it."

- Expected answer: it makes an element a flex container and lays its direct children out in a row (or column), making it easy to space and align them. We used it for the header and navigation in earlier sessions.
- Common gap: learners think `flex` goes on the children. Clarify: it goes on the **parent** container.

Question 4: "What is a media query and what keyword do we use to break *down* from desktop to mobile?"

- Expected answer: a block of CSS that only applies when a condition is true, usually screen width. `@media (max-width: 767px) { ... }`.

- Common gap: forgetting the parentheses around the condition. It is `@media (max-width: 767px)`, not `@media max-width: 767px`.

Question 5: "How do we link a stylesheet, and where does the line go?"

- Expected answer: `<link rel="stylesheet" href="css/style.css">` inside the `<head>`.
- Common gap: confusing `<link>` (for CSS) with `<a>` (for page links). Emphasise: **link for stylesheets, a for anchors.**

Bridge to today (say this):

> *"Everything you just answered is a tool. Today we put all the tools on the workbench at once and build one finished, branded website. The trick professionals use is organisation — one file, one palette, one spacing system, reused everywhere. That is what makes a site look 'designed' instead of 'decorated'."*

If more than a third of the class cannot answer Questions 2 and 3, spend 8 extra minutes re-demonstrating the box model and flexbox on a single sample page before continuing into Section F.

F. FULL LECTURE NOTES

F.1 What "styling a whole website" really means

The idea. Up to now you have styled single elements in isolation — one heading red, one image rounded. A *website* is different: dozens of elements across several pages that must look like they belong together.

Plain-English definition. Styling a website means choosing a small set of rules — colours, fonts, sizes and spacing — and applying them the same way everywhere, so every page feels like one brand.

Why it exists. A visitor does not judge one button; they judge the whole experience. If the Home page uses one blue and the Contact page uses a different blue, the visitor senses carelessness and stops trusting the business. Consistency is what separates an amateur site from a professional one.

Analogy. Think of a **tailor's shop**. Adaeze does not cut a fresh pattern for every customer in a random way; she has house measurements, house stitching, house colours. The result is that everything from her shop is recognisably *hers*. A stylesheet is your house style: cut once, applied everywhere.

What it does. It takes the HTML skeleton and adds colour, type, space and depth. The structure (HTML) never changes; only the presentation changes.

Important misconception. "More decoration = better design." Wrong. Beginner sites often have fourteen colours and five fonts. Professional sites usually have **three to five colours** and **two fonts**. Restraint looks expensive; clutter looks cheap.

How a learner recognises good styling. Spacing is even, colours repeat consistently, text is readable, buttons look clickable, and every page matches. If you can move from any page to any other and not feel a "jump", the styling is working.

Limitation. CSS cannot fix bad content or bad structure. If the HTML has no headings and no semantic sections, styling can only hide the problem, not solve it. Session 8's clean HTML is what makes today possible.

F.2 One shared external stylesheet

The idea. Every page links to the *same* CSS file. There is only one `style.css` for the whole site.

Why not one file per page? If each page had its own stylesheet, changing the brand colour would mean editing four files and hoping you did not forget one. With a shared stylesheet you edit the colour **once** and all four pages update instantly.

Analogy. A shared stylesheet is like the **school uniform rule**. Every class wears the same uniform; if the school changes the badge, every class changes together. Per-page CSS is like every class inventing its own uniform — chaos.

The linking line (goes in the `<head>` of *every* page):

```
<link rel="stylesheet" href="css/style.css">
```

Line by line:

- `<link>` — the tag that connects the page to an outside resource.

- `rel="stylesheet"` — *relationship* is "stylesheet"; without this the browser does not know what kind of file it is.
- `href="css/style.css"` — the path. It means "from the HTML file, go into the `css` folder and find `style.css`". This is a **relative path** (Section K explains why that matters).

Where does `style.css` live? Inside a `css/` folder, sibling to the HTML files:

```
adaeze-fashion/
├── index.html
├── shop.html
├── about.html
├── contact.html
├── css/
│   └── style.css     ← the one shared file
└── images/
    └── ...
```

Worked example — proving the link works:

HTML (`index.html` head):

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion | Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

CSS (`css/style.css`):

```
body {
  background-color: lightyellow;
}
```

Expected browser result: Open `index.html` and the whole page background turns pale yellow. If it does not, the link or the path is wrong — check Section K.1.

Consequence of getting it wrong: If the path is wrong (e.g. `href="style.css"` when the file is in a `css/` folder), the page loads but *completely unstyled*, and the Console shows a red `net::ERR_FILE_NOT_FOUND` for the stylesheet.

F.3 CSS custom properties — the colour palette in one place

The idea. Instead of typing `#2c1810` fifteen times, you name it once as `--color-dark`, then use that name everywhere.

Plain-English definition. A **CSS custom property** (also called a *CSS variable*) is a named value you store on `:root` and reuse with `var()`.

Why it exists. When a client says "change the brown to navy", you change **one line** instead of hunting through hundreds of lines. This is how real design systems stay consistent.

Analogy. It is like naming a jar "SUGAR" in a kitchen rather than writing the word sugar on every recipe. Rename the contents once; every recipe still says "add sugar".

The Adaeze Fashion palette:

```
:root {
  /* -- Brand colours -- */
  --color-dark:   #2c1810;   /* deep cocoa brown — headers, footer, headings */
  --color-gold:   #d4a574;   /* warm tan/gold — accents, buttons, logo */
  --color-gold-dark: #b8845a;/* darker gold — hover state */
  --color-cream:  #f5f0eb;   /* soft cream — section backgrounds */
  --color-white:  #ffffff;   /* pure white — cards */
  --color-text:   #333333;   /* body text */
  --color-muted:  #666666;   /* secondary text */
  --color-border: #dddddd;   /* input borders, card outlines */
}
```

- `:root` means "the very top of the document" — a safe place to store global values.
- Each line is `--name: value;`. Note the **double hyphen** at the start — that is required.
- Names are your choice, but meaningful names like `--color-gold` beat meaningless ones like `--c1`.

Using a variable with `var()`:

```
body {
  color: var(--color-text);
  background-color: var(--color-white);
}
```

Expected browser result: the text is charcoal `#333` on white. Change `--color-text` to `#000` up top and refresh — the text darkens everywhere at once.

Worked example — proving the power:

```
:root {
  --color-gold: #d4a574;
}
.button {
  background-color: var(--color-gold);
}
.logo {
  color: var(--color-gold);
}
```

Now edit `:root` to `--color-gold: #1e6091;` and refresh. The button *and* the logo turn blue together. That is the whole point.

Misconception. Learners sometimes write `var(--color-gold);` as if it were a full declaration. `var()` is only the *value*; it must still sit after a property: `color: var(--color-gold);`.

Limitation. Variables are not magic storage for arbitrary logic; they simply hold values. Also, very old browsers (Internet Explorer) do not support them — we do not support IE in this course, so we are safe.

F.4 The typography scale — two fonts, a set of sizes

The idea. Choose **two fonts** (one for headings, one for body) and a **short list of sizes**, then never invent a new size in the middle of the page.

Why it exists. Random font sizes are the fastest way to make a site look messy. A scale makes headings feel like a family.

Analogy. A **choir**. You do not have 200 people singing 200 different notes; you have sections (soprano, alto) singing a limited set of notes that harmonise. A typography scale is your limited set of "notes".

Our two fonts:

```
:root {
  --font-heading: Georgia, "Times New Roman", serif;
  --font-body: "Segoe UI", Arial, sans-serif;
}
```

- `Georgia` is a serif font (small "feet" on the letters) — it feels elegant and traditional, good for a fashion brand.
- `"Segoe UI"` is a clean sans-serif (no feet) — easy to read for body text.
- Note the **font stack**: several names separated by commas. If the first font is not on the device, the browser tries the next. This is why we include `Arial` and `serif` / `sans-serif` as fallbacks.

The size scale:

```
:root {
  --size-base: 16px;   /* normal paragraph text */
  --size-h3:   22px;
  --size-h2:   32px;
  --size-h1:   48px;
  --size-small:14px;
}
```

Applying the scale:

```
body {
  font-family: var(--font-body);
  font-size: var(--size-base);
  line-height: 1.6;
}
h1, h2, h3 {
  font-family: var(--font-heading);
  color: var(--color-dark);
}
h1 { font-size: var(--size-h1); }
h2 { font-size: var(--size-h2); }
h3 { font-size: var(--size-h3); }
```

- `line-height: 1.6` means each line of text is 1.6 times the font size tall. Around **1.5–1.7** is comfortable for reading; the default is often too tight.
- Grouping `h1, h2, h3` with commas applies the shared rules to all three, then each gets its own size. This is a **reusable** pattern.

Expected browser result: headings appear in Georgia dark brown at clearly different sizes; body text is a readable Segoe UI.

Misconception. Learners think the browser's default `h1` is "the only h1 size". It is not — you can set it to anything. Also, a heading's size should not be used to make *text* look bigger; use a heading for a real heading, and change paragraph size if you need to.

F.5 The spacing system — repeat a few numbers

The idea. Instead of inventing `padding: 17px` here and `padding: 23px` there, you pick a small set of spacing values and reuse them.

Why it exists. Even spacing is a huge part of why a design "feels right". Random spacing makes a page feel jumpy even when colours are fine.

Analogy. A **bricklayer** does not lay one row with 2mm mortar and the next with 8mm. Consistent gaps create a solid wall. Consistent spacing creates a solid layout.

```
:root {
  --space-1: 8px;
  --space-2: 16px;
  --space-3: 24px;
  --space-4: 32px;
  --space-5: 48px;
  --space-6: 64px;
}
```

Applying it:

```
section {
  padding: var(--space-5) var(--space-3); /* 48px top/bottom, 24px sides */
}
h1 { margin-bottom: var(--space-3); }      /* 24px under the heading */
p  { margin-bottom: var(--space-2); }      /* 16px under paragraphs */
```

Expected browser result: sections have generous, even breathing room; headings sit a comfortable distance above their text.

Misconception. "Margin and padding do the same thing." They do not: padding pushes *outward from inside* the element (space between content and border), margin pushes *other elements away* (space outside the border). Mixing them up is one of the top CSS mistakes, covered in Section K.

F.6 Styling the header and navigation (flexbox)

The target look: a dark brown bar across the top. On the left, the logo "Adaeze Fashion" in gold. On the right, four nav links (Home, Shop, About, Contact) in cream, turning gold when hovered.

The HTML we are styling (from Session 8):

```
<header>
  <a href="index.html" class="logo">Adaeze Fashion</a>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="shop.html">Shop</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```

The CSS:

```
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: var(--color-dark);
  padding: var(--space-2) var(--space-3);
}

.logo {
  font-family: var(--font-heading);
  color: var(--color-gold);
  font-size: 24px;
  font-weight: bold;
  text-decoration: none;
}

nav ul {
  display: flex;
  list-style: none;
  gap: var(--space-3);
  margin: 0;
  padding: 0;
}

nav a {
  color: var(--color-cream);
  text-decoration: none;
  font-size: var(--size-base);
  transition: color 0.3s ease;
}

nav a:hover {
  color: var(--color-gold);
}
```

Line by line:

- `header { display: flex; ... }` turns the header into a flex container so the logo and the nav sit in a row.
- `justify-content: space-between` pushes them to opposite ends — logo on the left, nav on the right.
- `align-items: center` lines them up vertically in the middle.
- `nav ul { display: flex; }` lays the list items out in a row instead of a vertical bulleted list.
- `list-style: none` removes the bullet dots. `margin: 0; padding: 0;` remove the browser's default list indentation.
- `gap: var(--space-3)` puts a 24px gap *between* each nav item (cleaner than margins).
- `transition: color 0.3s ease` makes the colour change smooth over 0.3 seconds instead of jumping. (`ease` means "start and finish gently".)
- `nav a:hover` applies only while the mouse is over a link.

Expected browser result: a dark bar with the gold logo left and cream links right; hovering a link fades it to gold. Removing the mouse fades it back.

Misconception. Learners add `text-decoration: none` to `nav` instead of `nav a`. The underline belongs to the anchor element, so you must target the `a` itself.

F.7 Styling the hero section and buttons

The idea. The **hero** is the big welcome block at the top of the Home page — a headline, a sentence and a call-to-action button.

The HTML:

```
<section class="hero">
  <h1>Adaeze Fashion</h1>
  <p>Bespoke Nigerian fashion, tailored in Port Harcourt.</p>
  <a href="shop.html" class="button">Shop Our Collection</a>
</section>
```

The CSS:

```
.hero {
  text-align: center;
  padding: var(--space-6) var(--space-3);
  background-color: var(--color-cream);
}

.hero h1 {
  font-size: var(--size-h1);
  color: var(--color-dark);
  margin-bottom: var(--space-2);
}

.hero p {
  font-size: 20px;
  color: var(--color-muted);
  margin-bottom: var(--space-4);
}

.button {
  display: inline-block;
  background-color: var(--color-gold);
  color: var(--color-dark);
  padding: var(--space-2) var(--space-4);
  text-decoration: none;
  font-weight: bold;
  border-radius: 6px;
  transition: background-color 0.3s ease, transform 0.2s ease;
}

.button:hover {
  background-color: var(--color-gold-dark);
  transform: translateY(-2px);
}
```

Line by line:

- `text-align: center` centres all the text and the button inside.
- `padding: var(--space-6) var(--space-3)` gives a tall hero (64px top/bottom, 24px sides). The first value is vertical, the second horizontal — a handy CSS shorthand.
- `.button` has `display: inline-block`. This is important: an `<a>` is normally inline and ignores vertical padding. `inline-block` lets it behave like a small box that can have padding, width and height.
- `border-radius: 6px` rounds the corners.
- `transition` lists two properties: background colour *and* transform. The transform `translateY(-2px)` lifts the button up by 2px on hover — a subtle "come and click me" effect.

Expected browser result: a cream band with a big brown headline, a grey sentence and a rounded gold button. Hovering the button darkens it slightly and lifts it 2px.

Misconception. Learners forget `text-decoration: none` on `.button`, so the button still has a link underline. Also, `transform: translateY(-2px)` moves the element *up* because the y-axis grows downward; a negative value moves up.

F.8 Product cards with CSS Grid

The idea. Products sit in a **grid** — a neat arrangement of equal cards. The brief asks specifically for CSS Grid here.

Why Grid and not flexbox? Flexbox is great for a single row that wraps unpredictably. Grid is great when you want *columns that line up perfectly*, like a shop shelf. For a product catalogue, Grid gives even, tidy columns.

The HTML (one card shown; repeat for each product):

```
<section class="products">
  <h2>Our Collection</h2>
  <div class="product-grid">
    <article class="product">
      <img src="images/product-1.jpg" alt="Ankara gown in gold and brown">
      <h3>Ankara Gown</h3>
      <p class="price">₦18,000</p>
      <a href="contact.html" class="button">Order</a>
    </article>
    <!-- more cards... -->
  </div>
</section>
```

The CSS:

```
.products {
  max-width: 1100px;
  margin: 0 auto;
  padding: var(--space-5) var(--space-3);
}

.products h2 {
  text-align: center;
  margin-bottom: var(--space-4);
}

.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: var(--space-4);
}

.product {
  background-color: var(--color-white);
  border: 1px solid var(--color-border);
  border-radius: 10px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
  padding: var(--space-3);
  text-align: center;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.product:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 18px rgba(0, 0, 0, 0.15);
}

.product img {
  width: 100%;
  height: 220px;
  object-fit: cover;
  border-radius: 8px;
}

.product h3 {
  margin: var(--space-2) 0 var(--space-1);
}

.price {
  color: var(--color-gold-dark);
  font-weight: bold;
  font-size: 18px;
  margin-bottom: var(--space-2);
}
```

Line by line:

- `max-width: 1100px; margin: 0 auto;` is the classic **centering trick**: the content never grows past 1100px, and `margin: 0 auto` (auto left/right margins) centres it on wide screens.
- `display: grid` turns the container into a grid.
- `grid-template-columns: repeat(3, 1fr)` means "make 3 columns, each taking 1 fraction of the space" — three equal columns.
- `gap: var(--space-4)` spaces the cards evenly both ways.
- `box-shadow: 0 4px 10px rgba(0,0,0,0.08)` gives a soft shadow: 4px down, 10px blur, black at 8% opacity. `rgba` means red-green-blue-alpha; the alpha (last number) controls transparency from `0` (invisible) to `1` (solid).
- `object-fit: cover` makes every product image fill its box neatly without stretching or squashing.

Expected browser result: three equal product cards in a row, each with a rounded photo, a name, a gold price and an Order button. Hovering a card lifts it and deepens its shadow.

Misconception. `width: 100%` on an image makes it as wide as its *container*, not the screen — that is correct here. But if you forget a fixed `height` with `object-fit`, images of different shapes make the cards different heights, which looks messy. Also, `1fr` must be lowercase; `1FR` will not work.

Limitation. A fixed 3-column grid looks cramped on a phone. We fix that with a media query in Section F.10.

F.9 Styling the footer and the contact form

Footer. Same across every page — dark bar, cream text, gold social links.

```
footer {
  background-color: var(--color-dark);
  color: var(--color-cream);
  text-align: center;
  padding: var(--space-4) var(--space-3);
  margin-top: var(--space-6);
}

footer p {
  margin-bottom: var(--space-2);
}

.social-links a {
  color: var(--color-gold);
  text-decoration: none;
  margin: 0 var(--space-2);
}

.social-links a:hover {
  text-decoration: underline;
}
```

Contact form. Labels above inputs, full-width fields, a clear focus state, and a submit button.

The HTML:

```
<form action="#" method="post" class="contact-form">
  <label for="name">Your Name</label>
  <input type="text" id="name" name="name" required>

  <label for="email">Email Address</label>
  <input type="email" id="email" name="email" required>

  <label for="message">Message</label>
  <textarea id="message" name="message" rows="5" required></textarea>

  <button type="submit" class="button">Send Message</button>
</form>
```

The CSS:

```
.contact-form {
  max-width: 520px;
  margin: 0 auto;
  padding: var(--space-3);
}

.contact-form label {
  display: block;
  margin-bottom: var(--space-1);
  font-weight: bold;
  color: var(--color-dark);
}

.contact-form input,
.contact-form textarea {
  width: 100%;
  padding: var(--space-2);
  margin-bottom: var(--space-3);
  border: 1px solid var(--color-border);
  border-radius: 6px;
  font-size: var(--size-base);
  font-family: var(--font-body);
}

.contact-form input:focus,
.contact-form textarea:focus {
  outline: none;
  border-color: var(--color-gold);
  box-shadow: 0 0 0 3px rgba(212, 165, 116, 0.25);
}
```

Line by line:

- `label { display: block; }` makes each label sit on its own line above its input (labels are inline by default).
- `width: 100%` makes inputs fill the form width. Note the box model: with the reset's `box-sizing: border-box`, the padding and border are *included* in that 100%, so inputs do not overflow. This is exactly why the reset matters (Section F.11).
- `font-family: var(--font-body)` — inputs do **not** inherit the page font by default, so we set it explicitly; otherwise the form looks like a different website.
- `:focus` styles what happens when the user clicks into a field. `outline: none` removes the default blue ring, and we replace it with a gold border and a soft gold glow — this is both prettier and still clearly visible (an accessibility win we revisit in Session 10).
- `rgba(212, 165, 116, 0.25)` is the gold colour at 25% opacity, used as a soft glow via `box-shadow`.

Expected browser result: a tidy, centred form; each field has a light border and rounds the corners; clicking into a field shows a gold outline. The "Send Message" button looks like the other buttons.

Important honesty note (say this to the class): this form only *displays*. With `action="#"` it does not actually send email anywhere — sending needs a server or a form service. We note this now and it returns as a case study in Session 10. Do not promise learners that the form delivers messages.

F.10 Responsive styling (bringing back media queries)

We styled for desktop. Now we let it adapt. The two breakpoints used across the course: **tablet at** `max-width: 1023px` and **mobile at** `max-width: 767px`.

```
@media (max-width: 1023px) {
  .product-grid {
    grid-template-columns: repeat(2, 1fr);
  }
  .hero h1 { font-size: 38px; }
}

@media (max-width: 767px) {
  header {
    flex-direction: column;
    gap: var(--space-2);
  }
```

```
  nav ul {
    flex-direction: column;
    align-items: center;
    gap: var(--space-1);
  }
  .product-grid {
    grid-template-columns: 1fr;
  }
  .hero h1 { font-size: 30px; }
  .hero p  { font-size: 16px; }
}
```

- `grid-template-columns: repeat(2, 1fr)` on tablet gives two cards per row.
- `grid-template-columns: 1fr` on mobile gives a single column — one card per row.
- `header { flex-direction: column; }` stacks the logo above the nav on small screens instead of squeezing them side by side.
- `nav ul { flex-direction: column; }` stacks the links vertically on mobile.

Expected browser result: three columns on desktop, two on tablet, one on mobile; the header stacks on mobile. Reduce the browser width and the layout reflows at each breakpoint.

Misconception. Learners put a media query in the *middle* of other rules. Keep all media queries **at the bottom** of the stylesheet, in order from large to small, so they cleanly override the desktop rules.

F.11 The reset — starting from a clean slate

Different browsers add their own default margins and sizes. A short **reset** makes browsers behave the same.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- `*` means "every element".
- Removing all default margins/padding gives us full control.
- `box-sizing: border-box` is the important one: it tells the browser that an element's declared `width` *includes* its padding and border. Without it, `width: 100%` plus `padding: 16px` makes an element *wider* than 100% and it overflows. With `border-box`, 100% means 100% — a huge time-saver.

Misconception. "The reset is optional." For beginners it is close to mandatory, because it removes a whole class of confusing overflow bugs.

F.12 Organising the stylesheet

A stylesheet that is in a random order is hard to maintain. Professionals write it in a **logical order with comment headers**, so anyone (including future-you) can find things.

The order we use all course long:

```
/* ===== 1. Custom properties (colours, fonts, spacing) ===== */
/* ===== 2. Reset & base ===== */
/* ===== 3. Typography ===== */
/* ===== 4. Layout helpers (containers) ===== */
/* ===== 5. Header & navigation ===== */
/* ===== 6. Hero ===== */
/* ===== 7. Buttons ===== */
/* ===== 8. Product grid ===== */
/* ===== 9. Forms ===== */
/* ===== 10. Footer ===== */
/* ===== 11. Responsive media queries ===== */
```

Why comments matter. In three weeks you will not remember why `.product` has `overflow: hidden`. A one-line comment such as `/* keeps long titles from breaking the card */` saves minutes of guesswork. Comment the *why*, not the obvious.

Reusable classes. Notice we made one `.button` class and used it on the hero button, each product's Order link and the form submit button. Write the rule once, reuse it everywhere. That is the professional habit that keeps CSS short and consistent.

F.13 Complete reference stylesheet (the target for today)

This is the full `css/style.css` the learner is building toward. It combines everything above into one organised file.

```
/* ===== 1. Custom properties ===== */
:root {
  --color-dark:      #2c1810;
  --color-gold:      #d4a574;
  --color-gold-dark: #b8845a;
  --color-cream:     #f5f0eb;
  --color-white:     #ffffff;
  --color-text:      #333333;
  --color-muted:     #666666;
  --color-border:    #dddddd;

  --font-heading: Georgia, "Times New Roman", serif;
  --font-body:    "Segoe UI", Arial, sans-serif;

  --size-base:  16px;
  --size-h3:    22px;
  --size-h2:    32px;
  --size-h1:    48px;

  --space-1: 8px;
  --space-2: 16px;
  --space-3: 24px;
  --space-4: 32px;
  --space-5: 48px;
  --space-6: 64px;
}

/* ===== 2. Reset & base ===== */
* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  font-family: var(--font-body);
  font-size: var(--size-base);
  line-height: 1.6;
  color: var(--color-text);
  background-color: var(--color-white);
}

/* ===== 3. Typography ===== */
h1, h2, h3 { font-family: var(--font-heading); color: var(--color-dark); }
h1 { font-size: var(--size-h1); }
h2 { font-size: var(--size-h2); }
h3 { font-size: var(--size-h3); }

/* ===== 4. Layout helpers ===== */
.container { max-width: 1100px; margin: 0 auto; padding: 0 var(--space-3); }

/* ===== 5. Header & navigation ===== */
header {
  display: flex; justify-content: space-between; align-items: center;
  background-color: var(--color-dark); padding: var(--space-2) var(--space-3);
}
.logo {
  font-family: var(--font-heading); color: var(--color-gold);
  font-size: 24px; font-weight: bold; text-decoration: none;
}
nav ul { display: flex; list-style: none; gap: var(--space-3); margin: 0; padding: 0; }
nav a { color: var(--color-cream); text-decoration: none; transition: color 0.3s ease; }
nav a:hover { color: var(--color-gold); }

/* ===== 6. Hero ===== */
.hero { text-align: center; padding: var(--space-6) var(--space-3); background-color: var(--color-cream); }
.hero h1 { margin-bottom: var(--space-2); }
.hero p  { font-size: 20px; color: var(--color-muted); margin-bottom: var(--space-4); }
```

```
/* ===== 7. Buttons ===== */
.button {
  display: inline-block; background-color: var(--color-gold); color: var(--color-dark);
  padding: var(--space-2) var(--space-4); text-decoration: none; font-weight: bold;
  border-radius: 6px; transition: background-color 0.3s ease, transform 0.2s ease;
}
.button:hover { background-color: var(--color-gold-dark); transform: translateY(-2px); }

/* ===== 8. Product grid ===== */
.products { max-width: 1100px; margin: 0 auto; padding: var(--space-5) var(--space-3); }
.products h2 { text-align: center; margin-bottom: var(--space-4); }
.product-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: var(--space-4); }
.product {
  background-color: var(--color-white); border: 1px solid var(--color-border);
  border-radius: 10px; box-shadow: 0 4px 10px rgba(0,0,0,0.08);
  padding: var(--space-3); text-align: center;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}
.product:hover { transform: translateY(-4px); box-shadow: 0 8px 18px rgba(0,0,0,0.15); }
.product img { width: 100%; height: 220px; object-fit: cover; border-radius: 8px; }
.price { color: var(--color-gold-dark); font-weight: bold; font-size: 18px; }

/* ===== 9. Forms ===== */
.contact-form { max-width: 520px; margin: 0 auto; padding: var(--space-3); }
.contact-form label { display: block; margin-bottom: var(--space-1); font-weight: bold; color: var(--color-dark); }
.contact-form input, .contact-form textarea {
  width: 100%; padding: var(--space-2); margin-bottom: var(--space-3);
  border: 1px solid var(--color-border); border-radius: 6px;
  font-size: var(--size-base); font-family: var(--font-body);
}
.contact-form input:focus, .contact-form textarea:focus {
  outline: none; border-color: var(--color-gold); box-shadow: 0 0 0 3px rgba(212,165,116,0.25);
}

/* ===== 10. Footer ===== */
footer { background-color: var(--color-dark); color: var(--color-cream); text-align: center; padding: var(--space-4) var(--space-3);
.social-links a { color: var(--color-gold); text-decoration: none; margin: 0 var(--space-2); }
.social-links a:hover { text-decoration: underline; }

/* ===== 11. Responsive ===== */
@media (max-width: 1023px) {
  .product-grid { grid-template-columns: repeat(2, 1fr); }
  .hero h1 { font-size: 38px; }
}
@media (max-width: 767px) {
  header { flex-direction: column; gap: var(--space-2); }
  nav ul { flex-direction: column; align-items: center; gap: var(--space-1); }
  .product-grid { grid-template-columns: 1fr; }
  .hero h1 { font-size: 30px; }
  .hero p  { font-size: 16px; }
}
```

F.14 Summary of the session

- One **shared external stylesheet** styles every page; link it in the `<head>` of each file.
- **CSS custom properties** store the palette, fonts and spacing in one place.
- A **typography scale** (two fonts, a few sizes) keeps headings consistent.
- A **spacing system** (a few repeated values) keeps layouts calm.
- **Flexbox** for the header/nav; **Grid** for the product cards.
- **Borders, shadows, radii and hover states** add depth and feedback.
- **Reusable classes** (like `.button`) avoid repetition.
- **Comment and order** the stylesheet so it is maintainable.
- **Media queries** at the bottom make everything responsive.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- The instructor's finished Adaeze Fashion site open in Chrome (the target).
- The project folder open in VS Code with the four HTML files and an empty `css/style.css`.
- The projector showing VS Code and Chrome side by side (or quickly tabbing).

G.2 Demonstration 1 — link the stylesheet and prove it (3 minutes)

Say: "Every page must point to the same stylesheet. Watch how one line on each page controls the whole site."

1. In `index.html` `<head>`, type:

```
<link rel="stylesheet" href="css/style.css">
```

2. In `css/style.css`, type a single obvious rule:

```
body { background-color: lightyellow; }
```

3. Save both, refresh in Chrome.
   - **Expected result:** the whole page turns pale yellow.
4. Copy the same `<link>` line into `shop.html`, `about.html` and `contact.html`. Refresh each.
   - **Expected result:** all four pages are pale yellow.
5. **Inject an error on purpose:** change `href` to `href="style.css"` (dropping the `css/` folder). Refresh.
   - **Expected result:** the page goes back to white; Console shows `Failed to load resource: net::ERR_FILE_NOT_FOUND`.
   - **Say:** "This is the number-one styling bug. If a page suddenly loses all its styling, check the link path first." Fix it back.
6. Replace the yellow test rule with the real `:root` variables and base styles from Section F.13, explaining each block as you paste it.

Questions to ask:

- "If I change the palette variable, how many files do I edit?" (One.)
- "Why does the yellow rule not belong in our final file?" (It was a test to prove the link worked.)

G.3 Demonstration 2 — build the header, hero and products live (9 minutes)

Say: "Now I will style the three sections you see most: the top bar, the welcome band, and the products."

1. Paste the **reset & base** block. Point out `box-sizing: border-box` and explain the overflow it prevents.
2. Paste the **typography** block. Show the headings changing font on refresh.
3. Paste the **header & navigation** block. Refresh. Point to: logo left, links right, gold-on-hover.
   - **Inject an error:** target `nav { text-decoration: none; }` instead of `nav a`. The underlines remain.
   - **Expected result:** links still underlined. **Say:** "The underline lives on the anchor, so I must style `nav a`." Correct it.
4. Paste the **hero** and **buttons** blocks. Refresh and hover the button — show the lift.
5. Paste the **product grid** block. Refresh — three cards appear in a row.
   - **Inject an error:** change `grid-template-columns: repeat(3, 1fr)` to `repeat(3, 1fr)` *misspelled* as `grid-template-column: ...` (singular). Refresh.
   - **Expected result:** cards stack in one column (the rule is ignored). **Say:** "One wrong word and a whole rule is silently ignored. Learn to check the property name." Correct it.

Explain the *reasoning* aloud at each step: "I use `var()` so the palette is central; I use Grid here because I want equal columns; I use `object-fit: cover` so photos do not stretch."

G.4 Demonstration 3 — footer, form and responsive (3 minutes)

1. Paste the **footer** block. Refresh — dark footer across the bottom.

2. Paste the **form** block and click into a field to show the gold focus ring.
3. Paste the **responsive** block. Open DevTools (`F12`), toggle the device toolbar (`Ctrl+Shift+M`), drag to a phone width.
    - **Expected result:** the layout reflows — one product column, stacked header.
4. Click every nav link to confirm all four pages now look consistent.

Final question to the class: "What is the *one* tool that let us restyle the whole site without touching the HTML?" (The shared stylesheet + variables.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding step by step, learners convert their own plain Session 8 Adaeze Fashion site into a designed, consistent, responsive website using one stylesheet.

Starting materials: The learner's `adaeze-fashion/` folder from Session 8, with an empty `css/style.css` file.

Procedure (instructor calls each step; learners follow; instructor walks the room):

1. Confirm all four HTML files are open-able and that `css/style.css` exists. In each HTML file, add inside `<head>` :

    ```
    <link rel="stylesheet" href="css/style.css">
    ```

2. In `style.css` , add the `:root` block from Section F.13 (all colours, fonts and spacing). Save.
3. Add the reset and base `body` rules. Save, refresh. **Pause and predict:** "What should change right now?" (Almost nothing visible yet — margins tighten, font becomes Segoe UI.)
4. Add the typography block. Refresh. Headings become Georgia and shrink to our scale.
5. Add the header & navigation block. Refresh. **Pause and predict:** "Where will the logo and the four links sit?" (Logo left, links right, in a row.) Hover a link — it fades to gold.
6. Add the hero and button blocks. Refresh. **Pause and predict:** "What happens when you hover the button?" (It darkens and lifts 2px.)
7. Add the product grid block. Refresh on `shop.html` . **Pause and predict:** "How many cards per row?" (Three, on a wide screen.)
8. Add the footer block. Refresh.
9. Add the form block and open `contact.html` . Click into the Name field.
10. Add the responsive block. Press `F12` , then `Ctrl+Shift+M` , and drag to phone width. Watch the layout change.
11. Visit all four pages in turn and confirm the header, footer and colours match.

Expected result: a cohesive Four-page site — dark brown header and footer, cream hero, gold buttons, three product cards on desktop, one on mobile, and a styled form.

Points to pause on and predict (write predictions on paper first):

- After step 4: "Will the paragraph text change colour?" (No — only headings were targeted.)
- After step 7: "If the screen is 900px wide, how many cards?" (Two — the tablet media query is not in yet, so still three until step 10; this is a good discussion point.)

Troubleshooting while walking the room:

- Nothing is styled → the `<link>` is missing, misspelled, or in the wrong folder. See Section K.1.
- Products stack even on desktop → the `product-grid` class name is misspelled in the HTML or CSS. See Section K.3.
- Nav links still underlined → you styled `nav` not `nav a` . See Section K.4.
- Inputs overflow the form → `box-sizing: border-box` is missing from the reset. See Section K.5.
- ₦ shows as `?` → file encoding; save as UTF-8 (Section K.6).

Completion checklist (learner ticks each):

- ☐ `<link>` to `css/style.css` present in all four pages
- ☐ `:root` variables define colours, fonts and spacing
- ☐ Reset includes `box-sizing: border-box`

- [ ] Header/nav styled with flexbox and a hover state
- [ ] Hero and `.button` styled
- [ ] Product grid uses `display: grid`
- [ ] Footer styled and consistent
- [ ] Contact form styled with a visible focus state
- [ ] At least two media queries at the bottom
- [ ] All four pages look consistent on desktop and mobile

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Style a *new* single page for a different Port Harcourt business — **"Mama Ngozi's Kitchen"**, a small restaurant — from scratch, without copying the Adaeze Fashion stylesheet. This proves you can apply the palette-and-scale approach to any brand.

Scenario: Mama Ngozi serves local dishes. She wants a warm, appetising look. Her brand colours are **deep green** `#1b4332`, **warm orange** `#e07a1f` and **off-white** `#faf7f2`. She has one page listing three dishes with prices.

Starting information (given):

- Use the same HTML structure you know: `header` (logo + nav), a hero, a menu section with three cards, a footer.
- Dishes: "Jollof Rice & Chicken — ₦3,500", "Egusi Soup & Pounded Yam — ₦4,000", "Pepper Soup — ₦2,800".
- Fonts: use **Georgia** for headings and **Arial** for body.

Requirements:

1. Create `mama-ngozi/index.html` and `mama-ngozi/css/style.css`; link them.
2. Define a `:root` block with **three colours**, two fonts and at least four spacing values.
3. Style the header with flexbox and a hover state on the nav links.
4. Style a hero with a heading, one sentence and a call-to-action button.
5. Style a menu using **CSS Grid** to show **three equal cards** on desktop.
6. Add a media query that shows **one card per row** below 767px.
7. Add a soft `box-shadow` and `border-radius` to the cards.

Time allowance: 15 minutes.

Constraints: no copying the Adaeze `style.css`; the palette must actually be the green/orange/off-white given; all colours must be defined as custom properties (no hard-coded hex inside the component rules).

Success criteria: the page uses Mama Ngozi's colours, has a working flex header, a three-column grid that becomes one column on mobile, and a consistent look.

Hints if stuck:

- Hint 1: Green `#1b4332` is the "dark" colour (header, footer, headings). Orange `#e07a1f` is the accent (buttons, prices). Off-white `#faf7f2` is the page/hero background.
- Hint 2: Your grid line is `grid-template-columns: repeat(3, 1fr);` and the mobile override is `grid-template-columns: 1fr;`.
- Hint 3: Reuse one `.button` class for the hero button *and* the card buttons, exactly as in Adaeze Fashion.

Instructor answer key (do not show learners first):

```
/* ===== Mama Ngozi's Kitchen ===== */
:root {
  --green:  #1b4332;
  --orange: #e07a1f;
  --cream:  #faf7f2;
  --text:   #333333;
  --font-heading: Georgia, serif;
  --font-body: Arial, sans-serif;
  --sp-1: 8px; --sp-2: 16px; --sp-3: 24px; --sp-4: 32px;
}

* { margin: 0; padding: 0; box-sizing: border-box; }
```

```
body { font-family: var(--font-body); color: var(--text); line-height: 1.6; }

h1, h2, h3 { font-family: var(--font-heading); color: var(--green); }

header {
  display: flex; justify-content: space-between; align-items: center;
  background-color: var(--green); padding: var(--sp-2) var(--sp-3);
}
.logo { color: var(--orange); font-size: 24px; font-weight: bold; text-decoration: none; }
nav ul { display: flex; list-style: none; gap: var(--sp-3); }
nav a { color: var(--cream); text-decoration: none; transition: color 0.3s ease; }
nav a:hover { color: var(--orange); }

.hero { text-align: center; padding: var(--sp-4) var(--sp-3); background-color: var(--cream); }
.hero p { margin-bottom: var(--sp-4); }

.button {
  display: inline-block; background-color: var(--orange); color: #fff;
  padding: var(--sp-2) var(--sp-3); text-decoration: none; border-radius: 6px; font-weight: bold;
}
.button:hover { background-color: #b85f14; }

.menu { max-width: 1000px; margin: 0 auto; padding: var(--sp-4) var(--sp-3); }
.menu-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: var(--sp-3); }
.dish {
  background-color: #fff; border-radius: 10px; padding: var(--sp-3); text-align: center;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}
.price { color: var(--orange); font-weight: bold; }

footer { background-color: var(--green); color: var(--cream); text-align: center; padding: var(--sp-3); margin-top: var(--sp-4); }

@media (max-width: 767px) {
  .menu-grid { grid-template-columns: 1fr; }
  header { flex-direction: column; gap: var(--sp-2); }
}
```

Marking guidance (out of 10):

- Correct `:root` with 3 colours + 2 fonts + spacing (2)
- Header/nav flex + hover (2)
- Hero + button + hover (2)
- Grid three-up, one-up on mobile via media query (3)
- Shadows/radius and overall consistency (1)

J. REAL-WORLD CASE STUDY

Situation. *Adaeze* runs Adaeze Fashion from a shop off Rumuola Road, Port Harcourt. She sews bespoke gowns and has a strong Instagram following. A year ago she paid a young man ₦40,000 for a website. It "worked" but looked broken: three different shades of brown, a heading that overflowed on her phone, and a nav bar that jumped to a different position on every page.

Relevant facts.

- Customers told her the site "looked like a scam" and bounced.
- The young man had written separate, slightly different CSS for every page.
- The colours were hard-coded in dozens of places, so nothing matched.

Decision point. Should Adaeze pay for a whole new site, or can the existing one be fixed?

Possible approaches.

1. Pay another developer ₦80,000 for a brand-new site.
2. Keep the HTML and rewrite the CSS into one organised, shared stylesheet with a palette and a scale — the exact skill learned today.

3. Leave it and rely only on Instagram.

Recommended solution (for this stage). Approach 2. The structure (HTML) was fine; only the presentation was chaotic. A single shared stylesheet with `:root` variables, a typography scale and a spacing system would make every page consistent and fix the mobile overflow with one media query.

Trade-off.

- Approach 1 costs money and time and may repeat the same mistakes.
- Approach 2 is cheap, fast and permanent — and it is a service the learner can now offer.
- Approach 3 loses the trust and reach a website brings.

Professional connection. A large part of real front-end work is *refactoring* messy CSS into a tidy, token-based system. Being able to say "I will centralise your colours and spacing so your site looks consistent everywhere" is a billable, professional sentence — and it is exactly what today's lesson made possible.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this as the live troubleshooting guide. When a learner hits a problem, ask them to check the matching row before giving the answer.

K.1 A page has no styling at all

Symptoms: One page (or all pages) shows plain black text and blue underlined links. **Likely causes:** (a) The `<link>` tag is missing from that page's `<head>`. (b) The `href` path is wrong (missing `css/`). (c) The file name is misspelled (`style.css` vs `styles.css`). **How to investigate:** Press `F12` → Console. Look for a red `net::ERR_FILE_NOT_FOUND` naming the stylesheet. Compare the spelling exactly. **Corrective steps:** Add or fix the `<link>` line in every page; save; refresh with `Ctrl+F5`. **How to confirm resolved:** The page's colours and fonts return. **Prevention:** Put the exact same `<link>` line in all pages; keep `style.css` inside the `css/` folder.

K.2 My colours/fonts do not change even after editing `:root`

Symptoms: You change a variable but the page looks the same. **Likely cause:** The component is using a **hard-coded** value (e.g. `color: #2c1810;`) instead of `var(--color-dark)`. **How to investigate:** In DevTools, click the element and read which value wins in the Styles panel. **Corrective steps:** Replace the hard-coded value with the matching `var(...)`. **Prevention:** Decide from the start that all colours come from `:root`.

K.3 Product cards stack even on a wide screen

Symptoms: Cards appear in one vertical column instead of three. **Likely cause:** A class-name mismatch — the HTML says `class="product-grid"` but the CSS rule is `.products-grid`. **How to investigate:** Compare the class name in the HTML and the selector in the CSS, character by character (hyphens and plurals are the usual culprits). Or inspect the container in DevTools and check whether the grid rule is greyed out. **Corrective steps:** Make both names identical. **Prevention:** Type the class in the HTML first, then copy it into the CSS.

K.4 Navigation links keep their underline

Symptoms: The nav links are still underlined despite styling the navigation. **Likely cause:** The rule targets `nav` instead of `nav a`. The underline belongs to the anchor. **Corrective steps:** Change `nav { text-decoration: none; }` to `nav a { text-decoration: none; }`. **Prevention:** Remember: style the `a`, not its container.

K.5 Form inputs spill past the edge of the page

Symptoms: Inputs are wider than the form and cause a horizontal scrollbar. **Likely cause:** `box-sizing: border-box` is missing from the reset, so padding adds to the 100% width. **How to investigate:** In DevTools, hover the input and watch the box-model diagram — if width grows past 100%, this is it. **Corrective steps:** Add `* { box-sizing: border-box; }` at the top of the stylesheet. **Prevention:** Always include the reset.

K.6 The Naira sign shows as `?` or a box

Symptoms: `₦` renders as `?` or a tofu box. **Likely cause:** The file was saved in a non-UTF-8 encoding. **Corrective steps:** In VS Code, click the encoding in the bottom-right, choose **Save with Encoding → UTF-8**, and keep `<meta charset="UTF-8">` in the `<head>`. **Prevention:** Always keep the charset line.

K.7 Spacing looks random

Symptoms: Some sections are tight, others huge. **Likely cause:** You used one-off pixel values instead of the spacing variables. **Corrective steps:** Replace ad-hoc values with `var(--space-2)`, `var(--space-3)`, etc. **Prevention:** Restrict yourself to the handful of spacing values.

Distinguish likely vs confirmed: Every "likely cause" becomes *confirmed* only after you check it — read the Console error, compare the class names, inspect the element. Teach learners to confirm before fixing so they do not change the wrong thing.

L. SESSION ASSIGNMENT

Assignment title: "Style the Adaeze Fashion Website"

Purpose: Apply a complete, organised stylesheet to a real multi-page site — the direct preparation for the capstone.

Realistic scenario: Adaeze has her four HTML pages ready from last week but no design. She hires you to give the site a professional, consistent, mobile-friendly look before she shows it to customers.

Detailed instructions:

1. In your `adaeze-fashion/` project, create or open `css/style.css`.
2. Add the `<link rel="stylesheet" href="css/style.css">` line to the `<head>` of **all four** HTML pages.
3. Build the stylesheet in the order from Section F.12, with a comment header for each section.
4. Define a `:root` block with your brand colours, two fonts and a spacing scale.
5. Style the header, navigation, hero, buttons, product grid (using Grid), contact form and footer.
6. Add at least two media queries at the bottom (tablet and mobile).
7. Reuse one `.button` class everywhere a button appears.
8. Test every page on desktop and on a phone width (DevTools device toolbar).

Required deliverables:

- `adaeze-fashion/css/style.css` (organised and commented).
- All four HTML pages linked to it.
- A photo or written note of the site viewed at desktop width and phone width.

Submission format: Files in the class project folder / shared drive; screenshots or written notes in your workbook.

Deadline: Before Session 10.

Marking rubric (30 marks):

Criterion	5 — Excellent	3 — Good	1 — Needs work
Stylesheet organisation	Sectioned, commented, logical order	Mostly organised	Random order, few comments
Colour palette & type scale	Clear palette + scale via variables	Palette present, partly reused	Hard-coded, inconsistent
Header & navigation	Flex, aligned, hover state	Mostly consistent	Unstyled or broken
Hero, buttons & product grid	Grid three-up, hover effects	Present but minor issues	Missing or not grid
Form styling	Styled with clear focus state	Styled but plain	Unstyled
Responsiveness	Two breakpoints, clean on mobile	One breakpoint	Not responsive
Total	**30**		

Common mistakes to avoid: forgetting `<link>` on some pages; hard-coding colours; styling `nav` instead of `nav a`; omitting `box-sizing: border-box`; no media queries.

Instructor reference solution: the complete stylesheet in Section F.13, plus the HTML heads:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion | Home</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Short answer. Why do all pages of a website link to the *same* stylesheet instead of each having its own?

Q2. Multiple choice. Which line correctly links an external stylesheet stored in a `css/` folder? a) `<a href="css/style.css">` b) `<style src="css/style.css">` c) `<link rel="stylesheet" href="css/style.css">` d) `<stylesheet href="css/style.css">`

Q3. Fill the gap. Write the CSS that stores a brand colour in a variable and then applies it to the body text.

```
:root { ____: #2c1810; }
body  { color: ____(____); }
```

Q4. Explain in your own words. What does `box-sizing: border-box` do, and why does it stop form inputs from overflowing?

Q5. Error detection. Find and fix the mistake:

```
nav { text-decoration: none; }
```

when the goal is to remove underlines from the navigation *links*.

Q6. Scenario. A client says "change the brown to navy everywhere." In a site built with `:root` variables, how many lines do you edit, and which one?

Q7. Practical prediction. Given `header { display: flex; justify-content: space-between; align-items: center; }`, describe where a logo and a nav list will appear.

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. So the design updates in one place and stays consistent across every page — edit once, all pages change together. **A2.** (c) `<link rel="stylesheet" href="css/style.css">`. **A3.** `--color-dark: #2c1810;` and `color: var(--color-dark);`. (Accept any sensible variable name.) **A4.** It makes an element's declared width include its padding and border. Without it, `width: 100%` plus padding adds *beyond* 100%, so inputs spill past the form and cause a horizontal scrollbar. **A5.** The underline is on the anchor, so change the selector: `nav a { text-decoration: none; }`. **A6.** One line — the palette variable in `:root` (e.g. `--color-dark`). Every rule using `var(--color-dark)` updates automatically. **A7.** The logo appears on the far left and the nav list on the far right, both vertically centred, in a single row.

N. REFLECTION AND RECAP

Plain-English summary. Today you stopped decorating single elements and started designing a whole website. You created one shared stylesheet, stored the brand's colours, fonts and spacing as CSS custom properties, built a typography scale, and styled the header, navigation, hero, product grid, buttons, footer and contact form. You organised the file with comment headers in a logical order, reused one button class everywhere, and added media queries so the whole site adapts from desktop to phone — all without touching the HTML structure you built last week.

Five important takeaways:

1. One shared external stylesheet keeps a multi-page site consistent.
2. CSS custom properties put the palette, fonts and spacing in one place.
3. Restraint wins: three to five colours and two fonts look professional.
4. Reuse classes (one `.button`) instead of writing the same rule repeatedly.
5. Keep media queries at the bottom, in order from large to small.

Questions to help you recognise what you can now do:

- Can I link one stylesheet to every page without help?
- Can I change my whole site's colour by editing one variable?
- Can I explain why `nav a`, not `nav`, controls the links?
- Can I make a grid of cards that becomes one column on a phone?

Connection to the next session: Your site is now beautiful but still *still* — it looks good but does nothing when touched. In Session 10 we add gentle interactivity with CSS (hover, transitions, simple animation), embed a Google Map on the contact page and a YouTube video on the shop page, and touch on accessibility. That is the difference between a poster and an experience.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner links the stylesheet to all four pages unaided, with the correct relative path.
- Learner uses `var()` throughout rather than hard-coded colours.
- Learner can point to `:root` and explain that changing one line updates the site.
- Learner chooses flexbox for the header and Grid for the products, and can say why.
- Learner reuses the `.button` class instead of duplicating rules.
- Learner tests every page and both screen sizes, not just the Home page.

Common misconceptions to watch for and correct:

- "Each page needs its own CSS." → One shared file prevents drift.
- "More colours and fonts look more designed." → Restraint looks professional.
- "`text-decoration: none` on `nav` will work." → Style the anchor.
- "The reset is unnecessary." → Without `box-sizing: border-box`, layout bugs multiply.
- "Margin and padding are the same." → Inside vs outside the border.

Suggested remedial exercises for learners who struggle:

1. **Link hunt:** give a four-page site where one page is missing the `<link>`; ask the learner to find and fix it using the Console.
2. **Palette swap drill:** hand over a finished stylesheet and ask the learner to reskin it to a new brand by editing only `:root`.
3. **Selector fix-up:** provide five rules with the wrong selector (`nav` vs `nav a`, `.product` vs `.products`) and ask for corrections.

Differentiation:

- **Level 1 (supported):** Provide a partially completed `style.css` with `:root` and the reset already in place, and guide the learner through pasting each section.
- **Level 2 (standard):** The exercise and assignment as written.
- **Level 3 (challenge):** Add a `:root` dark-mode variant toggled by a `.dark` class on `<body>`, add `:focus-visible` styles for keyboard users, and introduce CSS `clamp()` for fluid heading sizes.

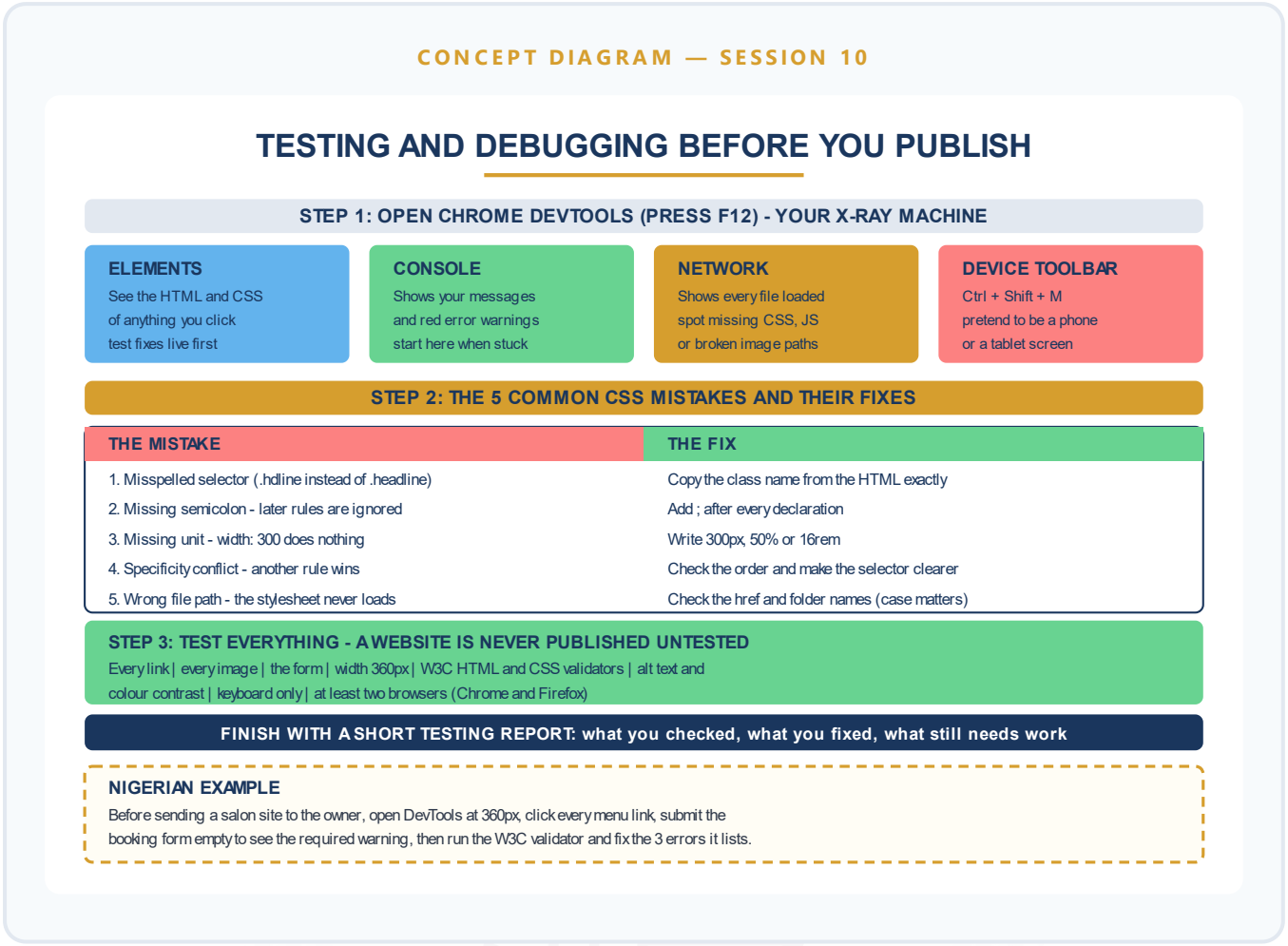
Timing guidance: The practical (Section H) is the priority. If the room is slow, compress the case study (Section J) to a two-minute mention and shorten the independent exercise to just the `:root` and header. Protect at least 25 minutes of hands-on styling time.

END OF SESSION 9

Next: Session 10 — Interactivity and Media.

SESSION 10 OF 12

INTERACTIVITY AND MEDIA



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 10 — INTERACTIVITY AND MEDIA

A. SESSION IDENTIFICATION

Item	Detail
Session number	10 of 12
Session title	Interactivity and Media
Week / Module	Week 4 — Module: Publishing and Capstone
Recommended duration	105 minutes
Difficulty level	Beginner (builds on Sessions 4–9)

Item	Detail
Prerequisite knowledge	Sessions 1–9. Learners can build a styled, responsive multi-page site with a shared stylesheet, custom properties and media queries.
Connection to previous work	In Session 9 the Adaeze Fashion site became *beautiful* but *static* — nothing responds to the mouse, and there is no map or video. This session adds gentle **CSS-only interactivity** (hover, focus, transitions, simple animation and a show/hide trick) and richer **media** (responsive images, a Google Map, a YouTube video and a favicon). We deliberately stop short of JavaScript; real behaviour is a later course.
Preparation before class	Academy PCs on; VS Code and Chrome; each learner's styled site from Session 9 open; projector; a printed "Media & Interactivity Checklist"; **Internet required** for the map and video embeds; a sample square logo image for the favicon; the instructor's finished site ready to show.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Explain the difference between** `:hover`, `:focus` **and** `:active` and when each applies.
2. **Add smooth** `transition` **s** so hover and focus changes animate instead of jumping.
3. **Apply a simple** `transform` (translate/scale/rotate) for a hover effect.
4. **Create a basic** `@keyframes` **animation** and attach it to an element.
5. **Show and hide content using CSS only** (the `:hover` reveal and the checkbox hack), and explain its limits.
6. **Make images responsive** with `max-width: 100%; height: auto;` and describe `srcset`, `sizes` and `loading="lazy"` at a basic level.
7. **Embed a Google Map** on the contact page using an `<iframe>` copied from Google Maps.
8. **Embed a YouTube video responsively** using the `aspect-ratio` (or padding-bottom) technique.
9. **Add a favicon** so the browser tab shows the brand mark.
10. **Apply three accessibility touches** — keep visible focus outlines, write meaningful `alt` text, and ensure sufficient colour contrast — and explain why each matters.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Sharing is fine — one types, one checks the browser, then swap.
- Instructor computer connected to a projector or large screen.

Software

- Visual Studio Code (2022 or later).
- Google Chrome (any recent version).

Accounts

- None required. The map and video embeds are copied from public pages and need no login.

Sample files

- The learner's `adaeze-fashion/` project from Session 9 (four pages, `css/style.css`, `images/`).
- A square logo image, e.g. `images/logo.png` (roughly 512×512), to convert into a favicon.
- The instructor's finished reference site and `style.css`.

Worksheets

- "Media & Interactivity Checklist" (printed, 1 page).

Tools

- Chrome DevTools (`F12`) — used lightly to test hover/focus and to check the video size.

Internet

- **Required this session.** Google Maps and YouTube both fetch their content from the internet. If the academy internet fails, use the instructor's pre-saved embed code and explain that it will work once connectivity returns.

Alternative for learners who lack a facility

- Any laptop with Chrome, VS Code and internet works. Offline learners can still write all the CSS and paste the iframe code; the map/video will appear once online.

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the learner's finished Session 9 site. Slowly hover a nav link and a product card, click into a form field, then — in a second tab — open the instructor's finished site where a button lifts, a card zooms, a "Delivery info" panel slides open on hover, a map is embedded on the contact page and a video sits neatly on the shop page.

What the instructor says:

> *"Last week our site looked good — but it was like a shop with the lights off and the doors locked. Watch what happens with small CSS touches: the link glows, the button lifts, the card breathes, a hidden panel reveals itself, and now there is a map so customers can find Adaeze and a video so they can see the clothes move.*
>
> *All of this today is still just HTML and CSS — no JavaScript. There is one rule I want you to carry out of this room: real, clever interactivity — menus that open on click, forms that validate, buttons that calculate — needs a programming language called JavaScript, and that is a whole later course. Today we learn what CSS alone can do, and how to add media like a map and a video."*

Expected learner response: Learners will enjoy the hover effects and ask "How did you make the panel open?" Accept the excitement, and be honest: "Some of these tricks are simple; some have limits — I will show you both."

Why this opener works: It motivates the topic with visible, tactile effects while immediately setting an ethical boundary — the learner knows exactly how far CSS goes and what JavaScript is for, which prevents both over-promising and confusion later.

E. PREREQUISITE REVIEW (7 minutes)

Run as a quick oral round-robin. Questions on the projector.

Question 1: "Why do all pages of a site use the same stylesheet?"

- Expected answer: consistency and one-place updates.
- Common gap: learners think it is only about saving typing. Emphasise *consistency* first.

Question 2: "What are CSS custom properties and how do we use them?"

- Expected answer: named values stored on `:root` , used with `var(--name)` .
- Common gap: writing `var` without `()` or forgetting the double hyphen. Show one live.

Question 3: "What CSS did we already use to change a nav link's colour when the mouse is over it?"

- Expected answer: the `:hover` pseudo-class, e.g. `nav a:hover { color: gold; }` .
- Common gap: some learners think that needed JavaScript. Confirm it is pure CSS.

Question 4: "What does `display: grid` do and where did we use it?"

- Expected answer: lays children out in a grid; we used it for the product cards (`grid-template-columns: repeat(3, 1fr)`).
- Common gap: confusing grid with flex. Grid = two dimensions (rows *and* columns); flex = mainly one direction.

Question 5: "What is a relative path and why does it matter?"

- Expected answer: a path relative to the current file (`images/photo.jpg`); it makes files work on any machine, including the web server.

- Common gap: mixing `\` (Windows) and `/` (web). On the web we always use forward slashes `/`.

Bridge to today (say this):

> *"Now that the site is consistent and responsive, we bring it to life. We will add small movements that guide the eye, reveal hidden information, embed a map and a video, and — importantly — make sure we do not break accessibility while we do it. Accessibility means a person using a keyboard or a screen reader can still use your site. That is not optional; it is professional."*

If more than a third of the class cannot answer Question 3, re-demonstrate a single `:hover` rule before continuing.

F. FULL LECTURE NOTES

F.1 Pseudo-classes — reacting to the user's state

The idea. A **pseudo-class** is a keyword you add to a selector to style an element *in a particular state* — when the mouse is over it, when it is focused, when it is being clicked, or when it is the first child, and so on.

Plain-English definition. A pseudo-class lets one element look different depending on what the user is doing with it, without any extra class names or JavaScript.

Why it exists. Users need feedback. If a button looks identical whether or not you are hovering it, and whether or not it is focused, people do not know what is clickable or where they are. Feedback is usability.

Analogy. Think of a **nylon shirt under a light**. It is the same shirt, but it *looks* different depending on the light. The shirt did not change; its appearance reacted to a condition. Pseudo-classes are the "light".

The three we use today:

Pseudo-class	Applies when…	Example
`:hover`	the mouse pointer is over the element	`a:hover { color: gold; }`
`:focus`	the element is selected (clicked into, or reached with the Tab key)	`input:focus { border-color: gold; }`
`:active`	the element is being pressed down right now	`.button:active { transform: scale(0.98); }`

Syntax note. You write them attached to a selector with a colon: `selector:hover { ... }`. The gap between `selector` and `:hover` matters — `a :hover` (with a space) means something completely different (a hovered *descendant* of `a`). No space: the element itself.

Worked example:

```
nav a:hover { color: var(--color-gold); }
.contact-form input:focus { border-color: var(--color-gold); }
.button:active { transform: scale(0.98); }
```

Expected browser result: hover a nav link → it turns gold; click into an input → its border turns gold; press and hold a button → it shrinks slightly under your finger.

Misconception. `:hover` does not exist on touchscreens in the same way; on a phone there is no mouse. This is one reason we also style `:focus` and never rely on hover for essential information. Always make sure important content is visible *without* hover.

Limitation. Pseudo-classes only respond to *state*. They cannot store information, do calculations, or respond to a click that *toggles* something permanently — that needs JavaScript (or the checkbox trick in F.5).

F.2 Transitions — making changes smooth

The idea. A **transition** tells the browser: "when this property changes, do not jump — animate it over a short time."

Why it exists. An instant colour snap feels cheap and jarring. A 0.3-second fade feels polished and helps the eye follow the change.

Analogy. A **dimmer switch** versus a normal light switch. Both end at "light on", but the dimmer glides and feels expensive. A transition is the dimmer switch for CSS values.

Syntax:

```
selector {
  transition: property duration timing-function;
}
```

- `property` — what to animate (`color`, `background-color`, `transform`, or `all`).
- `duration` — how long, e.g. `0.3s` or `300ms`.
- `timing-function` — the feel; `ease` (gentle start and end) is the common default, `ease-in-out` is even softer, `linear` is mechanical.

Worked example:

```
.button {
  background-color: var(--color-gold);
  transition: background-color 0.3s ease, transform 0.2s ease;
}
.button:hover {
  background-color: var(--color-gold-dark);
  transform: translateY(-2px);
}
```

Line by line:

- The transition lists **two** properties separated by a comma: background colour over 0.3s and transform over 0.2s.
- `translateY(-2px)` moves the element 2px **up** (negative y is up) on hover.
- Because the transition is on the base `.button` (not on `:hover`), the animation plays **both** on entering and leaving the hover state.

Expected browser result: the button darkens and lifts gently over a third of a second; release the mouse and it glides back down.

Misconception. Learners put `transition` inside the `:hover` rule. Then the animation runs *into* hover but snaps back instantly. The transition belongs on the **base** selector.

Performance note. `color`, `background-color` and `transform` are cheap to animate. Animating `width`, `height` or `left/top` forces the browser to recalculate layout and can stutter. Prefer `transform` and `opacity` where possible.

F.3 Transforms — moving, scaling and rotating

The idea. `transform` changes how an element is drawn — moved, scaled, rotated or skewed — **without affecting the position of other elements**.

Why it exists. It lets you create motion (lift, zoom, turn) cheaply and without breaking the layout around it. That last point is the key advantage: unlike changing `margin` or `width`, a transform does not push neighbours around.

The functions we use:

Function	Effect	Example
`translateY(-2px)`	move up 2px	button lift
`scale(1.05)`	grow 5%	image zoom on hover
`rotate(90deg)`	turn 90°	an arrow that rotates

Worked example — an image that zooms on hover:

```
.product img {
  transition: transform 0.4s ease;
}
.product:hover img {
  transform: scale(1.06);
}
```

- Note the selector `.product:hover img` — "when the **card** is hovered, transform its **image**". This is a descendant selector combined with a pseudo-class, a very common professional pattern. Hover anywhere on the card and the photo zooms.

Expected browser result: moving the mouse over a product card gently enlarges its photo by 6%; moving away shrinks it back.

Important overflow note. If the zoomed image spills outside the card, add `overflow: hidden;` to the card so the overflow is clipped neatly:

```
.product { overflow: hidden; }
```

Misconception. Learners think `transform` deletes the element or changes the document flow. It does neither — it is purely visual.

F.4 Keyframe animations — motion without a trigger

The idea. A transition needs a *change* (like hover) to fire. An **animation** with `@keyframes` can play by itself, in a loop or once, the moment the page loads.

Why it exists. Sometimes you want motion that is not caused by the user: a gentle fade-in as the hero appears, a soft pulsing badge on a "Sale" tag.

Analogy. A transition is a door that swings when pushed. An animation is a wind-up toy that keeps moving on its own once you set it going.

Syntax:

```
@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}

.hero {
  animation: fadeIn 1s ease-in;
}
```

- `@keyframes fadeIn` defines the motion and gives it a name.
- `from { opacity: 0; }` is the start (invisible), `to { opacity: 1; }` is the end (fully visible).
- `animation: fadeIn 1s ease-in;` attaches it: name, duration, feel.
- `opacity: 0` means fully transparent; `1` means fully solid.

A slightly richer example — a gentle float loop:

```
@keyframes float {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-6px); }
  100% { transform: translateY(0); }
}
.badge {
  animation: float 3s ease-in-out infinite;
}
```

- Using `0% / 50% / 100%` gives three stages. `infinite` makes it loop forever.

Expected browser result: the hero fades in once on load; the badge bobs gently up and down forever.

Misconception / important caution. Do not over-animate. Constant motion is distracting and can be uncomfortable for some users. Use animation sparingly (one or two per page). Also, some users set an operating-system preference for reduced motion; we can respect it:

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none; transition: none; }
}
```

This small block is a mark of a thoughtful designer.

F.5 Showing and hiding content with CSS only

The idea. CSS can reveal or hide a panel without JavaScript. Two beginner-safe ways: a `:hover` reveal and the "checkbox hack".

Way 1 — hover reveal. The simplest. A hidden panel appears while you hover its parent.

```
.delivery-info {
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.4s ease;
}
.hero:hover .delivery-info {
  max-height: 200px;
}
```

- We animate `max-height` from `0` to a generous `200px` and clip with `overflow: hidden`.
- **Limitation:** this only works with a mouse. On a phone or with a keyboard there is no hover, so a touch user may never see the panel.

Way 2 — the checkbox hack. A hidden checkbox stores an on/off state; CSS reacts to `:checked`. This works on click (including on touch) and with the keyboard.

The HTML:

```
<input type="checkbox" id="menu-toggle" class="visually-hidden">
<label for="menu-toggle" class="menu-btn">Menu</label>
<nav class="mobile-nav">
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="shop.html">Shop</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>
```

The CSS:

```
.visually-hidden {
  position: absolute;
  width: 1px; height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
}

.menu-btn {
  display: none;   /* hidden on desktop */
  cursor: pointer;
  color: var(--color-cream);
}

.mobile-nav {
  display: none;
}

#menu-toggle:checked ~ .mobile-nav {
  display: block;
}
```

Line by line:

- The checkbox is visually hidden but still focusable (better for accessibility than `display: none`).

- The `<label>` is clickable because its `for="menu-toggle"` matches the checkbox's `id`; clicking the label toggles the checkbox.
- `#menu-toggle:checked ~ .mobile-nav` uses the **general sibling combinator** `~` ("a following sibling") and the `:checked` pseudo-class: "when the checkbox is checked, show the nav that follows it."
- On a real mobile menu you would only show `.menu-btn` inside a mobile media query.

Expected browser result: clicking the word "Menu" shows the list; clicking again hides it.

Honest limitation (say this clearly): the checkbox hack is a clever trick with real drawbacks — it cannot easily close the menu when you tap a link, it becomes messy with multiple menus, and it does not give screen readers helpful "expanded/collapsed" information. **Real, reliable interactivity — open/close menus, tabs, sliders, live search, form validation — is done with JavaScript.** That is taught in the Web Development course. Treat the checkbox hack as an introduction to the *idea* of state, not as the professional solution.

Misconception. Learners think CSS can "remember" a click. It cannot. The checkbox hacks works only because a form control (the checkbox) is holding the state for us. Without a control to hold state, CSS cannot.

F.6 Responsive images

The idea. An image should never be wider than its container, and on small screens it should shrink instead of overflowing.

Why it exists. Large photos are one of the top causes of slow, broken-looking mobile pages. Making images fluid is step one.

The core rule:

```
img {
  max-width: 100%;
  height: auto;
}
```

- `max-width: 100%` stops an image growing past its container.
- `height: auto` keeps the image's natural proportions so it does not squash.

The `width` / `height` attributes — preventing layout jump. In the HTML, add the image's real dimensions:

```
<img src="images/hero.jpg" alt="Adaeze Fashion studio in Port Harcourt" width="1200" height="600">
```

The browser reserves a 2:1 space *before* the image loads, so the page does not jump when it appears (a nice performance and usability win). The CSS rule above still keeps it fluid.

Lazy loading. Images far down the page need not load until the user scrolls to them:

```
<img src="images/product-1.jpg" alt="Ankara gown" loading="lazy" width="600" height="800">
```

`loading="lazy"` tells the browser to defer off-screen images, speeding up the first view.

A brief word on `srcset` and `sizes` (awareness only). For serious performance, you can offer several sizes of the same image and let the browser choose:

```
<img
  src="images/product-1-600.jpg"
  srcset="images/product-1-400.jpg 400w, images/product-1-800.jpg 800w"
  sizes="(max-width: 767px) 100vw, 300px"
  alt="Ankara gown" loading="lazy">
```

This means: "here are 400px and 800px versions; on phones use full width, otherwise a 300px slot." You do not need to master this today — just know it exists, because it is how large commercial sites stay fast.

Expected browser result: resize the window and images shrink with their column, never causing a horizontal scrollbar.

Misconception. Learners think setting `width: 100%` on *all* images is always best. It is not — sometimes you want a small logo to stay small. `max-width: 100%` is safer because it caps size without forcing full width.

F.7 Embedding a Google Map (contact page)

The idea. Google lets you copy a ready-made `<iframe>` that shows a map of any place, including a pin and address. You paste it into your page.

Why it exists. Customers need to *find* the business. A map removes the "where exactly are you?" phone call, which is huge for a small Port Harcourt shop.

What an `iframe` is. An **iframe (inline frame)** is a small window inside your page that shows *another* web page — here, Google's map. It is the same idea for the video in F.8.

The steps (in the browser):

1. Go to **google.com/maps**.
2. Search for the place, e.g. "*24 Ebony Road, Port Harcourt*". A marker appears.
3. Click **Share** (or the three-dot menu) → **Embed a map**.
4. Choose a size, then click **Copy HTML**.
5. Paste the copied `<iframe>` into your `contact.html` where the map should appear.

The embed code (the `src` will be a long Google link in real use; shown shortened here):

```
<iframe
  src="https://www.google.com/maps/embed?pb=..."
  width="600" height="450"
  style="border:0;"
  allowfullscreen=""
  loading="lazy"
  referrerpolicy="no-referrer-when-downgrade"
  title="Map showing Adaeze Fashion, Port Harcourt">
</iframe>
```

Line by line:

- `src` — the special Google link that loads the map. This is why internet is required.
- `width` / `height` — the default size; we override this with CSS below.
- `style="border:0"` — removes the default frame border.
- `allowfullscreen` — lets the map expand to full screen.
- `loading="lazy"` — the map loads only when scrolled near (better speed).
- `title` — describes the iframe for screen-reader users; **always add this**.

Making the map responsive (fill the width):

```
.map-container {
  max-width: 800px;
  margin: 0 auto;
}
.map-container iframe {
  width: 100%;
  height: 380px;
  border: 0;
  border-radius: 10px;
}
```

Expected browser result: a rounded map showing a pin on 24 Ebony Road, sized to the page and shrinking with it on mobile.

Accuracy note. You cannot build a map by hand — the embed is Google's own page shown through an iframe. If the internet is down, the area shows blank or a "cannot load" message. That is expected, not a bug in your code.

Limitation. The embed shows a *map*. Google's free embed does **not** give you live turn-by-turn directions or unlimited styling. For a basic "find us" map, it is perfect.

F.8 Embedding a YouTube video responsively (aspect-ratio)

The idea. Copy YouTube's embed `<iframe>` for the video, then wrap it so it keeps its 16:9 shape and shrinks on small screens.

Why it exists. Video is the best way for a fashion brand to show fabric moving and colours in real light. A responsive video keeps the layout tidy on a phone without squashing or letterboxing badly.

The steps (in the browser):

1. Open the video on youtube.com.
2. Click **Share** → **Embed**.
3. Click **Copy**.
4. Paste the `<iframe>` into `shop.html`, wrapped in a container.

The modern technique — `aspect-ratio`:

```
<div class="video-container">
  <iframe
    src="https://www.youtube.com/embed/VIDEO_ID"
    title="Adaeze Fashion — Ankara collection preview"
    allow="accelerometer; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
    allowfullscreen>
  </iframe>
</div>
```

```
.video-container {
  max-width: 760px;
  margin: 0 auto;
}
.video-container iframe {
  width: 100%;
  aspect-ratio: 16 / 9;
  border: 0;
  border-radius: 10px;
}
```

- `aspect-ratio: 16 / 9` tells the box to keep a widescreen shape; when the width changes, the height follows automatically. This is the clean, modern way.
- `VIDEO_ID` is the short code after `watch?v=` (or after `youtu.be/`) in the video's URL. Note `embed` and `watch?v=` are different links — the embed one is what an iframe needs.

The older technique (still common) — the padding-bottom trick: if a browser does not support `aspect-ratio`, use this:

```
.video-container {
  position: relative;
  width: 100%;
  height: 0;
  padding-bottom: 56.25%; /* 9 ÷ 16 = 0.5625 → 16:9 */
  overflow: hidden;
}
.video-container iframe {
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  border: 0;
}
```

- `padding-bottom: 56.25%` builds a box whose height is 56.25% of its width — exactly 16:9. The iframe is then stretched to fill that box absolutely.
- Know both; modern sites increasingly use `aspect-ratio`.

Expected browser result: a rounded video that fills the column on desktop and shrinks neatly on mobile, always keeping a movie-like shape.

Accuracy note. The video streams from YouTube's servers, so internet is required. If a video is private, embeddable is disabled by its owner, or the `VIDEO_ID` is wrong, you will see "Video unavailable". Check the video is public and embeds are allowed.

Limitation. You are embedding *someone's* video on *their* terms (YouTube branding, suggestions, ads may appear). For a fully controlled player you would host the video yourself — a topic for later.

F.9 Favicon basics

The idea. A **favicon** ("favourite icon") is the tiny square icon shown on the browser tab and in bookmarks. It is your brand's smallest, most-seen logo.

Why it exists. A tab with a real icon looks professional and is easy to find among ten open tabs; a blank page icon looks unfinished.

The file. A small square image, ideal size **512×512** (browsers scale it down). PNG or ICO both work; `.ico` is the traditional format. For best results, keep it simple — a logo mark, not tiny text.

Linking it (in the `<head>` of every page):

```
<link rel="icon" type="image/png" href="images/favicon.png">
```

Line by line:

- `rel="icon"` — tells the browser this is the tab icon.
- `type="image/png"` — the file type.
- `href="images/favicon.png"` — the relative path to the icon.

Expected browser result: after a refresh, the Adaeze Fashion "A" mark appears on the browser tab.

Misconception. Learners expect the icon to update instantly. Browsers **cache** favicons aggressively; if it does not change, close the tab and reopen, or hard-refresh with `Ctrl+F5`. This is normal, not a code error.

Limitation. The favicon does not change the page itself, only the tab. And a busy, detailed logo will look like a blur at 16×16 pixels — keep it simple.

F.10 Accessibility — the professional non-negotiable

The idea. Accessibility (often shortened to "a11y") means building so that *everyone* can use your site — including people using a keyboard instead of a mouse, people with low vision, and people using a screen reader.

Why it exists. Roughly, disabled people are a large part of every market, and accessible sites are also easier for *everyone*. More importantly, it is the right thing to do: a business website that only works for sighted mouse users excludes real customers.

Three touches we apply today:

1. Keep a visible focus outline. When a keyboard user presses `Tab`, the focused element must be clearly visible. Never do this alone:

```
/* BAD — removes the only sign of keyboard focus */
a:focus { outline: none; }
```

If you dislike the default ring, **replace** it with something at least as clear:

```
a:focus-visible {
  outline: 3px solid var(--color-gold);
  outline-offset: 2px;
}
```

`:focus-visible` shows the ring for keyboard users but not when a mouse user clicks — the best of both worlds.

2. Write meaningful `alt` text. `alt` is the text that describes an image for screen-reader users (and shows if the image fails to load).

```
<!-- BAD: no description -->
<img src="images/product-1.jpg" alt="">

<!-- BAD: "image" tells the user nothing -->
<img src="images/product-1.jpg" alt="image">

<!-- GOOD: describes the content -->
<img src="images/product-1.jpg" alt="Ankara gown in gold and brown, full length">

<!-- GOOD: decorative image, intentionally empty -->
<img src="images/divider-line.png" alt="">
```

Rule of thumb: describe the *content and function* of the image. If an image is purely decorative, an **empty** `alt=""` is correct (it tells the screen reader to skip it).

3. Ensure sufficient colour contrast. Text must stand out from its background. The widely used guideline is a contrast ratio of at least **4.5:1** for normal body text and **3:1** for large text. Light grey text on white, or gold text on cream, often fails.

- Good: charcoal `#333` on cream `#f5f0eb`.
- Risky: light gold `#d4a574` on cream — use dark brown text instead for body copy.
- The Chrome DevTools colour picker shows the contrast ratio and a tick or cross; we explore this properly in Session 11.

Expected result after applying all three: you can Tab through the whole page and always see where you are; a screen reader announces each image sensibly; body text is easy to read.

Misconception. "Accessibility is extra work for a few people." In practice it is small habits that make the site better for everyone — clearer focus, better descriptions, readable text. It is part of doing the job well, not an add-on.

F.11 Summary of the session

- `:hover`, `:focus` and `:active` give feedback on user state — pure CSS.
- `transition` makes changes smooth; put it on the **base** rule, not on `:hover`.
- `transform` moves/scales/rotates without disturbing layout; pair with `overflow: hidden` for zooms.
- `@keyframes` + `animation` create motion that can run by itself — use sparingly.
- Show/hide with CSS only: the `:hover` reveal (mouse-only) or the checkbox hack (works on click) — but **real interactivity needs JavaScript, a later course**.
- Responsive images: `max-width: 100%; height: auto;`, plus `width` / `height`, `loading="lazy"` and (awareness) `srcset`.
- Embed a **Google Map** and a **YouTube video** by pasting their `<iframe>`s; make the video responsive with `aspect-ratio`.
- Add a **favicon** to every page.
- Accessibility: keep a visible focus outline, write meaningful `alt` text, ensure contrast ≥ 4.5:1.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- The finished Session 9 project open in VS Code, with `index.html`, `shop.html`, `contact.html` and `css/style.css`.
- Chrome open on the site; internet connected.
- A YouTube video URL and a Google Maps location ready to embed.
- `images/logo.png` present for the favicon.

G.2 Demonstration 1 — transitions and transforms (5 minutes)

Say: "Watch the difference between a snap and a glide."

1. In `style.css`, find `.button`. Add:

```
transition: background-color 0.3s ease, transform 0.2s ease;
```

2. Refresh and hover the hero button. **Expected result:** the colour change glides and the button lifts 2px.
3. **Inject an error on purpose:** move the `transition` line *inside* `.button:hover`, save, refresh. Now the button still jumps in but snaps out.
    - **Say:** "See how the exit snaps? The transition must live on the base rule so both directions animate." Move it back.
4. Add the image zoom:

```
.product { overflow: hidden; }
.product img { transition: transform 0.4s ease; }
.product:hover img { transform: scale(1.06); }
```

- Refresh, hover a product card. **Expected result:** the photo zooms within the rounded card; nothing spills out.

5. **Inject an error:** remove `overflow: hidden`. The zoomed image spills over the card edges. **Say:** "This is why we clip." Restore it.

Questions: "Which two properties are we animating?" (background-color, transform.) "Why is `transform` cheaper than changing `width`?" (It does not force the browser to re-lay-out the page.)

G.3 Demonstration 2 — animation and the checkbox reveal (5 minutes)

Say: "A transition needs a trigger. An animation can run on its own."

1. Add the `@keyframes fadeIn` block and apply it to `.hero`. Refresh — the hero fades in once.
2. Then build the checkbox reveal in `about.html`:
   - Paste the hidden checkbox, label and `.mobile-nav` markup from F.5.
   - Paste the CSS, adding `#menu-toggle:checked ~ .mobile-nav { display: block; }`.
3. Refresh, click "Menu". **Expected result:** the list appears and disappears on click.
4. **Ask the class the honest question:** "Will this close automatically when I click a link? Can it tell a screen reader 'expanded'?" (No.) **Say:** "That is why real menus use JavaScript. Use this trick to understand state, not to ship a menu."

Questions: "Does CSS remember a click?" (No — the checkbox holds the state.) "Which selector links the checked box to the nav?" (`#menu-toggle:checked ~ .mobile-nav`.)

G.4 Demonstration 3 — media: video, map, favicon, accessibility (5 minutes)

Say: "Now real media from the internet."

1. On `shop.html`, paste the YouTube `.video-container` markup, replacing `VIDEO_ID` with a real one, and add the CSS with `aspect-ratio: 16 / 9`. Refresh. **Expected result:** a widescreen, rounded video.
2. Resize the window (or use `Ctrl+Shift+M`) to phone width. **Expected result:** the video shrinks, keeping its shape. Point out `aspect-ratio`.
3. On `contact.html`, paste the Google Maps `<iframe>` and add the `.map-container` CSS. Refresh.
   - **If offline:** the map area shows a loading/blank message. **Say:** "This confirms the map comes from Google over the internet — not from our code."
4. In `index.html` `<head>`, add the favicon `<link>`. Hard-refresh (`Ctrl+F5`). Point to the tab icon.
   - If it does not appear, close and reopen the tab — explain favicon caching.
5. Accessibility: `Tab` through the page and point out the focus ring; deliberately set `a:focus { outline: none; }`, Tab again, and let the class feel how lost they are; then replace it with `:focus-visible { outline: 3px solid var(--color-gold); }`.

Final question: "Which of today's features would break without internet?" (The map and the video.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding, learners add transitions, a hover reveal (or checkbox reveal), a responsive image, an embedded map, a responsive video and a favicon to the Adaeze Fashion site.

Starting materials: The learner's Session 9 project; internet connected; `images/logo.png` available.

Procedure (instructor calls each step; learners follow; instructor walks the room):

1. **Transitions.** In `style.css`, add a `transition` to `.button` and to `nav a`. Save, refresh, hover. **Pause and predict:** "Will the *exit* be smooth too?" (Yes — it is on the base rule.)
2. **Card zoom.** Add `.product { overflow: hidden; }`, the image transition and `.product:hover img { transform: scale(1.06); }`. Hover a card.
3. **Focus state.** Add `.contact-form input:focus` and `a:focus-visible` styles. Tab through the page and click into a field.
4. **Hover reveal.** On `about.html`, wrap a paragraph in `.delivery-info` and add the hover CSS from F.5. Hover to reveal. **Pause and predict:** "Will this work on a phone?" (No — no hover on touch.)

5. **Responsive images.** Ensure `img { max-width: 100%; height: auto; }` exists; add `loading="lazy"` and `width` / `height` to the product images. Shrink the window.
6. **Video.** On `shop.html`, paste the YouTube embed wrapped in `.video-container`, add the `aspect-ratio` CSS, and test at phone width.
7. **Map.** On `contact.html`, embed a Google Map (search "24 Ebony Road, Port Harcourt") in `.map-container`. Refresh.
8. **Favicon.** Add the favicon `<link>` to all four pages; hard-refresh; find the tab icon.
9. **Accessibility pass.** Check each product image has meaningful `alt`; Tab through and confirm a visible focus ring.

Expected result: the site now glides on hover, zooms product photos, reveals a panel, shows a responsive video and a Port Harcourt map, carries a tab icon, and is keyboard-navigable.

Points to pause on and predict:

- After step 2: "What happens to the card's neighbours when the image scales?" (Nothing — transform does not affect layout.)
- After step 6: "If the browser window is very narrow, what shape is the video?" (Still 16:9 — `aspect-ratio` holds it.)

Troubleshooting while walking the room:

- Hover effect jumps back on exit → the `transition` is inside `:hover`; move it to the base rule (K.1).
- Video stretches or has black bars wrong → use `aspect-ratio: 16 / 9` on the iframe inside a wrapper (K.2).
- Map or video blank → no internet, wrong `src`, wrong `VIDEO_ID`, or the video is not embeddable (K.3).
- Favicon will not update → browser cache; hard-refresh or reopen the tab (K.4).
- Zoomed image spills out → missing `overflow: hidden` on the card (K.5).
- Tab key shows no focus ring → an earlier `outline: none` was left in; replace with `:focus-visible` (K.6).

Completion checklist (learner ticks each):

- ☐ Smooth transition on buttons and nav links
- ☐ Product image zooms on hover, clipped by the card
- ☐ Visible `:focus` state on inputs and links
- ☐ A show/hide reveal working (hover or checkbox)
- ☐ Images responsive with `loading="lazy"` and dimensions
- ☐ YouTube video embedded and responsive 16:9
- ☐ Google Map embedded on the contact page
- ☐ Favicon linked on all pages
- ☐ Every image has meaningful `alt` text
- ☐ Page is fully usable with the keyboard (Tab)

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Build a *new* one-page "product spotlight" for **"Ngozi's Place"**, a small Port Harcourt hair-and-beauty shop, that uses today's CSS-only interactivity and media skills — without copying the Adaeze Fashion code.

Scenario: Ngozi wants a single page that shows one featured product, a "Why choose us" panel that opens on hover, an embedded video, and a tab icon. She has no budget for a developer beyond what you can do with HTML and CSS.

Starting information (given):

- Brand colours: **deep plum** `#4a1942`, **rose** `#c96a8a`, **cream** `#fdf7f9`.
- One product: "Shea Butter Hair Cream — ₦2,500".
- A YouTube video URL (the instructor provides one to embed).
- A square `logo.png` for the favicon.

Requirements:

1. Create `ngozi/index.html` and `ngozi/css/style.css`; link them.
2. Style a header and a product card using a `:root` palette.
3. Add a **transition** and a **transform** so the product image zooms on hover (clip it with `overflow: hidden`).

4. Add a **"Why choose us" panel** that is hidden until you hover the card (the `:hover` reveal technique).
5. Add a `@keyframes` **fade-in** that plays on the page's hero once on load.
6. Embed a **responsive YouTube video** using `aspect-ratio: 16 / 9`.
7. Add a **favicon** link.
8. Add `alt` text to the image and a visible `:focus-visible` outline on links.

Time allowance: 15 minutes.

Constraints: no copying Adaeze's `style.css`; colours must be custom properties; the zoom must not spill outside the card.

Success criteria: the image zooms smoothly, the panel reveals on hover, the video is widescreen and responsive, the tab shows the logo, and you can Tab to the links with a visible outline.

Hints if stuck:

- Hint 1: Zoom recipe — `.card { overflow: hidden; }`, `.card img { transition: transform .4s ease; }`, `.card:hover img { transform: scale(1.05); }`.
- Hint 2: Reveal recipe — panel has `max-height: 0; overflow: hidden; transition: max-height .4s ease;` and `.card:hover .panel { max-height: 160px; }`.
- Hint 3: Video recipe — wrap the iframe in `.video-container` and set `.video-container iframe { width: 100%; aspect-ratio: 16 / 9; border: 0; }`.

Instructor answer key (do not show learners first):

```
:root {
  --plum:  #4a1942;
  --rose:  #c96a8a;
  --cream: #fdf7f9;
  --text:  #2b2b2b;
}
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: "Segoe UI", Arial, sans-serif; color: var(--text); background: var(--cream); }

header { display: flex; justify-content: space-between; align-items: center; background: var(--plum); padding: 16px 24px; }
.logo { color: var(--rose); font-weight: bold; text-decoration: none; }

@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
.hero { text-align: center; padding: 48px 24px; animation: fadeIn 1s ease-in; }
.hero h1 { color: var(--plum); }

.card {
  max-width: 360px; margin: 0 auto; background: #fff; border-radius: 12px;
  overflow: hidden; box-shadow: 0 4px 12px rgba(0,0,0,0.12); padding: 16px; text-align: center;
}
.card img { width: 100%; height: 260px; object-fit: cover; transition: transform 0.4s ease; }
.card:hover img { transform: scale(1.05); }

.panel { max-height: 0; overflow: hidden; transition: max-height 0.4s ease; color: #555; }
.card:hover .panel { max-height: 160px; margin-top: 12px; }

.price { color: var(--rose); font-weight: bold; font-size: 20px; }

.video-container { max-width: 720px; margin: 32px auto; }
.video-container iframe { width: 100%; aspect-ratio: 16 / 9; border: 0; border-radius: 10px; }

a:focus-visible { outline: 3px solid var(--rose); outline-offset: 2px; }
```

```
<!-- head of ngozi/index.html -->
<link rel="stylesheet" href="css/style.css">
<link rel="icon" type="image/png" href="images/logo.png">
```

Marking guidance (out of 10):

- Palette via `:root`, consistent (2)
- Transition + transform zoom, clipped by `overflow: hidden` (3)
- Hover reveal panel (2)
- Responsive video with `aspect-ratio` (2)

- Favicon + `alt` text + `:focus-visible` (1)

J. REAL-WORLD CASE STUDY

Situation. *Tunde* owns a small event-decoration business in Port Harcourt, *Tunde Decor*. He has a one-page website his cousin made. The hero has a constantly spinning logo, every image has `alt="image"`, and hovering does nothing. When a customer uses a keyboard (because their mouse broke), they cannot tell where they are on the page. Another customer, who uses a screen reader, hears "image, image, image" and gives up.

Relevant facts.

- The spinning logo runs forever and distracts; it even made one visitor dizzy.
- The blank/meaningless alt text hides all the content from screen-reader users.
- The form fields have had their focus outline removed, so keyboard users are lost.
- He has not embedded a map, so customers keep phoning to ask for directions.

Decision point. What are the highest-value fixes he can make with CSS and HTML only?

Possible approaches.

1. Rebuild the whole site from scratch (expensive, unnecessary).
2. Make targeted fixes: one tasteful hover transition, meaningful `alt` text, a visible `:focus-visible` outline, a Google Map embed, and remove the infinite spinning (or respect `prefers-reduced-motion`).
3. Add lots of JavaScript effects (out of scope and risky for a beginner).

Recommended solution (for this stage). Approach 2. Almost all of it is pure CSS/HTML and directly uses today's skills: transitions for feedback, `alt` text for accessibility, `:focus-visible` for keyboard users, `@media (prefers-reduced-motion: reduce)` to calm the animation, and a map embed so customers can find him.

Trade-offs.

- Approach 1 wastes money and may repeat the same accessibility mistakes.
- Approach 2 is fast, cheap and visibly improves trust and usability.
- Approach 3 adds complexity and can break accessibility further if done poorly.

Professional connection. Real front-end work is often *fixing and polishing*, not building from zero. A designer who notices missing `alt` text, removed focus rings and endless animations — and fixes them with small, careful edits — is far more valuable than one who only adds flashy effects. This is exactly the judgement today's lesson builds.

K. COMMON MISTAKES AND TROUBLESHOOTING

K.1 Hover effect animates in but snaps back out

Symptoms: The colour/movement glides on hover but jumps instantly on mouse-out. **Likely cause:** The `transition` was written inside the `:hover` rule. **Corrective steps:** Move the `transition` to the **base** selector. **Prevention:** Always put the transition on the element's normal rule. **Confirm resolved:** Both entering and leaving are smooth.

K.2 Video looks stretched or squashed

Symptoms: The video is distorted or the black bars are wrong. **Likely cause:** A fixed `height` fighting the width, or no aspect ratio set. **Corrective steps:** Remove fixed heights; set `width: 100%; aspect-ratio: 16 / 9;` on the iframe inside a wrapper. **Prevention:** Let the wrapper control width and `aspect-ratio` control shape. **Confirm resolved:** The video keeps a movie shape at every window size.

K.3 Map or video area is blank

Symptoms: An empty grey box where the map/video should be. **Likely causes:** (a) No internet. (b) Wrong `src` / wrong `VIDEO_ID`. (c) The video is private or embeds are disabled. **How to investigate:** Open the iframe `src` in a new browser tab. If it loads there, the embed code is fine and the issue is elsewhere; if it errors, the link is wrong or the media is restricted. **Corrective**

steps: Re-copy the embed code; ensure the video is public; restore internet. **Prevention:** Test embeds immediately after pasting. **Confirm resolved:** The map shows a pin / the video shows a play button.

K.4 Favicon will not change

Symptoms: The old or no tab icon appears. **Likely cause:** The browser cached the favicon. **Corrective steps:** Hard-refresh (`Ctrl+F5`) or close and reopen the tab; confirm the `href` path and file exist. **Prevention:** Keep `favicon.png` inside `images/` and reference it correctly. **Confirm resolved:** The brand mark appears on the tab.

K.5 The zoomed image spills outside the card

Symptoms: On hover, the enlarged image overflows its rounded card. **Likely cause:** The card has no `overflow: hidden;`. **Corrective steps:** Add `overflow: hidden;` to the card. **Confirm resolved:** The zoom stays inside the rounded card.

K.6 No focus outline when tabbing

Symptoms: Pressing `Tab` shows no visible highlight. **Likely cause:** A rule like `a:focus { outline: none; }` (perhaps copied from the internet) removed it. **Corrective steps:** Remove it, or replace with `a:focus-visible { outline: 3px solid var(--color-gold); outline-offset: 2px; }`. **Prevention:** Never remove a focus outline without providing a visible replacement. **Confirm resolved:** You can see where focus is at every Tab press.

K.7 Screen reader cannot tell what an image is

Symptoms: A screen reader reads "image" or reads nothing useful. **Likely cause:** Missing or meaningless `alt` text. **Corrective steps:** Write a short description of the image's content/function; use `alt=""` only for purely decorative images. **Confirm resolved:** The description makes sense when read aloud on its own.

Distinguish likely vs confirmed: Confirm before fixing — open the embed's `src` in a tab, check the `overflow` value in DevTools, look for the removed outline rule. Guessing wastes time.

L. SESSION ASSIGNMENT

Assignment title: "Bring the Adaeze Fashion Site to Life"

Purpose: Practise CSS-only interactivity and media embedding on the running project — directly reusable in the capstone.

Realistic scenario: Adaeze says the site now looks good but feels flat, and customers keep calling for directions. She asks you to add gentle hover effects, embed a short collection video and a map, and make sure the site still works for everyone.

Detailed instructions:

1. In `css/style.css`, add smooth `transition`s to `.button` and `nav a`.
2. Add a hover effect to the images inside product cards using `transform` (with `overflow: hidden` on the card).
3. Add a `@keyframes` fade-in to the hero on the Home page, and include a `@media (prefers-reduced-motion: reduce)` block that calms animations.
4. Add a show/hide reveal on the About page (hover reveal **or** the checkbox hack). Write a one-line comment stating which technique you used and its limitation.
5. Ensure all images are responsive (`max-width: 100%`), have `width` / `height`, and use `loading="lazy"` and meaningful `alt` text.
6. On `shop.html`, embed a YouTube video responsively with `aspect-ratio: 16 / 9`.
7. On `contact.html`, embed a Google Map of a real Port Harcourt location.
8. Add a favicon `<link>` to all four pages.
9. Add a visible `:focus-visible` outline to links and form fields.

Required deliverables:

- Updated `css/style.css`.
- Updated `index.html`, `about.html`, `shop.html`, `contact.html`.
- A short note in your workbook listing: the video you embedded, the place you mapped, which show/hide technique you chose, and how you checked keyboard focus.

Submission format: Files in the class project folder / shared drive; note in the workbook.

Deadline: Before Session 11.

Marking rubric (25 marks):

Criterion	5 — Excellent	3 — Good	1 — Needs work
Transitions & hovers	Smooth, purposeful, on base rules	Present, minor snapping	Missing or snapping
Transform / animation	Clean zoom + keyframe, clip correct	Present but rough	Missing
Show/hide technique	Works and limitation explained	Works, no explanation	Not working
Media embeds	Video responsive + map accurate	One embed or minor issues	Missing/broken
Accessibility	`alt`, focus, lazy, contrast all handled	Two of the three	Largely ignored
Total	**25**		

Common mistakes to avoid: transition inside `:hover`; missing `overflow: hidden`; fixed video height; blank `alt`; removed focus outlines.

Instructor reference solution: the transitions, zoom, keyframe, responsive-image, video, map, favicon and focus code shown in Sections F.6–F.10, plus the `prefers-reduced-motion` block from F.4.

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Short answer. In one sentence each, when does `:hover`, `:focus` and `:active` apply?

Q2. Multiple choice. Where should the `transition` property be written so a hover effect is smooth *both* when entering and leaving? a) inside the `:hover` rule b) on the base element rule c) inside a `@keyframes` block d) in the `:root` block

Q3. Explain in your own words. Why does `transform: scale(1.05)` on a hovered image not push the neighbouring cards around?

Q4. Fill the gap. Complete the rule that makes an embedded video keep a widescreen shape:

```
.video-container iframe { width: 100%; __________: 16 / 9; }
```

Q5. Error detection. Find and fix the two problems in this accessibility code:

```
a:focus { outline: none; }
```

```
<img src="images/gown.jpg" alt="image">
```

Q6. Scenario. A client wants a menu that opens when tapped and closes automatically when a link is tapped. Using CSS only (the checkbox hack), can you fully achieve this? Explain why or why not, and say what the correct tool is.

Q7. Practical identification. Name the attribute that (a) describes an image for a screen reader, (b) defers loading of an off-screen image, and (c) tells the browser a video keeps a 16:9 shape.

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. `:hover` applies while the mouse pointer is over the element; `:focus` applies when the element is selected (clicked into or reached with the Tab key); `:active` applies while the element is being pressed down. **A2.** (b) on the base element rule — so both the enter and exit animate. **A3.** Because `transform` is a purely visual change; unlike changing `width` or `margin`, it does not affect the document layout, so neighbours stay put. **A4.** `aspect-ratio: 16 / 9;` **A5.** Problem 1: `outline: none` removes the keyboard focus indicator — replace with something visible, e.g. `a:focus-visible { outline: 3px solid var(--color-gold); outline-offset: 2px; }`. Problem 2: `alt="image"` is meaningless — write a real description, e.g. `alt="Ankara gown in gold and brown"`. **A6.** No. The checkbox hack holds an on/off state, but CSS cannot detect that you clicked a *link* to close the menu, and it does not

tell screen readers the menu expanded/collapsed. Reliable open/close menus need **JavaScript** (taught in the Web Development course). **A7.** (a) `alt` ; (b) `loading="lazy"` ; (c) `aspect-ratio` .

N. REFLECTION AND RECAP

Plain-English summary. Today you gave the site a sense of touch. You used `:hover` , `:focus` and `:active` to give feedback, `transition` to make changes glide, `transform` to lift and zoom without disturbing the layout, and `@keyframes` for motion that runs by itself. You learned two CSS-only ways to show and hide content — and, importantly, their limits: real interactivity needs JavaScript, which is a later course. You made images responsive and lazy, embedded a real Google Map and a responsive YouTube video, added a favicon, and applied three accessibility essentials: a visible focus outline, meaningful `alt` text and sufficient contrast.

Five important takeaways:

1. `:hover` / `:focus` / `:active` give state-based feedback — pure CSS.
2. Put `transition` on the base rule, and prefer `transform` / `opacity` for smoothness.
3. Real interactivity (menus, validation, sliders) needs JavaScript; CSS tricks have limits.
4. Embeds are other people's pages shown in an `<iframe>` ; make the video responsive with `aspect-ratio` .
5. Never break accessibility: keep focus visible, describe images, keep contrast ≥ 4.5:1.

Questions to help you recognise what you can now do:

- Can I make a button lift smoothly on hover without JavaScript?
- Can I explain why the checkbox hack is not a real menu?
- Can I embed a map and a video that both behave on a phone?
- Can I Tab through my own site and always see where I am?

Connection to the next session: Your site is now designed, responsive and lively — but is it *correct*? Are all the links working? Do all the images load? Does it pass the validators? In Session 11 we become testers and detectives: we use Chrome DevTools to find and fix a broken page, validate our HTML and CSS with the W3C tools, and check accessibility with confidence.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner writes the `transition` on the base rule and can explain why.
- Learner uses `transform` for the zoom and adds `overflow: hidden` without being told.
- Learner can state, unprompted, that real interactivity requires JavaScript.
- Learner embeds the video with `aspect-ratio` and the map accurately, and tests both.
- Learner adds meaningful `alt` text rather than "image", and does not delete the focus outline.
- Learner tests with the keyboard (Tab) as well as the mouse.

Common misconceptions to watch for and correct:

- "CSS can make a proper menu / remember clicks." → Only via a control like a checkbox; real behaviour needs JavaScript.
- "Removing the outline makes it look cleaner." → It makes the site unusable for keyboard users; replace, never remove.
- " `alt` is optional / can be 'image'." → It carries the content for screen readers.
- "Hover works on phones." → There is no hover on touch; never hide essential content behind hover alone.
- "Transitions belong on `:hover` ." → They belong on the base rule.

Suggested remedial exercises for learners who struggle:

1. **Snap vs glide:** give one button with the transition misplaced and one correct; ask the learner to identify which is smooth on exit.
2. **Alt-text rewrite:** provide five images with poor `alt` values and ask for meaningful replacements (and one decorative `alt=""`).

3. **Embed fix-up:** hand a page whose video is stretched and whose map is blank; ask the learner to fix the aspect ratio and check the `src`.

Differentiation:

- **Level 1 (supported):** Provide the CSS blocks pre-written to paste, and guide the learner through the two embeds only.
- **Level 2 (standard):** The exercise and assignment as written.
- **Level 3 (challenge):** Add a CSS-only "back to top" using `scroll-behavior: smooth` and an anchor link; add a second `@keyframes` (slide-in) with a delay; test contrast ratios in DevTools and adjust the palette to pass 4.5:1.

Timing guidance: Embeds need internet and can be slow; if connectivity is poor, do the video first (smaller payload) and demonstrate the map only on the projector. Protect the transitions and accessibility steps — those carry the real learning. Compress the case study to a two-minute mention if time is short.

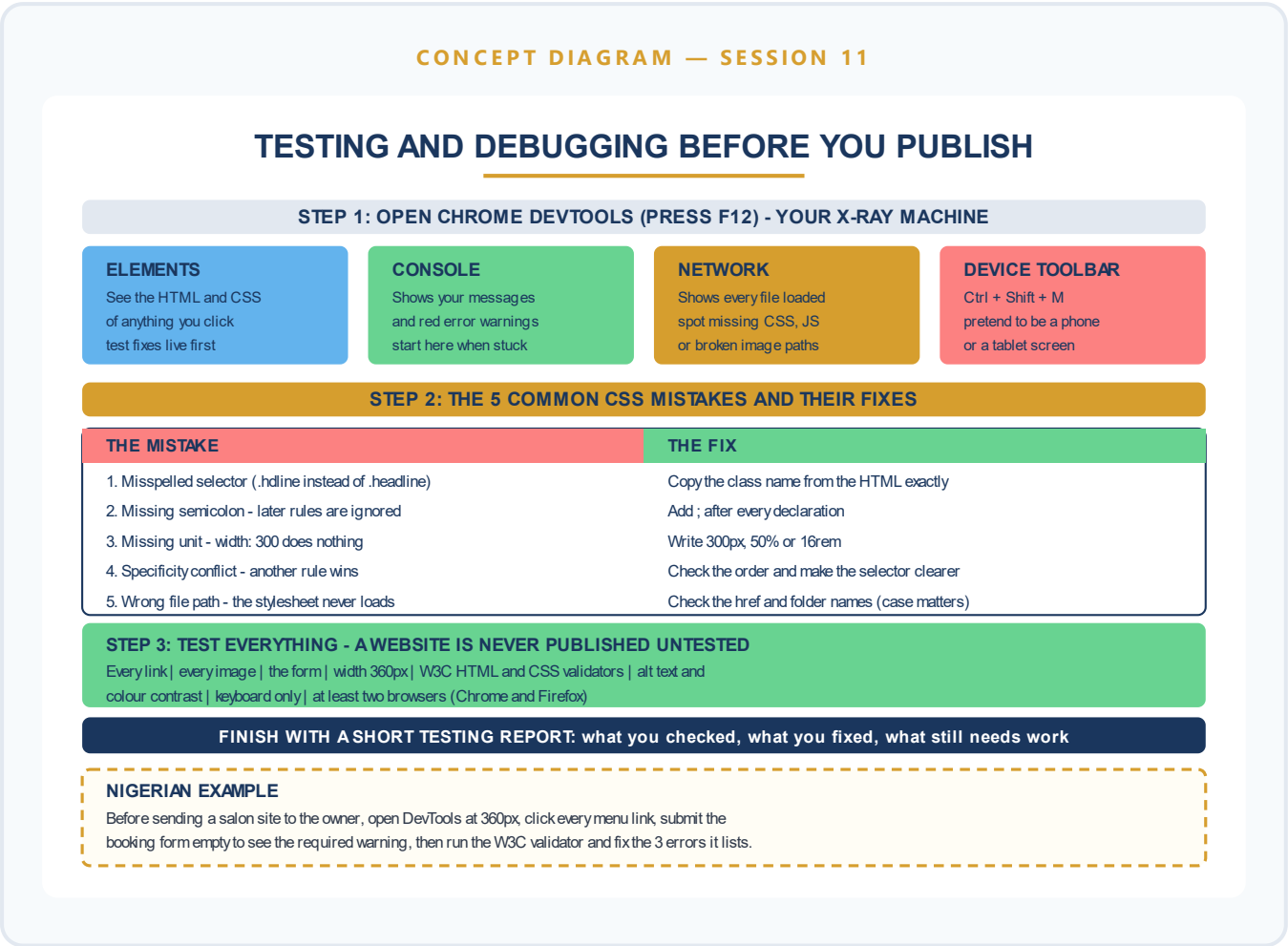
END OF SESSION 10

Next: Session 11 — Testing and Debugging.

SESSION 11 OF 12

TESTING AND DEBUGGING

CONCEPT DIAGRAM — SESSION 11



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 11 — TESTING AND DEBUGGING

A. SESSION IDENTIFICATION

Item	Detail
Session number	11 of 12
Session title	Testing and Debugging
Week / Module	Week 4 — Module: Publishing and Capstone
Recommended duration	105 minutes
Difficulty level	Beginner (builds on Sessions 1–10)

Item	Detail
Prerequisite knowledge	Sessions 1–10. Learners can build and style a responsive multi-page site and add CSS interactivity and media.
Connection to previous work	In Session 10 the Adaeze Fashion site became lively — but lively is not the same as *correct*. A beautiful site with a broken link, a missing image or an easy-to-miss CSS typo still fails the customer. This session teaches the professional habit of testing before publishing: using Chrome DevTools, reading and fixing HTML/CSS errors, running the W3C validators, and checking accessibility and cross-browser basics.
Preparation before class	Academy PCs on; VS Code and Chrome; each learner's Session 10 project open; projector; a printed "Testing Checklist" and "Common Errors Reference" per learner; **internet required** for the W3C validators; a deliberately broken practice page (files in Section F.10) prepared and shared so every learner can debug a real broken page alongside the instructor.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Describe what testing and debugging mean** and why a website must never be published untested.
2. **Open and navigate Chrome DevTools** — the Elements, Console and Network panels and the device toolbar.
3. **Inspect an element and test CSS changes live** in the Elements panel, then apply the confirmed fix to the source code.
4. **Diagnose five categories of CSS mistake** — a misspelled selector, a missing semicolon, a missing unit, a specificity conflict and a wrong file path.
5. **Check every link, image and page** on desktop and on a simulated mobile width.
6. **Check a contact form** — labels, required fields, focus and layout.
7. **Validate HTML and CSS** using the W3C validators and interpret the results.
8. **Run basic accessibility checks** — colour contrast, `alt` text and form labels.
9. **Perform basic cross-browser testing** in at least two browsers.
10. **Identify and fix a set of deliberate bugs** in a provided broken page, and produce a short testing report.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops). Sharing is fine — one drives DevTools, one reads the errors aloud, then swap.
- Instructor computer connected to a projector or large screen.

Software

- Visual Studio Code (2022 or later).
- Google Chrome (any recent version) with DevTools.
- Mozilla Firefox (optional but recommended for cross-browser testing).

Accounts

- None required.

Sample files

- The learner's `adaeze-fashion/` project from Session 10.
- The **broken practice page** prepared by the instructor: `bug-hunt/index.html` and `bug-hunt/css/style.css` (contents in Section F.10). Share it before the session.

Worksheets

- "Testing Checklist" (printed, 1 page — the same list appears in Section H).

- "Common Errors Reference" (printed, 1 page — based on Sections F.4, F.5 and K).

Tools

- Chrome DevTools (built in — press `F12`; `Ctrl+Shift+M` for the device toolbar).
- The W3C HTML validator (validator.w3.org) and CSS validator (jigsaw.w3.org/css-validator).

Internet

- **Required** for the W3C validators and for testing the map/video embeds from Session 10.

Alternative for learners who lack a facility

- Any laptop with Chrome and VS Code works. If there is no internet at all, the instructor pastes pages into the validators on the projector, and learners do the DevTools work offline (which is the bulk of the session).

D. OPENING ACTIVITY (6 minutes)

Instructor setup: On the projector, open the deliberately broken practice page (Section F.10). It should look clearly wrong: an unstyled block, a broken image icon, a link that goes nowhere, and a heading that is nearly invisible against its background. Do **not** reveal any code yet.

What the instructor says:

> *"Nobody built this page on purpose to be broken — but every one of these problems is the kind of thing real designers create by accident every single day: one missing letter, one missing semicolon, one wrong file name. On the surface it looks like a disaster. In truth there are only a handful of small mistakes, and we can find every one of them in minutes.*
>
> *Here is the secret: you do not fix bugs by staring and guessing. You fix them by investigating — using a tool that shows you exactly what the browser thinks your code says. That tool is Chrome DevTools, and by the end of today you will use it like a professional. Let us hunt these bugs together."*

Expected learner response: Learners will point at the broken image or the invisible heading and volunteer guesses. Accept guesses, then say: "Good instinct — but let us not guess. Let us *prove* it."

Why this opener works: It normalises mistakes (reducing shame and fear), makes the abstract idea of "debugging" concrete by showing a real broken page, and immediately promises a tool that makes the task feel achievable rather than overwhelming.

E. PREREQUISITE REVIEW (7 minutes)

Fast oral round-robin. Questions on the projector.

Question 1: "Where should every page's stylesheet link go, and why does a page sometimes lose *all* its styling?"

- Expected answer: in the `<head>`; a missing or wrong `href` breaks all styling.
- Common gap: learners check only one page. Remind them every page needs its own `<link>`.

Question 2: "What are the four layers of the box model?"

- Expected answer: content, padding, border, margin.
- Common gap: this matters today because *missing units* (e.g. `padding: 20;`) is a common bug that silently breaks the box model.

Question 3: "What does `:hover` do and where have we used it?"

- Expected answer: styles a state while the mouse is over the element; used on nav links and buttons.
- Common gap: confusing `:hover` with `:focus`; remind them `:focus` is for keyboard and clicking into fields.

Question 4: "Why do all pages share one stylesheet, and what is a *relative* path?"

- Expected answer: consistency and one-place updates; a path relative to the current file, using forward slashes.
- Common gap: using backslashes; today this is a category of bug we test for.

Question 5: "Name one accessibility practice we applied last session."

- Expected answer: visible focus outline, meaningful `alt` text, or sufficient contrast.
- Common gap: none serious; this is a warm-up for the accessibility checks in Section F.7.

Bridge to today (say this):

> *"Every one of those answers is something that can break — and today we learn to catch it before a customer does. There is a saying in our trade: 'If it works on your machine, you have tested nothing.' Today we test like the customer, on their screen and their browser, and we fix what we find."*

If more than a third of the class cannot answer Question 1, quickly re-demonstrate the `<link>` path on one page before continuing.

F. FULL LECTURE NOTES

F.1 What testing and debugging mean

The idea. Testing is checking that everything on the site works and looks right — links, images, pages, forms, layout and accessibility. **Debugging** is finding and fixing the errors that testing reveals.

Plain-English definition. Testing is the *check*; debugging is the *cure*. They go together: you test to discover problems, then debug to solve them.

Why they exist. A website is not "done" when it looks nice on your own screen. It is done when it works for a real customer on a different device, a different browser and a flaky internet connection. Untested sites break in public — which costs the business money and trust.

Analogy. Think of a **tailor before a wedding**. She does not hand over the aso-ebi because the fabric is beautiful; she checks the seams, the hem, the fit — and fixes what is off *before* the customer wears it in public. Testing is the fitting; debugging is the alteration.

What testing covers (our checklist today):

- Do all links go to the right place?
- Do all images load?
- Does every page work, on desktop and on a phone width?
- Does the form display and behave, with labels and required fields?
- Is the page reasonably fast to load?
- Does the code validate?
- Is it accessible — contrast, alt text, labels?
- Does it behave in more than one browser?

Misconception. "If it works, it is fine." *Works on my machine* is not the standard. The standard is *works for the customer on their device*. This is why we simulate other devices and browsers deliberately.

Important limitation. No amount of testing proves a site is perfect; it reduces surprises. Even large companies ship bugs. The professional habit is: **test every time you change anything**, not once at the end.

F.2 Chrome DevTools — your investigation toolkit

The idea. DevTools is a set of panels built into Chrome that shows you the *live* truth of what the browser loaded and how it interpreted your code.

Why it exists. You cannot see the DOM, the computed styles, the loaded files or the errors just by looking at a rendered page. DevTools exposes all of it.

How to open it: press `F12`, or right-click any part of the page and choose **Inspect**, or press `Ctrl + Shift + I`. To open directly on the Console: `Ctrl + Shift + J`.

The four panels we use today:

Panel	What it shows	Why we use it
Elements	The live DOM (HTML) and every CSS rule that applies to the selected element	Inspect structure; test CSS fixes live
Console	Errors (in red), warnings (yellow) and any output	Find broken paths, JavaScript errors, missing files
Network	Every file the page loaded, its size and status	Find 404s (missing files) and heavy images
Device toolbar	A resizable viewport simulating phones/tablets	Test responsive layout quickly

Two extra aids worth knowing:

- **Colour-picker contrast checker:** in Elements, click any colour swatch to see the contrast ratio and whether it passes. Used in Section F.7.
- **Lighthouse / Performance:** a built-in audit for speed, accessibility and more. We only *mention* it today; deeper use is beyond this course.

Analogy. DevTools is the **mechanic's diagnostic computer** plugged into the car. The car (page) may look fine parked outside, but the computer shows exactly which sensor is failing. It does not fix the car for you — it tells you where to look.

Misconception. "DevTools changes are permanent." They are **temporary**: edits made in the panel last only until you refresh. That is a *feature* — you can experiment freely, then copy the confirmed fix back into your real file. **The panel is the experiment bench; VS Code is the workshop.**

F.3 Inspecting and testing CSS live in the Elements panel

The idea. Click an element on the page, and DevTools shows its HTML and the full list of CSS rules affecting it, with the winning value highlighted.

The steps:

1. Press `F12` to open DevTools.
2. Click the **Elements** tab.
3. Click the small **arrow icon** (top-left of the panel), then click any element on the page — or right-click the element and choose **Inspect**. The element's HTML highlights in the DOM tree.
4. Below the HTML, the **Styles** pane lists every CSS rule that applies, in order, with the **winning declaration** shown at the top.
5. Tick/untick a checkbox to enable/disable a rule; click a value and type a new one to try a fix instantly.
6. When you find the fix, **write it into your real** `style.css` and save.

Reading the Styles pane (this is where debugging happens):

- **Struck-through** declarations were **overridden** by something more specific. That is the visual sign of a specificity conflict.
- A rule shown as **greyed/with a warning** is invalid (e.g., a missing unit) and was ignored.
- The **Computed** tab shows the final values the browser actually used (e.g., the real pixel height).

Worked example: a heading looks blue but you expected brown.

1. Right-click the heading → **Inspect**.
2. In **Styles**, find a struck-through `color: #2c1810;` and an active `color: blue;` above it.
3. That active rule is more specific (or comes later). Untick it — the heading turns brown. **Confirmed.**
4. Fix the losing rule in your source (make the selector more specific, or remove the conflicting rule).

Expected result: you can point at any element and see exactly which rule decides its look — no guessing.

Misconception. Learners think the *first* rule in the file always wins. Cascading works by **specificity and source order**, not by which rule you *intended*. DevTools settles the argument by showing the truth.

Limitation. DevTools edits do not save. If you refresh, you lose them — always transfer the fix to VS Code.

F.4 Reading HTML errors

HTML is forgiving: browsers often "fix" small mistakes silently and show *something*. That is dangerous because a mistake can hide for a long time. Here are the ones to look for.

1. Unclosed tags.

```
<!-- Wrong -->
<p>Welcome to Adaeze Fashion.
<!-- Right -->
<p>Welcome to Adaeze Fashion.</p>
```

A missing `</p>` can swallow the next elements into the paragraph, changing the layout.

2. Incorrect nesting (tags crossed like scissors).

```
<!-- Wrong -->
<p>We <strong>love fashion</p></strong>
<!-- Right -->
<p>We <strong>love fashion</strong></p>
```

Open and close in the correct order — like brackets: last opened, first closed.

3. Missing `alt` on images.

```
<!-- Wrong -->
<img src="images/workshop.jpg">
<!-- Right -->
<img src="images/workshop.jpg" alt="Adaeze's tailoring workshop in Port Harcourt">
```

`alt` carries the image's meaning for screen readers and shows if the image fails.

4. Wrong file path.

```
<!-- Wrong: points to a file that does not exist -->
<img src="images/photo.png">
<!-- Right: matches the real file name -->
<img src="images/photo.jpg">
```

A file-name mismatch is one of the most common bugs; the image shows a broken icon.

5. Missing `<!DOCTYPE html>`.

```
<!-- Right — always at the very top -->
<!DOCTYPE html>
<html lang="en">
```

Without it, browsers may switch to "quirks mode" and render oddly.

6. Missing essential `<head>` lines.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion | About</title>
</head>
```

No charset → `₦` and apostrophes can display wrongly. No viewport meta → the site will not scale properly on phones.

How to catch HTML errors quickly. The Console shows some (e.g., failed resources); the W3C validator (F.6) catches the rest, including unclosed tags and wrong nesting. Also inspect in DevTools: the DOM tree shows how the browser *actually* interpreted your markup, which often reveals a mis-nested tag.

Misconception. "The page shows, so the HTML is fine." Not necessarily. Browsers paper over errors; validators and the DOM tree reveal them.

F.5 Reading CSS mistakes

A single wrong character can silently disable a whole rule. Learn to recognise these five.

1. Misspelled selector (a typo that targets nothing).

```
/* HTML has class="product" ... */
.prodcut { width: 300px; }   /* Wrong: matches nothing */
.product { width: 300px; }   /* Right */
```

The rule looks fine but never fires. In DevTools the element simply has no styles from it. **Confirm** by comparing the class in the HTML with the selector character by character.

2. Missing semicolon.

```
/* Wrong */
.product {
  color: #2c1810
  padding: 16px;
}
/* Right */
.product {
  color: #2c1810;
  padding: 16px;
}
```

The missing semicolon often causes the *next* declaration to be ignored too.

3. Missing unit.

```
/* Wrong — the browser cannot guess "20" of what */
padding: 20;
/* Right */
padding: 20px;
```

Lengths need a unit (`px` , `%` , `rem` , `em` , `vw`). The unitless `0` is the only exception (`margin: 0;` is fine).

4. Specificity conflict.

```
p        { color: #333333; }   /* any paragraph */
#about p { color: #cccccc; }   /* MORE specific — this one wins */
```

The second rule is more specific (an id beats a tag), so the text is light grey, not charcoal — even if you *wanted* it charcoal. In DevTools the losing rule is **struck-through**. Fix it by making the winning rule what you want, or by increasing the specificity of the rule you intend to win (usually by adding a class), not by piling on `!important` .

5. Wrong file path (for the stylesheet or a background image).

```
<!-- Wrong -->
<link rel="stylesheet" href="css/styles.css">
<!-- Right -->
<link rel="stylesheet" href="css/style.css">
```

```
/* Wrong: background image not found */
.hero { background-image: url(../images/hero.jpeg); }
/* Right */
.hero { background-image: url(../images/hero.jpg); }
```

A wrong stylesheet path = no styling at all; a wrong background path = a silently missing background, visible in the Console as a 404.

How DevTools helps with all five:

- **Elements → Styles:** struck-through = overridden (specificity); greyed/invalid = typo/unit; missing rule = selector typo.
- **Console:** red `net::ERR_FILE_NOT_FOUND` = wrong path.
- **Network:** a `404` status next to a file = wrong path.

Misconception. "If the CSS is in the file, it applies." No — invalid declarations are ignored, and selectors that match nothing do nothing. Only the *computed* styles matter, and DevTools shows them.

F.6 The W3C validators

The idea. The **W3C** (World Wide Web Consortium) publishes free tools that check your code against the official standards and report every error and warning.

Why they exist. Browsers are lenient; the validators are strict. Strict checking catches subtle problems (unclosed tags, duplicate ids, invalid attribute values) before they bite across different browsers.

HTML validator — validator.w3.org:

1. Open the site.
2. Choose the **"Validate by Direct Input"** tab.
3. Paste your full HTML (from `<!DOCTYPE html>` to `</html>`).
4. Click **Check**.
5. Read the results: it lists each error with a line number and a short explanation, then warnings.

CSS validator — jigsaw.w3.org/css-validator:

1. Open the site, choose **"By direct input"**.
2. Paste your CSS.
3. Click **Check** and read the report.

How to use the results like a professional:

- Fix errors from the **top down** and **re-validate** after each fix — one error can cause several later ones to disappear.
- Read what each error *means*; do not just hunt for the line. The message tells you the rule.
- Advance from errors to warnings, but a "clean" pass means **no errors** (warnings are advisory).
- A missing `alt` might appear as an error or a warning depending on the checker — treat it as something to fix regardless.

Expected result: after fixing, the HTML validator shows *"Document checking completed. No errors or warnings to show."* and the CSS validator shows *"Congratulations! No Error Found."*

Misconception. "Validation is optional polish." Most professionals treat valid, standards-compliant code as a baseline — it is more predictable across browsers and easier for the next person to maintain.

Limitation. Validators check *standards compliance*, not *design quality* or whether the content is correct. A valid page can still be ugly or have the wrong prices. Validation is necessary, not sufficient.

F.7 Accessibility checks

The idea. Check that the site is usable by people with different needs — low vision, colour blindness, keyboard-only use, or a screen reader.

Why. It is both ethical and practical: accessible sites reach more customers and are easier for everyone.

Three checks we run today:

1. Colour contrast. Text must stand out from its background — at least **4.5:1** for normal text, **3:1** for large text.

- In DevTools, inspect a text element, click its colour swatch, and read the contrast ratio.
- Example of a failure: light grey `#cfcfcf` on cream `#fdf7f9` — far too faint. Fix by darkening the text.
- Failures are common with gold-on-cream and grey-on-white palettes.

2. `alt` text. Every meaningful image needs a description; decorative images use `alt=""`.

- Quick audit: in the Elements panel, click through the images, or in VS Code search the HTML for `alt=`. Every `<img>` must have one.
- A screen reader reads the `alt`; ask yourself "does this make sense read aloud on its own?"

3. Form labels. Every input needs a `<label>` connected by `for` / `id`, so a screen reader (and a tap on a small screen) knows what the field is.

```
<label for="email">Email Address</label>
<input type="email" id="email" name="email" required>
```

The `for="email"` must match the input's `id="email"`. If it does not match, the label is not connected — a common, quiet bug.

Bonus check — keyboard only: press `Tab` repeatedly. You should be able to reach every link, button and field, and always *see* where you are (the `:focus-visible` outline from Session 10).

Expected result: every image described, every field labelled and focusable, and all body text passing 4.5:1.

Misconception. "Accessibility is only for a few users." Poor contrast, tiny targets and missing labels frustrate *everyone* on a bright phone screen or a rushed day.

F.8 Cross-browser basics

The idea. Different browsers occasionally render the same code slightly differently. Test in at least two so you are not surprised when a customer opens your site in Firefox or Edge.

Why. Chrome is the most-used browser, but not the only one. A site that only works in Chrome fails the customers using something else.

What to do:

1. Test the site in **Chrome** (primary), then in **Firefox** and/or **Edge** (on Windows, Edge is built in).
2. Look for: missing styles, shifted layout, fonts looking different, broken spacing, or an embed that will not load.
3. Note any difference and decide whether it matters.

Typical difference to watch for (from our lessons):

- `aspect-ratio` is well supported in current browsers but was not in much older ones — this is exactly why Session 10 also taught the `padding-bottom` fallback.
- Some CSS features need vendor prefixes (like `-webkit-`) in older browsers; for the simple CSS on this course, modern Chrome/Firefox/Edge agree closely.

Expected result: the site looks essentially the same in Chrome and at least one other browser.

Misconception. "If it works in Chrome, it works everywhere." Not guaranteed. Testing is how you find out. If you only have Chrome, at least use the DevTools device toolbar (which simulates different *devices*, though not other browsers' engines).

Limitation. You cannot test every browser and old version by hand. Real teams use automated services; for this course, two modern browsers is a professional and realistic standard.

F.9 Loading speed basics

The idea. A page that loads slowly loses visitors. You do not need to be an expert; you need a few basic checks.

Why. Many customers in Port Harcourt browse on mobile data. Heavy pages are expensive for them and slow to open, so they leave.

The basic checks:

1. **Image file sizes.** Open DevTools → **Network** tab, refresh, and look at the sizes. A single photo above roughly **300–500 KB** is usually too heavy for a gallery image. Fix by resizing images to the size you actually display, and compressing them.
2. **File formats.** Use **JPG** for photos, **PNG** for graphics needing transparency, and modern formats like WebP when available.
3. **Lazy loading.** Off-screen images should use `loading="lazy"` (added in Session 10).
4. **Fewer, smaller files** load faster than many large ones. Remove unused images and duplicate stylesheets.

Expected result: image sizes in the Network tab are sensible; the page opens in a second or two on a normal connection.

Misconception. "A bigger, higher-resolution image always looks better." On a web page the opposite is often true: an images resized to the display size looks identical but loads far faster.

Limitation. This is a taste of performance work, not a full performance course. Tools like Lighthouse give deeper reports; we only point the way today.

F.10 The broken page — debugging a real problem

The instructor shares a broken practice page. It contains a deliberate set of bugs spanning every category above. Learners debug it *with* the instructor, using DevTools and the validators.

F.10.1 The broken files

`bug-hunt/index.html` :

```
<html>
<head>
  <title>Adaeze Fashion - Visit Us</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <a href="index.html" class="logo">Adaeze Fashion</a>
    <nav>
      <ul>
        <li><a href="index.html">Home</a>
        <li><a href="shop.html">Shop</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contct.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main id="about">
    <h1>Visit Adaeze Fashion</h1>
    <p class="intro">We are a bespoke tailoring studio in Port Harcourt.

    <img src="images/workshop.png">

    <div class="product">
      <h3>Opening hours</h3>
      <p>Monday to Saturday, 9am to 6pm.</p>
      <p>We <strong>look forward to seeing you</p></strong>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Adaeze Fashion</p>
  </footer>
</body>
```

`bug-hunt/css/style.css` :

```
body
  font-family: "Segoe UI", Arial, sans-serif;
  color: #333
}

.product {
  width: 100px
  padding: 20;
  border-radius: 8px;
}

.prodcut h3 {
  color: #2c1810;
}

.intro {
  color: #cfcfcf;
}

nav a {
  color: #ffffff;
  text-decoration: none
}
```

F.10.2 The deliberate bugs (instructor reference — do not reveal at once)

#	File	Bug	Symptom
1	HTML	Missing `<!DOCTYPE html>`	May render in quirks mode
2	HTML	Missing `<meta charset="UTF-8">` and viewport meta	`₦` /apostrophes may look wrong; not mobile-scaled
3	HTML	`<a href="contct.html">` typo	Contact link goes nowhere (file not found)
4	HTML	First `<li>` never closed	Browser auto-fixes, but invalid
5	HTML	`<p class="intro">` not closed	The image and next block get pulled in
6	HTML	`<img>` missing `alt`	No description for screen readers
7	HTML	Image path `workshop.png` (file is `workshop.jpg`)	Broken image icon
8	HTML	Crossed tags: `<strong>...</p></strong>`	Invalid nesting
9	CSS	Missing `{` after `body`	The `body` rule is discarded
10	CSS	`color: #333` missing semicolon	May break the next declaration
11	CSS	`width: 100px` missing semicolon	Next declaration may be ignored
12	CSS	`padding: 20;` missing unit	Declaration ignored
13	CSS	`.prodcut` typo (HTML uses `product`)	`h3` not styled as intended
14	CSS	`.intro { color: #cfcfcf; }` low contrast	Text nearly invisible on white
15	CSS	`text-decoration: none` missing semicolon	Minor cascade issue

F.10.3 Investigating with DevTools (live walkthrough)

1. **Open the Console (`F12` → Console).** Expect a red error: `Failed to load resource: net::ERR_FILE_NOT_FOUND ... workshop.png`. This **confirms bug 7** immediately. In the **Network** tab the same file shows status **404**.
2. **Inspect the paragraph `.intro`** (right-click → Inspect). In **Styles**, the rule `.intro { color: #cfcfcf; }` is active. Click the colour swatch and read the contrast ratio — it fails. This **confirms bug 14**. Untick it and the text darkens; the fix is to choose a readable colour.
3. **Inspect the `<h3>` "Opening hours".** The intended rule `.prodcut h3` is **not** in the Styles list at all. Comparing the selector (`.prodcut`) with the HTML class (`product`) **confirms bug 13**. This is a typo, found by comparing, not by staring.
4. **Look at the `<div class="product">` box.** Its padding is missing and its width looks wrong. In Styles, `padding: 20;` shows as **invalid/greyed** (no unit) and `width: 100px` may be dropped. This **confirms bugs 10, 11 and 12**. Test the fix live: type `padding: 20px;` — the box breathes.
5. **Scan the DOM tree for `<body>`.** The `body` rule is absent from Styles. The source shows `body` is missing its `{`. This **confirms bug 9**. Add the brace and the font/colour apply.
6. **Look at the DOM tree at the top of `<main>`.** The browser has *moved* the image and the product block **inside** the `<p class="intro">` paragraph, because the paragraph was never closed. This **confirms bug 5**. Close the paragraph and the nesting returns to normal.
7. **Hover the broken image.** The icon plus the Console 404 **confirms bug 7** again from the visual side. Fix the path in the source.
8. **See the crossed tags** in the DOM: `<p>We <strong>look forward to seeing you</p></strong>` — the browser reorders it, but the validator will flag it. This **confirms bug 8**.
9. **Run the W3C HTML validator** on the file. It reports the missing DOCTYPE, the missing charset, the unclosed `<li>`, the unclosed `<p>`, the missing `alt`, and the crossed tags — bugs **1, 2, 4, 5, 6 and 8** — each with a line number.
10. **Run the CSS validator.** It flags the missing brace, missing semicolons and missing unit — bugs **9, 10, 11, 12 and 15**.

F.10.4 The fixed files

`bug-hunt/index.html` (fixed):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Adaeze Fashion - Visit Us</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    <a href="index.html" class="logo">Adaeze Fashion</a>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="shop.html">Shop</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main id="about">
    <h1>Visit Adaeze Fashion</h1>
    <p class="intro">We are a bespoke tailoring studio in Port Harcourt.</p>

    <img src="images/workshop.jpg" alt="Adaeze's tailoring workshop in Port Harcourt">

    <div class="product">
      <h3>Opening hours</h3>
      <p>Monday to Saturday, 9am to 6pm.</p>
      <p>We <strong>look forward to seeing you</strong>.</p>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Adaeze Fashion</p>
  </footer>
</body>
</html>
```

`bug-hunt/css/style.css` (fixed):

```
body {
  font-family: "Segoe UI", Arial, sans-serif;
  color: #333333;
}

.product {
  width: 100%;
  max-width: 400px;
  padding: 20px;
  border-radius: 8px;
}

.product h3 {
  color: #2c1810;
}

.intro {
  color: #555555;
  line-height: 1.6;
}

nav a {
  color: #ffffff;
  text-decoration: none;
}
```

What changed, and why:

- DOCTYPE and `<html lang="en">` added → standards mode and a language for screen readers.

- charset + viewport added → correct Naira signs and proper mobile scaling.
- `contct.html` → `contact.html` → the link now works.
- All tags properly closed and nested correctly → predictable rendering and valid code.
- `.intro` colour darkened to `#555` → passes contrast.
- `.product` width changed to `100%` with a `max-width` → responsive; `padding: 20px` with a unit.
- `.prodcut` → `.product` → the heading is styled.
- Every missing semicolon and the missing `{` fixed → all rules apply.

Expected result: the page now loads with no Console errors, the image shows, the heading is styled, the paragraph is readable, the layout is responsive, and both validators pass.

Teaching note: the power was never in typing faster — it was in *investigating* with DevTools and the validators, then fixing the **confirmed** cause. Emphasise this repeatedly.

F.11 Summary of the session

- **Testing** checks; **debugging** fixes. Test like the customer, not like yourself.
- **DevTools** (Elements / Console / Network / device toolbar) shows the browser's live truth; panel edits are temporary — copy fixes to your file.
- HTML errors: unclosed tags, wrong nesting, missing `alt`, wrong path, missing DOCTYPE/charset/viewport.
- CSS errors: misspelled selectors, missing semicolons, missing units, specificity conflicts, wrong paths.
- **W3C validators** catch standards errors — fix top-down and re-validate.
- **Accessibility:** contrast ≥ 4.5:1, meaningful `alt`, labels tied by `for` / `id`, plus keyboard Tab testing.
- **Cross-browser:** test at least Chrome and one other.
- **Speed basics:** watch image sizes in the Network tab; resize and compress; lazy-load.

G. INSTRUCTOR DEMONSTRATION (15 minutes)

G.1 What the instructor prepares

- The broken `bug-hunt/` page open in Chrome, with DevTools pinned open.
- The same files open in VS Code (broken versions).
- Both W3C validator pages open in other tabs.
- The learner's own site open in another tab for the final tests.

G.2 Demonstration 1 — investigate, do not guess (8 minutes)

Say: "Watch how I find a bug without changing a single line yet. First we *look*."

1. Open DevTools → **Console**. Point at the red `net::ERR_FILE_NOT_FOUND ... workshop.png`.
    - **Expected:** one clear error naming the missing file. **Say:** "Confirmed bug number one: a wrong image name."
2. Switch to **Network**, refresh, and point at the `404` beside `workshop.png`.
3. Right-click the `.intro` paragraph → **Inspect**. Point at `.intro { color: #cfcfcf; }` in Styles. Click the colour swatch and show the **contrast ratio failing**.
    - **Say:** "Confirmed: the text is too light. I will not guess a colour; the tool tells me it fails."
4. Inspect the "Opening hours" `<h3>`. **Ask:** "Is there any rule styling this heading?" (No.) Compare `product` in the HTML with `.prodcut` in the CSS. **Say:** "A one-letter typo — the rule targets nothing."
5. Inspect the `.product` box. Point at the ignored `padding: 20;` (no unit) and the dropped `width: 100px` (missing semicolon).
6. Scan the DOM tree and show how the browser pulled the image and product block *inside* the unclosed `.intro` paragraph.
7. **Inject a fresh bug on purpose:** open Styles and change a colour to something clashing, then refresh to show the change is lost. **Say:** "DevTools edits do not save — the panel is for experiments. The real fix goes in VS Code."

Questions: "Which panel told us about the missing image?" (Console/Network.) "Which panel showed the contrast failure?" (Elements → colour picker.)

G.3 Demonstration 2 — validate and fix (7 minutes)

Say: "Now we fix the confirmed bugs, top-down, and let the validators confirm every one."

1. Open **validator.w3.org**, choose **Validate by Direct Input**, paste the broken HTML, check. Read out the errors with their line numbers (missing DOCTYPE, unclosed `<li>`, unclosed `<p>`, crossed `<strong>`, missing `alt`).
2. In VS Code, fix them **in order from the top**, saving as you go.
3. Re-paste and re-check. **Expected:** far fewer errors, then none.
4. Repeat with **jigsaw.w3.org/css-validator** for the CSS: fix the missing `{`, the missing semicolons and the missing unit.
5. Back in Chrome, refresh and show the fixed page: image loads (no red error), heading styled, paragraph readable, layout responsive.
6. **Ask:** "What would have happened if we had published before testing?" (Broken image, broken link, unreadable text — customers lost.)

Common errors to highlight live: the misspelled selector (comparing `product` vs `prodcut`), and the specificity conflict shown as a struck-through rule.

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: With the instructor guiding, learners run a full test of the Adaeze Fashion site and fix what they find, then produce a short testing report.

Starting materials: The learner's Session 10 project; the bug-hunt page already debugged in Section G as practice; internet connected for the validators.

Procedure (instructor calls each step; learners follow; instructor walks the room):

1. **Open the site in Chrome.** On the Home page, click every nav link and every button. Note any that go to the wrong page or nowhere.
   - **Expected:** every link reaches its page.
2. **Check the Console.** Press `F12` → Console. Note any red errors (usually a wrong path).
   - **Expected:** no red errors; if there is one, it names the missing file.
3. **Check every image.** Scroll each page; look for broken-image icons; in the Network tab note 404s.
4. **Check the form.** On `contact.html`, click each field; confirm the `:focus` ring shows, labels are connected (`for` / `id`), and required fields refuse to be left blank.
5. **Test responsive layout.** Press `Ctrl+Shift+M`. View at **desktop**, at roughly **768px (tablet)** and at **375px (phone)**. Look for overflow, overlapping text or cramped cards.
   - **Pause and predict:** "At 375px, how many product cards per row?" (One.)
6. **Validate HTML.** Paste each page's HTML into validator.w3.org; fix and re-check.
7. **Validate CSS.** Paste `style.css` into jigsaw.w3.org/css-validator; fix and re-check.
8. **Accessibility pass.** Check body-text contrast in DevTools; confirm every `<img>` has meaningful `alt`; confirm every input has a matching label. Press `Tab` through the page and confirm a visible focus ring.
9. **Cross-browser quick test.** Open the site in Edge or Firefox and compare with Chrome.
10. **Write the testing report** (see the template below) listing what you tested, what you found and how you fixed it.

Expected result: a site with working links, all images loading, a sensible form, clean responsive layouts, passing validators, good contrast, labelled inputs, and no Console errors.

Testing report template (learners fill this in):

#	What I tested	What I found	How I fixed it	Confirmed?
1	All nav links			

#	What I tested	What I found	How I fixed it	Confirmed?
2	All images			
3	Console errors			
4	Contact form			
5	Mobile layout (375px)			
6	HTML validation			
7	CSS validation			
8	Contrast / alt / labels			
9	Second browser			

Troubleshooting while walking the room:

- Something unstyled → check the `<link>` path (K.1) and the Console.
- Element not styled as expected → inspect it; look for a struck-through rule (K.3).
- A whole rule ignored → missing semicolon or unit (K.2).
- Validator flags hundreds of errors → fix only the first few, re-validate, repeat (K.5).
- Label not connected → `for` and `id` do not match (K.6).

Completion checklist (learner ticks each):

- ☐ Every link tested and working
- ☐ Every image loads (no broken icons, no 404s)
- ☐ Console free of red errors
- ☐ Form labels, focus and required fields verified
- ☐ Responsive layout checked at 375px, 768px and desktop
- ☐ HTML validated to zero errors
- ☐ CSS validated to zero errors
- ☐ Contrast checked and passing
- ☐ All images have meaningful `alt`; all inputs labelled
- ☐ Tested in a second browser
- ☐ Testing report completed

I. INDEPENDENT CLASS EXERCISE (15 minutes)

Task: Act as a professional tester for a *new*, deliberately flawed page: **"Chinedu Electronics"**. Find every bug, fix it in the source, and justify each fix.

Scenario: Chinedu's cousin built a product page quickly and published it. Customers complain that one photo never shows, a button does nothing, and some text is faint. Chinedu pays you to test and fix it before he shares the link widely.

The broken page (given to learners) — `chinedu/index.html`:

```
<html>
<head>
  <title>Chinedu Electronics</title>
  <link rel="stylesheet" href="css/main.css">
</head>
<body>
  <h1>Chinedu Electronics</h1>
  <p class="tag">Quality gadgets in Port Harcourt.
  <img src="images/powerbank.png">
  <div class="card">
    <h2>Power Bank 20000mAh</h2>
    <p class="price">₦12,000</p>
```

```
    <a href="order.html" class="button">Order Now</a>
  </div>
</body>
```

The broken stylesheet (given) — `chinedu/css/main.css`:

```
body
  font-family: Arial, sans-serif;
  color: #333
}

.card {
  padding: 20;
  border-radius: 8px
}

.tag {
  color: #e6e6e6;
}

.button {
  background-color: #1e6091;
  color: white;
  padding: 10px 20px
}

.buton:hover {
  background-color: #164a70;
}
```

Requirements:

1. Inspect with DevTools and list each bug with a short explanation (file, symptom, cause).
2. Fix each bug in the source (write the corrected HTML and CSS).
3. State, for each fix, how you *confirmed* it (which panel or validator).
4. Do not improve the design beyond fixing the bugs — fix only what is broken. (Good testers do not sneak in redesigns.)

Time allowance: 15 minutes.

Success criteria: all bugs found and fixed; each fix justified; the page loads with no Console errors, a working Order link, a visible image, readable text, and applied padding.

Keyboard hint: count the bugs — there are eight.

Instructor answer key (do not show learners first):

Bugs and fixes:

1. **Missing** `<!DOCTYPE html>` → add at the top. (Symptom: quirks mode; found via validator.)
2. **Missing** `<meta charset>` **and viewport meta** → add both in `<head>`. (Symptom: odd characters, poor mobile scaling; noted by validator/observation.)
3. `<p class="tag">` **not closed** → add `</p>` after "Port Harcourt." (Symptom: image/card pulled into the paragraph; seen in the DOM tree.)
4. `<img>` **missing** `alt` **, and path** `powerbank.png` → change to `powerbank.jpg` (or correct file) and add `alt="Chinedu Electronics 20000mAh power bank"`. (Symptom: broken image and Console 404.)
5. **CSS** `body` **missing** `{` → add it. (Symptom: body font/colour not applied; found via CSS validator.)
6. `color: #333` **missing semicolon** and `padding: 20;` **missing unit** → `color: #333333;` and `padding: 20px;`. (Symptom: declarations ignored; found via validator and Styles pane.)
7. `.buton:hover` **typo** (should be `.button`) → rename. (Symptom: hover does not work; found by comparing selector with the HTML class.)
8. `.tag { color: #e6e6e6; }` **fails contrast** → darken to e.g. `#555555`. (Symptom: nearly invisible text; found via DevTools colour picker contrast check.)

Corrected HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Chinedu Electronics</title>
  <link rel="stylesheet" href="css/main.css">
</head>
<body>
  <h1>Chinedu Electronics</h1>
  <p class="tag">Quality gadgets in Port Harcourt.</p>
  <img src="images/powerbank.jpg" alt="Chinedu Electronics 20000mAh power bank">
  <div class="card">
    <h2>Power Bank 20000mAh</h2>
    <p class="price">₦12,000</p>
    <a href="order.html" class="button">Order Now</a>
  </div>
</body>
</html>
```

Corrected CSS:

```
body {
  font-family: Arial, sans-serif;
  color: #333333;
}

.card {
  padding: 20px;
  border-radius: 8px;
}

.tag {
  color: #555555;
}

.button {
  background-color: #1e6091;
  color: #ffffff;
  padding: 10px 20px;
}

.button:hover {
  background-color: #164a70;
}
```

Marking guidance (out of 10): one mark per bug found and fixed (8), plus one mark for a clear confirmation note and one for not redesigning (2).

J. REAL-WORLD CASE STUDY

Situation. *Ngozi* runs a small online store selling shoes from Port Harcourt. She launched her website in a hurry and shared the link on WhatsApp. Within a day she got messages: "I clicked 'Buy' and nothing happened", "Your prices page is blank", "I can't read the small grey text on my phone". She lost several sales and had to apologise.

Relevant facts.

- The "Buy" button linked to `produts.html` — a typo for `products.html`.
- One product image was `shoe.ong` (a typo for `shoe.png`), shown as a broken icon.
- The description text was light grey `#dddddd` on white — a clear contrast failure.
- She never tested on a phone, and never checked the Console.

Decision point. What should Ngozi change about *how* she works, not just *this* page?

Possible approaches.

1. Fix the three visible bugs only and share the link again immediately.

2. Fix the bugs **and** adopt a habit: open the Console, click every link, view at phone width, check contrast, and validate — before every update.
3. Delete the site and give up on selling online.

Recommended solution (for this stage). Approach 2. The bugs are easy to fix, but the real lesson is the *habit*: the Console would have named `shoe.ong` instantly; clicking the button would have exposed `produts.html`; the device toolbar would have shown the mobile view; the colour-picker would have flagged the contrast. Ten minutes of testing would have saved days of damage.

Trade-offs.

- Approach 1 fixes today's problems but invites the next batch.
- Approach 2 takes a little longer each time but protects trust and sales — and is the professional standard.
- Approach 3 over-reacts; the site is fixable.

Professional connection. Entry-level front-end roles are full of "test and fix" work: QA checks, bug tickets, cross-browser passes, accessibility audits. A junior who can open DevTools, read the Console, run a validator and explain a fix is genuinely employable. Today's skills are exactly those.

K. COMMON MISTAKES AND TROUBLESHOOTING

Use this as the live guide. When a learner hits a problem, point them at the relevant row before answering.

K.1 A page has no styling

Symptoms: Plain black text, blue underlined links. **Likely cause:** Missing or wrong `<link>` `href`, or the file was renamed. **Investigate:** Console → red `net::ERR_FILE_NOT_FOUND`. **Fix:** Correct the path/name in every page's `<head>`. **Confirm:** Styling returns; no red error.

K.2 A whole CSS rule is ignored

Symptoms: Some properties work, one or two do not. **Likely causes:** Missing semicolon on the previous line; missing unit; a missing `{` or `}`. **Investigate:** DevTools → the declaration is greyed/invalid; the CSS validator flags it. **Fix:** Add the semicolon/unit/brace. **Confirm:** The property applies after refresh; validator passes.

K.3 A style does not “win”

Symptoms: You set a colour but another rule overrides it. **Likely cause:** Specificity conflict — a more specific selector (e.g. an id) or a later rule wins. **Investigate:** DevTools → the losing declaration is struck through. **Fix:** Adjust the winning rule or make your intended rule more specific (e.g. add a class); avoid `!important`. **Confirm:** Your value now appears un-struck.

K.4 An image is broken

Symptoms: Broken-image icon in place of the photo. **Likely cause:** Wrong file name or path; file not uploaded. **Investigate:** Console/Network 404 naming the file; compare spelling and folder. **Fix:** Correct the path; ensure the file exists in the folder. **Confirm:** The image displays; no 404.

K.5 The validator shows dozens of errors

Symptoms: An overwhelming list. **Likely cause:** One early error cascading into many. **Investigate:** Read the first two or three errors only. **Fix:** Fix top-down, save, and **re-validate** between fixes. **Confirm:** The list shrinks quickly, then reaches zero.

K.6 A form label is not connected to its field

Symptoms: Clicking the label does nothing; screen readers announce an unlabelled field. **Likely cause:** The label's `for` does not match the input's `id`. **Investigate:** Compare the two values character by character. **Fix:** Make them identical, e.g. `for="email"` ↔ `id="email"`. **Confirm:** Clicking the label focuses the field.

K.7 It looks fine on desktop but breaks on mobile

Symptoms: Overflow, overlapping text or cramped layout on a phone. **Likely cause:** Missing viewport meta, or missing mobile media query. **Investigate:** DevTools device toolbar at 375px. **Fix:** Add the viewport meta and a `max-width: 767px` media query.

Confirm: The layout reflows cleanly at phone width.

Distinguish likely vs confirmed: Never fix on a hunch. Open the Console, inspect the element, or run the validator and *see* the cause before editing. Confirming first is what separates a professional from a guesser.

L. SESSION ASSIGNMENT

Assignment title: "Testing and Debugging Report"

Purpose: Practise the full testing workflow on the running Adaeze Fashion project and document it — the exact process used before publishing the capstone.

Realistic scenario: Adaeze is about to launch the site. She asks you for a short written assurance that every page has been tested, with the problems found and fixed.

Detailed instructions:

1. Test **all pages** of your Adaeze Fashion site:
   - click every link and button;
   - confirm every image loads;
   - open the Console and note any red errors.
2. Test the **contact form**: labels connected, `:focus` visible, required fields enforced.
3. Test the **responsive layout** with the device toolbar at **375px**, **768px** and desktop.
4. **Validate** every HTML page (validator.w3.org) and the stylesheet (jigsaw.w3.org/css-validator) to zero errors.
5. Run an **accessibility pass**: contrast ≥ 4.5:1 on body text, meaningful `alt` on every image, a label on every input, and a visible keyboard focus ring.
6. **Cross-browser:** open the site in a second browser and note any differences.
7. Produce a **Testing Report** (one to two pages) containing:
   - a testing checklist with tick boxes;
   - a bug log table: *what I tested · what I found · how I fixed it · confirmed?*;
   - validator screenshots or pasted results (both "no errors" outcomes);
   - a short paragraph on what you learned about testing.

Required deliverables:

- The updated, corrected project files.
- The written Testing Report.

Submission format: Files in the class project folder / shared drive; report in your workbook or a shared document.

Deadline: Before Session 12.

Marking rubric (25 marks):

Criterion	5 — Excellent	3 — Good	1 — Needs work
Testing coverage	All pages, links, images, form	Most tested	Minimal testing
Error identification	All key issues found with confirmed causes	Most found	Few found
Error resolution	All fixed and re-tested	Most fixed	Few fixed
Validation	HTML **and** CSS both pass	One passes	Neither passes
Documentation	Clear, organised report with bug log	Mostly complete	Vague or missing
Total	**25**		

Common mistakes to avoid: fixing without confirming the cause; validating once and not re-checking; only testing the Home page; ignoring contrast and `alt`; leaving a stray `!important` instead of fixing specificity.

Instructor reference solution: apply the fixes shown in Sections F.5, F.10 and I to the learner's own site; a strong report mirrors Section H's template with a filled bug log and two "no errors" validator results.

M. KNOWLEDGE CHECK

(Learner version — answers in the Instructor Key below.)

Q1. Short answer. Explain the difference between testing and debugging in one sentence each.

Q2. Multiple choice. Which DevTools panel tells you that an image file failed to load (a 404)? a) Elements b) Console and Network c) Styles d) device toolbar

Q3. Explain in your own words. Why are changes you make in the Elements panel temporary, and why is that useful?

Q4. Error detection. Name the error in each snippet: a) `.prodcut { color: red; }` when the HTML uses `class="product"` b) `padding: 20;` c) `body { color: #333 }` (semicolon missing) d) `.tag { color: #dddddd; }` on a white background (text colour)

Q5. Practical. In DevTools, how do you tell that a CSS declaration has been overridden by a more specific rule, and what is the professional fix (not `!important`)?

Q6. Scenario. A learner's page renders with no styling at all in the browser. List the three checks you would run, in order, and what each would show.

Q7. Accessibility identification. What three things do we check for accessibility, and what is the minimum contrast ratio for normal body text?

INSTRUCTOR ANSWER KEY (Q1–Q7)

A1. *Testing* is checking that links, images, pages, the form, layout, accessibility and speed all work; *debugging* is finding and fixing the errors that testing reveals. **A2.** (b) Console and Network (the Console shows the error; the Network panel shows the 404). **A3.** They are temporary because the panel edits the live page in memory, not the saved file. This is useful because you can experiment freely, confirm which fix works, and only then write it into `style.css` — you can break nothing permanently. **A4.** (a) misspelled selector (targets nothing); (b) missing unit (should be `20px`); (c) missing semicolon after `#333`; (d) contrast failure (too light) — darken it. **A5.** The declaration appears **struck through** in the Styles pane. The professional fix is to adjust the winning rule or increase the specificity of the rule you intend to win (usually by adding/using a class) — not to use `!important`. **A6.** (1) Check the Console for a red `net::ERR_FILE_NOT_FOUND` on the stylesheet — confirms a wrong path. (2) Compare the `href` in the `<head>` with the real file name and folder. (3) Confirm the file is saved and the `<link>` exists on that page. **A7.** Colour contrast (≥ **4.5:1** for normal text), meaningful `alt` text on images, and form labels connected by `for` / `id` (plus keyboard/focus testing).

N. REFLECTION AND RECAP

Plain-English summary. Today you became a tester and a detective. You learned that "it works on my machine" is not the standard; the standard is "it works for the customer". You used Chrome DevTools — the Console, Network, Elements and device toolbar — to see what the browser really did, inspected elements to test CSS fixes live (remembering that panel edits are temporary), and learned to diagnose misspelled selectors, missing semicolons, missing units, specificity conflicts and wrong file paths. You ran the W3C HTML and CSS validators, checked contrast, alt text and labels, tested in a second browser, and looked at image sizes for speed. Most importantly, you debugged a genuinely broken page by *confirming* each cause before fixing it.

Five important takeaways:

1. Test like the customer — every page, every screen size, more than one browser.
2. Investigate before fixing: the Console, Styles pane and validators reveal the true cause.
3. DevTools edits are temporary; the permanent fix goes in your source file.
4. Fix validator errors top-down and re-validate after each fix.
5. Accessibility — contrast, `alt`, labels, focus — is part of testing, not an afterthought.

Questions to help you recognise what you can now do:

- Can I find a broken image or link using DevTools without guessing?

- Can I tell when a CSS rule is being ignored or overridden, and why?
- Can I get my HTML and CSS to pass the W3C validators?
- Can I check my own site for contrast, alt text and labels?

Connection to the next session: You now have a designed, responsive, lively and *tested* website. In Session 12 — the final session — you assemble your complete capstone (3–5 pages), publish it online with free hosting, present it to the class and answer viva questions. Everything you have tested up to now is exactly what will carry you through. You are ready.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner opens DevTools unaided and reads the Console error text rather than guessing.
- Learner inspects an element and correctly identifies a struck-through (overridden) rule.
- Learner confirms a cause before fixing ("the Console says the file name is wrong").
- Learner fixes validator errors top-down and re-validates.
- Learner checks mobile width with the device toolbar and catches overflow.
- Learner tests at least a second browser and reports a real observation.
- Learner treats accessibility (contrast, `alt`, labels) as part of testing.

Common misconceptions to watch for and correct:

- "It works on my machine, so it works." → Test like the customer on other sizes and browsers.
- "DevTools saved my change." → Panel edits vanish on refresh; copy the fix to your file.
- "The validator is just for experts." → It is a basic professional habit.
- "Solving specificity means adding `!important`." → Fix the rule or its specificity instead.
- "Accessibility is extra." → It is part of quality, and it helps everyone.

Suggested remedial exercises for learners who struggle:

1. **Console reading drill:** give five pages, each with one wrong path, and ask learners to name the missing file from the Console alone.
2. **Struck-through hunt:** provide a page with three specificity conflicts and ask learners to identify each losing rule in the Styles pane and fix it without `!important`.
3. **Validator top-down drill:** hand a page with one early error that causes five later ones; ask learners to fix only the first and re-validate to watch the list collapse.

Differentiation:

- **Level 1 (supported):** Provide the bug-hunt page with a written list of *categories* (not exact locations) to find, and walk the learner through DevTools step by step.
- **Level 2 (standard):** The exercise and assignment as written.
- **Level 3 (challenge):** Use Lighthouse in DevTools to run an accessibility/performance audit, fix the top three items it reports, and test in Firefox and Edge documenting any genuine differences.

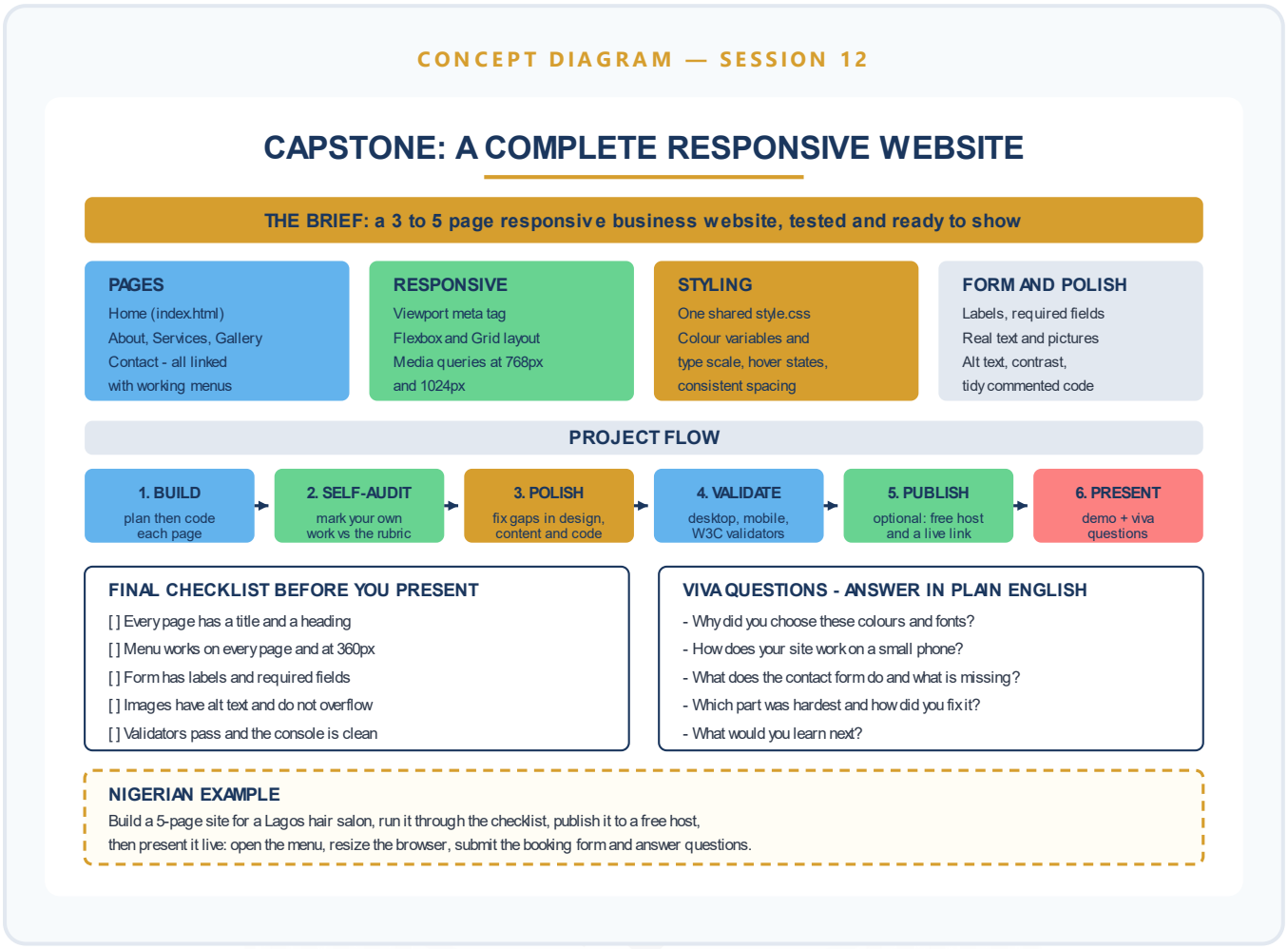
Timing guidance: The DevTools investigation is the heart of the session — protect it. If the room is slow, run only the HTML validator live and assign the CSS validation as homework. Internet failure is the main risk; if the validators are unreachable, teach the Console/Elements work first and defer validation, since most of the learning is offline.

END OF SESSION 11

Next: Session 12 — Final Project: Capstone.

SESSION 12 OF 12

FINAL PROJECT: CAPSTONE (FINALISE AND PRESENT)



CYBER ELIAS ACADEMY — WEB DESIGN

SESSION 12 — FINAL PROJECT: CAPSTONE (FINALISE AND PRESENT)

A. SESSION IDENTIFICATION

Item	Detail
Session number	12 of 12 (final session)
Session title	Final Project: Capstone — Finalise, Publish and Present
Week / Module	Week 4 — Module: Publishing and Capstone
Recommended duration	105 minutes

Item	Detail
Difficulty level	Beginner, integrating **all** skills from Sessions 1–11
Prerequisite knowledge	Sessions 1–11. Learners can plan a site, build semantic multi-page HTML, style it with an organised external stylesheet, make it responsive, add CSS interactivity and media, and test and debug it.
Connection to previous work	Everything in the course leads here. The website the learner has been building since Session 7 (planning) and Session 8 (structure) and styling since Session 9 becomes a complete **3–5 page responsive business website**. This session is where the learner finalises it, verifies it against a professional rubric, optionally publishes it, and presents it with a viva.
Preparation before class	Academy PCs on; VS Code and Chrome; every learner's project accessible; projector for presentations; **printed capstone briefs (one per learner), printed rubrics (one per learner), and certificate sign-off sheets**; internet for optional publishing (GitHub/Netlify); a timer visible to the class; the instructor's own finished Adaeze Fashion site ready as the benchmark.

B. LEARNING OUTCOMES

By the end of this session, the learner will be able to:

1. **Finalise a complete 3–5 page responsive business website** that meets the capstone brief.
2. **Audit their own site against a professional rubric** and identify any last gaps.
3. **Apply final polish** to structure, design, responsiveness, content, the form, code quality and accessibility.
4. **Validate and test** the finished site on desktop and mobile, and explain the results.
5. **Optionally publish the website** to a free host and obtain a public URL.
6. **Present the website to the class** confidently, demonstrating its features.
7. **Answer viva questions** about their design and technical decisions in plain English.
8. **Reflect honestly** on their learning and describe realistic next steps.

C. REQUIRED MATERIALS

Hardware

- One computer per learner (academy desktops).
- Instructor computer connected to a projector or large screen for presentations.

Software

- Visual Studio Code (2022 or later).
- Google Chrome (any recent version); Firefox or Edge optional.
- (For optional publishing) a web browser plus a GitHub account, or a Netlify account.

Accounts

- Each learner needs a **free GitHub account** if they will publish via GitHub Pages (recommended). Netlify is an acceptable alternative. Create accounts **before** this session where possible to save time.

Internet

- **Required** for optional publishing and for re-checking the Session 10 map/video embeds.

Printed Materials

- Capstone brief (one per learner).
- Capstone rubric (one per learner — the table in Section K).
- Certificate sign-off sheet (one per learner).
- Timer or clock for the presentation schedule.

Sample files

- The instructor's finished Adaeze Fashion reference site (Home, Shop, About, Contact) as the benchmark.

Tools

- Chrome DevTools (`F12`), the W3C validators (validator.w3.org, jigsaw.w3.org/css-validator), and a free hosting dashboard.

Alternative for learners who lack a facility

- Any laptop with VS Code and Chrome works. If a learner cannot create a hosting account, the instructor publishes their site from an academy account, or the learner presents locally and publishes afterwards with support.

D. OPENING ACTIVITY (5 minutes)

Instructor setup: On the projector, show three images/tabs: (1) a blank page, (2) the learner's own Session 8 *skeleton* if it still exists, and (3) the instructor's finished Adaeze Fashion site.

What the instructor says:

> *"Four weeks ago, most of you had never written a line of code. Today you will present a real, working business website that you built yourselves — a site for Adaeze Fashion that a real customer could use. Look at where we started and where we are.*
>
> *There will be no new lessons today. Today is about finishing well and presenting well. I am going to give you a brief and a rubric — the same kind of checklist a real client uses — and you are going to bring your site up to that standard, publish it if you can, and show it to us. A beautiful website you cannot explain is not enough; a site you understand and can defend is the goal. Let us finish strong — From Zero to Expert, Together."*

Expected learner response: A mix of nervousness and pride. Reassure them: the rubric is not a trick; it lists exactly what we practised, and the viva is a conversation, not an interrogation.

Why this opener works: It frames the session as celebration and consolidation rather than new content, reduces performance anxiety with transparency, and reminds learners of the journey — which builds the confidence needed to present.

E. PREREQUISITE REVIEW (7 minutes)

Run fast. This doubles as the final revision before the viva, so let several learners answer each.

Question 1: "What are the three building blocks of a website, and what does each do?"

- Expected: HTML (structure/content), CSS (appearance), JavaScript (behaviour — not used in this course's sites).
- Gap handling: remind them our capstone is HTML + CSS only, and JavaScript is the next course.

Question 2: "What are the four layers of the box model?"

- Expected: content, padding, border, margin.
- Gap: padding (inside) vs margin (outside).

Question 3: "What does `display: flex` do and where did we use it? What about `display: grid` ?"

- Expected: flex for the header/nav (one direction); grid for the product cards (rows and columns).
- Gap: flex goes on the parent container.

Question 4: "What is a media query, and which breakpoints did we use?"

- Expected: CSS that applies under a condition, usually screen width; `1023px` (tablet) and `767px` (mobile).
- Gap: forgetting the parentheses.

Question 5: "Why do all pages share one stylesheet?"

- Expected: consistency and one-place updates.
- Gap: the value is *consistency* first.

Question 6: "How do you test a site before publishing?"

- Expected: click links, check images, use the device toolbar, check the Console, validate HTML/CSS, check contrast/alt/labels, try a second browser.
- Gap: testing only on the Home page.

Question 7: "What is a relative path, and why does it matter when publishing?"

- Expected: a path relative to the current file, with forward slashes; absolute paths break on a web server.
- Gap: backslashes vs forward slashes.

Question 8: "In one sentence, what did you learn that surprised you most?"

- Accept any sincere answer. This warms the room and seeds the reflection.

Bridge to today: "Every answer you just gave is something on the rubric. If you can explain them — as you just did — you can pass the viva. Now finish your site and show it."

If the room is weak on the media-query or path questions, spend five extra minutes on those two, since they cause the most capstone failures.

F. FULL LECTURE NOTES

F.1 What the capstone is and why it matters

The idea. The **capstone** is the final project that proves you can do the whole job end to end: plan, build, style, make responsive, add media, test, and present a complete business website.

Plain-English definition. A capstone is a single, substantial piece of work that draws together every skill from the course, the way a final year project draws together a degree.

Why it exists. Knowing individual skills (a media query, a flexbox header) is not the same as producing a finished website under your own judgement. The capstone tests the *whole*, including decisions, testing and explanation. This is what employers and clients actually pay for.

Analogy. Learning to sew a hem, a collar and a sleeve is not the same as producing a finished gown that fits and looks right. The capstone is your finished gown.

What "done" means here. A **3–5 page responsive business website** for Adaeze Fashion (or a business you choose), containing:

- a Home page (welcome/intro and a call to action),
- a Shop/Products page (a grid of products with prices),
- an About page (the business story and values),
- a Contact page (a styled form, and a map from Session 10),
- consistent navigation on every page,
- a responsive layout,
- images with meaningful `alt` text,
- valid HTML and CSS,
- tested on mobile and desktop.

Misconception. "The capstone must be perfect." No. The standard is *competent and explainable*. Minor issues that you can identify and fix yourself are acceptable. A polished site you cannot explain is *not* passing.

Limitation and honesty. This course, and therefore this capstone, covers **HTML and CSS only**. That makes you an entry-level front-end web designer — someone who can build a clean, responsive, static business website. It does **not** make you a full-stack developer. We revisit this clearly in the wrap-up.

F.2 Milestone checkpoints — what should already be done before Session 12

Use this table at the start as a self-audit. Anything unticked is today's priority.

Checkpoint	Introduced in	Should be complete by
Business chosen; site plan and wireframes sketched	Session 7	Before Session 8
3-page HTML structure: header, nav, main, footer; pages linked	Session 8	Before Session 9
Shared stylesheet linked to every page; palette, type scale, spacing	Session 9	Before Session 10
Responsive (tablet + mobile media queries)	Sessions 6 & 9	Before Session 10
CSS interactivity: hovers, transitions, a reveal; map and video embeds; favicon	Session 10	Before Session 11
Site tested: links, images, form, responsive, validators, accessibility	Session 11	Before Session 12
All pages present (3–5); project folder organised	Sessions 7–11	Entering Session 12
Corrected bugs logged; validation passing	Session 11	Entering Session 12

Entering Session 12, the minimum baseline is: all pages built and styled, navigation working on every page, responsive, a contact form, images with `alt` text, and the site having been tested at least once. Learners below this baseline should switch to the **Level 1 supported track** (Section O) immediately.

F.3 Finalising the website — the last 10%

The difference between a good student site and a passing capstone is usually a short list of small, high-value items.

Structure and semantics (the skeleton must be right).

- Every page starts with `<!DOCTYPE html>` and `<html lang="en">`.
- The `<head>` contains `<meta charset="UTF-8">`, the viewport meta, a descriptive `<title>` **and** the stylesheet `<link>`.
- Use semantic elements: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.
- The same header/nav and footer appear on every page, with identical links.

Consistency (the brand must be one).

- All pages use the same palette, fonts and spacing — ideally driven by `:root` variables.
- One `.button` class reused everywhere.
- Active nav link styled so the visitor knows which page they are on (optional but professional).

Responsiveness (it must work in a pocket).

- Test at **375px (phone)**, **768px (tablet)** and desktop.
- No horizontal scrolling; cards become one column; header stacks; text does not overflow.

Content quality (it must read well).

- Real, specific content — not "Lorem ipsum" and not "text here".
- Correct Naira prices, a real Port Harcourt address and phone number.
- No obvious spelling mistakes; headings in a sensible order (one `<h1>` per page).

The form (it must display properly).

- Labels connected by `for` / `id`; required fields; a clear `:focus` state; a styled submit button.
- **Honesty:** the form displays and validates required fields, but HTML alone cannot send it. Sending requires a service (e.g. Formspree, Netlify Forms) or a server. Do not claim it delivers email.

Code quality (it must be clean).

- Indented, commented, organised files; a logical folder structure.

Validation and testing (it must be correct).

- HTML and CSS pass the W3C validators.
- Console free of red errors; every link and image works.

Accessibility (it must be usable by everyone).

- Meaningful `alt` text; passing contrast (≥ 4.5:1 body text); visible focus outlines; labels.

Organised folder structure (the professional habit):

```
adaeze-fashion/
├── index.html          (Home)
├── shop.html           (Products)
├── about.html
├── contact.html
├── css/
│   └── style.css
├── images/
│   ├── logo.png
│   ├── favicon.png
│   ├── hero.jpg
│   └── product-1.jpg ...
```

F.4 Publishing (optional but recommended — required for the certificate)

The CEA certificate requires a **published** website, so we attempt this where accounts and internet allow.

What "publishing" means. Right now your site lives only on your computer; the address in the browser bar starts with `file:///C:/...`. Publishing copies your files to a web server so anyone can open them at a public `https://...` address.

Option A — GitHub Pages (free, recommended).

1. Create a free account at github.com (if you do not have one).
2. Click **New repository**; name it, e.g. `adaeze-fashion`; make it **Public**; create it.
3. Upload your site files (drag-and-drop in the browser is fine for our small site): open `index.html` must sit at the **top level**, not inside an extra folder.
4. Go to **Settings** → **Pages**; under **Build and deployment**, set **Source** to *Deploy from a branch*, branch `main`, folder `/ (root)`; save.
5. Wait a minute or two. Your site appears at `https://<your-username>.github.io/adaeze-fashion/`.
6. Open that URL, click through every page, and confirm the images load.

Option B — Netlify (free, drag-and-drop).

1. Create a free account at netlify.com.
2. On the dashboard, drag your whole project **folder** onto the "drag and drop to deploy" area.
3. Netlify gives you a public URL immediately (you can rename it in settings).

The single most common publishing bug: absolute paths. A path like `C:\Users\Name\project\images\photo.jpg` works only on your machine. On the server it breaks. **Always use relative paths** (`images/photo.jpg`) so the site works everywhere. This is why Session 11 tested for it.

Other publishing checks:

- `index.html` must be the Home page (servers look for it by default).
- File names are **case-sensitive** on servers (`Logo.PNG` is not `logo.png`). Match them exactly.
- No spaces in file names; use hyphens (`product-1.jpg`).

Expected result: a public URL that opens your site on any device, with working pages, images, map and video.

Misconception. "Once published, it is finished." No — you can update files and the published site updates (GitHub Pages takes a minute; Netlify is near-instant). Publishing is a step, not a finish line.

F.5 Preparing the presentation and viva

The presentation (3–4 minutes per learner). You are not reading your code aloud. You are *showing* a business owner what you built and why.

A simple 4-part structure:

1. **The business and the goal (30s):** who Adaeze Fashion is and what the site must achieve.
2. **The tour (90s):** walk through Home → Shop → About → Contact, pointing out the navigation, the product grid, the responsiveness (drag the window live), and the form.

3. **The decisions (60s):** two or three choices and *why* — the palette, the fonts, why grid for products, why the mobile breakpoint.
4. **The honesty (30s):** one thing you are proud of, one thing you would improve, and what the site does *not* do (e.g. the form does not send email without a service).

The viva. After (or during) your presentation, the instructor asks a few questions from the list in Section L. The viva exists to make sure you **understand your own work**. Answer in plain English; if you do not know, say so honestly and reason aloud — that is respected far more than a bluff.

Presentation tips for a Nigerian classroom:

- Speak clearly and a little slower than usual.
- Point at the screen so the audience follows.
- If something breaks live, stay calm, open DevTools, and treat it as a real-world moment — explaining the fix *is* showing competence.

Misconception. "The viva is a trap." It is the opposite: it is your chance to show that the work is *yours* and that you understand it. A site you can explain is worth far more than one you cannot.

F.6 What comes after this course (introduction)

You are finishing the **Web Design** course. That is a real achievement and a real, marketable skill — but it is a *foundation*, not the ceiling. Section N explains exactly what you are now qualified to do and what to learn next. For now: finish, present, and be proud.

F.7 Summary of the session

- The capstone is a **3–5 page responsive business website** that draws together every skill.
- Final polish focuses on structure, consistency, responsiveness, content, the form, code quality, validation and accessibility.
- **Publish** (GitHub Pages or Netlify) using **relative paths** and a top-level `index.html`.
- **Present** in four parts and answer the **viva** honestly.
- You are an **entry-level front-end web designer (HTML/CSS)** — not a full-stack developer. JavaScript and frameworks come next.

G. INSTRUCTOR DEMONSTRATION (12 minutes)

There is no new coding demonstration today. Instead, two short demonstrations set the learners up to succeed.

G.1 Demonstration 1 — the benchmark tour and self-audit (6 minutes)

Say: "Before you finish your own, let me show you exactly what I am looking for, using my reference site."

1. Open the instructor's Adaeze Fashion site in Chrome.
2. Walk through every page, pointing out each rubric item out loud:
   - semantic structure (view the Elements panel to show `<header>`, `<nav>`, `<main>`, `<footer>`);
   - the shared stylesheet and `:root` variables;
   - the product grid becoming one column when the window narrows (drag it live);
   - the styled form with a visible `:focus` ring and label `for` / `id`;
   - the map and video from Session 10;
   - the favicon in the tab.
3. **Inject a final "polish" bug on purpose:** rename `style.css` in the reference to `styles.css` temporarily and refresh. The site loses all styling.
   - **Say:** "Even at the final minute, one wrong character can break everything. That is why we re-test after every change — right up to publishing." Restore it.
4. Open DevTools Console to show **no red errors**, then briefly show a passing W3C validation result.

5. **Ask:** "Which of these will you check on your own site in the next 15 minutes?" (Let learners call out items; this seeds their self-audit.)

G.2 Demonstration 2 — publishing walkthrough (6 minutes)

Say: "A website on your laptop is a website nobody can visit. Watch how fast we put it online."

1. Open github.com, create (or open) a public repository named `adaeze-fashion-demo`.
2. Upload the instructor's files (drag-and-drop), making sure `index.html` is at the top level.
3. Go to **Settings** → **Pages**, set the branch to `main` and folder to `/ (root)`, and save.
4. Wait, refresh the Pages section, and open the generated URL.
    - **Expected:** the live site appears at `https://<username>.github.io/adaeze-fashion-demo/`.
5. Click through the pages; show the map and video loading over the internet.
6. **Common trap to show:** deliberately set an **absolute** image path in one page (e.g. `C:\...\images\hero.jpg`), redeploy (or just explain), and show the broken image on the live site. **Say:** "This is the number-one publishing mistake — always relative paths."

Questions: "Why must `index.html` be at the top level?" (Servers look for it as the default home page.) "Why do relative paths matter more online than offline?" (Absolutes point at your own computer, which the server does not have.)

H. GUIDED PRACTICAL EXERCISE (18 minutes)

Objective: A guided self-audit and final-polish clinic. Learners run the rubric against their own site, fix the highest-value gaps, and prepare to present.

Starting materials: The learner's project, open in VS Code and Chrome; the printed rubric; internet for re-checking embeds and validating.

Procedure (instructor walks the room continuously; learners work through the checklist, pausing for short group checks):

1. **Structure audit (3 min).** On each page, confirm `<!DOCTYPE html>`, `<html lang="en">`, charset, viewport, title and the stylesheet `<link>`. Fix any missing.
2. **Consistency audit (2 min).** Confirm header/nav and footer are identical on every page, and that every page links the same `style.css`. Check that `:root` drives the colours.
3. **Responsive audit (4 min).** Press `Ctrl+Shift+M`; check **375px, 768px, desktop**. **Pause and predict:** "At 375px, is there any horizontal scrollbar?" (There should not be.) Fix any overflow.
4. **Content audit (2 min).** Read each page aloud in your head. Fix spelling, replace placeholder text, confirm Naira prices and a real Port Harcourt address.
5. **Form audit (2 min).** Click each field; confirm the label focuses the field, the `:focus` ring shows, and required fields are enforced.
6. **Code quality (1 min).** Confirm files are indented and commented, and the folder structure matches Section F.3.
7. **Validation (2 min).** Validate one HTML page and the CSS. Fix the first error, re-validate, repeat. **Expected:** "No errors".
8. **Accessibility (2 min).** Check body-text contrast in DevTools; confirm every `<img>` has meaningful `alt`; Tab through and confirm a visible focus ring.
9. **Presentation rehearsal (2 min with a partner).** Give your 4-part pitch to a neighbour in 2 minutes; the neighbour asks one viva question from Section L. Swap.
10. **Publishing (where accounts/internet allow).** Begin the GitHub Pages steps from G.2. If no account, the instructor publishes on your behalf, or you publish after class.

Expected result: every learner has a visibly improved, self-audited site, a validated code base, and a rehearsed presentation.

Points to pause on and predict:

- After step 3: "Which section breaks first on a phone?" (Often the header or a fixed-width element.)
- After step 7: "Will fixing the first validator error remove others?" (Usually yes — re-validate to confirm.)

Troubleshooting while walking the room:

- Still no styling on one page → missing/wrong `<link>` (K worth checking).

- Horizontal scroll on mobile → a fixed width or a missing `box-sizing: border-box`.
- Validator flood → fix top-down, re-validate.
- Live site images broken → absolute paths; change to relative.
- Overwhelmed learner → switch to Level 1 (Section O): template + one-on-one.

Completion checklist (learner ticks each):

- ☐ All pages have complete `<head>` and correct `<link>`
- ☐ Header/nav and footer identical on every page
- ☐ No horizontal scroll at 375px
- ☐ Real content, correct Naira prices, no placeholders
- ☐ Form labels connected, `:focus` visible, required enforced
- ☐ HTML and CSS validated to zero errors
- ☐ Contrast, `alt` text and focus checked
- ☐ Presentation rehearsed once with a partner
- ☐ Published (or scheduled with the instructor)

I. INDEPENDENT CLASS EXERCISE (30 minutes) — THE CAPSTONE WORK SESSION

Task: Independently finalise your complete 3–5 page responsive business website to the standard in the brief (Section K), then present it and answer viva questions.

Scenario: You are the hired web designer. Adaeze Fashion (or your chosen Port Harcourt business — a church, school, salon, restaurant or shop) needs a finished website by the end of this session. The client will judge it against the rubric and ask you questions about it.

Independent work (about 20 minutes):

- Complete or fix anything still outstanding from the audit in Section H.
- Make sure the site meets every deliverable in the brief (Section K).
- Prepare your 4-part presentation.

Presentation and viva (about 10 minutes total for the class; 3–4 minutes each as time allows):

- Present using the four-part structure in F.5.
- Answer viva questions from Section L.
- The instructor records the rubric score (Section K) and the viva score (5 marks).

Success criteria:

- 3–5 pages, each complete and consistent.
- Responsive on desktop and mobile.
- A styled, labelled, working (display) contact form.
- Valid HTML and CSS; no Console errors; tested in a second browser if available.
- Published with a public URL (where possible).
- Presented clearly and defended in the viva.

Answer key / evaluator's guide: There is no single "correct website". The instructor evaluates against the **capstone rubric (Section K)** and the **viva model answers (Section L)**. A strong submission looks like the instructor's reference site: consistent brand, grid of products, semantic structure, valid code, and a learner who can explain every choice.

J. REAL-WORLD CASE STUDY

Situation. *Emmanuel* finished the CEA Web Design course last term. Two weeks later, a friend opening a barbershop in Rumuola asked him to build a simple website. Emmanuel sat down on a Saturday morning, reused his own capstone structure, swapped the content and colours, tested it on his phone and his laptop, published it with GitHub Pages, and shared the link that afternoon. His friend paid him, and the shop started getting booking messages from the site.

Relevant facts.

- Emmanuel did not invent anything new; he *reused a proven structure* and adjusted it.
- He tested on two devices and validated before publishing — so nothing was broken on launch.
- He was honest that the contact form displayed but did not yet send email, and offered to add that later for a small fee.

Decision point. Why was Emmanuel able to deliver in one day when he had never charged for a website before?

Possible explanations.

1. He was born a "tech person".
2. He had practised the *whole process* repeatedly — plan, structure, style, responsive, test, publish — until it became a routine he could repeat for any small business.
3. He got lucky.

Recommended answer. Explanation 2. The capstone is not a one-off exam; it is a *rehearsal of a repeatable process*. Because Emmanuel had built the Adaeze Fashion site end-to-end, the second site was mostly substitution: new text, new photos, new palette, same structure. That is exactly how entry-level freelance work begins.

Trade-offs.

- Reusing a structure is fast and reliable, but he must still change the content, colours and images so no two clients look identical.
- Being honest about the form's limits protects his reputation and can lead to paid follow-up work.

Professional connection. Many real front-end jobs are *adapting* known patterns rather than inventing new ones. The confidence to say "I can build you a clean, responsive 4-page website and publish it" — and to actually do it — is precisely what this course certifies.

K. THE CAPSTONE BRIEF AND RUBRIC

K.1 The capstone brief (learner copy)

Client: Adaeze Fashion — a bespoke clothing business in Port Harcourt (or an approved alternative business such as Chinedu Electronics, Mama Ngozi's Kitchen, Ngozi's Salon, a church or a school).

Project: Design, build, test and (where possible) publish a **3–5 page responsive business website**.

Required pages:

- **Home** — a welcome hero with a clear message and a call-to-action button.
- **Shop / Products** — a grid of at least three products/services with names and Naira prices.
- **About** — the business story, values and (optionally) a team section.
- **Contact** — a styled contact form and an embedded Google Map of the business location.
- (Optional 5th page) — Gallery, Testimonials or Blog.

Requirements:

1. Consistent navigation on every page linking all pages correctly.
2. A responsive layout: desktop, tablet and mobile (no horizontal scrolling).
3. At least one meaningful image per page, all with descriptive `alt` text.
4. A styled contact form with connected labels, a visible focus state and required fields.
5. Semantic HTML (`header` , `nav` , `main` , `section` , `footer`).
6. A single shared external stylesheet, organised and commented, with a `:root` palette.
7. **Valid HTML and CSS** (W3C validators, zero errors).
8. **Tested** on mobile and desktop (and a second browser if available).
9. (Optional, required for certificate) **Published** to a public URL.

Honest scope note: This is an **HTML/CSS** website. The contact form displays and validates required fields but does not send messages without a form service or server. Interactive menus, live search and form-sending require **JavaScript** — the next course. Do not claim otherwise.

K.2 The full capstone rubric (30 marks + 5 viva = 35)

#	Criterion (marks)	Excellent	Good	Developing	Not yet
1	**Structure & semantics (4)**	All pages have valid `<!DOCTYPE>`, `<head>`, semantic `header/nav/main/section/footer`; logical heading order; everything closed and nested correctly.	Semantics mostly correct; one or two minor structural slips.	Some semantic elements used but inconsistent; a few nesting errors.	Little/no semantic structure; structural errors throughout.
2	**Visual design (4)**	Cohesive palette (3–5 colours), two fonts, consistent spacing; clear hierarchy; professional and branded.	Generally consistent; minor colour/spacing inconsistencies.	Basic styling; palette or spacing inconsistent.	Little styling or clashing colours/fonts.
3	**Layout & responsiveness (4)**	Flex header + grid products; reflows cleanly at all sizes; no overflow; works well at 375px, 768px and desktop.	Responsive with minor issues (small overflow or awkward gaps).	Partly responsive; breaks at some sizes.	Not responsive; horizontal scrolling on mobile.
4	**Content quality (3)**	Real, specific content; correct Naira prices; real Port Harcourt details; no placeholders; no spelling errors.	Mostly real content; a few weak spots.	Some placeholder text or thin content.	Placeholder/empty content; many errors.
5	**Contact form (3)**	Labels connected by `for` / `id`; required fields; visible `:focus`; styled button; layout tidy.	Form works and is styled with one small gap (e.g. missing focus styling).	Form present but labels unconnected or unstyled.	Form missing or broken.
6	**Code quality (4)**	Files indented, commented, well organised; logical folder structure; reusable classes; no obvious duplication.	Mostly tidy; a few long/uncommented sections.	Messy in places; inconsistent indentation.	Disorganised; hard to read; duplicated rules.
7	**Validation (4)**	HTML **and** CSS pass both W3C validators with zero errors.	One passes; the other has minor warnings/small errors.	Both attempted with several errors.	Not validated, or many errors.
8	**Testing & cross-device (4)**	Tested on mobile + desktop (+ second browser); Console clean; links/images all work; issues fixed and re-tested.	Tested on two sizes; a small issue remains.	Tested only on desktop, or issues unfixed.	Not tested; broken links/images.
	Subtotal	**30**			
9	**Viva / demo (5)**	Explains every decision confidently and accurately; demonstrates features; honest about limits; handles questions well.	Explains most decisions; minor uncertainty.	Partial explanation; struggles with some questions.	Cannot explain own work.

Total: 35 marks. Pass standard: **≥ 50% overall AND a functional, responsive website AND a passing viva.** A polished site the learner cannot explain cannot pass on the viva criterion alone.

K.3 Deliverables checklist

- ☐ 3–5 page responsive website in an organised folder
- ☐ Consistent navigation, all links working
- ☐ At least one image per page, all with `alt` text
- ☐ Responsive layout verified at 375px, 768px and desktop
- ☐ Styled, labelled contact form

- [ ] Valid HTML and CSS (zero errors)
- [ ] Tested in Chrome and a second browser; Console clean
- [ ] **Published with a public URL** (for the certificate)
- [ ] Site presented and viva answered
- [ ] Written reflection (Section M / assignment)

L. VIVA QUESTION LIST WITH MODEL ANSWERS

(Instructor asks 4–6 of these per learner. Learners may see the list beforehand so the viva is about understanding, not surprise.)

1. Why did you choose these colours and fonts? *Model answer:* "I chose three or four colours — a dark colour for the header/footer, an accent for buttons, and a light background — so the site looks consistent. I used two fonts: one for headings and one for body, so text stays readable. I stored the colours as variables in `:root` so I could change them in one place."

2. Walk me through the structure of one of your HTML pages. *Model answer:* "It starts with `<!DOCTYPE html>` and `<html lang='en'>`. The `<head>` has the charset, viewport, title and stylesheet link. In the `<body>` I have a `<header>` with the logo and `<nav>`, a `<main>` holding the page content in sections, and a `<footer>`. I used semantic elements so the code is clear and accessible."

3. How did you link your pages and your stylesheet? *Model answer:* "Each page links the others through the `<nav>` using relative paths like `shop.html`. Every page has one `<link rel='stylesheet' href='css/style.css'>` in the head, so all pages share the same design and update together."

4. How does your site adapt to a phone? *Model answer:* "I used the viewport meta tag and media queries. Below 1023px the product grid becomes two columns; below 767px it becomes one column and the header stacks. I tested it with the Chrome device toolbar at 375px and 768px."

5. Show me how your stylesheet is organised. *Model answer:* "It goes in order: custom properties, reset and base, typography, layout, header and nav, hero, buttons, product grid, forms, footer, then media queries at the bottom. Each section has a comment header so it is easy to find things."

6. Why did you use grid for the products and flexbox for the header? *Model answer:* "The products need neat rows and columns that line up, which grid does well with `grid-template-columns: repeat(3, 1fr)`. The header is a simple single row with space between the logo and the nav, which is what flexbox is best at."

7. What does your contact form do when someone clicks 'Send'? *Model answer (honest):* "Right now it displays and will not submit if a required field is empty, because of the `required` attribute. But HTML alone cannot send the message anywhere — that needs a form service like Formspree or a server. I would add that as a paid next step."

8. What did you test, and what did you fix? *Model answer:* "I clicked every link, checked every image, opened the Console for errors, tested the form, checked the site at phone and desktop widths, and ran the W3C validators. I fixed a wrong image file name and a missing semicolon that was breaking a rule."

9. What is a relative path and why does it matter? *Model answer:* "A path relative to the current file, like `images/hero.jpg`, using forward slashes. Absolute paths like `C:\\...` only work on my computer, so the images break when I publish. Relative paths work everywhere."

10. Tell me about one bug you found and how you found it. *Model answer:* "My product heading was not styled. I inspected it in DevTools and saw the rule `.prodcut h3` did not match my class `product` — a spelling mistake. I fixed the selector and it worked."

11. Did your page pass validation? How do you know? *Model answer:* "Yes. I pasted each page into validator.w3.org and the CSS into jigsaw.w3.org/css-validator, and both showed no errors. I fixed errors from the top down and re-validated."

12. What would you add next if you learned JavaScript? *Model answer:* "A working menu that opens and closes on click, real form submission and validation, and maybe an image slider. Right now the menu uses a CSS checkbox trick, which is not ideal."

13. What are you most proud of, and what would you improve? *Model answer:* personal but specific — e.g. "I am proud that the product grid is responsive and clean. I would improve the About page content and make the form actually send messages."

14. Why can't you call yourself a full-stack developer after this course? *Model answer:* "Because I have learned HTML and CSS, which build and style the front of a website. Full-stack also needs JavaScript, a server, a database and deployment logic — that is the next course."

Scoring the viva (5 marks): 5 = confident and accurate on all; 4 = strong with minor hesitation; 3 = correct on most; 2 = partial; 1 = cannot explain; 0 = no viva.

M. SESSION ASSIGNMENT

Assignment title: "Final Reflection and Portfolio Handover"

Purpose: Close the course professionally: reflect on the learning journey and submit a clean, complete portfolio for certification.

Realistic scenario: To receive the CEA Certificate of Web Design Competence, the learner must submit a tidy project folder and a short written reflection, exactly as a freelancer would hand over files and notes to a client.

Detailed instructions:

1. Ensure your project folder contains: all HTML pages, `css/style.css`, and an `images/` folder with all assets.
2. Publish the site (where possible) and copy the public URL into your workbook.
3. Write a reflection of **250–400 words** answering the reflection questions in Section N:
    - three specific things you learned;
    - two things you found difficult and how you dealt with them;
    - one thing you are proud of in your website;
    - one thing you would improve next;
    - one thing you want to learn next (be specific);
    - how you felt in Session 1 compared with now.
4. Include a short **self-audit**: paste the deliverables checklist (K.3) with your ticks.

Required deliverables:

- Complete, organised project folder (and published URL if available).
- Written reflection (250–400 words).
- Completed self-audit checklist.

Submission format: Project folder in the class folder / shared drive; reflection and URL in your workbook or shared document.

Deadline: End of Session 12.

Marking rubric (10 marks):

Criterion	3 — Excellent	2 — Good	1 — Developing	0 — Missing
Reflection depth (3)	Specific, honest, thoughtful	Mostly specific	Vague	No reflection
Portfolio completeness (3)	All files, organised, published	Most files, mostly tidy	Incomplete/messy	Not submitted
Self-audit accuracy (2)	Accurate, matches reality	Mostly accurate	Inaccurate	None
Professional presentation (2)	Clear, neat, on time	Acceptable	Sloppy/late	Not submitted

Common mistakes to avoid: forgetting to include the `css/` or `images/` folder; leaving the published URL out; vague reflection ("I learned a lot"); submitting with absolute paths still present.

Instructor reference solution: a well-organised folder following Section F.3, a live URL, and a reflection that names concrete skills (flexbox, media queries, validation) and honest gaps (no JavaScript yet).

N. REFLECTION, RECAP AND COURSE WRAP-UP

N.1 Reflection questions (for learners)

1. What are three specific things you can now do that you could not do in Session 1?

2. What was the hardest part of building your website, and how did you get past it?
3. What is one thing in your site you are genuinely proud of, and why?
4. If you had one more day, what would you improve first?
5. What is one thing you want to learn next, and why?
6. How do you feel now compared with the first day?

N.2 Plain-English summary

Today you finished. You finalised a complete 3–5 page responsive business website, audited it against a professional rubric, tested and validated it, and — where possible — published it to the world. You stood up, showed it to the class and explained your decisions under questioning. That is exactly what an entry-level web designer does.

N.3 Five important takeaways from the whole course

1. HTML is the structure, CSS is the appearance — together they build a website.
2. Consistency comes from one shared stylesheet, a palette, a type scale and a spacing system.
3. Responsive design (flexbox, grid, media queries) is not optional; most visitors are on phones.
4. Testing and debugging before publishing is what separates a professional from an amateur.
5. Clean, validated, accessible code is a professional habit, not a luxury.

N.4 The honest wrap-up — what this certificate means

Be clear-eyed about this. Completing this course makes you a competent beginner and an entry-level front-end web designer using HTML and CSS. It does not make you a full-stack developer.

What you can now do:

- Build a clean, responsive, multi-page business website with HTML and CSS.
- Apply consistent branding, typography and spacing.
- Use flexbox, grid and media queries for responsive layouts.
- Add CSS interactivity, embed maps and videos, and handle basic accessibility.
- Test, debug and validate a site, and publish it to free hosting.
- Explain your technical decisions in plain English.

What this course does NOT cover (and does not claim to):

- **JavaScript** — the language that adds real interactivity (menus, form sending, sliders).
- Backend development, servers and databases.
- Frameworks and libraries (React, Vue, etc.).
- Advanced performance, security or SEO.

Your realistic next steps at CEA:

1. **Web Development course** — learn JavaScript, then how websites talk to servers. This is the natural continuation of what you have learned.
2. **Frameworks** — after JavaScript, learn a modern front-end framework (React or Vue) to build larger applications.
3. **Complementary courses** — **Graphic Design** (stronger visuals), **Digital Marketing** (get visitors), and **Content Creation** (photos, video, writing).
4. **Practice and start small** — rebuild the Adaeze Fashion site for a real local business, publish it, and keep a portfolio of live URLs.

You are not an expert yet — and that is exactly where every expert started. You are a *competent beginner with a real, marketable skill*. That is a genuine achievement. Keep building, keep testing, and keep learning — From Zero to Expert, Together.

N.5 Certificate note

The **CEA Certificate of Web Design Competence** is awarded when the learner has: attended at least 10 of 12 sessions, completed assignments satisfactorily, published a 3–5 page responsive website, scored at least 50% overall, and presented the site to the class. Certificates include a verification code checkable at the academy's verification page.

O. INSTRUCTOR EVALUATION NOTES

Observable signs of genuine competence:

- Learner presents without reading code, explaining *why* rather than *what*.
- Learner demonstrates responsiveness live (dragging the window) with confidence.
- Learner answers viva questions accurately and honestly, including admitting limits.
- Learner's site passes validation and has a clean Console.
- Learner can reproduce the process for a new business (as in the Emmanuel case study).
- Learner helps a struggling peer — a strong sign of understanding.

Common misconceptions to watch for and correct:

- "My site is finished because it looks fine to me." → Test on other sizes/browsers and validate.
- "The form sends emails already." → HTML forms need a service/server; be honest.
- "JavaScript and HTML/CSS are the same thing." → They are different; JavaScript is next.
- "Passing this course makes me a full-stack developer." → It makes you an entry-level front-end designer.
- "Publishing needs paid hosting." → Free hosting (GitHub Pages, Netlify) is enough for this level.

Common capstone failure points and remediation:

Failure point	Likely cause	Remediation
Only 2 pages	Ran out of time / poor planning	Pair with a learner who finished; add one simple page from a reused template
Not responsive; overflow on mobile	Missing viewport meta or media queries	Re-teach Session 6/9 media queries; provide a mobile media-query template
Images broken after publishing	Absolute paths	Convert all paths to relative; re-test the live URL
Validator full of errors	Unclosed tags/missing attributes	Fix top-down; provide the Common Errors Reference from Session 11
Form labels not connected	`for` / `id` mismatch	Demonstrate the fix; re-test clicking labels
No `alt` text / poor contrast	Skipped accessibility	Run the Session 11 accessibility checks together
Cannot explain own code	Copied without understanding	Viva remediation: walk through each page slowly; require re-explanation
Site not published	No account / no internet	Publish from an academy account; or learner publishes with support after class

Suggested remedial exercises:

1. **Rebuild sprint:** rebuild a 3-page site from the instructor's structure in one sitting to consolidate the process.
2. **Validate-and-fix drill:** take a flawed page and get it to zero validator errors.
3. **Viva rehearsal:** practise explaining three decisions aloud to a peer before re-assessment.

Differentiation:

- **Level 1 — Supported:** Provide a starter template (all pages, `:root`, reset and a basic grid), allow extra time, and offer one-on-one support. Accept a 3-page site. Focus on a working, tidy, responsive result over features.
- **Level 2 — Independent (standard):** The brief and rubric as written; present and viva.
- **Level 3 — Challenge:** Add a functioning form service (Formspree/Netlify Forms), a second `@keyframes`, a gallery page with a CSS grid, or manager a smooth-scroll "back to top". Help a struggling peer and present both sites.

Timing guidance: Decide at the start whether publishing happens this session or is scheduled afterwards; it can consume a lot of the 105 minutes. If time is tight, prioritise: (1) finalise and self-audit, (2) present and viva, (3) publish as follow-up. Keep the presentation short (3–4 minutes each) to fit the class; run a strict but kind schedule.

ASSESSMENT RECORD AND CERTIFICATE SIGN-OFF

Learner Name: _____

Date: _____

Business / Project: _____

Rubric criterion	Marks available	Marks awarded
1. Structure & semantics	4	____
2. Visual design	4	____
3. Layout & responsiveness	4	____
4. Content quality	3	____
5. Contact form	3	____
6. Code quality	4	____
7. Validation	4	____
8. Testing & cross-device	4	____
Website subtotal	**30**	____
9. Viva / demo	5	____
TOTAL	**35**	____

Percentage: _____ % **Grade:** _____ **Result:** Competent / Not Yet Competent

Published URL: _____

Certificate requirement checks:

- ☐ Attended at least 10 of 12 sessions
- ☐ Assignments completed satisfactorily
- ☐ 3–5 page responsive website published
- ☐ Overall score ≥ 50%
- ☐ Website presented to the class

Instructor Signature: _____

Certificate Awarded: Yes / No **Certificate Code:** _____

Comments:

END OF SESSION 12

COURSE COMPLETION NOTE

Congratulations — you have completed the **Cyber Elias Academy Web Design course**. Over twelve sessions you went from *"What is HTML?"* to a published, responsive, tested business website you built and can explain. You can now:

- plan, build and style a multi-page website with HTML and CSS;
- make it responsive for phones, tablets and desktops;
- add CSS interactivity and embed media;
- test, debug and validate your work; and

- publish it online for the world to see.

This makes you an **entry-level front-end web designer (HTML/CSS)** — not yet a full-stack developer. The next step is the **Web Development course** to learn **JavaScript**, and after that **frameworks**. Keep practising, keep building for real local businesses, and keep testing before you publish.

From Zero to Expert, Together.

End of the Web Design course — well done.